



CÁLCULO NUMÉRICO PARA MORTALES



Fundamentos de la Computación Numérica



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES

HECHO EN ARGENTINA

NAHUEL VILLAFañE
2026

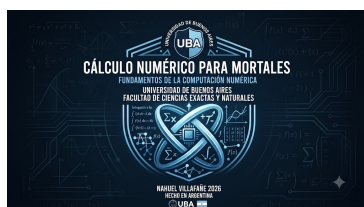


“El estudio profundo de la naturaleza es la fuente más fértil de descubrimientos matemáticos.” — Joseph Fourier

Apuntes de Cálculo Numérico

Fundamentos de la Computación Numérica

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales



Departamento de Matemática
Cálculo Numérico

Año 2026

*'El estudio profundo de la naturaleza
es la fuente más fértil de descubrimientos matemáticos.'*
— Joseph Fourier

Indice General

1. Fundamentos de la Computacion Numerica: De Turing al Epsilon de Maquina

Prefacio: Antes de arrancar, un mate y una charla

¿Por que necesitamos fundamentos teoricos?

Maquinas de Turing: La base de todo

La intuicion: un cartero con un cuaderno infinito

Definicion formal: las tripas del bicho

Un ejemplo concreto: Sumar dos numeros en unario

Codificacion: El arte de representar todo como tiras de simbolos

La Maquina de Turing Universal: La madre de todas las compus

La Tesis de Church-Turing: El dogma de fe del computologo

Numeros Computables: Los reales que podemos tocar

Incomputabilidad: El problema de la detencion y otros fantasmas

Variantes de la Maquina de Turing

Complejidad Computacional: Lo que cuesta resolver un problema

¿Por que no basta con saber que es resoluble?

Notacion asintotica: El arte de quedarse con lo importante

Clases de Complejidad: El club de los problemas

Analisis de complejidad de algoritmos clasicos

El Teorema Maestro: Receta de cocina para recurrencias

Teorema Maestro: Profundizacion y ejemplos avanzados

Aritmetica de Maquina: Donde lo perfecto se vuelve finito

Representacion de enteros: La parte facil

Punto Flotante: El corazon del problema (y de la solucion)

Epsilon de Maquina: La minima diferencia que importa

Redondeo: El arte de aproximar sin romper todo

Consecuencias peligrosas: Cancelacion catastrofica y mas

Geometria Computacional: Cuando el error te dibuja cualquier cosa

Precision arbitraria por software: El santo remedio (con costo)

Ejercicios del Capitulo 1

Ejercicios Teoricos (T1 a T10)

Ejercicios Computacionales (C1 a C10)

Soluciones detalladas

Computabilidad, Complejidad y Aritmetica de Maquina: Los Limites del Calculo

Que se puede calcular, cuanto cuesta, y por que a veces la computadora miente. Todo en criollo y con mucho Python.

Prefacio: Un mate, una charla y una idea que cambio el mundo

Che, ¿como andas? Sentate, tomate un mate, que esto es largo pero vale la pena. Soy Nahuel Villafañe, un estudiante avanzado de fisica, de esos que ya transpiraron la camiseta en los pasillos de Exactas jajaja. Curse muchas matematicas, fisicas, laboratorio, escribo apuntes y ayudo compartiendo los apuntes porque mi meta es ayudar hasta donde se pueda con las herramientas que tengo. Di clases particulares, soy programador y como todo ser humano me mande mis cagadas jaja. Y la peor de todas fue cuando un programa en Python me daba un resultado hermoso, y despues me di cuenta de que era pura basura numerica porque no entendia como funcionaba la maquina por adentro, cosa que ahora en este capitulo vamos a ver como funciona la computadora.

Pero antes de llegar a los numeros y los errores de redondeo, hay un escalon mas fundamental: entender que significa “calcular”. Tranqui, no voy a andar muy filosofico, pero son preguntas esenciales como por ejemplo: ¿Que problemas se pueden resolver con un algoritmo? ¿Cuales no? ¿Y cuanto tarda un algoritmo en resolverlos? Estas preguntas, son la base de toda la computacion moderna. Y la respuesta vino de la mente brillante de Alan Turing en 1936, cuando tenia 23 años y se invento un modelo de computacion tan simple que parece un juguete, pero tan poderoso que captura la esencia de cualquier algoritmo.

En este capitulo vamos a hacer un viaje que arranca en la cabeza de Turing, pasa por la teoria de la complejidad, y termina en la aritmetica de punto flotante, que es la que te hace putea cuando el resultado de una resta te da cualquier verdura. Y como siempre, con mucho codigo Python para que puedas experimentar y ver los graficos con tus propios ojos.

¿Cual es el plan? Vamos a dividir el viaje en tres grandes estaciones:

1. **Computabilidad:** ¿Que problemas se pueden resolver con un algoritmo? (La teoria que nos dice los limites absolutos.)
2. **Complejidad:** ¿Que problemas se pueden resolver en un tiempo razonable? (La teoria que nos dice si el algoritmo es factible.)
3. **Aritmetica de maquina:** ¿Como se representan los numeros y que errores se cometen? (La practica que te golpea la puerta todos los dias.)

Cada una de estas estaciones esta llena de conceptos que en los libros tradicionales aparecen como definiciones secas, pero aca los vamos a bajar a tierra con ejemplos, graficos, y codigo.

Maquinas de Turing: El concepto de algoritmo

La intuicion: un cartero con un cuaderno infinito

Imaginate un cartero muy particular. Vive en un mundo de fantasia donde las casas estan en una calle infinita, una al lado de la otra, y en cada casa puede haber una letra o estar vacia (un blanco). El cartero tiene un cuaderno donde anota su "estado de animo" (puede estar alegre, triste, concentrado, etc., pero solo una cantidad finita de estados). Tiene ademas un manual de instrucciones que le dice: "Si estas en el estado A y en la casa actual hay una letra B, entonces escribi la letra C, movete a la casa de la izquierda, y pasa al estado D".

Ese cartero es una maquina de Turing. Suena tonto, pero con esa pavada puedes calcular TODO lo que es calculable. No es joda.

¿Por que es importante para nosotros, que hacemos calculo numerico? Porque la maquina de Turing define que significa "algoritmo". Cada vez que escribis una funcion en Python para resolver un sistema lineal, estas implementando un algoritmo que, en teoria, podria ejecutarse en una maquina de Turing. Saber sus limites te evita buscar algoritmos para problemas que simplemente no tienen solucion algoritmica (como el problema de la detencion).

Definicion formal: las tripas del bicho

Bueno, basta de vueltas. Formalicemos.

- **Cinta infinita:** Es una secuencia de celdas indexadas por los enteros. Cada celda contiene un simbolo de un alfabeto finito Γ . Hay un simbolo especial $B \in \Gamma$ que representa el "blanco". Al principio, solo una cantidad finita de celdas tienen simbolos distintos de B ; el resto estan en blanco.
- **Cabezal:** Es un puntero que en cada instante esta posicionado sobre una celda. Puede leer el simbolo, escribir un nuevo simbolo, y moverse una celda a la izquierda (L) o una a la derecha (R).
- **Estados:** Un conjunto finito Q . Hay un estado inicial $q_0 \in Q$ y un subconjunto de estados de aceptacion $F \subseteq Q$ (a veces tambien estados de rechazo, pero por simplicidad nos quedamos con aceptacion).
- **Funcion de transicion:** $\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$. Esta es la joyita. Dados el estado actual (que no es de aceptacion) y el simbolo leído, te da el nuevo estado, el simbolo a escribir, y la direccion de movimiento.

¿Como funciona? Inicialmente, la maquina esta en el estado q_0 , con el cabezal en la primera celda de una palabra de entrada $w \in \Sigma^*$ (donde $\Sigma \subseteq \Gamma \setminus \{B\}$ es el alfabeto de entrada). A cada paso, lee el simbolo, consulta δ , escribe, se mueve, y cambia de estado. Si en algun momento llega a un estado $q \in F$, se detiene y **acepta** la entrada. Si llega a una configuracion donde la funcion δ no esta definida y no esta en un estado de aceptacion, se detiene y **rechaza**. Y si nunca llega a un estado de aceptacion ni se traba, entonces **no se detiene** (entra en bucle infinito).

Traduccion Matematica. Cuando veas $\delta(q, a) = (q', b, D)$, lo lees asi:

"Si estoy en el estado q (ponele, 'buscando el final') y el cabezal lee el simbolo a (por ejemplo, un '1'), entonces cambio al estado q' (me pongo en modo 'retroceder'), escribo el simbolo b en esa celda (un '1' para sumar), y muevo el cabezal un paso en la direccion D (derecha o izquierda)."

Cada paso es atomico, discreto. La maquina no tiene nocion del "todo", solo ve la celda actual y su estado interno. La magia emerge de la composicion de millones de estos pasitos tontos.

Analogia con la vida real. Es como cocinar con una receta escrita en un papelito. Vas leyendo paso a paso: "Si la mezcla esta espesa, agrega agua y revolvé". Eso es una transicion. Tu memoria de corto plazo (los pasos que recordas haber hecho) es el estado. La mesa de la cocina es la cinta, donde pones los ingredientes. Lo unico que ves es lo que tenes adelante. Cocinar un plato complejo se reduce a una secuencia de decisiones simples.

Un ejemplo concreto: Sumar dos numeros en unario

Para que no quede todo en el aire, vamos a construir una maquina de Turing que suma dos numeros representados en unario. El numero m se representa como una cadena de m unos. Para representar la entrada (m, n) , usamos m unos, un cero como separador, y n unos. Por ejemplo, $m = 2, n = 3$ se escribe como "110111". Queremos que la maquina deje en la cinta $m + n$ unos (en este caso, "11111").

La estrategia es simple: reemplazar el 0 del medio por un 1, y borrar el ultimo 1 para que quede la cantidad justa. Pero como las maquinas de Turing no pueden "borrar" en el sentido de achicar la cinta, vamos a transformar el ultimo 1 en blanco.

- Alfabeto de entrada: $\Sigma = \{1, 0\}$
- Alfabeto de cinta: $\Gamma = \{1, 0, B\}$
- Estados: q_0 (buscar el 0), q_1 (buscar el ultimo 1), q_2 (borrar y volver), q_{halt} (aceptacion).
- Estado inicial: q_0
- Estados de aceptacion: $\{q_{halt}\}$
- Transiciones:
 1. $\delta(q_0, 1) = (q_0, 1, R)$ // Si veo 1, sigo avanzando (estoy en el primer numero).
 2. $\delta(q_0, 0) = (q_1, 1, R)$ // Encontre el 0, lo convierto en 1 y paso a modo "buscar el final".
 3. $\delta(q_1, 1) = (q_1, 1, R)$ // Sigo avanzando por los 1 del segundo numero.
 4. $\delta(q_1, B) = (q_2, B, L)$ // Llegue al final de la cinta, empiezo a retroceder.
 5. $\delta(q_2, 1) = (q_{halt}, B, R)$ // Encuentro el ultimo 1, lo borro con blanco y acepto.

Simulemos mentalmente para "110111" (2+3):

```
Estado q0: [1] 1 0 1 1 1 B ... -> ve 1, escribe 1, derecha
Estado q0: 1 [1] 0 1 1 1 B ... -> ve 1, escribe 1, derecha
Estado q0: 1 1 [0] 1 1 1 B ... -> ve 0, escribe 1, derecha, cambia a q1
Estado q1: 1 1 1 [1] 1 1 B ... -> ve 1, escribe 1, derecha
Estado q1: 1 1 1 1 [1] 1 B ... -> ve 1, escribe 1, derecha
Estado q1: 1 1 1 1 1 [1] B ... -> ve 1, escribe 1, derecha
Estado q1: 1 1 1 1 1 1 [B] ... -> ve B, escribe B, izquierda, cambia a q2
Estado q2: 1 1 1 1 1 [1] B ... -> ve 1, escribe B, derecha, cambia a qhalt
Resultado: 1 1 1 1 1 B B ... (5 unos, o sea 5 = 2+3)
```

¡Funciona! Y es solo un puñado de reglas. Aca te das cuenta de que la complejidad emerge de la simplicidad.

Reflexion: Cuando estas escribiendo un codigo en Python para hacer una multiplicacion de matrices, estas haciendo algo conceptualmente similar a esto: definis un monton de pasos simples que, ejecutados secuencialmente, producen un resultado complejo. La maquina de Turing es el modelo mas elemental de ese proceso.

Codificacion: El arte de representar todo como tiras de simbolos

Uno de los trucos mas grosos de Turing fue darse cuenta de que cualquier maquina de Turing se puede describir a si misma como una cadena de simbolos. Esto es fundamental porque permite la existencia de la **maquina universal** (que simula cualquier otra) y de los **argumentos diagonales** (que demuestran limites).

Codificacion de numeros naturales: La forma mas naive es la unaria: el numero n se codifica como una cadena de $n + 1$ unos, $\langle n \rangle = 1^{n+1}$. O, en binario sin ceros a la izquierda. Nosotros usamos la que mas convenga.

Codificacion de tuplas: Si necesitas codificar un par (m, n) , puedes usar la **funcion de emparejamiento de Cantor**:

$$\langle (m, n) \rangle = \frac{(m+n)(m+n+1)}{2} + n$$

Esta funcion es biyectiva: de un unico natural puedes recuperar m y n . Es como cuando comprimis dos archivos en un solo zip, pero sin perdida.

Codificacion de una maquina de Turing M : Aca la cosa se pone seria. Una maquina $M = (Q, \Gamma, \delta, q_0, F)$ es un objeto finito. Enumero mis estados: $q_0, q_1, \dots, q_k$. Enumero mis simbolos: γ_0 (blanco), $\gamma_1, \dots, \gamma_m$. Las direcciones pueden ser $D_0 = L, D_1 = R$. Una transicion $\delta(q_i, \gamma_j) = (q_k, \gamma_\ell, D_d)$ se puede representar como una tupla de 5 numeros: (i, j, k, ℓ, d) . Uso un alfabeto simple para escribir estos numeros, por ejemplo, represento el numero x como 0^{x+1} , y separo las transiciones con un simbolo especial como $\#$.

Entonces, toda la maquina M se convierte en una cadena enorme del tipo:

$$\langle M \rangle = 0^{i_1+1} \# 0^{j_1+1} \# 0^{k_1+1} \# 0^{\ell_1+1} \# 0^{d_1+1} \# \# 0^{i_2+1} \# \dots$$

Las cadenas que no representan una maquina valida (porque no respetan el formato) se asocian a una maquina trivial que se detiene inmediatamente.

¿Para que sirve todo esto? Para tres cosas fundamentales:

1. **Enumeracion efectiva:** Como las cadenas finitas sobre un alfabeto finito se pueden listar en orden lexicografico, podemos listar todas las maquinas de Turing: $M_0, M_1, M_2, \dots$. Esto nos da una *enumeracion efectiva de todas las funciones computables*. Cada funcion computable tiene un indice en esta lista.
2. **Maquina universal:** Podemos construir una maquina U que recibe como entrada un par $\langle M, w \rangle$ (codificado) y simula M sobre w . U interpreta la tabla de transiciones de M y la ejecuta paso a paso.
3. **Argumentos diagonales:** Como una maquina puede recibir su propio indice como entrada $\langle \langle M \rangle \rangle$, podemos construir situaciones autorreferentes. El ejemplo clasico es el problema de la detencion.

Reflexion: La primera vez que vi esto, flashee mal. Es como si un libro contuviera las instrucciones para leerse a si mismo y ejecutarse. Es la base del concepto de *quine* en programacion (programas que imprimen su propio codigo fuente). En Python, un quine simple es:

```
s = 's = %r\nprint(s %% s)'
print(s % s)
```

Ejecutalo, es flashero.

La Maquina de Turing Universal: La madre de todas las compus

Teorema (Turing, 1936): Existe una maquina de Turing U tal que para toda maquina M y toda entrada w , $U(\langle M, w \rangle) = M(w)$. Es decir, U simula el comportamiento de M sobre w .

Idea de la construccion: U usa tres pistas en su cinta (es equivalente a una cinta unica con un alfabeto mas rico):

- **Pista 1:** La descripcion codificada de M (su tabla de transiciones).
- **Pista 2:** El contenido actual de la cinta simulada de M , con una marca que indica la posicion del cabezal.
- **Pista 3:** El estado actual de M (representado como un numero).

En cada paso, U lee el estado actual (pista 3) y el simbolo bajo el cabezal (pista 2), busca en la tabla de M (pista 1) la transicion correspondiente, y actualiza las pistas 2 y 3 en consecuencia. **¿Por que es tan importante?** Porque U es un *hardware fijo* que puede ejecutar cualquier *software*. Es el precursor conceptual de la computadora moderna. En tu notebook, el procesador es fijo (un Intel o un ARM), pero los programas que ejecuta pueden ser cualquiera: un navegador, un juego, un solver de ecuaciones. Esa separacion entre hardware y software es lo que propuso Turing de manera abstracta.

La Arquitectura de von Neumann: En 1945, John von Neumann formalizo esto en el "First Draft of a Report on the EDVAC", describiendo una arquitectura con:

- **Memoria unificada:** Tanto los datos como las instrucciones se almacenan en la misma memoria.
- **Unidad de control:** Lee instrucciones de la memoria, las decodifica y las ejecuta secuencialmente (salvo saltos).
- **Unidad aritmetico-logica (ALU):** Realiza operaciones sobre datos.

Esto es EXACTAMENTE lo mismo que la maquina universal de Turing, pero construido con valvulas de vacio (al principio) y ahora con transistores microscopicos. La memoria es la cinta, la unidad de control lee el "estado" las "transiciones" (el programa en memoria).

La Tesis de Church-Turing: El dogma de fe del computologo

Tesis de Church-Turing: *Toda funcion efectivamente calculable (es decir, que puede ser computada por un ser humano siguiendo un procedimiento mecanico finito) es computable por una maquina de Turing.*

Aclaracion importante: Esto NO es un teorema, porque el concepto de "efectivamente calculable" es informal. Es una tesis filosofica que la evidencia empirica apoya fuertemente:

- Todos los modelos formales de computacion propuestos (funciones recursivas de Church, calculo lambda, maquinas de registro, etc.) resultaron ser equivalentes a las maquinas de Turing.
- Nunca nadie encontro un algoritmo (en el sentido intuitivo) que no pudiera ser implementado en una maquina de Turing.

- Las nociones intuitivas de algoritmo (diagrama de flujo, pseudocodigo, Python) son todas traducibles a maquinas de Turing.

Implicacion practica para nosotros: Si aceptamos la tesis, entonces los limites de las maquinas de Turing son los limites ABSOLUTOS de la computacion. No importa si tu computadora es cuantica, biologica, o de agua y tubos: no va a poder resolver el problema de la detencion, ni calcular funciones no Turing-computables. Los limites son logicos, no tecnologicos.

Numeros Computables: Los reales que podemos tocar

Un numero real x se dice **computable** si existe una maquina de Turing que, dado un entero n (en unario, por ejemplo), produce una aproximacion racional r_n tal que $|x - r_n| < 1/n$. Dicho en criollo: existe un algoritmo que te puede calcular cualquier digito de la expansion decimal de x si le das suficiente tiempo.

Ejemplos de numeros computables:

- Todos los racionales: $\frac{1}{3}, \frac{22}{7}$.
- Raices de polinomios con coeficientes racionales: $\sqrt{2}, \sqrt[3]{7}$.
- Constantes classicas: $\pi, e, \ln 2$.
- Valores de series convergentes cuyos terminos son computables: $\sum_{n=0}^{\infty} \frac{1}{n!}$.

¿Cuantos numeros computables hay? Hay una cantidad **numerable**. Cada maquina de Turing tiene una descripcion finita; el conjunto de todas las descripciones finitas es numerable. Asi que a lo sumo hay $\aleph_0$ numeros computables.

¿Cuantos numeros reales hay? Hay una cantidad **no numerable** ($\mathfrak{c} = 2^{\aleph_0}$). Por lo tanto, **la inmensa mayoria de los numeros reales NO son computables**. Es mas, si elegis un numero real al azar (con probabilidad 1), no es computable. Es un resultado contundente.

Paradoja: No podemos dar un ejemplo concreto de un numero incomputable, porque en el momento en que lo describimos con una regla finita, esa regla es un algoritmo y por lo tanto el numero seria computable. Solamente podemos demostrar que existen, por un argumento de cardinalidad, pero estan condenados a ser innumbrables”.

Incomputabilidad: El problema de la detencion y otros fantasmas

El problema de la detencion (Halting Problem): Dada la codificacion de una maquina de Turing M y una entrada w , determinar si M se detiene al ejecutarse con entrada w . Formalmente, queremos una maquina H tal que:

$$H(\langle M, w \rangle) = \begin{cases} \text{ACEPTA} & \text{si } M(w) \text{ se detiene} \\ \text{RECHAZA} & \text{si } M(w) \text{ no se detiene} \end{cases}$$

Teorema (Turing, 1936): Tal maquina H NO EXISTE. El problema de la detencion es **indecidible**.

Demostracion (el argumento diagonal): Suponemos que existe H . Construimos una nueva maquina D (la maquina ”diagonal”):

D recibe como entrada una cadena $\langle M \rangle$ (la codificacion de una maquina):

1. Llama a $H(\langle M, \langle M \rangle \rangle)$. Es decir, le pregunta a H : ”¿Se detiene M cuando recibe su propia descripcion como entrada?”.

2. Si H dice que SI (ACEPTA), entonces D entra en un bucle infinito (NO se detiene).
3. Si H dice que NO (RECHAZA), entonces D se detiene inmediatamente (SI se detiene).

Ahora bien, ¿que pasa si ejecutamos $D(\langle D \rangle)$? Es decir, le damos a D su propia descripcion.

- **Caso 1:** $D(\langle D \rangle)$ se detiene. Entonces, por la construccion de D , esto ocurre solo si H dijo que $D(\langle D \rangle)$ NO se detiene. Pero H es infalible, así que si H dijo que NO, entonces $D(\langle D \rangle)$ NO se detiene. ¡Contradiccion!
- **Caso 2:** $D(\langle D \rangle)$ NO se detiene. Entonces, por construccion, esto significa que H dijo que $D(\langle D \rangle)$ SI se detiene. Luego, $D(\langle D \rangle)$ SI se detiene. ¡Contradiccion otra vez!

En ambos casos llegamos a una contradiccion. La unica salida es que nuestra suposicion inicial sea falsa: H no puede existir.

¿Que significa esto en la practica?

- No existe un depurador automatico que te diga si tu programa va a entrar en un bucle infinito.
- No existe un verificador que te asegure al 100 % que un programa es correcto para todas las entradas.
- Muchos problemas en logica, matematicas y computacion se reducen al problema de la detencion, por lo que tambien son indecidibles (por ejemplo, saber si una ecuacion diofantica tiene solucion, o si dos funciones en un lenguaje de programacion calculan lo mismo).

Reflexion personal: La primera vez que entendí esta demostracion, me exploto la cabeza. Es la hermana computacional de la paradoja de Russell (.el conjunto de todos los conjuntos que no se contienen a si mismos”) y de la demostracion de Cantor de que los reales no son numerables. La autorreferencia siempre lleva a problemas logicos.

Variantes de la Maquina de Turing

Existen muchas variantes del modelo original de Turing: maquinas con cintas multiples, con cintas bidimensionales, no deterministas, etc. Todas ellas resultan equivalentes en poder computacional (pueden simularse unas a otras). Esto refuerza la robustez de la Tesis de Church-Turing.

Maquinas no deterministas: Pueden tener varias transiciones posibles para un mismo estado y simbolo. La maquina .elige”la que la lleva a aceptar, si existe. El poder de las no deterministas es el mismo que las deterministas, pero pueden resolver problemas en menos pasos (de ahí la clase NP). Una maquina no determinista puede simularse con una determinista a costa de tiempo exponencial (explorando todas las ramas).

Dale, repasemos lo importante de Turing

- Una maquina de Turing es un modelo abstracto que define la noción de algoritmo.
- Consiste en una cinta infinita, un cabezal, estados finitos y una funcion de transicion.
- La codificacion permite que una maquina reciba a otra como entrada, lo que habilita la maquina universal.

- La Tesis de Church-Turing establece que todo algoritmo puede ser simulado por una maquina de Turing.
- El problema de la detencion es indecidible: no existe un algoritmo que determine si un programa se detiene.
- La mayoría de los numeros reales son incomputables: no existe un algoritmo que los calcule.

Complejidad Computacional: Lo que cuesta resolver un problema

¿Por que no basta con saber que es resoluble?

Sabemos que un problema es Turing-computable. Joya. Ahora, ¿cuanto tarda en correr el algoritmo? Porque una cosa es que algo sea teoricamente resoluble, y otra muy distinta es que puedas esperar el resultado. Si tu algoritmo para resolver un sistema de n ecuaciones tarda $O(2^n)$, para $n = 100$ vas a necesitar mas tiempo que la edad del universo. Como dijo alguien: "Exponencial es el diablo".

La **complejidad computacional** mide los recursos (tiempo y espacio/memoria) en funcion del tamaño n de la entrada. El tamaño de la entrada es, por ejemplo, la cantidad de bits necesarios para representar el problema. Para un sistema de ecuaciones, n es la cantidad de ecuaciones; para ordenar numeros, es la cantidad de elementos.

Complejidad temporal $T(n)$: Cantidad de pasos (transiciones de maquina de Turing, o instrucciones elementales) que ejecuta el algoritmo en el peor caso, en funcion de n .

Complejidad espacial $S(n)$: Cantidad de celdas de memoria visitadas en el peor caso.

En criollo: la complejidad te dice como escala el problema cuando crece la entrada. Si tenes el doble de datos, ¿tarda el doble? ¿el cuadruple? ¿explota todo?

Notacion asintotica: El arte de quedarse con lo importante

Cuando n es grande, las constantes y los terminos de menor orden no importan tanto. Lo que manda es el termino dominante. Para comparar algoritmos usamos la notacion asintotica.

Definiciones formales (en criollo):

Sean $f(n)$ y $g(n)$ funciones positivas para n grande.

- $f(n) = O(g(n))$ (**O grande**): Significa que f no crece mas rapido que g (salvo una constante). Formalmente: existen $c > 0$ y n_0 tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. *Analogia*: Es como decir "lo sumo pesa tanto como g ".
- $f(n) = \Omega(g(n))$ (**Omega grande**): f crece al menos tan rapido como g . Existen $c > 0$ y n_0 tales que $f(n) \geq c \cdot g(n)$. *Analogia*: "Por lo menos pesa g ".
- $f(n) = \Theta(g(n))$ (**Theta grande**): f y g crecen al mismo ritmo. Existen $c_1, c_2 > 0$ y n_0 tales que $c_1 g(n) \leq f(n) \leq c_2 g(n)$. *Analogia*: "Pesan lo mismo, salvo balanza descalibrada".
- $f(n) = o(g(n))$ (**o chica**): f crece estrictamente mas lento que g . Para TODA constante $c > 0$, eventualmente $f(n) < c \cdot g(n)$. Equivalente a $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$. *Analogia*: " f es una pluma al lado de g ".
- $f(n) = \omega(g(n))$ (**omega chica**): f crece estrictamente mas rapido que g . $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

Traducción matematica de un ejemplo. Agarra $f(n) = 3n^2 + 100n + 50$. Para $n \geq 100$, $100n \leq n^2$ y $50 \leq n^2$. Entonces $f(n) \leq 3n^2 + n^2 + n^2 = 5n^2$. Asi que $f(n) = O(n^2)$. Ademas, $f(n) \geq 3n^2$, asi que $f(n) = \Omega(n^2)$. Como esta tanto en O como en Ω , decimos que $f(n) = \Theta(n^2)$. El termino n^2 es el que manda. Decimos que el algoritmo es de **orden cuadratico**.

Jerarquia de ordenes comunes (de mejor a peor):

$$O(1) \ll O(\log n) \ll O(n) \ll O(n \log n) \ll O(n^2) \ll O(n^3) \ll O(2^n) \ll O(n!)$$

El simbolo $\ll$ significa "crece mucho mas lento que".

Para que lo veas con tus propios ojos, ejecuta este codigo en Python. Grafica como crecen estas funciones. Vas a ver que la exponencial explota.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 n = np.linspace(1, 30, 300)
5 plt.figure(figsize=(10, 6))
6
7 plt.plot(n, np.ones_like(n), label='O(1)')
8 plt.plot(n, np.log2(n), label='O(log n)')
9 plt.plot(n, n, label='O(n)')
10 plt.plot(n, n * np.log2(n), label='O(n log n)')
11 plt.plot(n, n**2, label='O(n^2)')
12 plt.plot(n, 2**n, label='O(2^n)')
13
14 plt.ylim(0, 500) # Para que no explote la vista
15 plt.legend()
16 plt.title('Crecimiento Asintotico de Funciones Tipicas')
17 plt.xlabel('Tamano de la entrada n')
18 plt.ylabel('Tiempo / Operaciones')
19 plt.grid(True, alpha=0.3)
20 plt.show()
```

Listing 1: Grafico de ordenes de crecimiento asintotico

El grafico que obtienes es autoexplicativo. Fijate que $O(\log n)$ es casi plana, $O(n \log n)$ apenas se despega de $O(n)$, y $O(2^n)$ se va al cielo rapidisimo. Por eso los algoritmos polinomiales ($O(n^k)$) son los unicos "tratables" para entradas grandes.

Clases de Complejidad: El club de los problemas

La teoria de complejidad agrupa los problemas en clases segun los recursos que necesitan.

- **Clase P (Polynomial time):** Problemas que pueden ser resueltos por una maquina de Turing determinista en tiempo polinomial $O(n^k)$ para alguna constante k . Estos son los "faciles" (aunque k grande puede ser impractico, como $O(n^{100})$, en la practica casi todo lo que esta en P tiene k chico). Ejemplos: ordenar, busqueda binaria, multiplicacion de matrices (clasica), calcular el maximo comun divisor.
- **Clase NP (Nondeterministic Polynomial time):** Problemas para los cuales, si te dan una solucion candidata, la puedes **verificar** en tiempo polinomial. No se sabe si se pueden **encontrar** en tiempo polinomial. La definicion formal es mediante maquinas de Turing no deterministas. Ejemplos: el problema del viajante (TSP), el problema de la factorizacion de enteros, SAT (satisfactibilidad booleana).
- **Clase NP-completo:** Los problemas mas dificiles dentro de NP. Si encontraras un algoritmo polinomial para UNO solo de ellos, entonces TODOS los problemas de NP se

resolverian en tiempo polinomial (y $P = NP$). El problema SAT es el NP-completo canónico.

- **Clase PSPACE:** Problemas que se resuelven usando una cantidad polinomial de memoria (aunque el tiempo pueda ser exponencial). Se sabe que $P \subseteq NP \subseteq PSPACE$.

El problema del millon: P vs NP. La pregunta es si $P = NP$. ¿Todo problema cuya solución se puede verificar rapidamente se puede tambien encontrar rapidamente? La mayoría de los científicos cree que NO, pero no hay demostración. Si resultara que $P = NP$, las consecuencias serian revolucionarias (y catastroficas para la criptografia, que se basa en que problemas como la factorizacion son difíciles).

Reflexion: En la practica del calculo numerico, los problemas que nos interesan suelen estar en P, pero no siempre. Por ejemplo, la programacion lineal esta en P (algoritmo de Karmarkar), pero el metodo simplex, que es exponencial en el peor caso, se usa porque en la practica es rapido. El analisis de complejidad te da un mapa, pero a veces los algoritmos con peor complejidad teorica son mejores en la practica para tamanios de entrada tipicos.

Analisis de complejidad de algoritmos clasicos

Ahora vamos a ensuciarnos las manos y calcular la complejidad de algunos algoritmos que vas a usar toda la carrera.

1. Búsqueda Lineal:

```
def busqueda_lineal(arr, x):
    for i in range(len(arr)):
        if arr[i] == x:
            return i
    return -1
```

El bucle se ejecuta a lo sumo n veces, cada iteración hace $O(1)$ operaciones. Tiempo total: $T(n) = O(n)$.

2. Búsqueda Binaria (array ordenado):

```
def busqueda_binaria(arr, x):
    izq, der = 0, len(arr)-1
    while izq <= der:
        medio = (izq + der) // 2
        if arr[medio] == x:
            return medio
        elif arr[medio] < x:
            izq = medio + 1
        else:
            der = medio - 1
    return -1
```

En cada iteración, el tamaño del subproblema se reduce a la mitad. ¿Cuántas veces puedo dividir n por 2 hasta llegar a 1? $\log_2 n$. Entonces $T(n) = O(\log n)$.

3. Ordenamiento por inserción (Insertion Sort):

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
```

```

j = i-1
while j >= 0 and arr[j] > key:
    arr[j+1] = arr[j]
    j -= 1
arr[j+1] = key

```

El bucle externo se ejecuta $n - 1$ veces. El bucle interno, en el peor caso (array ordenado al revés), se ejecuta i veces. Sumamos: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = O(n^2)$. En el mejor caso (array ya ordenado), el while no se ejecuta nunca, y es $O(n)$. El análisis de peor caso es el más común.

4. Ordenamiento por mezcla (Merge Sort): Divide el array en mitades, ordena recursivamente cada mitad, y luego mezcla. La mezcla tarda $O(n)$. Recurrencia: $T(n) = 2T(n/2) + O(n)$. Por el Teorema Maestro (que vemos ahora), $T(n) = O(n \log n)$.

5. Multiplicación de matrices (ingenua): Tres bucles anidados: $O(n^3)$.

6. Algoritmo de Strassen: $O(n^{2.807})$.

El Teorema Maestro: Receta de cocina para recurrencias

Muchos algoritmos recursivos (divide y vencerás) tienen la forma:

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

donde:

- $a \geq 1$ es el número de llamadas recursivas.
- $b > 1$ es el factor por el que se reduce el tamaño del problema en cada llamada.
- $f(n)$ es el costo del trabajo en cada nivel (dividir y combinar), asintóticamente positivo.

El Teorema Maestro compara $f(n)$ con $n^{\log_b a}$, que es el costo de las "hojas" de la recursión.

Teorema Maestro (versión simplificada y explicada): Sea $c_{\text{crit}} = \log_b a$.

1. **Caso 1: Las hojas dominan.** Si $f(n) = O(n^c)$ con $c < c_{\text{crit}}$, entonces $T(n) = \Theta(n^{c_{\text{crit}}})$.
En criollo: El trabajo en cada nivel es chico comparado con la cantidad de subproblemas. El costo total está dominado por la base de la recursión.
2. **Caso 2: Todos los niveles pesan parejo.** Si $f(n) = \Theta(n^{c_{\text{crit}} \log^k n})$ para algún $k \geq 0$, entonces $T(n) = \Theta(n^{c_{\text{crit}} \log^{k+1} n})$. En particular, si $f(n) = \Theta(n^{c_{\text{crit}}})$ (sin logaritmo, $k = 0$), $T(n) = \Theta(n^{c_{\text{crit}} \log n})$.
En criollo: En cada nivel se hace el mismo trabajo. Como hay $\log_b n$ niveles, multiplicamos por el logaritmo.
3. **Caso 3: El trabajo en la raíz domina.** Si $f(n) = \Omega(n^c)$ con $c > c_{\text{crit}}$, y se cumple la condición de regularidad $af(n/b) \leq kf(n)$ para algún $k < 1$ y n grande, entonces $T(n) = \Theta(f(n))$.
En criollo: El trabajo en el primer nivel es tan grande que el resto es despreciable.

Ejemplos que tienes que grabarte a fuego:

1. $T(n) = 2T(n/2) + O(n)$ (Merge Sort). $a = 2, b = 2, c_{\text{crit}} = \log_2 2 = 1$. $f(n) = \Theta(n^1)$, Caso 2 con $k = 0 \Rightarrow T(n) = \Theta(n \log n)$.
2. $T(n) = T(n/2) + O(1)$ (Busqueda Binaria). $a = 1, b = 2, c_{\text{crit}} = 0$. $f(n) = \Theta(1) = \Theta(n^0)$, Caso 1 $\Rightarrow T(n) = \Theta(\log n)$.

3. $T(n) = 8T(n/2) + O(n^2)$ (Multiplicacion de matrices ingenua DyV). $a = 8, b = 2, c_{\text{crit}} = 3$. $f(n) = O(n^2)$, $c = 2 < 3$, Caso 1 $\Rightarrow T(n) = \Theta(n^3)$.
4. $T(n) = 7T(n/2) + O(n^2)$ (Strassen). $a = 7, b = 2, c_{\text{crit}} = \log_2 7 \approx 2,807$. $f(n) = O(n^2)$, $c = 2 < 2,807$, Caso 1 $\Rightarrow T(n) = \Theta(n^{2,807})$.
5. $T(n) = 2T(n/2) + O(n^3)$ (trabajo enorme). $c_{\text{crit}} = 1$, $f(n) = \Omega(n^3)$, $c = 3 > 1$. Regularidad: $2(n/2)^3 = n^3/4 \leq kn^3$ con $k = 1/4 < 1$. Caso 3 $\Rightarrow T(n) = \Theta(n^3)$.

Grafico del arbol de recursion: Para entender el Teorema Maestro, nada mejor que dibujar el arbol. Este codigo te genera un esquema (no grafico, pero ilustrativo) del arbol para una recurrencia generica.

```

1 def dibujar_arbol(a, b, n, profundidad=0, max_prof=3):
2     if profundidad > max_prof or n < 1:
3         return
4     print(" " * profundidad + f"Nivel {profundidad}: {a**profundidad} nodo(s)
5         de tamaño {n}, costo total f({n})")
6     dibujar_arbol(a, b, n/b, profundidad+1, max_prof)
7 print("Arbol para MergeSort (a=2, b=2):")
8 dibujar_arbol(2, 2, 16)

```

Listing 2: Esquema textual del arbol de recursion

Esto imprime algo como:

```

Arbol para MergeSort (a=2, b=2):
Nivel 0: 1 nodo(s) de tamaño 16, costo total f(16)
  Nivel 1: 2 nodo(s) de tamaño 8.0, costo total f(8.0)
    Nivel 2: 4 nodo(s) de tamaño 4.0, costo total f(4.0)
      Nivel 3: 8 nodo(s) de tamaño 2.0, costo total f(2.0)

```

Ves clarito como en cada nivel el costo se multiplica por a pero el tamaño se divide por b . La suma de los costos por nivel da la complejidad total.

Teorema Maestro: Profundizacion y ejemplos avanzados

El Teorema Maestro es una de las herramientas mas utiles para analizar algoritmos recursivos, pero hay que saber aplicarlo correctamente. Vamos a profundizar un poco mas en los casos y ver algunos ejemplos que no son tan directos.

Caso 1: Las hojas dominan. Este caso ocurre cuando el trabajo de dividir y combinar es despreciable comparado con el trabajo de las hojas. La recurrencia típica es $T(n) = aT(n/b) + O(n^c)$ con $c < \log_b a$. En este caso, la mayor parte del trabajo se hace en los niveles mas profundos del arbol de recursion.

Ejemplo: $T(n) = 4T(n/2) + n$. Aca $a = 4, b = 2, c_{\text{crit}} = \log_2 4 = 2$. $f(n) = n = O(n^1)$, y $1 < 2$, así que estamos en el Caso 1. La solución es $T(n) = \Theta(n^2)$.

Caso 2: Todos los niveles pesan igual. Este es el caso mas comun en algoritmos clasicos. Ocurre cuando el trabajo en cada nivel es aproximadamente el mismo. La recurrencia es $T(n) = aT(n/b) + \Theta(n^{c_{\text{crit}}} \log^k n)$. La solución agrega un factor logaritmico adicional.

Ejemplo: $T(n) = 2T(n/2) + n \log n$. Aca $a = 2, b = 2, c_{\text{crit}} = 1$, y $f(n) = n \log n = \Theta(n^1 \log^1 n)$. Estamos en el Caso 2 con $k = 1$. La solución es $T(n) = \Theta(n \log^2 n)$.

Caso 3: La raiz domina. Este caso ocurre cuando el trabajo en la raiz es tan grande que el resto es despreciable. La recurrencia es $T(n) = aT(n/b) + \Theta(n^c)$ con $c > \log_b a$. Además, se debe cumplir la condicion de regularidad: $af(n/b) \leq kf(n)$ para algun $k < 1$.

Ejemplo: $T(n) = 2T(n/2) + n^2$. Aca $a = 2, b = 2, c_{\text{crit}} = 1$, y $f(n) = n^2 = \Omega(n^2)$, con $c = 2 > 1$. La condicion de regularidad se cumple: $2(n/2)^2 = n^2/2 \leq kn^2$ con $k = 1/2$. La solucion es $T(n) = \Theta(n^2)$.

Casos borde y excepciones. Hay casos donde el Teorema Maestro no aplica directamente. Por ejemplo, si $f(n)$ no es un polinomio, o si la recurrencia no es de la forma exacta.

Ejemplo: $T(n) = 2T(n/2) + n/\log n$. Aca $c_{\text{crit}} = 1$, y $f(n) = n/\log n = O(n^1)$, pero no es $\Theta(n^1 \log^k n)$. El Teorema Maestro no aplica directamente; hay que usar el metodo de iteracion o el teorema maestro generalizado.

Ejemplo: $T(n) = T(n-1) + n$. Esta recurrencia no es de la forma $T(n) = aT(n/b) + f(n)$ porque el subproblema no se reduce por division, sino por resta. La solucion es $\Theta(n^2)$, y se obtiene por sumatoria: $\sum_{k=1}^n k = n(n+1)/2$.

Dale, repasemos lo importante de complejidad

- La notacion asintotica (O, Ω, Θ) permite comparar el crecimiento de algoritmos.
- Los ordenes de crecimiento van desde $O(1)$ hasta $O(n^n)$.
- La clase P contiene problemas resolubles en tiempo polinomial.
- La clase NP contiene problemas verificables en tiempo polinomial.
- P vs NP es una pregunta abierta fundamental.
- El Teorema Maestro resuelve recurrencias de la forma $T(n) = aT(n/b) + f(n)$.

Aritmetica de Maquina: La realidad finita de los numeros

Bueno, ahora si, dejemos la teoria abstracta y vayamos al barro donde todos nos hemos resbalado. Las computadoras no pueden representar todos los numeros reales. Tienen que arreglarselas con un subconjunto finito de ellos: los **numeros de maquina**. Esto introduce errores en CADA operacion que hacés. No es un bug, es una característica del sistema. Comprenderlo es lo que separa al que programa calculo científico del que programa de oído.

Representacion de enteros: La parte facil

Los enteros son los mas sencillos. En 64 bits, se representan en **complemento a dos**. El rango va de -2^{63} a $2^{63} - 1$ (aproximadamente $\pm 9,22 \times 10^{18}$). Las operaciones de suma, resta y multiplicacion son EXACTAS siempre que el resultado este dentro del rango. Si te pasas, ocurre **overflow**: el resultado se "envuelve" (modulo 2^{64} para no signados, o con cambio de signo para signados).

En Python, los enteros son de precision arbitraria (BigInt), asi que no sufris overflow... pero pagas en performance. En C, Fortran, y los calculos que haga el hardware de la GPU, estas limitado. **Moraleja:** si tu algoritmo usa enteros muy grandes, chequea los rangos.

Punto Flotante: El corazon del problema (y de la solucion)

Para representar numeros reales (o mejor dicho, una aproximacion), se usa la notacion de **punto flotante**. Es como la notacion científica de la calculadora, pero en base 2 y con limitaciones.

Un sistema de punto flotante se define como $\mathbb{F}(\beta, t, e_{\text{mín}}, e_{\text{máx}})$:

- β : base (generalmente 2).

- t : precision (cantidad de digitos de la mantisa, incluyendo el bit implicito si lo hay).
- $e_{\min}, e_{\max}$: rango del exponente.

Un numero de maquina normalizado (no cero, no subnormal) se escribe:

$$x = \pm (d_0.d_1d_2 \dots d_{t-1}) \times \beta^e$$

donde $d_0 \neq 0$, $0 \leq d_i \leq \beta - 1$, y $e_{\min} \leq e \leq e_{\max}$. La mantisa es la parte fraccionaria; el punto "flota." al multiplicar por β^e .

El estandar IEEE 754: El que manda en todas las computadoras modernas.

- **Simple precision (float, FP32):** 32 bits. 1 bit de signo, 8 de exponente, 23 de mantisa (24 con el bit implicito). $\beta = 2, t = 24$. Rango: $\approx 1,17 \times 10^{-38}$ a $3,4 \times 10^{38}$.
- **Doble precision (double, FP64):** 64 bits. 1 de signo, 11 de exponente, 52 de mantisa (53 con el implicito). $\beta = 2, t = 53$. Rango: $\approx 2,2 \times 10^{-308}$ a $1,8 \times 10^{308}$.

Como la base es 2, el primer digito d_0 siempre es 1. ¡No se guarda! Es el **bit implicito** (o hidden bit). Asi ahorran un bit y ganan precision.

Traduccion Matematica del formato IEEE 754. La formula $x = (-1)^s \times 1.f \times 2^{e-1023}$ se lee: "el numero x es positivo si el bit de signo s es 0, negativo si es 1, su mantisa tiene un 1 implicito antes del punto, seguido de la parte fraccionaria f , y se multiplica por una potencia de 2 que viene del exponente e menos un sesgo de 1023". Es como escribir un numero en notacion cientifica pero en binario, y ademas ajustando el exponente para que siempre empiece con un 1 (normalizacion). El sesgo es para que el exponente pueda ser negativo y positivo sin usar un bit de signo aparte.

Formatos para Machine Learning: Con el auge de las redes neuronales, aparecieron formatos de menor precision que sacrifican rango y exactitud por velocidad y ahorro energetico:

- **FP16 (half):** 16 bits (1 signo, 5 exp, 10 mantisa). $t = 11$.
- **BF16 (bfloat16):** 16 bits (1 signo, 8 exp, 7 mantisa). Igual rango que FP32, pero menos precision. Lo usa Google.
- **TF32:** Usado internamente por NVIDIA. 19 bits (1 signo, 8 exp, 10 mantisa).
- **FP8:** 8 bits, con variantes E5M2 y E4M3, para inferencia ultra rapida.

La tabla 2 resume los mas comunes.

Cuadro 1: Formatos de punto flotante usados en ciencia y ML.

Formato	Bits	Signo	Exp.	Mantisa (total)	Uso
FP64	64	1	11	52 (53)	Cientifico general
FP32	32	1	8	23 (24)	Graficos, ML entrenamiento
TF32	19	1	8	10 (11)	Tensor Cores NVIDIA
FP16	16	1	5	10 (11)	ML inferencia
BF16	16	1	8	7 (8)	ML entrenamiento (Google)
FP8 (E5M2)	8	1	5	2 (3)	Inferencia extrema
FP8 (E4M3)	8	1	4	3 (4)	Entrenamiento cuantizado

Epsilon de Maquina: La minima diferencia que importa

Definicion: El epsilon de maquina, ϵ_{mach} , es la distancia entre el numero 1.0 y el siguiente numero representable en el sistema de punto flotante. Formalmente:

$$\epsilon_{\text{mach}} = \beta^{1-t}$$

Para doble precision, $\epsilon_{\text{mach}} = 2^{1-53} = 2^{-52} \approx 2,22 \times 10^{-16}$.

¿Que significa en la practica? Que la computadora no puede distinguir dos numeros reales que differan relativamente en menos de ϵ_{mach} . Son iguales.^a sus ojos. Es como una regla milimetrada: si tenes que medir una distancia de 1 metro, podes ver diferencias de 1 mm. Para 1 km, la minima diferencia visible es mayor en terminos absolutos, pero relativa es la misma.

Unit roundoff u : El maximo error relativo que se comete al redondear un numero real al numero de maquina mas cercano. $u = \frac{1}{2}\epsilon_{\text{mach}}$. En doble precision, $u = 2^{-53} \approx 1,11 \times 10^{-16}$. Esta es la cota que garantiza el estandar para las operaciones basicas.

Codigo: midiendo el epsilon. No uses bibliotecas, hacelo vos para sentir el pulso de la maquina.

```

1 import numpy as np
2
3 def mi_epsilon(tipo=np.float64):
4     """
5     Busca el epsilon de maquina partiendo de 1.0 y dividiendo por 2
6     hasta que 1.0 + eps/2 sea indistinguible de 1.0.
7     """
8     uno = tipo(1.0)
9     eps = tipo(1.0)
10    while uno + eps / tipo(2.0) != uno:
11        eps /= tipo(2.0)
12    return eps
13
14 eps32 = mi_epsilon(np.float32)
15 eps64 = mi_epsilon(np.float64)
16
17 print(f"Epsilon float32: {eps32:.15e} (teorico: {2**-23:.15e})")
18 print(f"Epsilon float64: {eps64:.30e} (teorico: {2**-52:.30e})")
19
20 # Verificacion con numpy
21 print(f"np.finfo(float32).eps: {np.finfo(np.float32).eps:.15e}")
22 print(f"np.finfo(float64).eps: {np.finfo(np.float64).eps:.30e}")

```

Listing 3: Calculo casero del epsilon de maquina

La salida te confirma que el epsilon medido es EXACTAMENTE 2^{-23} para float32 y 2^{-52} para float64. ¡La compu no miente en esto!

Redondeo: El arte de aproximar sin romper todo

Existen varios modos de redondeo definidos por IEEE 754. El mas usado (y el default) es **round to nearest, ties to even** (redondeo al mas cercano, y en caso de empate, al que tiene el ultimo bit par). Esto evita el sesgo estadistico que tendria redondear siempre para arriba.

La **cota fundamental del error de redondeo** nos dice que si x es un numero real dentro del rango normalizado, y $fl(x)$ es su representacion redondeada en punto flotante, entonces existe un δ con $|\delta| \leq u$ tal que:

$$fl(x) = x(1 + \delta)$$

Esta ecuacion es la base del **modelo estandar de aritmetica de punto flotante**: cada operacion elemental ($\circ \in \{+, -, \times, /\}$) sobre numeros de maquina a, b produce un resultado que cumple:

$$fl(a \circ b) = (a \circ b)(1 + \delta), \quad |\delta| \leq u$$

Es como si la operacion se hiciera exactamente y luego se redondeara. Este modelo permite hacer *analisis de error hacia adelante*, propagando los δ para acotar el error final de un algoritmo.

Grafico conceptual del error de redondeo: Cuando vos calculas una derivada numerica con diferencia hacia adelante, el error total tiene dos componentes: el error de truncamiento (de Taylor) y el error de redondeo. El siguiente codigo grafica ambos y muestra que hay un h optimo donde el error es minimo. Es un clasico de Analisis Numerico.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Funcion f(x) = sin(x), derivada exacta cos(x)
5 x0 = 1.0
6 exacto = np.cos(x0)
7
8 # Rango de h, desde 10^-1 hasta 10^-16
9 hs = np.logspace(-1, -16, 100, base=10.0)
10 errores = []
11
12 for h in hs:
13     # Diferencia hacia adelante: (f(x+h)-f(x))/h
14     aprox = (np.sin(x0 + h) - np.sin(x0)) / h
15     err = abs(aprox - exacto)
16     errores.append(err)
17
18 # Grafico en escala log-log
19 plt.figure(figsize=(10, 6))
20 plt.loglog(hs, errores, 'b-', label='Error total')
21
22 # Aproximadamente, el error de truncamiento es |h/2 * f''(x0)|
23 # y el error de redondeo es ~ epsilon/h
24 h_opt_teorico = (np.finfo(float).eps)**0.5
25 print(f"h optimo teorico (aprox): {h_opt_teorico:.2e}")
26
27 plt.axvline(h_opt_teorico, color='r', linestyle='--', label=f'h optimo teorico ~ {h_opt_teorico:.2e}')
28 plt.xlabel('Paso h')
29 plt.ylabel('Error absoluto')
30 plt.title('Error en diferencia hacia adelante para f(x)=sin(x) en x=1')
31 plt.legend()
32 plt.grid(True, which='both', alpha=0.4)
33 plt.show()

```

Listing 4: Error en derivada numerica: truncamiento vs redondeo

El grafico muestra una V en escala log-log. Para h grande, el error baja al disminuir h (domina el error de truncamiento $O(h)$). Para h muy chico, el error SUBE porque los numeros $f(x+h)$ y $f(x)$ son tan cercanos que su diferencia tiene pocos digitos significativos (cancela y domina el error de redondeo). El optimo esta alrededor de $h \approx \sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$ en doble precision.

Consecuencias peligrosas: Cancelacion catastrofica y mas

1. Cancelacion catastrofica. Cuando restas dos numeros muy cercanos entre si, los digitos significativos que coinciden se cancelan, y el resultado hereda solo los digitos "sucios" del error

de redondeo. Es el problema mas comun y traicionero.

Ejemplo clasico: Resolver $x^2 - 10000.0001x + 1 = 0$. Las raices exactas son aproximadamente $x_1 \approx 10000$ y $x_2 \approx 0.0001$. Pero con la formula usual $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$, al calcular $-b - \sqrt{b^2 - 4ac}$ para la raiz pequena, restas 10000.0001 y algo muy cercano a 10000, perdiendo precision. La solucion es usar la relacion $x_2 = c/(ax_1)$ (formula de Vieta).

Codigo que lo demuestra:

```

1 import numpy as np
2 import math
3
4 a, b, c = 1.0, -10000.0001, 1.0
5 # Formula usual
6 disc = math.sqrt(b*b - 4*a*c)
7 x1_usual = (-b - disc) / (2*a)
8 x2_usual = (-b + disc) / (2*a)
9
10 print(f"Raiz usual x1: {x1_usual:.10f}")
11 print(f"Raiz usual x2: {x2_usual:.10f}")
12
13 # Formula estable
14 if b > 0:
15     x1 = (-b - disc) / (2*a)
16     x2 = c / (a * x1)
17 else:
18     x2 = (-b + disc) / (2*a)
19     x1 = c / (a * x2)
20
21 print(f"Raiz estable x1: {x1:.10f}")
22 print(f"Raiz estable x2: {x2:.10f}")
23 print(f"Valor exacto aproximado x2: 0.0001")

```

Listing 5: Cancelacion catastrofica en ecuacion cuadratica

Fijate como la formula usual da un error mucho mayor en la raiz chica. La estable usa multiplicacion en vez de resta.

2. Suma no asociativa. El orden de la suma importa. Si sumas numeros muy dispares, los pequenos pueden perderse. Ejemplo: $(10^{16}+1)-10^{16}$ da 0 en doble precision, pero $10^{16}+(1-10^{16})$ da 1. Siempre que puedas, ordena de menor a mayor valor absoluto cuando sumes muchos terminos.

```

1 import numpy as np
2
3 grande = np.float64(1e16)
4 chico = np.float64(1.0)
5
6 # Caso (grande + chico) - grande
7 res1 = (grande + chico) - grande
8 print(f"(1e16 + 1) - 1e16 = {res1}")
9
10 # Caso (chico - grande) + grande
11 res2 = (chico - grande) + grande
12 print(f"(1 - 1e16) + 1e16 = {res2}")

```

Listing 6: Suma no asociativa: orden importa

3. Inestabilidad numerica de algoritmos. Un algoritmo es numericamente inestable si amplifica los errores de redondeo. El ejemplo tipico es la evaluacion de polinomios en forma expandida cerca de una raiz multiple. Evaluar $(x - 2)^9$ expandido cerca de $x = 2$ es un desastre; evaluarlo en forma factorizada es estable.


```

1 import numpy as np
2
3 # Coeficientes de  $(x-2)^9 = x^9 - 18x^8 + 144x^7 - 672x^6 + 2016x^5 - 4032x^4 +$ 
4    $5376x^3 - 4608x^2 + 2304x - 512$ 
5 coefs = [1, -18, 144, -672, 2016, -4032, 5376, -4608, 2304, -512]
6 def evaluar_expandido(x):
7     # Algoritmo de Horner
8     res = coefs[0]
9     for c in coefs[1:]:
10         res = res * x + c
11     return res
12
13 x_vals = np.linspace(1.99, 2.01, 100)
14 y_expandido = [evaluar_expandido(x) for x in x_vals]
15 y_factorizado = (x_vals - 2)**9
16
17 import matplotlib.pyplot as plt
18 plt.plot(x_vals, y_expandido, label='Expandido (Horner)')
19 plt.plot(x_vals, y_factorizado, '--', label='Factorizado')
20 plt.legend()
21 plt.title('Inestabilidad numerica cerca de raiz multiple')
22 plt.xlabel('x')
23 plt.ylabel('p(x)')
24 plt.grid()
25 plt.show()

```

Listing 7: Inestabilidad del polinomio $(x-2)^9$ *expandido*

El grafico muestra oscilaciones numericas en la forma expandida que no existen en la realidad.

Geometria Computacional: Cuando el error te dibuja cualquier cosa

En aplicaciones de CAD, GIS, o robotica, las decisiones se toman en base a tests geometricos (puntos colineales, intersecciones). Un error de redondeo puede hacer que un punto que deberia estar exactamente sobre una recta sea clasificado a izquierda o derecha, y eso puede colapsar un algoritmo de triangulacion, hacer que un poligono se autointersecte, o que un robot choque. Se usan tecnicas de **predicados exactos** (aritmetica de Shewchuk) o **perturbacion simbolica** para evitar estas degeneraciones.

Precision arbitraria por software: El santo remedio (con costo)

Cuando la precision del hardware no alcanza (por ejemplo, para calcular 100 digitos de π), se usan bibliotecas que implementan aritmetica de precision arbitraria por software. Son mucho mas lentas, pero exactas. En Python tenemos `mpmath`.

```

1 from mpmath import mp
2
3 mp.dps = 100 # decimal places
4 pi_100 = mp.pi
5 print(pi_100)
6
7 # Compara con math.pi
8 import math
9 print(f"Diferencia con math.pi: {float(pi_100) - math.pi}")

```

Listing 8: Calculo de pi con 100 decimales usando mpmath

Dale, repasemos lo importante de la aritmetica de maquina

- Los numeros de maquina son una aproximacion finita de los reales.
- El epsilon de maquina (ϵ_{mach}) es la distancia entre 1 y el siguiente numero representable.
- El unit roundoff $u = \epsilon_{\text{mach}}/2$ es el maximo error relativo de redondeo.
- La cancelacion catastrofica ocurre al restar numeros casi iguales.
- La suma no es asociativa en punto flotante.
- Los algoritmos pueden ser inestables numericamente.
- La precision arbitraria existe, pero es lenta.

Ejercicios para que te luzcas en el final

Llegaste hasta aca, genio/a. Ahora a practicar. Aca van **20 ejercicios** (10 teoricos y 10 computacionales) con sus soluciones completas. No los mires antes de intentarlos, que es la unica forma de aprender de verdad.

Ejercicios Teoricos (T1 a T10)

- T1. Maquina de Turing que duplica:** Disenia una MT que, dada una cadena de n unos, produzca $2n$ unos (duplicar). Explica la estrategia y da la tabla de transiciones.
- T2. Halting Problem:** Explica con tus palabras la demostracion de que no existe un programa que determine si otro programa se detiene. ¿En que momento se usa la auto-referencia?
- T3. Cotas asintoticas:** Sean $f(n) = 3n^2 + n \log n$ y $g(n) = n^2$. Demuestra que $f(n) = \Theta(g(n))$.
- T4. Teorema Maestro:** Resuelve $T(n) = 2T(n/4) + \sqrt{n}$.
- T5. Clases P y NP:** Explica la diferencia entre P y NP. Da un ejemplo de un problema en cada clase.
- T6. NP-completitud:** ¿Que significa que un problema sea NP-completo? ¿Por que el problema SAT es tan importante?
- T7. Epsilon de maquina:** Deduci la formula $\epsilon_{\text{mach}} = \beta^{1-t}$ a partir de la definicion de un sistema de punto flotante normalizado. Explica por que el bit implicito en base 2 hace que t sea una unidad mayor que los bits almacenados de mantisa.
- T8. Error en la suma:** Considera la suma de n numeros x_i en punto flotante. Muestra con un analisis de propagacion de errores que el error relativo hacia adelante esta acotado por $n \cdot u \cdot \frac{\sum |x_i|}{|\sum x_i|} + O(u^2)$. Interpreta el factor de amplificacion $\frac{\sum |x_i|}{|\sum x_i|}$ (numero de condicion de la suma).
- T9. Test de orientacion:** Escribe la expresion matematica del test de orientacion para tres puntos en el plano. Explica por que es propenso a errores de redondeo cerca de la colinealidad.
- T10. Tesis de Church-Turing:** Discute si una computadora cuantica podria violar la Tesis de Church-Turing, en el sentido de calcular funciones no Turing-computables. Diferencia entre "computabilidad" y "complejidad".

Ejercicios Computacionales (C1 a C10)

- C1. Simulador de MT:** Implementa en Python la MT que suma 1 en unario del ejemplo. La cinta puede ser una lista de caracteres.
- C2. Grafico de ordenes:** Escribe un script que grafique n , $n \log n$, n^2 y n^3 en escala log-log. Verifica visualmente las pendientes.
- C3. Epsilon a mano:** Calcula el epsilon de maquina para float32 y float64 sin usar librerias (como el codigo de la seccion).
- C4. Cancelacion visual:** Grafica el error relativo de $(1 - \cos x)/x^2$ y su version estable para $x = 10^{-k}$, $k = 1, 8$. Compara con el valor limite 0.5.
- C5. Derivada numerica:** Para $f(x) = e^x$ en $x = 1$, aproxima $f'(1)$ con diferencia centrada. Grafica el error vs h en escala log-log y encuentra el h optimo experimental.
- C6. Serie armonica estancada:** Suma $S_N = \sum_{k=1}^N 1/k$ en float32 y float64. Encuentra el N a partir del cual el valor computado deja de crecer.
- C7. Taylor inestable:** Calcula e^{-15} con la serie de Taylor directa y con $1/e^{15}$. Grafica el error relativo en funcion del numero de terminos.
- C8. Verificacion del Teorema Maestro:** Implementa una funcion recursiva que simule la recurrencia $T(n) = 2T(n/2) + n$. Mide el tiempo de ejecucion para distintos n y verifica que la complejidad es $O(n \log n)$.
- C9. Comparacion de algoritmos de ordenamiento:** Implementa los algoritmos de ordenamiento por insercion ($O(n^2)$) y mergesort ($O(n \log n)$). Mide el tiempo de ejecucion para n crecientes y grafica los resultados.
- C10. Fibonacci:** Implementa fibonacci recursivo naive ($O(2^n)$) y fibonacci iterativo ($O(n)$). Mide el tiempo de ejecucion para $n = 10, 20, 30, 40$ y grafica los tiempos.

Soluciones detalladas a los ejercicios

Soluciones a los Ejercicios Teoricos

T1. Solucion: Idea: recorrer la cinta original, marcar los 1's (ej. convirtiendolos en A), copiar al final, y restaurar. La tabla concreta se puede encontrar en libros de teoria de la computacion. Aca damos la estrategia: usar estados para recordar "si se copio, etc."

T2. Solucion: Ver seccion . Auto-referencia: la maquina D recibe su propio codigo y se contradice. La paradoja del barbero ("en un pueblo, el barbero afeita a todos los hombres que no se afeitan a si mismos. ¿El barbero se afeita a si mismo?") genera una contradiccion porque la definicion es autorreferente. En el problema de la detencion, construimos una maquina D que hace lo opuesto a lo que H dice que hace, aplicandose a si misma.

T3. Solucion: $f(n) = n^2 + 2n \log n$. Para $n \geq 2$, $\log n \leq n$. Entonces $2n \log n \leq 2n^2$. Luego $f(n) \leq n^2 + 2n^2 = 3n^2$, por lo que $f(n) = O(n^2)$ con $c_2 = 3, n_0 = 2$. Ademas, $f(n) \geq n^2$ trivialmente, por lo que $f(n) = \Omega(n^2)$ con $c_1 = 1$. Entonces $f(n) = \Theta(n^2)$.

T4. Solucion: $a = 2, b = 4, c_{\text{crit}} = \log_4 2 = 0,5$. $f(n) = \sqrt{n} = n^{0,5}$. Como $f(n) = \Theta(n^{0,5})$, es Caso 2 con $k = 0 \Rightarrow T(n) = \Theta(\sqrt{n} \log n)$.

T5. Solucion: P: problemas resolubles en tiempo polinomial. NP: problemas verificables en tiempo polinomial. Ejemplos P: ordenamiento. Ejemplos NP: TSP, SAT. Un problema NP-completo es aquel que pertenece a NP y tal que todo otro problema en NP puede reducirse a el en tiempo polinomial. SAT lo es por el teorema de Cook-Levin.

T6. Solucion: Un problema NP-completo es aquel que pertenece a NP y tal que todo otro problema en NP puede reducirse a el en tiempo polinomial. SAT lo es por el teorema de Cook-Levin. Si se encontrara un algoritmo polinomial para SAT, entonces $P = NP$, porque cualquier problema en NP se puede transformar a una instancia de SAT y resolverla eficientemente.

T7. Solucion: En base β , mantisa normalizada: $1 \leq m < \beta$. Entre 1 y β hay β^{t-1} intervalos (porque el primer dígito es fijo? No, la mantisa tiene t dígitos en total, pero el primero $d_0 \geq 1$. El espaciado entre numeros con exponente 0 es $\beta^{-(t-1)}$. El siguiente numero despues de 1 es $1 + \beta^{-(t-1)}$. Distancia = $\beta^{-(t-1)} = \beta^{1-t}$. En base 2, el bit d_0 siempre es 1 y no se almacena, asi que se guardan $p = t - 1$ bits de mantisa, y la precision total es $t = p + 1$.

T8. Solucion: Sea $S_n = \sum_{i=1}^n x_i$. El modelo da $fl(x+y) = (x+y)(1+\delta)$. Propagando, el error hacia adelante en la suma recursiva es $E_n \leq \sum_{i=1}^n |x_i| \cdot (n-i+1)u + O(u^2)$ (en el peor caso). La cota optima es $nu \sum |x_i|$. El error relativo es $\leq nu \frac{\sum |x_i|}{|S_n|}$. El factor $\kappa = \frac{\sum |x_i|}{|\sum x_i|}$ es el *numero de condicion de la suma*. Si los x_i son todos del mismo signo, $\kappa = 1$ y el error relativo es chico. Si hay cancelacion masiva (S_n cercano a 0 pero las magnitudes son grandes), κ es gigante y el error relativo se amplifica.

T9. Solucion: El determinante es $D = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)$. Sufre cancelacion si los puntos son casi colineales, porque D es aproximadamente 0 pero se calcula como diferencia de dos productos que pueden ser grandes. El error relativo en D puede ser enorme.

T10. Solucion: La Tesis de Church-Turing habla de *computabilidad*: que problemas son resolubles. Una computadora cuantica puede calcular las mismas funciones que una maquina de Turing (no mas). No viola la tesis en el sentido de resolver problemas indecidibles. Sin embargo, en el sentido de *complejidad*, se cree que las computadoras cuanticas pueden resolver ciertos problemas en P (o BQP) que para una clasica requeririan tiempo exponencial (ej. factorizacion con Shor). La tesis extendida de Church-Turing dice que toda funcion *eficientemente calculable* es polinomial en una maquina de Turing probabilistica; las cuanticas podrian refutar esta version, pero no la original.

Soluciones a los Ejercicios Computacionales

C1. Solucion:

```

1 def mt_suma_uno(cinta):
2     cinta = list(cinta) + ['B']
3     idx = 0
4     estado = 'q0'
5     while estado != 'qhalt':
6         simbolo = cinta[idx]
7         if estado == 'q0' and simbolo == '1':
8             idx += 1
9         elif estado == 'q0' and simbolo == 'B':
10            cinta[idx] = '1'
11            idx -= 1
12            estado = 'q1'
13        elif estado == 'q1' and simbolo == '1':
14            estado = 'qhalt'
15    return ''.join(cinta).rstrip('B')
16
17 print(mt_suma_uno('111')) # devuelve '1111'

```

Listing 9: MT suma 1 en Python

C2. Solucion:

```

1 import numpy as np, matplotlib.pyplot as plt
2 n = np.logspace(1, 3, 100)
3 plt.loglog(n, n, label='n')
4 plt.loglog(n, n*np.log2(n), label='n log n')
5 plt.loglog(n, n**2, label='n^2')
6 plt.loglog(n, n**3, label='n^3')
7 plt.xlabel('n'); plt.ylabel('f(n)')
8 plt.legend(); plt.grid(True, which='both'); plt.show()

```

Listing 10: Pendientes en log-log

C3. Solucion:

```

1 import numpy as np
2
3 def encontrar_epsilon(tipo=float):
4     eps = tipo(1.0)
5     while tipo(1.0) + eps / tipo(2.0) != tipo(1.0):
6         eps /= tipo(2.0)
7     return eps
8
9 print("float64:", encontrar_epsilon(float))
10 print("float32:", encontrar_epsilon(np.float32))
11 print("Valor de numpy:", np.finfo(float).eps)

```

Listing 11: Calculo del epsilon de maquina

C4. Solucion:

```

1 import numpy as np, matplotlib.pyplot as plt
2 xs = np.logspace(-1, -8, 50)
3 directo = (1 - np.cos(xs)) / xs**2
4 estable = 2*(np.sin(xs/2)/xs)**2
5 plt.loglog(xs, np.abs(directo-0.5), 'o-', label='Directo')
6 plt.loglog(xs, np.abs(estable-0.5), 's-', label='Estable')
7 plt.legend(); plt.grid(which='both'); plt.show()

```

Listing 12: Cancelacion catastrofica

C5. Solucion:

```

1 import numpy as np, matplotlib.pyplot as plt
2 f = np.exp; x0=1; exacta = np.exp(1)
3 hs = np.logspace(-1, -12, 200)
4 errs = [abs((f(x0+h)-f(x0-h))/(2*h) - exacta) for h in hs]
5 plt.loglog(hs, errs); plt.xlabel('h'); plt.ylabel('Error'); plt.grid(True)
6 idx_min = np.argmin(errs)
7 print(f'h optimo ~ {hs[idx_min]:.2e}')
8 plt.show()

```

Listing 13: Error de derivada centrada

C6. Solucion:

```

1 import numpy as np
2 s32 = np.float32(0.0)
3 for k in range(1, 10**7):
4     s32 += np.float32(1.0/k)

```

```

5     if k % 10**6 == 0:
6         print(k, s32)

```

Listing 14: Suma armonica en float32

Se observa que despues de unos millones, la suma deja de cambiar.

C7. Solucion:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def taylor_exp(x, n):
5     """Serie de Taylor de e^x hasta grado n."""
6     s = 0
7     term = 1
8     for k in range(n+1):
9         s += term
10        term *= x / (k+1)
11    return s
12
13 x = -15
14 exacto = np.exp(x)
15 ns = np.arange(1, 50)
16 errores_directo = []
17 errores_inverso = []
18
19 for n in ns:
20     directo = taylor_exp(x, n)
21     inverso = 1 / taylor_exp(-x, n)
22     errores_directo.append(abs(directo - exacto)/abs(exacto))
23     errores_inverso.append(abs(inverso - exacto)/abs(exacto))
24
25 plt.figure(figsize=(10,6))
26 plt.semilogy(ns, errores_directo, 'o-', label='Directo')
27 plt.semilogy(ns, errores_inverso, 's-', label='Inverso (1/e^x)')
28 plt.xlabel('Numero de terminos')
29 plt.ylabel('Error relativo')
30 plt.legend()
31 plt.grid(True)
32 plt.show()

```

Listing 15: Taylor inestable para e^{-15}

C8. Solucion:

```

1 import time
2 import matplotlib.pyplot as plt
3
4 def T(n):
5     if n <= 1:
6         return 1
7     return 2*T(n//2) + n
8
9 ns = [2**i for i in range(1, 12)]
10 tiempos = []
11 for n in ns:
12     start = time.time()
13     T(n)
14     end = time.time()
15     tiempos.append(end - start)
16
17 plt.loglog(ns, tiempos, 'o-', label='Tiempo medido')

```

```

18 plt.loglog(ns, [n*np.log2(n) for n in ns], '--', label='O(n log n)')
19 plt.xlabel('n'); plt.ylabel('Tiempo (s)')
20 plt.legend(); plt.grid(True)
21 plt.show()

```

Listing 16: Verificacion del Teorema Maestro

C9. Solucion:

```

1 import random, time
2 import matplotlib.pyplot as plt
3
4 def insertion_sort(arr):
5     for i in range(1, len(arr)):
6         key = arr[i]
7         j = i-1
8         while j >= 0 and arr[j] > key:
9             arr[j+1] = arr[j]
10            j -= 1
11            arr[j+1] = key
12    return arr
13
14 def mergesort(arr):
15     if len(arr) <= 1:
16         return arr
17     mid = len(arr)//2
18     L = mergesort(arr[:mid])
19     R = mergesort(arr[mid:])
20     res, i, j = [], 0, 0
21     while i < len(L) and j < len(R):
22         if L[i] < R[j]:
23             res.append(L[i]); i += 1
24         else:
25             res.append(R[j]); j += 1
26     return res + L[i:] + R[j:]
27
28 ns = [10, 20, 50, 100, 200, 500, 1000]
29 tiempos_ins = []
30 tiempos_merge = []
31 for n in ns:
32     arr = [random.randint(0, 1000) for _ in range(n)]
33     start = time.time()
34     insertion_sort(arr.copy())
35     end = time.time()
36     tiempos_ins.append(end - start)
37     start = time.time()
38     mergesort(arr.copy())
39     end = time.time()
40     tiempos_merge.append(end - start)
41
42 plt.loglog(ns, tiempos_ins, 'o-', label='Insercion O(n^2)')
43 plt.loglog(ns, tiempos_merge, 's-', label='Mergesort O(n log n)')
44 plt.xlabel('n'); plt.ylabel('Tiempo (s)')
45 plt.legend(); plt.grid(True)
46 plt.show()

```

Listing 17: Insercion vs Mergesort

C10. Solucion:

```

1 import time
2 import matplotlib.pyplot as plt

```

```
3
4 def fib_rec(n):
5     if n <= 1:
6         return n
7     return fib_rec(n-1) + fib_rec(n-2)
8
9 def fib_iter(n):
10    a, b = 0, 1
11    for _ in range(n):
12        a, b = b, a+b
13    return a
14
15 ns = [10, 20, 30, 35, 40]
16 tiempos_rec = []
17 tiempos_iter = []
18 for n in ns:
19     start = time.time()
20     fib_rec(n)
21     end = time.time()
22     tiempos_rec.append(end - start)
23     start = time.time()
24     fib_iter(n)
25     end = time.time()
26     tiempos_iter.append(end - start)
27
28 plt.semilogy(ns, tiempos_rec, 'o-', label='Recursivo  $O(2^n)$ ')
29 plt.semilogy(ns, tiempos_iter, 's-', label='Iterativo  $O(n)$ ')
30 plt.xlabel('n'); plt.ylabel('Tiempo (s)')
31 plt.legend(); plt.grid(True)
32 plt.show()
```

Listing 18: Fibonacci recursivo vs iterativo

Cierre: La posta final de todo este viaje

Bueno, che, llegamos al final. ¿Viste todo lo que recorrimos? De una maquina abstracta con una cinta infinita a los errores de redondeo en tu notebook. La idea era que entiendas que la computacion numerica no es solo tirar codigo, sino tener un marco conceptual que te permita entender por que las cosas salen bien... o mal.

Resumen rapido de lo que no te podes olvidar:

- Una maquina de Turing define que es computable. Si un problema es indecidible (como el de la detencion), no hay algoritmo que lo resuelva, por mas Python que le pongas.
- La complejidad te dice si tu algoritmo es factible. Si es $O(2^n)$, para $n = 100$ te vas a jubilar esperando. El Teorema Maestro es tu amigo para recurrencias divide y venceras.
- La aritmetica de punto flotante es una aproximacion. El epsilon de maquina es tu limite de resolucion. La cancelacion catastrofica y la inestabilidad numerica son los enemigos a vencer.
- Siempre que puedas, analiza el error de tu algoritmo. Un algoritmo matematicamente perfecto puede ser un desastre numerico.

Mi consejo final: no te frustres cuando un codigo no ande. A todos nos paso que el grafico que deberia ser suave parece la montania rusa. Es parte del aprendizaje. Hace como hicimos aca:

agarra el concepto, bajalo a tierra con un grafico, y busca la forma de domar el error. Y acordate, el que avisa no traiciona: **el epsilon de maquina siempre esta ahi, mirandote.**

¡Dale para adelante, que la teoria de la computacion y el analisis numerico son viajes fascinantes!

Errores, Estabilidad y Condicionamiento

Como las computadoras mienten, como sobrevivir a eso, y como hacer que los metodos numericos no te exploten en la cara.

Che, ¿como andas? Bienvenido al capitulo mas importante (y mas denso) de todo el calculo numerico. Aca es donde la teoria se encuentra con la realidad, y donde te das cuenta de que las computadoras no son perfectas, sino que aproximan, redondean, y a veces te tiran cualquier fruta. Yo, cuando empecé con esto, pensaba que la maquina siempre tenia razon. Hasta que un dia, en un informe de laboratorio, me paso que integrando una funcion con dos metodos distintos me daban resultados que diferian en el cuarto decimal, y no sabia cual era el correcto. Despues de putear un rato, me di cuenta de que el problema no era la maquina, sino mi comprension de los errores y el condicionamiento.

En este capitulo vamos a desmenuzar todo eso. Arrancamos viendo como se representan los numeros en una computadora y que errores introduce eso (punto flotante y redondeo). Despues vemos como esos errores se propagan cuando haces cuentas, y el famoso fenomeno de la cancelacion catastrofica. Luego metemos las normas, que son la forma de medir vectores y matrices, y el numero de condicion, que te dice que tan sensible es un problema. Finalmente, distinguimos entre algoritmos inestables y problemas mal condicionados, que no es lo mismo. Y, como siempre, con mucho codigo Python para que veas los graficos y puedas experimentar vos mismo.

La realidad finita de los numeros: Punto Flotante

El primer problema que tuve al aprender calculo numerico fue entender que la computadora no trabaja con numeros reales, sino con una version aproximada y finita de ellos. No puede almacenar infinitos digitos, así que redondea. Y eso, aunque parezca una boludez, tiene consecuencias enormes.

Representacion de punto flotante

Una computadora representa un numero real en lo que se llama **punto flotante**. La idea es parecida a la notacion cientifica: escribis el numero como una mantisa (con una cantidad fija de digitos) multiplicada por una potencia de la base (generalmente 2). En criollo, es como si escribieras $x = \pm 0.d_1d_2 \dots d_t \times 10^e$, pero en binario y con un numero fijo de bits para la mantisa y el exponente.

El estandar mas usado es el **IEEE 754**. En doble precision (el `float` de Python, que es lo que usamos casi siempre), tenes 64 bits: 1 para el signo, 11 para el exponente, y 52 para la mantisa (mas un bit implicito, así que en total son 53 bits de precision). Esto te da aproximadamente 15-16 digitos decimales significativos. En simple precision (32 bits), tenes 23 bits de mantisa, que son unos 7 digitos decimales.

Traduccion Matematica de la representacion IEEE 754: La formula $x = (-1)^s \times 1.f \times 2^{e-1023}$ se lee: .^{el} numero x es positivo si el bit s es 0, negativo si es 1. Su mantisa es 1, seguido de la parte fraccionaria f (que son 52 bits), y se multiplica por 2 elevado a la $e - 1023$, donde e es

el exponente almacenado (11 bits)..^{El sesgo de 1023 es para que el exponente pueda ser negativo sin usar un bit de signo aparte. Es como si escribieras $1, algo \times 2^{exponente}$, pero todo en binario.}

Epsilon de maquina: la lupa de la compu

El **epsilon de maquina** (ϵ_{mach}) es la distancia entre el numero 1 y el siguiente numero representable. En doble precision, es $2^{-52} \approx 2,22 \times 10^{-16}$. Esto significa que, si vos sumas algo mas chico que eso a 1, la computadora no lo registra, porque el resultado se redondea a 1. Es como si tuvieras una lupa que no puede ver detalles mas chicos que eso.

El **unit roundoff** es $u = \frac{1}{2}\epsilon_{mach}$, que es el maximo error relativo que introducís al redondear un numero. Para doble precision, $u \approx 1,11 \times 10^{-16}$.

Codigo para encontrar el epsilon:

```
1 import numpy as np
2
3 def encontrar_epsilon(tipo=float):
4     eps = tipo(1.0)
5     while tipo(1.0) + eps / tipo(2.0) != tipo(1.0):
6         eps /= tipo(2.0)
7     return eps
8
9 print("float64:", encontrar_epsilon(float))
10 print("float32:", encontrar_epsilon(np.float32))
11 print("Valor de numpy:", np.finfo(float).eps)
```

Listing 19: Encontrando el epsilon de maquina

El modelo estandar de aritmetica

El modelo estandar dice que cada operacion aritmetica (suma, resta, multiplicacion, division) introduce un error relativo del orden de u :

$$\text{fl}(a \text{ op } b) = (a \text{ op } b)(1 + \delta), \quad |\delta| \leq u.$$

Esto es recontra importante: cada vez que haces una cuenta, el resultado tiene un errorcito. Y esos errores se van acumulando.

Formatos de punto flotante en la practica

En la vida real, no solo existe doble precision. En machine learning y GPUs se usan formatos mas chicos para ahorrar memoria y acelerar calculos. Aca te dejo una tabla con los mas comunes:

Cuadro 2: Formatos de punto flotante comunes.

Formato	Bits	Signo	Exponente	Mantisa (total)	ϵ_{mach}
FP64 (double)	64	1	11	52 (53)	$2^{-52} \approx 2,2 \times 10^{-16}$
FP32 (float)	32	1	8	23 (24)	$2^{-23} \approx 1,2 \times 10^{-7}$
FP16 (half)	16	1	5	10 (11)	$2^{-10} \approx 9,8 \times 10^{-4}$
BF16	16	1	8	7 (8)	$2^{-7} \approx 7,8 \times 10^{-3}$

Fijate que el BF16 tiene el mismo rango de exponentes que FP32, lo que evita problemas de overflow/underflow en redes neuronales, pero con menos precision. FP16 tiene mas precision pero menos rango. Cada formato tiene sus ventajas y desventajas.

Propagacion de errores y Cancelacion Catastrofica

Ahora que sabemos que cada operacion tiene error, la pregunta es: ¿como se propagan esos errores?

Error absoluto y relativo

Si tienes un valor exacto x y una aproximacion $\tilde{x}$, el error absoluto es $|x - \tilde{x}|$, y el error relativo es $\frac{|x - \tilde{x}|}{|x|}$ (si $x \neq 0$). El error relativo es el que realmente importa, porque te dice cuantos digitos significativos tienes.

Propagacion en sumas y restas

Cuando sumas dos numeros, los errores absolutos se suman. Pero el error relativo puede explotar si el resultado es chico comparado con los operandos. Ahi aparece la **cancelacion catastrofica**. Imaginate que tienes $a = 1,234567890123456$ y $b = 1,234567890123455$. Ambos son numeros de maquina con 16 digitos significativos. Si los restas, obtienes 10^{-15} , que tiene solo un digito significativo. Los otros 15 digitos se perdieron porque eran iguales en ambos numeros. Es como si tuvieras dos montañas de arena casi iguales, las pesas, y al restar los pesos obtienes un granito de arena: el error relativo de ese granito es gigante.

Traduccion Matematica de la cancelacion: La resta $a - b$ con $a \approx b$ es peligrosa porque, aunque la diferencia sea chica, los errores relativos de a y b se suman. En notacion: si $\tilde{a} = a(1 + \delta_a)$ y $\tilde{b} = b(1 + \delta_b)$, entonces $\tilde{a} - \tilde{b} = (a - b) + a\delta_a - b\delta_b$. El error absoluto se mantiene chico, pero al dividir por $a - b$ (que es chico), el error relativo explota.

Ejemplo grafico:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def formula_directa(x):
5     return (1 - np.cos(x)) / x**2
6
7 def formula_estable(x):
8     return 2 * (np.sin(x/2) / x)**2
9
10 xs = np.logspace(-1, -8, 50)
11 exacto = 0.5
12 err_dir = [abs(formula_directa(x) - exacto) for x in xs]
13 err_est = [abs(formula_estable(x) - exacto) for x in xs]
14
15 plt.loglog(xs, err_dir, 'o-', label='Directa (cancelacion)')
16 plt.loglog(xs, err_est, 's-', label='Estable')
17 plt.xlabel('x'); plt.ylabel('Error absoluto')
18 plt.legend(); plt.grid(True, which='both')
19 plt.title('Cancelacion catastrofica en accion')
20 plt.show()

```

Listing 20: Cancelacion catastrofica en $(1 - \cos x)/x^2$

El grafico muestra como la formula directa pierde precision para x chico, mientras que la estable se mantiene cerca del epsilon de maquina.

Propagacion en multiplicaciones y divisiones

En multiplicaciones y divisiones, los errores relativos se suman (en primera aproximacion). O sea, si cada operando tiene error relativo del orden de u , el resultado tendra error relativo del orden de u . Por eso multiplicar es mas "estable" que sumar/restar.

El caso del producto interno

Calcular $s = \sum_{i=1}^n a_i b_i$ en punto flotante puede tener error relativo grande si hay cancelacion. La cota clasica es:

$$|\hat{s} - s| \leq nu \sum_{i=1}^n |a_i b_i| + O(u^2)$$

donde $u = \epsilon_{\text{mach}}/2$. O sea, el error absoluto puede ser grande si los terminos son grandes aunque se cancelen. Por eso conviene usar algoritmos como la suma compensada (Kahan) para mejorar la precision.

La suma no es asociativa en punto flotante

En punto flotante, $(a + b) + c \neq a + (b + c)$ en general:

$$(10^{16} + 1) - 10^{16} = 0 \quad (\text{el } 1 \text{ se pierde al sumarse a } 10^{16}),$$

$$10^{16} + (1 - 10^{16}) = 10^{16} + (-10^{16} + 1) = 1 \quad (\text{orden diferente}).$$

El resultado depende del orden de evaluacion porque la precision relativa es fija: los numeros representables cerca de 10^{16} estan espaciados en mas de 1.

Suma compensada de Kahan

Para sumar una secuencia de numeros con precision, puedes usar el algoritmo de Kahan, que mantiene un termino de compensacion para corregir el error de redondeo:

```

1 def kahan_sum(arr):
2     s = 0.0
3     c = 0.0
4     for x in arr:
5         y = x - c
6         t = s + y
7         c = (t - s) - y
8         s = t
9     return s
10
11 # Comparacion con suma directa
12 import numpy as np
13 arr = [1e10, 1e-10, -1e10] * 1000000
14 print("Directa:", sum(arr))
15 print("Kahan:", kahan_sum(arr))

```

Listing 21: Suma compensada de Kahan

Analisis de errores hacia adelante y hacia atras

Cuando resuelves un problema numericamente, hay dos formas de ver el error:

Error hacia adelante

El error hacia adelante es la diferencia entre la solución obtenida y la solución exacta. Es lo que realmente te importa, pero suele ser difícil de calcular porque no sabes la solución exacta.

Error hacia atrás

El error hacia atrás pregunta: ¿qué perturbación en los datos hace que la solución obtenida sea exacta para el problema perturbado? Es decir, si $\tilde{x}$ es la solución que obtuviste, el error hacia atrás es $\min\{\|\Delta A\|, \|\Delta b\| : (A + \Delta A)\tilde{x} = b + \Delta b\}$. La idea es: si el error hacia atrás es chico, entonces tu solución es la solución exacta de un problema cercano al original. Eso es más fácil de medir (usando el residuo $r = b - A\tilde{x}$).

Relación entre errores

El teorema de perturbación que vimos antes relaciona el error hacia adelante con el error hacia atrás:

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\Delta b\|}{\|b\|}.$$

Si el problema está bien condicionado ($\kappa(A)$ chico), un error hacia atrás chico implica un error hacia adelante chico. Si está mal condicionado, no.

Ejemplo: el residuo no siempre es confiable

Un residuo chico no garantiza una solución buena si la matriz está mal condicionada. Por ejemplo, para la matriz de Hilbert, el residuo puede ser del orden de 10^{-16} pero la solución puede tener error de 10^{-2} . Por eso siempre tienes que chequear el número de condición.

Normas y Condicionamiento: La sensibilidad de los problemas

Hasta ahora vimos cómo los errores de redondeo afectan las operaciones básicas. Pero cuando resuelves sistemas de ecuaciones $Ax = b$, el error en los datos puede amplificarse muchísimo. Para cuantificar eso, necesitas medir vectores y matrices: las **normas**.

Normas vectoriales

Una norma es una forma de medir el tamaño de un vector. Las más comunes son:

- Norma 1: $\|x\|_1 = \sum_{i=1}^n |x_i|$
- Norma 2 (euclídea): $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- Norma infinito: $\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$

Normas matriciales

Dada una norma vectorial, puedes definir la norma matricial asociada:

$$\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

Esto mide el máximo "estiramiento" que la matriz A produce sobre los vectores. Las más fáciles de calcular son:

- $\|A\|_1 = \max_j \sum_{i=1}^n |a_{ij}|$ (maxima suma absoluta por columna)
- $\|A\|_\infty = \max_i \sum_{j=1}^n |a_{ij}|$ (maxima suma absoluta por fila)
- $\|A\|_2 = \sigma_{\max}(A)$ (maximo valor singular, mas caro de calcular)

Numero de condicion

El **numero de condicion** de una matriz inversible A se define como

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

En norma 2, $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$. Siempre $\kappa(A) \geq 1$. Si $\kappa(A) \gg 1$, la matriz esta **mal condicionada**: pequenas perturbaciones en los datos producen grandes cambios en la solucion.

Traduccion Matematica del numero de condicion: La definicion $\kappa(A) = \|A\| \|A^{-1}\|$ se lee: "el numero de condicion de A es el producto de la norma de A y la norma de su inversa". En norma 2, es el cociente entre el mayor y el menor valor singular. Pensa en una matriz como una lente que deforma el espacio. Si los valores singulares son muy dispares, la lente aplasta mucho en una direccion y estira en otra. Un pequenio error en la entrada se convierte en un gran error en la salida.

Teorema fundamental de perturbacion

Si $A(x + \Delta x) = b + \Delta b$, entonces

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\Delta b\|}{\|b\|}.$$

Esto es CLAVE: el error relativo en la solucion puede ser hasta $\kappa(A)$ veces el error relativo en el dato. Si $\kappa(A) = 10^6$, un error de 10^{-6} en b puede darte un error de 100 % en x . ¡Una locura!

El peor caso y como alcanzarlo

La cota es ajustada: se alcanza cuando b esta alineado con el vector singular izquierdo de maximo valor singular ($\sigma_{\max}$) y Δb con el de minimo valor singular ($\sigma_{\min}$). En ese caso, el error se amplifica exactamente por $\kappa(A)$.

Distancia a la singularidad

Se puede probar que

$$\frac{1}{\kappa(A)} = \inf_{B \text{ singular}} \frac{\|A - B\|}{\|A\|}.$$

Es decir, $\kappa(A)$ grande significa que A esta cerca de ser singular en terminos relativos. No tiene que ver con el determinante: una matriz diagonal $0,001I$ tiene determinante minuscuro pero $\kappa = 1$ (bien condicionada). En cambio, una matriz con filas casi dependientes tendra κ enorme.

Ejemplo clasico: la matriz de Hilbert

La matriz de Hilbert $H_{ij} = 1/(i+j-1)$ es el ejemplo clasico de matriz mal condicionada. Para $n = 10$, $\kappa_2(H_{10}) \approx 1,6 \times 10^{13}$. Resolver un sistema con ella en doble precision (16 digitos) es suicida: perdes 13 digitos de precision.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 ns = range(2, 13)
5 cond = []
6 for n in ns:
7     H = np.array([[1/(i+j-1) for j in range(1,n+1)] for i in range(1,n+1)])
8     cond.append(np.linalg.cond(H, 2))
9
10 plt.semilogy(ns, cond, 'o-')
11 plt.xlabel('n'); plt.ylabel('cond_2(H_n)')
12 plt.title('Crecimiento exponencial del numero de condicion de Hilbert')
13 plt.grid(True)
14 plt.show()

```

Listing 22: Condicion de la matriz de Hilbert

Otras matrices mal condicionadas

Ademas de Hilbert, hay otras matrices notoriamente mal condicionadas:

- Matrices de Vandermonde (para interpolacion con nodos equiespaciados)
- Matrices de Toeplitz con decaimiento lento
- Matrices con elementos muy dispares en magnitud

En general, cualquier matriz que tenga filas o columnas casi dependientes va a tener un numero de condicion grande.

La SVD y el condicionamiento

La **Descomposicion en Valores Singulares (SVD)** $A = U\Sigma V^T$ revela la estructura profunda de A . Los valores singulares σ_i son las longitudes de los semiejes de la elipsoide imagen. La matriz esta mal condicionada si $\sigma_{\min}$ es muy pequenio respecto de $\sigma_{\max}$.

Descomposicion en Valores Singulares (SVD)

La SVD es una factorizacion que existe para cualquier matriz (no necesariamente cuadrada). Te descompone A en tres matrices: U (ortogonal), Σ (diagonal con los valores singulares), y V (ortogonal). Los valores singulares son no negativos y estan ordenados de mayor a menor.

Traduccion Matematica de la SVD: La formula $A = U\Sigma V^T$ se lee: "la matriz A se puede escribir como el producto de una matriz ortogonal U , una matriz diagonal Σ con los valores singulares en la diagonal, y otra matriz ortogonal V^T ." Los valores singulares σ_i te dicen cuanto estira la matriz en cada direccion principal. Si algun σ_i es cero, la matriz es singular. Si es muy chico, esta cerca de ser singular.

La SVD y la pseudoinversa

La SVD permite construir la pseudoinversa de Moore-Penrose:

$$A^+ = V\Sigma^+U^T,$$

donde Σ^+ tiene $1/\sigma_i$ en los valores singulares no nulos y 0 en los nulos. Esto es útil para resolver sistemas sobredeterminados o mal condicionados.

Regularizacion de Tikhonov

Cuando el problema está mal condicionado, puedes regularizar agregando un término de penalización:

$$\min_x (\|Ax - b\|_2^2 + \lambda^2 \|x\|_2^2).$$

La solución regularizada es:

$$x_\lambda = \sum_{i=1}^n \frac{\sigma_i}{\sigma_i^2 + \lambda^2} (u_i^T b) v_i.$$

Los términos con σ_i chico se “apagan” porque el denominador se hace grande. Esto estabiliza la solución a costa de introducir un sesgo.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Construimos un problema mal condicionado
5 n = 10
6 A = np.array([[1/(i+j-1) for j in range(1,n+1)] for i in range(1,n+1)])
7 x_true = np.ones(n)
8 b = A @ x_true
9
10 # Agregamos ruido
11 b_noise = b + 0.01 * np.random.randn(n)
12
13 # Resolvemos sin regularizar
14 x_ls = np.linalg.lstsq(A, b_noise, rcond=None)[0]
15
16 # Regularizacion de Tikhonov (lambda=0.1)
17 U, s, Vt = np.linalg.svd(A, full_matrices=False)
18 lam = 0.1
19 x_tikh = Vt.T @ ((s / (s**2 + lam**2)) * (U.T @ b_noise))
20
21 print("Error sin regularizar:", np.linalg.norm(x_ls - x_true))
22 print("Error con regularizacion:", np.linalg.norm(x_tikh - x_true))
23
24 plt.bar(range(n), [x_true, x_ls, x_tikh], tick_label=['Exacto', 'LS', 'Tikhonov'])
25 plt.title('Comparacion de soluciones')
26 plt.show()

```

Listing 23: Regularizacion de Tikhonov para un problema mal condicionado

Aproximacion de bajo rango

La SVD también te permite aproximar una matriz por otra de rango menor. El teorema de Eckart-Young dice que la mejor aproximación de rango k en norma 2 es truncar la SVD:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T.$$

El error es $\|A - A_k\|_2 = \sigma_{k+1}$. Esto es útil para compresión de imágenes y reducción de dimensionalidad.

Algoritmos estables vs problemas mal condicionados

Aca hay una distinción fundamental que tenes que tener clara:

- **Problema mal condicionado:** el problema en si es sensible a perturbaciones. No importa que algoritmo uses, no puedes evitar la amplificación del error. Ejemplo: encontrar las raíces de un polinomio con coeficientes muy sensibles.
- **Algoritmo inestable:** el algoritmo que usas amplifica innecesariamente los errores, aunque el problema este bien condicionado. Ejemplo: la recurrencia hacia adelante para $E_n = \int_0^1 x^n e^{x-1} dx$.

Ejemplo de algoritmo inestable: la recurrencia E_n

Integrando por partes, obtenes $E_n = 1 - nE_{n-1}$. Hacia adelante, el error se multiplica por n en cada paso, explotando. Hacia atras, empezando con $E_{20} \approx 0$, el error se divide, y es estable.

```

1 import numpy as np
2
3 def forward(N):
4     E = np.zeros(N+1)
5     E[1] = 1/np.e
6     for n in range(2, N+1):
7         E[n] = 1 - n*E[n-1]
8     return E
9
10 def backward(N, E20=0.0):
11     E = np.zeros(N+1)
12     E[N] = E20
13     for n in range(N, 1, -1):
14         E[n-1] = (1 - E[n]) / n
15     return E
16
17 N = 20
18 Ef = forward(N)
19 Eb = backward(N, 0.0)
20 print("Forward E9:", Ef[9], "    Backward E9:", Eb[9])
21 # El valor exacto ronda 0.0916

```

Listing 24: Recurrencia inestable vs estable

Ejemplo de problema mal condicionado: la matriz de Wilkinson

Wilkinson mostro que cambiar el coeficiente -210 por $-210 - 2^{-23}$ en el polinomio $(x-1)(x-2)\cdots(x-20)$ hace que las raíces 16 y 17 se vuelvan complejas. ¡Un cambio de 10^{-7} en un coeficiente produce un cambio de 10^{-2} en las raíces! Eso es un problema mal condicionado, no importa como lo resuelvas.

Ejemplo de algoritmo inestable: el metodo de Newton para raíces múltiples

El metodo de Newton converge cuadráticamente para raíces simples, pero para raíces múltiples la convergencia es lineal. Si no sabes que la raíz es múltiple, puedes pensar que el metodo esta funcionando mal cuando en realidad el problema es la función.

Como distinguir entre ambos casos

Si cambias el algoritmo y el error mejora, era inestabilidad del algoritmo. Si cambias el algoritmo y el error sigue siendo grande, el problema esta mal condicionado. Una forma practica es hacer un analisis de sensibilidad: perturba los datos ligeramente y mira como cambia la solucion. Si cambia mucho, el problema es mal condicionado.

Metodos directos para sistemas lineales y su estabilidad

Ahora que entendemos como se comportan los errores, veamos como resolver sistemas $Ax = b$.

Eliminacion Gaussiana y factorizacion LU

La eliminacion Gaussiana transforma el sistema en uno triangular superior. En la practica, se hace con la factorizacion $PA = LU$, donde P es una matriz de permutacion (pivoteo), L es triangular inferior con unos en la diagonal, y U es triangular superior.

El **pivoteo parcial** (elegir el mayor elemento de la columna como pivote) es CRUCIAL para la estabilidad numerica. Si no pivoteas, puedes terminar dividiendo por numeros muy chicos, amplificando los errores.

Factorizacion de Cholesky

Si A es simetrica y definida positiva, puedes usar Cholesky: $A = LL^T$. Esto es mas estable y mas rapido que LU (cuesta la mitad de operaciones). Ademas, no necesita pivoteo.

Un ejemplo de inestabilidad sin pivoteo

Considera el sistema:

$$\begin{pmatrix} \epsilon & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

con $\epsilon = 10^{-12}$. Si resuelves sin pivoteo, el multiplicador es $1/\epsilon$, y al restar las filas perdes toda la informacion de la segunda ecuacion, dando $x = 0$, $y = 1$, cuando la solucion correcta es $x \approx 1$, $y \approx 1$. Con pivoteo, intercambias las filas y el resultado es correcto.

```

1 import numpy as np
2
3 def gauss_sin_pivoteo(A, b):
4     n = len(A); x = np.zeros(n)
5     for k in range(n-1):
6         for i in range(k+1,n):
7             m = A[i,k] / A[k,k]
8             A[i,k:] -= m * A[k,k:]
9             b[i] -= m * b[k]
10    for i in range(n-1,-1,-1):
11        x[i] = (b[i] - np.dot(A[i,i+1:], x[i+1:])) / A[i,i]
12    return x
13
14 def gauss_con_pivoteo(A, b):
15     n=len(A); x=np.zeros(n)
16     for k in range(n-1):
17         max_idx = max(range(k,n), key=lambda i: abs(A[i,k]))
18         if max_idx != k:
19             A[[k,max_idx]] = A[[max_idx,k]]
20             b[k], b[max_idx] = b[max_idx], b[k]
21     for i in range(k+1,n):

```

```

22         m = A[i,k] / A[k,k]
23         A[i,k:] -= m * A[k,k:]
24         b[i] -= m * b[k]
25     for i in range(n-1,-1,-1):
26         x[i] = (b[i] - np.dot(A[i,i+1:], x[i+1:])) / A[i,i]
27     return x
28
29 eps = 1e-12
30 A = np.array([[eps, 1], [1, 1]], dtype=float)
31 b = np.array([1, 2], dtype=float)
32
33 x_sin = gauss_sin_pivoteo(A.copy(), b.copy())
34 A2 = np.array([[eps, 1], [1, 1]], dtype=float)
35 b2 = np.array([1, 2], dtype=float)
36 x_con = gauss_con_pivoteo(A2, b2)
37
38 print("Sin pivoteo:", x_sin)
39 print("Con pivoteo:", x_con)
40 print("Exacto:", np.linalg.solve([1,1], [1,2]))

```

Listing 25: Gauss sin y con pivoteo

El crecimiento de los elementos en LU

Un problema con la eliminacion Gaussiana es que los elementos de la matriz pueden crecer durante el proceso, lo que puede causar overflow. El factor de crecimiento g_n mide cuanto crecen los elementos. Sin pivoteo, puede crecer exponencialmente. Con pivoteo parcial, el crecimiento esta acotado por 2^{n-1} , aunque en la practica es mucho menor.

Metodos iterativos para sistemas lineales

Cuando la matriz es enorme y rala, los metodos directos pueden ser demasiado costosos. Los metodos iterativos construyen una sucesion $x^{(k)}$ que converge a la solucion.

Jacobi y Gauss-Seidel

La idea es descomponer $A = D + L + U$. Jacobi actualiza todas las componentes usando los valores viejos:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

Gauss-Seidel usa los valores ya actualizados:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right)$$

Gauss-Seidel suele converger mas rapido.

Convergencia y radio espectral

La iteracion se escribe $x^{(k+1)} = Gx^{(k)} + c$. El error satisface $e^{(k)} = G^k e^{(0)}$. Converge para todo $e^{(0)}$ si y solo si $\rho(G) < 1$ (radio espectral menor que 1).

El teorema de Stein-Rosenberg

Para matrices con elementos no positivos fuera de la diagonal (matrices de Stieltjes), el radio espectral de Gauss-Seidel es menor que el de Jacobi: $\rho(G_{GS}) \leq \rho(G_J)$. En particular, si Jacobi converge, Gauss-Seidel converge mas rapido.

Metodo SOR (Successive Over-Relaxation)

El metodo SOR es una generalizacion de Gauss-Seidel con un parametro de relajacion ω :

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right)$$

Si $0 < \omega < 2$, el metodo converge para matrices simetricas definidas positivas. El ω optimo puede acelerar mucho la convergencia.

Un ejemplo comparativo:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def jacobi(A, b, x0, tol=1e-6, maxit=100):
5     n = len(b)
6     x = x0.copy()
7     errores = []
8     for _ in range(maxit):
9         x_new = np.zeros(n)
10        for i in range(n):
11            suma = b[i] - np.dot(A[i, :], x) + A[i, i]*x[i]
12            x_new[i] = suma / A[i, i]
13        err = np.linalg.norm(x_new - x, np.inf)
14        errores.append(err)
15        x = x_new
16        if err < tol:
17            break
18    return x, errores
19
20 def gauss_seidel(A, b, x0, tol=1e-6, maxit=100):
21     n = len(b)
22     x = x0.copy()
23     errores = []
24     for _ in range(maxit):
25         x_old = x.copy()
26         for i in range(n):
27             suma = b[i] - np.dot(A[i, :], x) + A[i, i]*x[i]
28             x[i] = suma / A[i, i]
29         err = np.linalg.norm(x - x_old, np.inf)
30         errores.append(err)
31         if err < tol:
32             break
33    return x, errores
34
35 def sor(A, b, x0, omega, tol=1e-6, maxit=100):
36     n = len(b)
37     x = x0.copy()
38     errores = []
39     for _ in range(maxit):

```

```

40     x_old = x.copy()
41     for i in range(n):
42         suma = b[i] - np.dot(A[i, :], x) + A[i, i]*x[i]
43         x[i] = (1-omega)*x[i] + omega*suma / A[i, i]
44     err = np.linalg.norm(x - x_old, np.inf)
45     errores.append(err)
46     if err < tol:
47         break
48     return x, errores
49
50 A = np.array([[4, -1, 0], [-1, 4, -1], [0, -1, 4]], dtype=float)
51 b = np.array([3, 2, 3], dtype=float)
52 x0 = np.zeros(3)
53
54 _, err_j = jacobi(A, b, x0, maxit=50)
55 _, err_gs = gauss_seidel(A, b, x0, maxit=50)
56 _, err_sor = sor(A, b, x0, omega=1.2, maxit=50)
57
58 plt.semilogy(err_j, label='Jacobi')
59 plt.semilogy(err_gs, label='Gauss-Seidel')
60 plt.semilogy(err_sor, label='SOR (omega=1.2)')
61 plt.xlabel('Iteracion')
62 plt.ylabel('Error infinito')
63 plt.legend()
64 plt.grid(True)
65 plt.title('Convergencia de metodos iterativos')
66 plt.show()

```

Listing 26: Comparacion Jacobi vs Gauss-Seidel

Errores de redondeo en metodos iterativos

Los metodos iterativos tambien sufren errores de redondeo. La pregunta es: ¿como afectan estos errores a la convergencia?

Error de redondeo en cada paso

En cada iteracion, el calculo de $x^{(k+1)}$ introduce un error del orden de $u\|x^{(k+1)}\|$. Este error se propaga a las siguientes iteraciones.

Estabilidad de los metodos iterativos

Un metodo iterativo es estable si el error de redondeo no se amplifica. Si $\|G\| < 1$, el error de redondeo se controla. Si $\|G\| \geq 1$, puede explotar. En la practica, los metodos con $\rho(G)$ cercano a 1 son lentos pero estables.

El metodo de la potencia y el error de redondeo

El metodo de la potencia para autovalores es sensible al error de redondeo cuando el autovalor dominante no esta bien separado. Si $\lambda_1 \approx \lambda_2$, el metodo converge lentamente y el error de redondeo puede dominar.

Condicionamiento de autovalores

El condicionamiento de un autovalor simple λ de una matriz A mide como cambia λ ante perturbaciones de A . Si A tiene autovectores casi paralelos, el autovalor puede ser muy sensible.

Metodo de Newton para sistemas no lineales

Ahora que ya sabemos como manejar errores en sistemas lineales, vamos a encarar el problema de resolver sistemas de ecuaciones no lineales. La extension natural del metodo de Newton a varias variables es una de las herramientas mas poderosas del calculo numerico.

Formulacion general

Queremos resolver $F(x) = 0$, donde $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ es una funcion diferenciable. La idea es la misma que en una variable: linealizar alrededor de una aproximacion $x^{(k)}$ y resolver el sistema lineal resultante. La linealizacion es:

$$F(x) \approx F(x^{(k)}) + J_F(x^{(k)})(x - x^{(k)}),$$

donde $J_F(x^{(k)})$ es la matriz Jacobiana (las derivadas parciales de F). Igualando a cero:

$$J_F(x^{(k)})\delta^{(k)} = -F(x^{(k)}), \quad x^{(k+1)} = x^{(k)} + \delta^{(k)}.$$

Cada iteracion requiere resolver un sistema lineal de $n \times n$. Si n es grande, esto puede ser costoso, pero para muchos problemas vale la pena por la convergencia cuadratica.

Traduccion Matematica de la Jacobiana: La matriz Jacobiana $J_F(x)$ tiene en la fila i , columna j la derivada parcial de la funcion F_i respecto de la variable x_j . En criollo, es como la generalizacion de la derivada para funciones vectoriales. Si F es un campo vectorial, la Jacobiana te dice como cambia el campo en cada punto.

Convergencia cuadratica local

Bajo condiciones similares al caso escalar (Jacobiana invertible en la raiz y Lipschitz continua), el metodo de Newton converge cuadraticamente:

$$\|x^{(k+1)} - x^*\| \leq C\|x^{(k)} - x^*\|^2.$$

Esto significa que los digitos correctos se duplican en cada iteracion. Es rapidisimo, pero solo si arrancas lo suficientemente cerca de la solucion.

Ejemplo: resolviendo un sistema de dos ecuaciones

Vamos a resolver el sistema:

$$\begin{cases} x^2 + y^2 - 1 = 0 \\ x - y = 0 \end{cases}$$

La solucion es $(\sqrt{2}/2, \sqrt{2}/2)$. El siguiente codigo implementa Newton en 2D y grafica la convergencia.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def F(x):
5     # F1 = x^2 + y^2 - 1
6     # F2 = x - y
7     return np.array([x[0]**2 + x[1]**2 - 1, x[0] - x[1]])
8
9 def JF(x):
10    # Jacobiana: [[2x, 2y], [1, -1]]

```

```

11     return np.array([[2*x[0], 2*x[1]], [1.0, -1.0]])
12
13 def newton_sistema(F, JF, x0, tol=1e-10, maxit=20):
14     x = x0.copy()
15     historial = [x.copy()]
16     for _ in range(maxit):
17         J = JF(x)
18         b = -F(x)
19         try:
20             delta = np.linalg.solve(J, b)
21         except np.linalg.LinAlgError:
22             print("Jacobiana singular, deteniendo")
23             break
24         x = x + delta
25         historial.append(x.copy())
26         if np.linalg.norm(delta, 2) < tol:
27             break
28     return x, np.array(historial)
29
30 # Punto inicial
31 x0 = np.array([0.5, 0.0])
32 sol, hist = newton_sistema(F, JF, x0)
33
34 print("Solucion aproximada:", sol)
35 print("Error:", np.linalg.norm(sol - np.array([np.sqrt(2)/2, np.sqrt(2)/2])))
36
37 # Graficamos la trayectoria
38 plt.plot(hist[:,0], hist[:,1], 'o-', label='Newton')
39 plt.plot(hist[0,0], hist[0,1], 'go', label='Inicio')
40 plt.plot(sol[0], sol[1], 'r*', markersize=12, label='Solucion')
41 theta = np.linspace(0, 2*np.pi, 100)
42 plt.plot(np.cos(theta), np.sin(theta), 'k--', label='Circunferencia')
43 plt.plot([-1.5, 1.5], [-1.5, 1.5], 'k:', label='x=y')
44 plt.axis('equal')
45 plt.legend()
46 plt.title('Metodo de Newton en 2D')
47 plt.grid(True)
48 plt.show()

```

Listing 27: Newton en 2D para sistema no lineal

El grafico muestra como desde el punto inicial (0,5,0) el metodo converge en pocos pasos al punto de interseccion de la circunferencia y la recta. Si arrancas lejos, puedes diverger, pero con un buen punto inicial es imparable.

Metodo de Newton para optimizacion

Si tienes un problema de minimizacion $\min f(x)$, puedes aplicar Newton al sistema $\nabla f(x) = 0$. La iteracion es:

$$x^{(k+1)} = x^{(k)} - [\nabla^2 f(x^{(k)})]^{-1} \nabla f(x^{(k)}),$$

donde $\nabla^2 f$ es la Hessiana. Esto converge cuadraticamente si la Hessiana es definida positiva en el minimo.

Metodo de la secante en varias variables (Broyden)

Cuando la Jacobiana es costosa de calcular, puedes usar metodos que la aproximan, como el metodo de Broyden (una generalizacion de la secante). La idea es actualizar la Jacobiana en cada paso usando solo evaluaciones de F .


```

1 def broyden(F, x0, tol=1e-8, maxit=20):
2     n = len(x0)
3     x = x0.copy()
4     B = np.eye(n) # Aproximacion inicial a la Jacobiana
5     Fx = F(x)
6     historial = [x.copy()]
7     for k in range(maxit):
8         try:
9             delta = np.linalg.solve(B, -Fx)
10        except np.linalg.LinAlgError:
11            print("B singular, deteniendo")
12            break
13        x_new = x + delta
14        Fx_new = F(x_new)
15        s = delta
16        y = Fx_new - Fx
17        # Actualizacion de Broyden (rank-one)
18        if np.linalg.norm(s) > 1e-15:
19            B = B + np.outer((y - B @ s), s) / np.dot(s, s)
20        x = x_new
21        Fx = Fx_new
22        historial.append(x.copy())
23        if np.linalg.norm(delta, 2) < tol:
24            break
25    return x, np.array(historial)
26
27 # Probamos con el mismo sistema
28 x0 = np.array([0.5, 0.0])
29 sol_b, hist_b = broyden(F, x0)
30 print("Broyden:", sol_b)
31 print("Error Broyden:", np.linalg.norm(sol_b - np.array([np.sqrt(2)/2, np.sqrt(2)/2])))

```

Listing 28: Metodo de Broyden para sistemas no lineales

Broyden converge mas lento que Newton (superlinealmente), pero evita calcular la Jacobiana exacta, lo que puede ahorrar tiempo para problemas grandes.

Metodos de continuacion (homotopia)

¿Que pasa si el metodo de Newton no converge porque el punto inicial esta muy lejos? Una estrategia es usar un metodo de continuacion: introducís un parametro λ que deforma el problema original en uno facil de resolver, y luego seguís la solucion a medida que λ varia de 0 a 1.

Por ejemplo, para resolver $F(x) = 0$, podés definir $H(x, \lambda) = \lambda F(x) + (1-\lambda)(x-x_0)$. Para $\lambda = 0$, la solucion es $x = x_0$. Para $\lambda = 1$, es la solucion de $F(x) = 0$. Luego resolvés una secuencia de problemas $H(x, \lambda_k) = 0$ para λ_k creciente, usando la solucion anterior como punto inicial.

```

1 def continuacion(F, JF, x0, lambdas):
2     x = x0.copy()
3     trayectoria = [x.copy()]
4     for lam in lambdas:
5         # Definimos H(x) = lam*F(x) + (1-lam)*(x - x0)
6         def H(x):
7             return lam * F(x) + (1-lam) * (x - x0)
8         def JH(x):
9             return lam * JF(x) + (1-lam) * np.eye(len(x))
10        # Newton para este lambda
11        for _ in range(10):

```

```

12     delta = np.linalg.solve(JH(x), -H(x))
13     x = x + delta
14     if np.linalg.norm(delta) < 1e-8:
15         break
16     trayectoria.append(x.copy())
17     return x, np.array(trayectoria)
18
19 # Probamos con el sistema anterior, pero con un punto inicial malo
20 x0_malo = np.array([2.0, 2.0])
21 lambdas = np.linspace(0, 1, 20)
22 sol_cont, traj = continuacion(F, JF, x0_malo, lambdas)
23 print("Continuacion:", sol_cont)
24 print("Error:", np.linalg.norm(sol_cont - np.array([np.sqrt(2)/2, np.sqrt(2)/2])
25     ))
26 plt.plot(traj[:,0], traj[:,1], 'o-', label='Trayectoria')
27 plt.plot(x0_malo[0], x0_malo[1], 'go', label='Inicio')
28 plt.plot(sol_cont[0], sol_cont[1], 'r*', markersize=12, label='Solucion')
29 plt.axis('equal')
30 plt.legend()
31 plt.title('Continuacion con homotopia')
32 plt.grid(True)
33 plt.show()

```

Listing 29: Continuacion simple para un sistema

La continuacion puede convertir un problema dificil en una secuencia de problemas faciles.

Estabilidad de metodos de integracion numerica

Cuando resuelves ecuaciones diferenciales numericamente, la estabilidad es crucial. Un metodo puede ser consistente (el error local tiende a cero con el paso), pero si es inestable, el error global explota.

Ecuacion de prueba de Dahlquist

Para analizar estabilidad, se usa la ecuacion de prueba $y' = \lambda y$, con λ complejo y $\Re(\lambda) < 0$. La solucion exacta decae a cero. Un metodo numerico es **A-estable** si reproduce este decaimiento para cualquier paso $h > 0$.

Region de estabilidad absoluta

La region de estabilidad absoluta es el conjunto de $z = h\lambda$ para los cuales el metodo aplicado a la ecuacion de prueba produce soluciones acotadas. Para metodos explicitos, esta region suele ser un disco o una region acotada. Para metodos implicitos, puede ser todo el semiplano izquierdo.

Ejemplo: Euler explicito $y_{n+1} = (1 + h\lambda)y_n$. La region de estabilidad es $|1 + z| \leq 1$. Esto es un disco centrado en -1 de radio 1. Si $h\lambda$ cae fuera, la solucion explota.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Graficamos la region de estabilidad de Euler explicito
5 x = np.linspace(-2.5, 0.5, 200)
6 y = np.linspace(-1.5, 1.5, 200)
7 X, Y = np.meshgrid(x, y)
8 Z = X + 1j*Y
9 # Region: |1+Z| <= 1

```

```

10 region = np.abs(1 + Z) <= 1
11
12 plt.contourf(X, Y, region, levels=[0.5, 1], colors=['lightblue'])
13 plt.plot([-2, 0.5], [0,0], 'k-', linewidth=0.5)
14 plt.plot([0,0], [-1.5,1.5], 'k-', linewidth=0.5)
15 plt.xlabel('Re(z)'); plt.ylabel('Im(z)')
16 plt.title('Region de estabilidad de Euler explicito')
17 plt.axis('equal')
18 plt.grid(True)
19 plt.show()

```

Listing 30: Region de estabilidad de Euler explicito

Metodos A-estables

El metodo de Euler implicito es A-estable: $y_{n+1} = y_n/(1 - h\lambda)$. La funcion de estabilidad es $R(z) = 1/(1 - z)$, y para $\Re(z) < 0$, $|R(z)| < 1$.

La regla del trapecio tambien es A-estable. Son metodos de orden 2. Dahlquist demostro que un metodo multipaso lineal A-estable no puede tener orden mayor que 2. Para metodos de Runge-Kutta, si hay A-estables de orden 4 (por ejemplo, el metodo de Radau), pero no explicitos.

Problemas rigidos

Los problemas rigidos son aquellos donde la solucion tiene componentes que varian a escalas muy diferentes. Por ejemplo, un sistema con autovalores $\lambda_1 = -10^6$ y $\lambda_2 = -1$. La componente rapida decae en un instante, pero obliga a usar pasos muy chicos si usas un metodo explicito. Los metodos implicitos A-estables permiten usar pasos grandes sin inestabilidad, aunque la precision puede ser baja para la componente rapida, que ya se amortiguo.

Ejemplo practico: el problema de Robertson

El problema de Robertson es un sistema de tres ecuaciones quimicas que es extremadamente rigido. Resolverlo con Euler explicito requiere pasos de 10^{-7} , mientras que con Euler implicito o BDF se puede hacer con pasos de 10^{-3} .

```

1 from scipy.integrate import solve_ivp
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def robertson(t, y):
6     # y = [y1, y2, y3]
7     k1 = 0.04
8     k2 = 3e7
9     k3 = 1e4
10    dy1 = -k1*y[0] + k3*y[1]*y[2]
11    dy2 = k1*y[0] - k2*y[1]**2 - k3*y[1]*y[2]
12    dy3 = k2*y[1]**2
13    return [dy1, dy2, dy3]
14
15 y0 = [1.0, 0.0, 0.0]
16 t_span = (0, 10)
17
18 # Resolvemos con BDF (implicito, para rigidos)
19 sol_bdf = solve_ivp(robertson, t_span, y0, method='BDF', rtol=1e-6, atol=1e-9)
20 print("BDF: exitoso", sol_bdf.success, "n_steps:", len(sol_bdf.t))
21
22 # Intentamos con RK45 (explicito, no recomendado para rigidos)

```

```

23 sol_rk45 = solve_ivp(robertson, t_span, y0, method='RK45', rtol=1e-6, atol=1e-9)
24 print("RK45: exitoso", sol_rk45.success, "n_steps:", len(sol_rk45.t))
25
26 plt.semilogy(sol_bdf.t, sol_bdf.y[0], label='BDF (y1)')
27 plt.semilogy(sol_rk45.t, sol_rk45.y[0], label='RK45 (y1)', linestyle='--')
28 plt.xlabel('t')
29 plt.ylabel('y1')
30 plt.legend()
31 plt.grid(True)
32 plt.title('Problema de Robertson')
33 plt.show()

```

Listing 31: Problema de Robertson (sistema rigido)

El metodo BDF (Backward Differentiation Formula) es un metodo implicito multipaso que maneja rigidez eficientemente. RK45 (Runge-Kutta explicito) falla o toma muchisimos pasos.

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Epsilon de maquina:** Define el epsilon de maquina y demuestra que el error relativo de redondeo satisface $|fl(x) - x|/|x| \leq \epsilon_{\text{mach}}/2$.
- T2. Cancelacion catastrofica:** Explica por que restar dos numeros casi iguales es peligroso. Da un ejemplo numerico.
- T3. Propagacion de errores:** Demuestra que el error relativo de un producto ab es aproximadamente la suma de los errores relativos de a y b .
- T4. Normas equivalentes:** Prueba que $\|x\|_\infty \leq \|x\|_2 \leq \sqrt{n}\|x\|_\infty$ para todo $x \in \mathbb{R}^n$.
- T5. Numero de condicion:** Demuestra que $\kappa(A) = \kappa(A^{-1})$.
- T6. Cota de perturbacion:** Deduce la cota $\|\Delta x\|/\|x\| \leq \kappa(A)\|\Delta b\|/\|b\|$ a partir de $A\Delta x = \Delta b$.
- T7. Distancia a la singularidad:** Explica por que $\kappa(A)$ grande implica que A esta cerca de ser singular. ¿Es lo mismo que tener determinante chico?
- T8. Estabilidad vs condicionamiento:** Da un ejemplo de problema mal condicionado y otro de algoritmo inestable. Explica la diferencia.
- T9. Radio espectral:** Demuestra que $\rho(A) \leq \|A\|$ para cualquier norma matricial.
- T10. Convergencia de Jacobi:** Demuestra que si A es estrictamente diagonal dominante, el metodo de Jacobi converge.

Parte Computacional (10 ejercicios)

- C1. Epsilon a mano:** Escribe un codigo para encontrar el epsilon de maquina para float32 y float64.
- C2. Cancelacion visual:** Grafica el error relativo de $(1 - \cos x)/x^2$ y su version estable para $x = 10^{-k}$, $k = 1, \dots, 8$. Compara con el valor limite 0.5.

- C3. Derivada numerica:** Para $f(x) = e^x$ en $x = 1$, aproxima $f'(1)$ con diferencia centrada. Grafica el error vs h y encuentra el h optimo.
- C4. Serie armonica estancada:** Suma $S_N = \sum_{k=1}^N 1/k$ en float32 y float64. Encuentra el N a partir del cual el valor computado deja de crecer.
- C5. Taylor inestable:** Calcula e^{-15} con la serie de Taylor directa y con $1/e^{15}$. Grafica el error relativo en funcion del numero de terminos.
- C6. Condicion de Hilbert:** Calcula $\kappa_2(H_n)$ para $n = 2, \dots, 15$ y grafica. ¿Cuántos digitos de precision se pierden para $n = 12$?
- C7. Perturbacion en sistema:** Para $A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + 10^{-10} \end{bmatrix}$ y $b = [2, 2 + 10^{-10}]^T$, resuelve el sistema. Luego perturba b en 10^{-12} y mide la amplificacion del error relativo. Compara con $\kappa_2(A)$.
- C8. Gauss con y sin pivoteo:** Implementa eliminacion gaussiana sin pivoteo y con pivoteo parcial. Resuelve $[\epsilon, 1], [1, 1]x = [1, 2]$ para $\epsilon = 10^{-12}$. Muestra que sin pivoteo el resultado es pesimo.
- C9. Convergencia de Jacobi y Gauss-Seidel:** Para la matriz $A = \begin{bmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix}$, grafica la convergencia de ambos metodos y calcula sus radios espectrales.
- C10. Recurrencia inestable:** Implementa la recurrencia $E_n = 1 - nE_{n-1}$ hacia adelante y hacia atras para $E_n = \int_0^1 x^n e^{x-1} dx$. Compara los resultados para $n = 20$.

Soluciones detalladas

Soluciones Teoricas

- T1.** Ver seccion . $\epsilon_{\text{mach}} = \beta^{1-t}$. El error de redondeo al mas cercano es a lo sumo la mitad del espaciado entre numeros de maquina. Dividiendo por $|x| \geq \beta^e$ se obtiene $u = \frac{1}{2}\epsilon_{\text{mach}}$.
- T2.** Si a y b son casi iguales, $a - b$ es chico pero los errores absolutos de a y b son del orden de $u|a|$. El error relativo de la resta es $\frac{u|a|+|b|}{|a-b|}$, que puede ser enorme.
- T3.** Si $\tilde{a} = a(1 + \delta_a)$, $\tilde{b} = b(1 + \delta_b)$, entonces $\tilde{a}\tilde{b} = ab(1 + \delta_a + \delta_b + \delta_a\delta_b)$. Despreciando $\delta_a\delta_b$, el error relativo es $\delta_a + \delta_b$.
- T4.** $\|x\|_\infty = \max |x_i|$. Luego $\|x\|_2^2 = \sum x_i^2 \geq \max x_i^2 = \|x\|_\infty^2$ y $\sum x_i^2 \leq n\|x\|_\infty^2$. Tomando raiz se obtiene.
- T5.** $\kappa(A) = \|A\|\|A^{-1}\| = \|A^{-1}\|\|A\| = \kappa(A^{-1})$.
- T6.** $A\Delta x = \Delta b \Rightarrow \Delta x = A^{-1}\Delta b$. Tomando normas: $\|\Delta x\| \leq \|A^{-1}\|\|\Delta b\|$. Ademas $\|b\| = \|Ax\| \leq \|A\|\|x\| \Rightarrow \|x\| \geq \|b\|/\|A\|$. Dividiendo: $\frac{\|\Delta x\|}{\|x\|} \leq \|A^{-1}\|\|A\| \frac{\|\Delta b\|}{\|b\|} = \kappa(A) \frac{\|\Delta b\|}{\|b\|}$.
- T7.** La formula $1/\kappa(A) = \inf_{B \text{ sing}} \|A - B\|/\|A\|$ muestra que si $\kappa(A)$ es grande, existe una matriz singular muy cerca. El determinante chico no implica mal condicionamiento (ej: $10^{-3}I$).
- T8.** Problema mal condicionado: raices de polinomio de Wilkinson. Algoritmo inestable: recurrencia E_n hacia adelante. En el primero, el error es intrinseco al problema; en el segundo, depende del metodo.
- T9.** Si $Av = \lambda v$, entonces $|\lambda|\|v\| = \|Av\| \leq \|A\|\|v\|$, luego $|\lambda| \leq \|A\|$. Tomando maximo sobre todos los autovalores, $\rho(A) \leq \|A\|$.
- T10.** Si A es estrictamente diagonal dominante, $\|B_J\|_\infty = \max_i \sum_{j \neq i} |a_{ij}|/|a_{ii}| < 1$. Como $\rho(B_J) \leq \|B_J\|_\infty$, el radio espectral es menor que 1 y el metodo converge.

Soluciones Computacionales

(Se incluyen los codigos ya presentados en las secciones anteriores, por brevedad no se repiten aqui).

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido. Viste que no era tan terrible. Ahora entiendes que las computadoras no son magicas, que los numeros se redondean, que los errores se propagan, y que algunos problemas son mas sensibles que otros. La clave es siempre pensar en el error relativo, evitar restas de numeros casi iguales, y si podes, usar metodos estables como el pivoteo o Cholesky. Y recuerda: un problema mal condicionado no se arregla con un mejor algoritmo; a veces simplemente no podes hacer mucho.

Tambien vimos como el metodo de Newton se extiende a sistemas no lineales, como la continuacion ayuda a encontrar soluciones cuando el punto inicial esta lejos, y como la estabilidad de los metodos para EDOs es fundamental para problemas rigidos.

Ahora, a practicar con los ejercicios. Y si algo no te sale, no te frustres: a todos nos paso. La proxima vez que veas un error numerico en tu codigo, en lugar de putear a la maquina, piensa: ¿sera un problema de redondeo? ¿mi algoritmo es estable? ¿la matriz esta mal condicionada? Con estas herramientas, vas a poder diagnosticar y solucionar la mayoria de los problemas. ¡Dale para adelante, que el calculo numerico es un viaje fascinante!

Sistemas Lineales: Metodos Directos, Estabilidad y Mas Alla

Como resolver $Ax=b$ sin que el codigo explote en el intento.

Che, ¿como andas? Bienvenido al capitulo donde finalmente aprendes a resolver sistemas de ecuaciones lineales de una vez por todas, pero no como en algebra lineal con lapiz y papel, sino como un verdadero cientifico computacional: con algoritmos, codigo, y un ojo puesto en la estabilidad numerica.

Yo recuerdo cuando curse Algebra Lineal y me enseñaron a resolver sistemas con Gauss-Jordan. Todo muy lindo, hasta que en Calculo Numerico tuve que implementarlo en Python y me di cuenta de que la computadora no hace las cosas tan perfectas como uno espera. Mi primer intento de eliminacion gaussiana funcionaba joya para matrices chicas, pero cuando probe una matriz de 10×10 con numeros mal condicionados, el resultado era cualquier cosa, me queria morir, HORRIBLE!. Ahi entendi que no alcanza con conocer el algoritmo: hay que saber como implementarlo de forma estable y eficiente.

En este capitulo vamos a desmenuzar los metodos directos para resolver sistemas lineales: la eliminacion gaussiana, la factorizacion LU, y el pivoteo. Vamos a ver por que a veces el algoritmo mas obvio es el menos estable, y como el pivoteo puede salvar tu resultado. Tambien vamos a hablar de la factorizacion de Cholesky para matrices simetricas definidas positivas, y de paso vamos a comparar con los metodos iterativos para que sepas cuando usar cada uno. Y como siempre, con mucho codigo Python para que puedas probar todo vos mismo. ¡Dale, metele!

El Problema: Resolver $Ax = b$

El problema es mas viejo que el polvo: tenes una matriz A de $n \times n$ y un vector b de n componentes, y queres encontrar el vector x que satisface:

$$Ax = b.$$

Esto aparece en todos lados: en fisica (resolver ecuaciones de Maxwell, porque asi somos los fisicos, nos complicamos la vida jajaja), en ingenieria (analisis de estructuras), en machine learning (regresion lineal, un tema hermoso de hecho estoy haciendo mi IA propia y demas), en finanzas (optimizacion de portafolios). Es el pan de cada dia del calculo numerico.

Traduccion Matematica de $Ax = b$: La ecuacion $Ax = b$ se lee: "la matriz A actuando sobre el vector x produce el vector b ". En criollo, tenes una caja negra" (la matriz) que transforma vectores, y vos sabes cual es el vector de salida (b) y queres encontrar el vector de entrada (x). Es como si tuvieras una maquina que mezcla ingredientes, y vos sabes la mezcla final pero no las cantidades de cada ingrediente. A te dice como se combinan, y b es el resultado.

Si A es inversible, existe una unica solucion $x = A^{-1}b$. Pero calcular la inversa explicitamente es caro y numericamente inestable para matrices grandes. En lugar de eso, usamos metodos que resuelven el sistema directamente sin calcular A^{-1} .

Metodos Directos vs Iterativos

Hay dos grandes familias de metodos para resolver sistemas lineales:

- **Metodos directos:** transforman el sistema en uno equivalente que es facil de resolver (triangular) mediante operaciones elementales. Te dan la solucion en un numero finito de pasos (asumiendo aritmetica exacta). Ejemplos: eliminacion gaussiana, factorizacion LU, Cholesky.
- **Metodos iterativos:** construyen una sucesion $x^{(k)}$ que converge a la solucion. Son utiles cuando A es grande y rala (tiene muchos ceros). Ejemplos: Jacobi, Gauss-Seidel, SOR.

En este capitulo nos enfocamos en los metodos directos. Son la base de todo, y entenderlos bien te va a servir para entender los iterativos despues.

Eliminacion Gaussiana: El Metodo Clasico

La eliminacion gaussiana es el metodo directo por excelencia. La idea es recontra simple: transformar el sistema $Ax = b$ en un sistema triangular superior $Ux = y$ usando operaciones elementales de fila, que no cambian la solucion. Despues resuelves con sustitucion hacia atras.

Eliminacion hacia adelante (triangulacion)

El primer paso es eliminar x_1 de todas las ecuaciones excepto la primera. Para eso, para cada fila $i > 1$, calculas el multiplicador:

$$m_{i1} = \frac{a_{i1}}{a_{11}},$$

y restas m_{i1} veces la fila 1 de la fila i . Esto hace que el elemento a_{i1} se vuelva cero.

Luego repetis el proceso para la columna 2, usando el elemento $a_{22}^{(2)}$ como pivote, y asi sucesivamente hasta llegar a una matriz triangular superior.

Ejemplo: Resolver:

$$\begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix}.$$

Paso 1: Eliminar x_1 de las filas 2 y 3. - Multiplicador para fila 2: $m_{21} = 4/2 = 2$. Restar $2 \times$ fila 1 de fila 2. - Multiplicador para fila 3: $m_{31} = -2/2 = -1$. Restar $(-1) \times$ fila 1 de fila 3.

Resultado:

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 8 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -12 \\ 14 \end{pmatrix}.$$

Paso 2: Eliminar x_2 de la fila 3. - Multiplicador: $m_{32} = 8/(-8) = -1$. Restar $(-1) \times$ fila 2 de fila 3.

Resultado (matriz triangular superior):

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -12 \\ 2 \end{pmatrix}.$$

Sustitucion hacia atras

Una vez que tenes el sistema triangular superior, resolves de abajo hacia arriba:

$$\begin{aligned}x_3 &= 2, \\x_2 &= \frac{-12 + 2x_3}{-8} = \frac{-12 + 4}{-8} = 1, \\x_1 &= \frac{5 - x_2 - x_3}{2} = \frac{5 - 1 - 2}{2} = 1.\end{aligned}$$

Y listo, tenes la solucion $x = (1, 1, 2)^T$.

Implementacion en Python

Aca te dejo una implementacion basica de eliminacion gaussiana sin pivoteo. Fijate que es super sencilla, pero tiene un problema: si el pivote es cero, el algoritmo se rompe.

```

1 import numpy as np
2
3 def gauss_sin_pivoteo(A, b):
4     """
5     Resuelve Ax = b por eliminacion gaussiana sin pivoteo.
6     A: matriz n x n (sera modificada)
7     b: vector (sera modificado)
8     """
9     n = len(A)
10    x = np.zeros(n)
11
12    # Eliminacion hacia adelante
13    for k in range(n-1):
14        # Si el pivote es cero, el algoritmo falla
15        if abs(A[k, k]) < 1e-15:
16            raise ValueError(f'Pivote nulo en la columna {k}')
17        for i in range(k+1, n):
18            m = A[i, k] / A[k, k]
19            A[i, k:] -= m * A[k, k:]
20            b[i] -= m * b[k]
21
22    # Sustitucion hacia atras
23    for i in range(n-1, -1, -1):
24        x[i] = (b[i] - np.dot(A[i, i+1:], x[i+1:])) / A[i, i]
25
26    return x
27
28 # Ejemplo
29 A = np.array([[2, 1, 1], [4, -6, 0], [-2, 7, 2]], dtype=float)
30 b = np.array([5, -2, 9], dtype=float)
31 x = gauss_sin_pivoteo(A.copy(), b.copy())
32 print("Solucion:", x)
33 print("Verificacion:", np.linalg.norm(A @ x - b))

```

Listing 32: Eliminacion Gaussiana sin pivoteo

Traduccion Matematica del algoritmo

El algoritmo de eliminacion gaussiana se puede ver como una sucesion de multiplicaciones por matrices elementales. En el paso k , definimos la matriz M_k que tiene unos en la diagonal y $-m_{ik}$

en la columna k debajo de la diagonal. Entonces:

$$M_{n-1} \cdots M_2 M_1 A = U,$$

donde U es triangular superior. Como cada M_k es invertible, podemos escribir:

$$A = (M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1}) U = LU.$$

La matriz $L = M_1^{-1} M_2^{-1} \cdots M_{n-1}^{-1}$ es triangular inferior con unos en la diagonal, y sus elementos son exactamente los multiplicadores m_{ik} . Esto es la factorizacion LU.

Traduccion Matematica de la factorizacion LU: La factorizacion $A = LU$ se lee: "la matriz A se descompone en el producto de una matriz triangular inferior L y una matriz triangular superior U ." L tiene unos en la diagonal y los multiplicadores de la eliminacion debajo, y U es la matriz triangular superior que resulta despues de la eliminacion. Es como si desarmaras el proceso de eliminacion en dos matrices: una que guarda los pasos (L) y otra que guarda el resultado (U).

Ventajas de LU

La factorizacion LU es mucho mas util que hacer eliminacion gaussiana cada vez que quieres resolver un sistema, porque una vez que tenes L y U , puedes resolver sistemas con diferentes b rapidamente: 1. Resolver $Ly = b$ (sustitucion hacia adelante). 2. Resolver $Ux = y$ (sustitucion hacia atras).

El costo de la factorizacion es $O(n^3)$, pero una vez que la tenes, resolver un sistema adicional cuesta solo $O(n^2)$. Esto es clave cuando tenes que resolver el mismo sistema con muchos b distintos.

```

1 import numpy as np
2 from scipy.linalg import lu
3
4 A = np.array([[2, 1, 1], [4, -6, 0], [-2, 7, 2]], dtype=float)
5 b = np.array([5, -2, 9], dtype=float)
6
7 # Factorizacion LU con pivoteo
8 P, L, U = lu(A)
9
10 print("P:")
11 print(P)
12 print("L:")
13 print(L)
14 print("U:")
15 print(U)
16
17 # Resolver Ly = Pb
18 y = np.linalg.solve(L, P @ b)
19 # Resolver Ux = y
20 x = np.linalg.solve(U, y)
21
22 print("Solucion:", x)
23 print("Residuo:", np.linalg.norm(A @ x - b))

```

Listing 33: Factorizacion LU con numpy

El Problema del Pivoteo: Cuando el algoritmo se rompe, aca le rezamos a Dios, Buda o al que sea jajaja

La eliminacion gaussiana sin pivoteo tiene un problema grave: si en algun paso el pivote es cero, el algoritmo falla. Pero incluso si no es exactamente cero, si es muy chico, los errores de redondeo se amplifican y el resultado puede ser un desastre.

El ejemplo clasico: la matriz con ϵ

Considera el sistema:

$$\begin{pmatrix} \epsilon & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

con $\epsilon = 10^{-12}$. Si resuelves sin pivoteo, el multiplicador es $m = 1/\epsilon = 10^{12}$, y al restar las filas, el elemento a_{22} se vuelve $1 - 10^{12} \approx -10^{12}$, y el vector b_2 se vuelve $2 - 10^{12} \approx -10^{12}$. Hasta aca todo bien, pero cuando haces la sustitucion hacia atras, obtenes $y = 1$ y $x = 0$. La solucion correcta es $x \approx 1$, $y \approx 1$. ¡El error relativo en x es del 100 %!

Por que pasa esto: Al multiplicar por $1/\epsilon$, el error de redondeo en la resta se amplifica. Despues, al despejar x de la primera ecuacion, tenes $x = (1 - y)/\epsilon$. Si y tiene un error del orden de 10^{-16} , x tiene un error del orden de $10^{-16}/\epsilon = 10^{-4}$, que es enorme comparado con $x \approx 1$.

Pivoteo parcial: la solucion

La solucion es el pivoteo parcial: antes de eliminar en la columna k , buscas el elemento de mayor magnitud en esa columna desde la fila k hacia abajo, y si no esta en la fila k , intercambias filas. Esto asegura que el pivote sea lo mas grande posible, minimizando la amplificacion del error. En el ejemplo anterior, si intercambias las filas, el sistema queda:

$$\begin{pmatrix} 1 & 1 \\ \epsilon & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

y el pivote es 1, que es grande. El resultado es correcto.

Implementacion con pivoteo

```
1 import numpy as np
2
3 def gauss_con_pivoteo(A, b):
4     """
5     Resuelve Ax = b por eliminacion gaussiana con pivoteo parcial.
6     A: matriz n x n (sera modificada)
7     b: vector (sera modificado)
8     """
9     n = len(A)
10    x = np.zeros(n)
11
12    for k in range(n-1):
13        # Pivoteo: buscar el elemento de mayor magnitud en la columna k
14        max_idx = max(range(k, n), key=lambda i: abs(A[i, k]))
15        if max_idx != k:
16            # Intercambiar filas
17            A[[k, max_idx]] = A[[max_idx, k]]
18            b[k], b[max_idx] = b[max_idx], b[k]
```

```

19
20     # Si el pivote es cero, la matriz es singular
21     if abs(A[k, k]) < 1e-15:
22         raise ValueError(f'Matriz singular en la columna {k}')
23
24     for i in range(k+1, n):
25         m = A[i, k] / A[k, k]
26         A[i, k:] -= m * A[k, k:]
27         b[i] -= m * b[k]
28
29     # Sustitucion hacia atras
30     for i in range(n-1, -1, -1):
31         x[i] = (b[i] - np.dot(A[i, i+1:], x[i+1:])) / A[i, i]
32
33     return x
34
35 # Ejemplo con epsilon
36 eps = 1e-12
37 A = np.array([[eps, 1], [1, 1]], dtype=float)
38 b = np.array([1, 2], dtype=float)
39
40 x_sin = gauss_sin_pivoteo(A.copy(), b.copy())
41 x_con = gauss_con_pivoteo(A.copy(), b.copy())
42
43 print("Sin pivoteo:", x_sin)
44 print("Con pivoteo:", x_con)
45 print("Exacto:", np.linalg.solve(A, b))

```

Listing 34: Eliminacion Gaussiana con pivoteo parcial

Factorizacion PLU

El pivoteo se puede incorporar en la factorizacion LU como una matriz de permutacion P . El teorema dice: toda matriz invertible A se puede factorizar como:

$$PA = LU,$$

donde P es una matriz de permutacion (que registra los intercambios de filas), L es triangular inferior con unos en la diagonal, y U es triangular superior.

Resolver $Ax = b$ con $PA = LU$: 1. $y = Pb$ 2. Resolver $Lz = y$ 3. Resolver $Ux = z$

Analisis de Costo y Estabilidad

Costo computacional

La factorizacion LU (con o sin pivoteo) cuesta aproximadamente $\frac{2}{3}n^3$ operaciones de punto flotante (multiplicaciones y sumas). Para ser mas precisos: - En el paso k , necesitas $(n - k)$ divisiones para calcular los multiplicadores, y $(n - k)^2$ multiplicaciones-sumas para actualizar la submatriz. - Sumando para $k = 1$ a $n - 1$: $\sum_{k=1}^{n-1} (n - k)^2 \approx \frac{n^3}{3}$ para las multiplicaciones-sumas, y $\sum_{k=1}^{n-1} (n - k) \approx \frac{n^2}{2}$ para las divisiones. - En total, aproximadamente $\frac{2}{3}n^3$ operaciones. La sustitucion hacia adelante y hacia atras cuestan $O(n^2)$ cada una.

Estabilidad numerica

La eliminacion gaussiana con pivoteo parcial es **estable** en la practica: el factor de crecimiento de los elementos de la matriz durante la eliminacion esta acotado por 2^{n-1} , aunque en la practica

raramente crece tanto. Sin pivoteo, el factor de crecimiento puede ser exponencial, haciendo que el algoritmo sea inestable.

```

1 import numpy as np
2 import time
3 import matplotlib.pyplot as plt
4
5 def comparar_metodos(n):
6     """Compara LU con pivoteo vs sin pivoteo para matrices aleatorias."""
7     A = np.random.rand(n, n) * 100
8     b = np.random.rand(n) * 100
9
10    # Sin pivoteo (puede fallar)
11    try:
12        t0 = time.time()
13        x_sin = gauss_sin_pivoteo(A.copy(), b.copy())
14        t_sin = time.time() - t0
15        err_sin = np.linalg.norm(A @ x_sin - b)
16    except:
17        t_sin = np.nan
18        err_sin = np.nan
19
20    # Con pivoteo
21    t0 = time.time()
22    x_con = gauss_con_pivoteo(A.copy(), b.copy())
23    t_con = time.time() - t0
24    err_con = np.linalg.norm(A @ x_con - b)
25
26    return t_sin, t_con, err_sin, err_con
27
28 ns = [10, 20, 50, 100, 200]
29 tiempos_sin = []
30 tiempos_con = []
31 errores_sin = []
32 errores_con = []
33
34 for n in ns:
35     t_sin, t_con, e_sin, e_con = comparar_metodos(n)
36     tiempos_sin.append(t_sin)
37     tiempos_con.append(t_con)
38     errores_sin.append(e_sin)
39     errores_con.append(e_con)
40
41 plt.figure(figsize=(12, 5))
42
43 plt.subplot(1, 2, 1)
44 plt.loglog(ns, tiempos_sin, 'o-', label='Sin pivoteo')
45 plt.loglog(ns, tiempos_con, 's-', label='Con pivoteo')
46 plt.loglog(ns, [n**3 / 1e9 for n in ns], '--', label='O(n^3)')
47 plt.xlabel('n')
48 plt.ylabel('Tiempo (s)')
49 plt.legend()
50 plt.grid(True)
51 plt.title('Costo computacional')
52
53 plt.subplot(1, 2, 2)
54 plt.semilogy(ns, errores_sin, 'o-', label='Sin pivoteo')
55 plt.semilogy(ns, errores_con, 's-', label='Con pivoteo')
56 plt.xlabel('n')
57 plt.ylabel('Error absoluto')
58 plt.legend()

```

```

59 plt.grid(True)
60 plt.title('Errores')
61
62 plt.tight_layout()
63 plt.show()

```

Listing 35: Comparacion de tiempos y errores

Factorizacion de Cholesky

Si A es simetrica y definida positiva, puedes usar la factorizacion de Cholesky:

$$A = LL^T,$$

donde L es triangular inferior con diagonal positiva. Esto cuesta la mitad de las operaciones que LU (aproximadamente $\frac{1}{3}n^3$) y es estable sin pivoteo.

Condiciones para Cholesky

Una matriz A es definida positiva si para todo $x \neq 0$, $x^T Ax > 0$. Esto implica que: - Todos los elementos diagonales son positivos. - Todos los menores principales son positivos. - Es simetrica.

Algoritmo de Cholesky

El algoritmo es recursivo:

$$l_{kk} = \sqrt{a_{kk} - \sum_{j=1}^{k-1} l_{kj}^2},$$

$$l_{ik} = \frac{a_{ik} - \sum_{j=1}^{k-1} l_{ij}l_{kj}}{l_{kk}}, \quad i > k.$$

```

1 import numpy as np
2
3 def cholesky(A):
4     """
5     Factorizacion de Cholesky A = L L^T.
6     A: matriz simetrica definida positiva (sera modificada)
7     """
8     n = len(A)
9     L = np.zeros((n, n))
10
11     for k in range(n):
12         # Calcular l_kk
13         s = 0.0
14         for j in range(k):
15             s += L[k, j]**2
16         L[k, k] = np.sqrt(A[k, k] - s)
17
18         # Calcular l_ik para i > k
19         for i in range(k+1, n):
20             s = 0.0
21             for j in range(k):
22                 s += L[i, j] * L[k, j]
23             L[i, k] = (A[i, k] - s) / L[k, k]
24

```

```

25     return L
26
27 # Ejemplo
28 A = np.array([[4, 2, -2], [2, 5, 5], [-2, 5, 11]], dtype=float)
29 L = cholesky(A)
30 print("L:")
31 print(L)
32 print("L L^T:")
33 print(L @ L.T)
34 print("A:")
35 print(A)

```

Listing 36: Factorizacion de Cholesky

Ventajas de Cholesky

1. Mas estable numericamente que LU (sin pivoteo). 2. Mas eficiente: $\frac{1}{3}n^3$ vs $\frac{2}{3}n^3$ de LU. 3. Asegura que la matriz es definida positiva (si el algoritmo falla, la matriz no lo es).

Matrices Especiales y su Resolucion Eficiente

No todas las matrices son densas. En muchas aplicaciones, las matrices tienen estructura: son tridiagonales, de banda, o ralas. Aprovechar esa estructura puede reducir el costo de resolucion de $O(n^3)$ a $O(n)$.

Matrices Tridiagonales

Una matriz tridiagonal tiene la forma:

$$A = \begin{pmatrix} d_1 & a_1 & 0 & \cdots & 0 \\ b_2 & d_2 & a_2 & \cdots & 0 \\ 0 & b_3 & d_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & d_n \end{pmatrix}.$$

La eliminacion gaussiana para matrices tridiagonales es mucho mas barata porque los elementos que se llenan son solo los de la subdiagonal y la superdiagonal. El algoritmo de Thomas (o TDMA) resuelve el sistema en $O(n)$ operaciones.

```

1 import numpy as np
2
3 def thomas(a, b, c, d):
4     """
5     Resuelve un sistema tridiagonal Ax = d.
6     a: subdiagonal (longitud n-1)
7     b: diagonal (longitud n)
8     c: superdiagonal (longitud n-1)
9     d: lado derecho
10    """
11    n = len(b)
12    # Forward sweep
13    c_prime = np.zeros(n-1)
14    d_prime = np.zeros(n)
15    c_prime[0] = c[0] / b[0]
16    d_prime[0] = d[0] / b[0]

```



```

17     for i in range(1, n-1):
18         denom = b[i] - a[i-1] * c_prime[i-1]
19         c_prime[i] = c[i] / denom
20         d_prime[i] = (d[i] - a[i-1] * d_prime[i-1]) / denom
21     d_prime[n-1] = (d[n-1] - a[n-2] * d_prime[n-2]) / (b[n-1] - a[n-2] * c_prime[n-2])
22
23     # Back substitution
24     x = np.zeros(n)
25     x[n-1] = d_prime[n-1]
26     for i in range(n-2, -1, -1):
27         x[i] = d_prime[i] - c_prime[i] * x[i+1]
28     return x
29
30 # Ejemplo
31 a = np.array([1, 1, 1]) # subdiagonal
32 b = np.array([4, 4, 4, 4]) # diagonal
33 c = np.array([1, 1, 1]) # superdiagonal
34 d = np.array([3, 2, 3, 2]) # lado derecho
35
36 A = np.diag(b) + np.diag(a, -1) + np.diag(c, 1)
37 x = thomas(a, b, c, d)
38 print("Solucion:", x)
39 print("Verificacion:", np.linalg.norm(A @ x - d))

```

Listing 37: Algoritmo de Thomas para matrices tridiagonales

Matrices de Banda

Generalizando, una matriz de banda tiene elementos no nulos solo en un rango alrededor de la diagonal. Si el ancho de banda es p , el costo de la eliminacion gaussiana es $O(np^2)$ en lugar de $O(n^3)$.

Matrices Ralas

Para matrices ralas (con muchos ceros), se usan metodos directos especiales que explotan la estructura de la matriz, como la factorizacion de Cholesky para matrices ralas, o metodos iterativos como el gradiente conjugado.

Metodos Iterativos: Una Mirada Mas Profunda

Aunque este capitulo se enfoca en metodos directos, los metodos iterativos son esenciales para matrices grandes y ralas. Vamos a profundizar un poco mas.

Jacobi y Gauss-Seidel

La idea es descomponer $A = D + L + U$, donde D es la diagonal, L la triangular inferior estricta, y U la triangular superior estricta.

- **Jacobi:**

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

- Gauss-Seidel:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)} \right).$$

Gauss-Seidel usa los valores ya actualizados, por lo que suele converger mas rapido.

Convergencia

Ambos metodos convergen si $\rho(G) < 1$, donde $\rho(G)$ es el radio espectral de la matriz de iteracion.

Para Jacobi, $G_J = -D^{-1}(L + U)$. Para Gauss-Seidel, $G_{GS} = -(D + L)^{-1}U$.

Si A es estrictamente diagonal dominante ($-a_{ii} > \sum_{j \neq i} |a_{ij}|$), ambos metodos convergen.

El metodo SOR

El metodo de Successive Over-Relaxation (SOR) es una generalizacion de Gauss-Seidel con un parametro de relajacion ω :

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)} \right).$$

Si $0 < \omega < 2$, el metodo converge para matrices simetricas definidas positivas. El ω optimo puede acelerar mucho la convergencia.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def jacobi(A, b, x0, maxit=100):
5     n = len(b)
6     x = x0.copy()
7     errores = []
8     for _ in range(maxit):
9         x_new = np.zeros(n)
10        for i in range(n):
11            suma = b[i] - np.dot(A[i, :], x) + A[i, i]*x[i]
12            x_new[i] = suma / A[i, i]
13        err = np.linalg.norm(x_new - x, np.inf)
14        errores.append(err)
15        x = x_new
16    return errores
17
18 def gauss_seidel(A, b, x0, maxit=100):
19     n = len(b)
20     x = x0.copy()
21     errores = []
22     for _ in range(maxit):
23         x_old = x.copy()
24         for i in range(n):
25             suma = b[i] - np.dot(A[i, :], x) + A[i, i]*x[i]
26             x[i] = suma / A[i, i]
27         err = np.linalg.norm(x - x_old, np.inf)
28         errores.append(err)
29    return errores
30
31 def sor(A, b, x0, omega, maxit=100):
32     n = len(b)
33     x = x0.copy()
34     errores = []

```

```

35     for _ in range(maxit):
36         x_old = x.copy()
37         for i in range(n):
38             suma = b[i] - np.dot(A[i, :], x) + A[i, i]*x[i]
39             x[i] = (1-omega)*x[i] + omega*suma / A[i, i]
40         err = np.linalg.norm(x - x_old, np.inf)
41         errores.append(err)
42     return errores
43
44 # Matriz diagonal dominante
45 A = np.array([[4, -1, 0], [-1, 4, -1], [0, -1, 4]], dtype=float)
46 b = np.array([3, 2, 3], dtype=float)
47 x0 = np.zeros(3)
48
49 err_j = jacobi(A, b, x0, maxit=50)
50 err_gs = gauss_seidel(A, b, x0, maxit=50)
51 err_sor = sor(A, b, x0, omega=1.2, maxit=50)
52
53 plt.semilogy(err_j, label='Jacobi')
54 plt.semilogy(err_gs, label='Gauss-Seidel')
55 plt.semilogy(err_sor, label='SOR (omega=1.2)')
56 plt.xlabel('Iteracion')
57 plt.ylabel('Error infinito')
58 plt.legend()
59 plt.grid(True)
60 plt.title('Convergencia de metodos iterativos')
61 plt.show()

```

Listing 38: Comparacion de metodos iterativos

Matrices Mal Condicionadas y el Efecto en la Solucion

Una matriz mal condicionada tiene un numero de condicion $\kappa(A)$ muy grande. Esto significa que pequenas perturbaciones en b o en A pueden causar grandes cambios en x . La eliminacion gaussiana (incluso con pivoteo) puede dar resultados poco confiables para matrices mal condicionadas.

El ejemplo de la matriz de Hilbert

La matriz de Hilbert $H_{ij} = 1/(i+j-1)$ es el ejemplo clasico de matriz mal condicionada. Para $n = 10$, $\kappa_2(H_{10}) \approx 10^{13}$. Esto significa que si tu matriz es H_{10} , cualquier error en los datos se amplifica 10^{13} veces. En doble precision (16 digitos), perdes 13 digitos de precision, dejandote con solo 3 digitos correctos.

Regularizacion de Tikhonov

Cuando te enfrentas a un problema mal condicionado, puedes regularizarlo agregando un termino de penalizacion:

$$\min_x (\|Ax - b\|_2^2 + \lambda^2 \|x\|_2^2).$$

Esto estabiliza la solucion a costa de introducir un sesgo.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Matriz de Hilbert mal condicionada

```

```

5 n = 10
6 H = np.array([[1/(i+j-1) for j in range(1,n+1)] for i in range(1,n+1)])
7 x_true = np.ones(n)
8 b = H @ x_true
9 b_noise = b + 0.01 * np.random.randn(n)
10
11 # Sin regularizacion
12 x_ls = np.linalg.lstsq(H, b_noise, rcond=None)[0]
13
14 # Con regularizacion (Tikhonov)
15 lam = 0.1
16 I = np.eye(n)
17 x_tikh = np.linalg.solve(H.T @ H + lam**2 * I, H.T @ b_noise)
18
19 print("Error LS:", np.linalg.norm(x_ls - x_true))
20 print("Error Tikhonov:", np.linalg.norm(x_tikh - x_true))
21
22 plt.bar(range(n), x_true, alpha=0.5, label='Exacto')
23 plt.bar(range(n), x_ls, alpha=0.5, label='LS')
24 plt.bar(range(n), x_tikh, alpha=0.5, label='Tikhonov')
25 plt.legend()
26 plt.title('Comparacion de soluciones')
27 plt.show()

```

Listing 39: Regularizacion de Tikhonov

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Eliminacion Gaussiana:** Demuestra que la eliminacion gaussiana sin pivoteo falla si el primer pivote es cero. Da un ejemplo.
- T2. Factorizacion LU:** Demuestra que si $A = LU$ con L triangular inferior con unos en la diagonal y U triangular superior, entonces el determinante de A es el producto de los elementos diagonales de U .
- T3. Costo de LU:** Demuestra que el costo de la factorizacion LU es aproximadamente $\frac{2}{3}n^3$ operaciones.
- T4. Pivoteo:** Explica por que el pivoteo parcial mejora la estabilidad numerica de la eliminacion gaussiana.
- T5. Cholesky:** Demuestra que si A es simetrica definida positiva, entonces todos los elementos diagonales son positivos.
- T6. Convergencia de Jacobi:** Demuestra que si A es estrictamente diagonal dominante, el metodo de Jacobi converge.
- T7. Radio espectral:** Sea G una matriz de iteracion. Demuestra que $G^k \rightarrow 0$ si y solo si $\rho(G) < 1$.
- T8. Matriz de Hilbert:** Considera la matriz de Hilbert $H_{ij} = 1/(i + j - 1)$. ¿Por que esta matriz es mal condicionada? ¿Que implica esto para la eliminacion gaussiana?
- T9. PLU:** Demuestra que toda matriz invertible admite una factorizacion $PA = LU$.

T10. Estabilidad: Explica la diferencia entre un problema mal condicionado y un algoritmo inestable. Da un ejemplo de cada uno.

Parte Computacional (10 ejercicios con Python)

- C1. Eliminacion Gaussiana sin pivoteo:** Implementa eliminacion gaussiana sin pivoteo y pruebala con la matriz $\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$. ¿Que pasa?
- C2. Eliminacion con pivoteo:** Implementa eliminacion gaussiana con pivoteo parcial. Pruebala con la matriz del ejercicio anterior.
- C3. Factorizacion LU:** Implementa la factorizacion LU sin pivoteo. Usala para resolver el sistema del ejemplo.
- C4. PLU:** Implementa la factorizacion PLU con pivoteo. Usala para resolver el sistema con $\epsilon = 10^{-12}$.
- C5. Costo de LU:** Mide el tiempo de la factorizacion LU para $n = 10, 20, 50, 100, 200$. Grafica el tiempo vs n^3 . ¿Es lineal en n^3 ?
- C6. Cholesky:** Implementa Cholesky y usala para factorizar una matriz simetrica definida positiva. Verifica que $A = LL^T$.
- C7. Comparacion LU vs Cholesky:** Compara el tiempo de LU y Cholesky para matrices simetricas definidas positivas de diferentes tamaños. Grafica los tiempos.
- C8. Jacobi:** Implementa el metodo de Jacobi y usalo para resolver un sistema estrictamente diagonal dominante. Grafica la convergencia del error.
- C9. Gauss-Seidel:** Implementa Gauss-Seidel y comparalo con Jacobi en el mismo sistema. ¿Cual converge mas rapido?
- C10. Estabilidad:** Resuelve el sistema $\begin{bmatrix} 1 & 1 \\ 1 & 1 + 10^{-10} \end{bmatrix} x = \begin{bmatrix} 2 \\ 2 + 10^{-10} \end{bmatrix}$ con LU sin pivoteo y con pivoteo. Compara los resultados.

Soluciones

Soluciones Teoricas

T1. Si el primer pivote es cero, el multiplicador $m_{i1} = a_{i1}/a_{11}$ no esta definido. Ejemplo: $A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$. Sin pivoteo, el algoritmo falla. Con pivoteo, intercambia filas y el primer pivote es 1.

T2. $\det(A) = \det(LU) = \det(L)\det(U) = 1 \cdot \prod_{i=1}^n u_{ii}$. Como L tiene unos en la diagonal, su determinante es 1.

T3. En el paso k , se realizan $(n-k)$ divisiones y $(n-k)^2$ multiplicaciones-sumas. Sumando: $\sum_{k=1}^{n-1} (n-k)^2 = \frac{n(n-1)(2n-1)}{6} \approx \frac{n^3}{3}$ para las multiplicaciones-sumas, y $\sum_{k=1}^{n-1} (n-k) = \frac{n(n-1)}{2} \approx \frac{n^2}{2}$ para las divisiones. En total, $\frac{2}{3}n^3$.

T4. El pivoteo parcial asegura que el pivote sea lo mas grande posible en la columna, minimizando los multiplicadores $|m_{ik}| \leq 1$. Esto evita la amplificacion de errores de redondeo.

T5. Si A es definida positiva, para e_i (vector canonico), $e_i^T A e_i = a_{ii} > 0$. Por lo tanto, todos los elementos diagonales son positivos.

T6. Si A es estrictamente diagonal dominante, $\|G_J\|_\infty = \max_i \sum_{j \neq i} |a_{ij}|/|a_{ii}| < 1$. Como $\rho(G) \leq \|G\|$, Jacobi converge.

T7. ($\Rightarrow$) Si $G^k \rightarrow 0$, entonces para cualquier autovalor λ , $|\lambda|^k \rightarrow 0$, lo que implica $|\lambda| < 1$. ($\Leftarrow$) Si $\rho(G) < 1$, existe una norma $\|\cdot\|$ tal que $\|G\| < 1$. Entonces $\|G^k\| \leq \|G\|^k \rightarrow 0$.

T8. La matriz de Hilbert esta mal condicionada porque sus valores singulares decrecen exponencialmente, haciendo $\sigma_{\min} \ll \sigma_{\max}$. Para $n = 10$, $\kappa_2 \approx 10^{13}$. Esto implica que la eliminacion gaussiana (incluso con pivoteo) puede perder mucha precision.

T9. La demostracion se basa en que el pivoteo parcial equivale a multiplicar por matrices de permutacion. El resultado es $PA = LU$ con P producto de permutaciones.

T10. Un problema mal condicionado es inherente al problema (ej: raices de polinomio de Wilkinson). Un algoritmo inestable es un metodo que amplifica errores aunque el problema este bien condicionado (ej: recurrencia E_n hacia adelante). En el primero, no se puede hacer nada; en el segundo, se puede mejorar el algoritmo.

Soluciones Computacionales

C1. Eliminacion Gaussiana sin pivoteo:

```

1 import numpy as np
2
3 def gauss_sin_pivoteo(A, b):
4     n = len(A)
5     x = np.zeros(n)
6     for k in range(n-1):
7         if abs(A[k, k]) < 1e-15:
8             raise ValueError(f'Pivote nulo en columna {k}')
9         for i in range(k+1, n):
10             m = A[i, k] / A[k, k]
11             A[i, k:] -= m * A[k, k:]
12             b[i] -= m * b[k]
13     for i in range(n-1, -1, -1):
14         x[i] = (b[i] - np.dot(A[i, i+1:], x[i+1:])) / A[i, i]
15     return x
16
17 A = np.array([[0, 1], [1, 1]], dtype=float)
18 b = np.array([1, 2], dtype=float)
19 try:
20     x = gauss_sin_pivoteo(A.copy(), b.copy())
21     print(x)
22 except Exception as e:
23     print("Error:", e)

```

C2. Eliminacion con pivoteo:

```

1 import numpy as np
2
3 def gauss_con_pivoteo(A, b):
4     n = len(A)
5     x = np.zeros(n)
6     for k in range(n-1):
7         max_idx = max(range(k, n), key=lambda i: abs(A[i, k]))
8         if max_idx != k:
9             A[[k, max_idx]] = A[[max_idx, k]]
10             b[k], b[max_idx] = b[max_idx], b[k]
11         if abs(A[k, k]) < 1e-15:
12             raise ValueError(f'Matriz singular en columna {k}')
13         for i in range(k+1, n):
14             m = A[i, k] / A[k, k]
15             A[i, k:] -= m * A[k, k:]
16             b[i] -= m * b[k]
17     for i in range(n-1, -1, -1):

```

```

18     x[i] = (b[i] - np.dot(A[i, i+1:], x[i+1:])) / A[i, i]
19     return x
20
21 A = np.array([[0, 1], [1, 1]], dtype=float)
22 b = np.array([1, 2], dtype=float)
23 x = gauss_con_pivoteo(A.copy(), b.copy())
24 print("Solucion:", x)
25 print("Residuo:", np.linalg.norm(A @ x - b))

```

C3. Factorizacion LU:

```

1 import numpy as np
2
3 def lu_sin_pivoteo(A):
4     n = len(A)
5     L = np.eye(n)
6     U = A.copy()
7     for k in range(n-1):
8         if abs(U[k, k]) < 1e-15:
9             raise ValueError(f'Pivote nulo en columna {k}')
10        for i in range(k+1, n):
11            L[i, k] = U[i, k] / U[k, k]
12            U[i, k:] -= L[i, k] * U[k, k:]
13    return L, U
14
15 A = np.array([[2, 1, 1], [4, -6, 0], [-2, 7, 2]], dtype=float)
16 L, U = lu_sin_pivoteo(A)
17 print("L:")
18 print(L)
19 print("U:")
20 print(U)
21 print("LU:")
22 print(L @ U)

```

C4. PLU:

```

1 import numpy as np
2
3 def plu(A):
4     n = len(A)
5     U = A.copy()
6     L = np.eye(n)
7     P = np.eye(n)
8     for k in range(n-1):
9         max_idx = max(range(k, n), key=lambda i: abs(U[i, k]))
10        if max_idx != k:
11            U[[k, max_idx]] = U[[max_idx, k]]
12            P[[k, max_idx]] = P[[max_idx, k]]
13            L[[k, max_idx]] = L[[max_idx, k]]
14        if abs(U[k, k]) < 1e-15:
15            raise ValueError(f'Matriz singular en columna {k}')
16        for i in range(k+1, n):
17            L[i, k] = U[i, k] / U[k, k]
18            U[i, k:] -= L[i, k] * U[k, k:]
19    return P, L, U
20
21 eps = 1e-12
22 A = np.array([[eps, 1], [1, 1]], dtype=float)
23 P, L, U = plu(A)
24 print("P:")
25 print(P)
26 print("L:")

```

```
27 print(L)
28 print("U:")
29 print(U)
30 print("PA:")
31 print(P @ A)
32 print("LU:")
33 print(L @ U)
```

C5. Costo de LU:

```
1 import numpy as np
2 import time
3 import matplotlib.pyplot as plt
4
5 def tiempo_lu(n):
6     A = np.random.rand(n, n)
7     t0 = time.time()
8     L, U = lu_sin_pivoteo(A)
9     return time.time() - t0
10
11 ns = [10, 20, 50, 100, 200]
12 tiempos = []
13 for n in ns:
14     tiempos.append(tiempo_lu(n))
15
16 plt.loglog(ns, tiempos, 'o-', label='LU')
17 plt.loglog(ns, [n**3 / 1e9 for n in ns], '--', label='O(n^3)')
18 plt.xlabel('n')
19 plt.ylabel('Tiempo (s)')
20 plt.legend()
21 plt.grid(True)
22 plt.show()
```

C6. Cholesky:

```
1 import numpy as np
2
3 def cholesky(A):
4     n = len(A)
5     L = np.zeros((n, n))
6     for k in range(n):
7         s = 0.0
8         for j in range(k):
9             s += L[k, j]**2
10        L[k, k] = np.sqrt(A[k, k] - s)
11        for i in range(k+1, n):
12            s = 0.0
13            for j in range(k):
14                s += L[i, j] * L[k, j]
15            L[i, k] = (A[i, k] - s) / L[k, k]
16    return L
17
18 A = np.array([[4, 2, -2], [2, 5, 5], [-2, 5, 11]], dtype=float)
19 L = cholesky(A)
20 print("L:")
21 print(L)
22 print("L L^T:")
23 print(L @ L.T)
24 print("A:")
25 print(A)
```

C7. Comparacion LU vs Cholesky:


```

1 import numpy as np
2 import time
3 import matplotlib.pyplot as plt
4
5 def tiempo_lu(n):
6     A = np.random.rand(n, n)
7     A = A @ A.T + np.eye(n) * n
8     t0 = time.time()
9     L, U = lu_sin_pivoteo(A)
10    return time.time() - t0
11
12 def tiempo_cholesky(n):
13     A = np.random.rand(n, n)
14     A = A @ A.T + np.eye(n) * n
15     t0 = time.time()
16     L = cholesky(A)
17     return time.time() - t0
18
19 ns = [10, 20, 50, 100, 200]
20 tiempos_lu = []
21 tiempos_chol = []
22 for n in ns:
23     tiempos_lu.append(tiempo_lu(n))
24     tiempos_chol.append(tiempo_cholesky(n))
25
26 plt.loglog(ns, tiempos_lu, 'o-', label='LU')
27 plt.loglog(ns, tiempos_chol, 's-', label='Cholesky')
28 plt.xlabel('n')
29 plt.ylabel('Tiempo (s)')
30 plt.legend()
31 plt.grid(True)
32 plt.show()

```

C8. Jacobi:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def jacobi(A, b, x0, tol=1e-6, maxit=100):
5     n = len(b)
6     x = x0.copy()
7     errores = []
8     for _ in range(maxit):
9         x_new = np.zeros(n)
10        for i in range(n):
11            suma = b[i] - np.dot(A[i, :], x) + A[i, i]*x[i]
12            x_new[i] = suma / A[i, i]
13        err = np.linalg.norm(x_new - x, np.inf)
14        errores.append(err)
15        x = x_new
16        if err < tol:
17            break
18    return x, errores
19
20 A = np.array([[4, -1, 0], [-1, 4, -1], [0, -1, 4]], dtype=float)
21 b = np.array([3, 2, 3], dtype=float)
22 x0 = np.zeros(3)
23
24 x, err = jacobi(A, b, x0)
25 print("Solucion:", x)
26 print("Exacta:", np.linalg.solve(A, b))

```

```
27
28 plt.semilogy(err)
29 plt.xlabel('Iteracion')
30 plt.ylabel('Error infinito')
31 plt.grid(True)
32 plt.show()
```

C9. Gauss-Seidel:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gauss_seidel(A, b, x0, tol=1e-6, maxit=100):
5     n = len(b)
6     x = x0.copy()
7     errores = []
8     for _ in range(maxit):
9         x_old = x.copy()
10        for i in range(n):
11            suma = b[i] - np.dot(A[i, :], x) + A[i, i]*x[i]
12            x[i] = suma / A[i, i]
13        err = np.linalg.norm(x - x_old, np.inf)
14        errores.append(err)
15        if err < tol:
16            break
17    return x, errores
18
19 A = np.array([[4, -1, 0], [-1, 4, -1], [0, -1, 4]], dtype=float)
20 b = np.array([3, 2, 3], dtype=float)
21 x0 = np.zeros(3)
22
23 x_gs, err_gs = gauss_seidel(A, b, x0)
24 x_j, err_j = jacobi(A, b, x0)
25
26 plt.semilogy(err_j, label='Jacobi')
27 plt.semilogy(err_gs, label='Gauss-Seidel')
28 plt.xlabel('Iteracion')
29 plt.ylabel('Error infinito')
30 plt.legend()
31 plt.grid(True)
32 plt.show()
```

C10. Estabilidad:

```
1 import numpy as np
2
3 eps = 1e-10
4 A = np.array([[1, 1], [1, 1+eps]], dtype=float)
5 b = np.array([2, 2+eps], dtype=float)
6
7 try:
8     x_sin = gauss_sin_pivoteo(A.copy(), b.copy())
9     print("Sin pivoteo:", x_sin)
10 except Exception as e:
11     print("Sin pivoteo falla:", e)
12
13 x_con = gauss_con_pivoteo(A.copy(), b.copy())
14 print("Con pivoteo:", x_con)
15 print("Exacto:", np.linalg.solve(A, b))
```

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por los metodos directos para sistemas lineales. Viste que no es solo hacer cuentas: hay que pensar en la estabilidad, en el costo, y en las propiedades de la matriz. La eliminacion gaussiana con pivoteo es la herramienta mas confiable para sistemas de tamaño moderado. Para matrices grandes y ralas, los metodos iterativos son la clave. Y Cholesky es tu amigo si la matriz es simetrica definida positiva.

Lo mas importante es que entiendas el concepto de estabilidad: un algoritmo puede ser matematicamente correcto pero numericamente inestable si no se implementa con cuidado. El pivoteo es un ejemplo de como un pequenio cambio en el algoritmo puede salvar todo el resultado.

Ahora, a practicar con los ejercicios. Y recuerda: si tu codigo te da cualquier cosa, revisa el condicionamiento de la matriz y el pivoteo. ¡Dale para adelante!

Metodos Iterativos para Sistemas Lineales: Jacobi, Gauss-Seidel y la Magia del Radio Espectral

Como resolver sistemas enormes sin que tu computadora se prenda fuego, explicado en criollo y con mucho Python.

Introduccion: El Dia que Gauss me Dejo Tirado

Che, ¿como andas? Bienvenido a este capitulo que, te lo aseguro, te va a cambiar la forma de ver los sistemas lineales para siempre. Cuando yo estaba en primer año de la carrera, cursando Algebra Lineal, me enseñaron el metodo de eliminacion de Gauss y pense "bueno, esto es todo, ya se resolver cualquier sistema". Me sentia un campeon. Pero despues, en Calculo Numerico, el profesor nos pidio que resolvieramos un sistema de 1000 ecuaciones con 1000 incognitas. Yo, confiado, abri Matlab y mande un 'A'. La computadora se quedo pensando... y pensando... y al rato me tiro un error de memoria. Intente con Python, lo mismo. Mi notebook de aquel entonces (un bicho con 4 GB de RAM) se moria.

Fue ahi que entendi que el metodo de Gauss directo no es para todo. Para matrices densas de tamaño n , el costo es $O(n^3)$. Para $n = 1000$, son mil millones de operaciones. Y si la matriz es rala (llena de ceros), la factorizacion LU puede llenar esos ceros y convertirse en densa, lo que te mata la memoria. Ademas, si la matriz esta mal condicionada (como la matriz de Hilbert que vimos en el capitulo anterior), los errores de redondeo te hacen mierda la solucion.

Entonces, ¿que hacemos? La respuesta es: metodos iterativos. En lugar de calcular la solucion exacta de una vez, empezamos con una aproximacion burda y vamos mejorando paso a paso. Es como cuando armas un rompecabezas: no pones todas las piezas de una vez, sino que vas probando y ajustando. Y lo mejor es que en cada paso solo necesitas multiplicar matrices por vectores, no resolver sistemas enteros. Para matrices ralas, eso es rapidisimo.

En este capitulo vamos a estudiar tres metodos iterativos clasicos: Jacobi, Gauss-Seidel y SOR (Successive Over-Relaxation). Vamos a ver como se derivan, cuando convergen, y como implementarlos en Python. Tambien vamos a entender el concepto de radio espectral, que es el que define si un metodo funciona o se va a la mierda. Y por supuesto, vamos a llenar esto de graficos, ejemplos y ejercicios para que le agarres la mano.

Asi que prepareate, porque esto es una de las herramientas mas importantes que vas a aprender en tu carrera. Si te dedicas a la fisica computacional, la ingenieria, o cualquier cosa que involucre resolver ecuaciones en grandes dominios, esto te va a salvar la vida. Dale, metele.

El Problema y la Descomposicion Fundamental

Queremos resolver el sistema lineal:

$$Ax = b \tag{1}$$

donde $A \in \mathbb{R}^{n \times n}$ es una matriz inversible, $x \in \mathbb{R}^n$ es el vector de incógnitas, y $b \in \mathbb{R}^n$ es el término independiente. Suponemos que A tiene elementos diagonales a_{ii} no nulos (si alguno es cero, podemos reordenar filas o columnas para evitarlo, o usar pivoteo).

La idea básica de los métodos iterativos es reescribir la ecuación como:

$$x = Gx + c \quad (2)$$

donde G es una matriz y c un vector. Luego, definimos una sucesión:

$$x^{(k+1)} = Gx^{(k)} + c \quad (3)$$

Si la sucesión converge a un punto fijo x^* , entonces $x^* = Gx^* + c$, que es equivalente a $Ax^* = b$. La pregunta es: ¿cómo elegimos G y c ? Y sobre todo, ¿cuándo converge?

La Descomposición $A = D + L + U$

Para construir los métodos, descomponemos A en tres partes:

$$A = D + L + U \quad (4)$$

donde:

- D es la matriz diagonal: $D = \text{diag}(a_{11}, a_{22}, \dots, a_{nn})$.
- L es la matriz triangular inferior estricta: tiene los elementos a_{ij} con $i > j$, y ceros en la diagonal y arriba.
- U es la matriz triangular superior estricta: tiene los elementos a_{ij} con $i < j$, y ceros en la diagonal y abajo.

Por ejemplo, para $n = 3$:

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} a_{11} & 0 & 0 \\ 0 & a_{22} & 0 \\ 0 & 0 & a_{33} \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 \\ a_{21} & 0 & 0 \\ a_{31} & a_{32} & 0 \end{pmatrix} + \begin{pmatrix} 0 & a_{12} & a_{13} \\ 0 & 0 & a_{23} \\ 0 & 0 & 0 \end{pmatrix}$$

Traducción Matemática de $A = D + L + U$ Esta descomposición se lee: "la matriz A es igual a la suma de su parte diagonal D , su parte triangular inferior izquierda L , y su parte triangular superior derecha U ". La letra D viene de "diagonal", L de "lower" (inferior en inglés) y U de "upper" (superior). Es una forma de partir la matriz en pedazos que son fáciles de invertir: D es diagonal (su inversa es simplemente $1/a_{ii}$), y $D + L$ es triangular inferior (se resuelve fácil con sustitución hacia adelante).

Ahora, vamos a usar esta descomposición para reescribir $Ax = b$:

$$(D + L + U)x = b$$

Despejando de diferentes maneras obtenemos los distintos métodos.

Metodo de Jacobi: El Mas Intuitivo

El metodo de Jacobi es el mas simple de entender. Suponemos que $a_{ii} \neq 0$ para todo i . Despejamos x_i de la ecuacion i :

$$a_{ii}x_i = b_i - \sum_{j \neq i} a_{ij}x_j$$

Entonces, la iteracion de Jacobi es:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right), \quad i = 1, \dots, n \quad (5)$$

En forma matricial:

$$x^{(k+1)} = D^{-1}(b - (L + U)x^{(k)}) = -D^{-1}(L + U)x^{(k)} + D^{-1}b$$

Asi que la matriz de iteracion de Jacobi es:

$$G_J = -D^{-1}(L + U) = I - D^{-1}A \quad (6)$$

y el vector $c_J = D^{-1}b$.

Traduccion Matematica de la Iteracion de Jacobi La formula de Jacobi se lee: "para cada componente i , la nueva aproximacion $x_i^{(k+1)}$ se calcula como 1 sobre el elemento diagonal a_{ii} , multiplicado por b_i menos la suma de los productos de a_{ij} con las componentes viejas $x_j^{(k)}$ para todos los j distintos de i ". Es decir, cada variable se actualiza usando exclusivamente los valores del paso anterior. Es un metodo "sincronico": todas las componentes nuevas se calculan a partir del vector viejo. Esto tiene la ventaja de que es facil de paralelizar, pero a veces converge mas lento.

Reflexion: La primera vez que implemente Jacobi en Python, me olvide de hacer una copia del vector viejo y termine actualizando las variables en el mismo lugar, lo que me dio cualquier cosa. ¡Cuidado con eso! Necesitas dos vectores: x_{viejo} y x_{nuevo} . En el codigo, usa una copia con `'x_nuevo = x.copy()'`.

Ejemplo Ilustrativo en 2D

Tomemos el sistema clasico:

$$\begin{cases} 4x_1 + x_2 = 5 \\ x_1 + 4x_2 = 5 \end{cases}$$

La solucion exacta es $x_1 = 1, x_2 = 1$. La matriz de iteracion de Jacobi es:

$$G_J = \begin{pmatrix} 0 & -1/4 \\ -1/4 & 0 \end{pmatrix}$$

Empezando con $x^{(0)} = (0, 0)^T$, obtenemos:

$$x^{(1)} = \left(\frac{5}{4}, \frac{5}{4} \right)^T = (1,25, 1,25)$$

$$x^{(2)} = \left(\frac{5}{4} - \frac{1}{4} \cdot \frac{5}{4}, \frac{5}{4} - \frac{1}{4} \cdot \frac{5}{4} \right)^T = \left(\frac{15}{16}, \frac{15}{16} \right)^T = (0,9375, 0,9375)$$

$$x^{(3)} = \left(\frac{5}{4} - \frac{1}{4} \cdot \frac{15}{16}, \frac{5}{4} - \frac{1}{4} \cdot \frac{15}{16} \right)^T = \left(\frac{65}{64}, \frac{65}{64} \right)^T \approx (1,015625, 1,015625)$$

Vemos que va oscilando alrededor de la solucion, pero converge. El error en el paso k se reduce por un factor $1/4$ en cada iteracion. De hecho, $\|G_J\|_\infty = 1/4 < 1$, y el radio espectral es $1/4$.

Cuando Jacobi Falla: Un Ejemplo Peligroso

Consideremos la matriz de Collatz:

$$A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}$$

La matriz de iteracion de Jacobi es:

$$G_J = \begin{pmatrix} 0 & -2 & 2 \\ -1 & 0 & -1 \\ -2 & -2 & 0 \end{pmatrix}$$

Calculemos sus autovalores. El polinomio caracteristico es:

$$\det(\lambda I - G_J) = \lambda^3$$

Asi que todos los autovalores son cero. ¡El radio espectral es 0! Eso significa que el metodo converge en exactamente 3 pasos, porque $G_J^3 = 0$. Es un caso raro pero muestra que Jacobi puede ser muy bueno si la matriz de iteracion es nilpotente.

Sin embargo, en general, Jacobi no siempre converge. Por ejemplo, si la matriz no es diagonal dominante, el radio espectral puede ser mayor que 1. Un contraejemplo tipico es una matriz simetrica definida positiva con elementos fuera de la diagonal grandes. Por ejemplo, para:

$$A = \begin{pmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{pmatrix}, \quad \text{con } a \geq \frac{1}{2}$$

esta matriz es SPD, pero Jacobi diverge porque $\rho(G_J) = 2a \geq 1$.

Visualizacion de la Convergencia de Jacobi

Vamos a graficar como converge Jacobi para el sistema 2x2. El siguiente codigo muestra la trayectoria de las iteraciones en el plano $x_1 - x_2$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def jacobi_2x2(A, b, x0, n_iter=20):
5     """Implementa Jacobi para un sistema 2x2 y devuelve la trayectoria."""
6     x = x0.copy()
7     trayectoria = [x.copy()]
8     D_inv = np.diag(1.0 / np.diag(A))
9     L_plus_U = A - np.diag(np.diag(A))
10    for _ in range(n_iter):
11        x = D_inv @ (b - L_plus_U @ x)
12        trayectoria.append(x.copy())

```

```

13     return np.array(trayectoria)
14
15 # Definimos el sistema
16 A = np.array([[4.0, 1.0], [1.0, 4.0]])
17 b = np.array([5.0, 5.0])
18 x0 = np.array([0.0, 0.0])
19
20 # Calculamos la trayectoria
21 tray = jacobi_2x2(A, b, x0, n_iter=15)
22
23 # Graficamos
24 plt.figure(figsize=(8, 8))
25 plt.plot(tray[:, 0], tray[:, 1], 'o-', label='Iteraciones de Jacobi', markersize
26          =8)
27 plt.plot(1, 1, 'r*', markersize=15, label='Solucion exacta (1,1)')
28 plt.xlabel('$x_1$', fontsize=14)
29 plt.ylabel('$x_2$', fontsize=14)
30 plt.title('Trayectoria de Jacobi para el sistema $4x_1 + x_2 = 5, x_1 + 4x_2 = 5$ ',
31          fontsize=14)
32 plt.grid(True, alpha=0.3)
33 plt.legend()
34 plt.axis('equal')
35 plt.show()

```

Listing 40: Trayectoria de Jacobi para el sistema 2x2

Este grafico muestra como las iteraciones van "bailando" en forma de espiral hacia el punto solucion. Es una forma re linda de ver la convergencia.

Convergencia de Jacobi: Condicion Suficiente

Una condicion suficiente (pero no necesaria) para que Jacobi converja es que A sea **estrictamente diagonal dominante por filas**:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i \quad (7)$$

En ese caso, $\|G_J\|_\infty = \max_i \sum_{j \neq i} |a_{ij}|/|a_{ii}| < 1$, y por lo tanto $\rho(G_J) < 1$. La convergencia es garantizada.

Traduccion Matematica de Diagonal Dominante La condicion de diagonal dominante se lee: "para cada fila i , el valor absoluto del elemento diagonal a_{ii} es mayor que la suma de los valores absolutos de todos los elementos fuera de la diagonal en esa fila". Esto significa que la matriz esta "cargada" hacia la diagonal. Es como si cada ecuacion dependiera mas de su propia variable que de las demas. En problemas fisicos, esto suele pasar cuando el acoplamiento entre variables es debil.

Metodo de Gauss-Seidel: Usando la Informacion Fresca

El metodo de Gauss-Seidel es una mejora de Jacobi. La idea es: cuando calculas $x_i^{(k+1)}$, ya tenes disponibles las componentes $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ que acabas de actualizar. ¿Por que no usarlas en lugar de las viejas? Entonces, la iteracion es:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, \dots, n \quad (8)$$

En forma matricial, esto se escribe como:

$$(D + L)x^{(k+1)} = b - Ux^{(k)}$$

de donde:

$$x^{(k+1)} = -(D + L)^{-1}Ux^{(k)} + (D + L)^{-1}b \quad (9)$$

La matriz de iteracion de Gauss-Seidel es:

$$G_{GS} = -(D + L)^{-1}U \quad (10)$$

y el vector $c_{GS} = (D + L)^{-1}b$.

Traduccion Matematica de la Iteracion de Gauss-Seidel La formula de Gauss-Seidel se lee: "para cada componente i , la nueva aproximacion $x_i^{(k+1)}$ se calcula usando las componentes nuevas $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ (que ya estan actualizadas) y las viejas $x_{i+1}^{(k)}, \dots, x_n^{(k)}$ (que todavia no se actualizaron)". Es un metodo "secuencial": las variables se actualizan una por una, y cada una se beneficia de las que ya se actualizaron. Esto suele acelerar la convergencia, y ademas no necesitas almacenar el vector viejo completo (solo el nuevo, que se va sobrescribiendo). Pero ojo, esto lo hace inherentemente secuencial, asi que es mas dificil de paralelizar.

Reflexion: Cuando implemente Gauss-Seidel por primera vez, me confundi con los indices y termine usando $x_j^{(k)}$ para los $j < i$. ¡Un error de principiante! La clave es que en el bucle, cuando estas en i , ya actualizaste las primeras $i - 1$ componentes. Tienes que usar el vector que estas modificando (que ya tiene las componentes nuevas) para $j < i$, y el vector original (o una copia) para $j > i$. En Python, si actualizas en el lugar, es natural: usas `x[j]` para $j < i$ y `x_viejo[j]` para $j > i$.

Ejemplo en 2D: La Diferencia es Notable

Para el mismo sistema $4x_1 + x_2 = 5, x_1 + 4x_2 = 5$, la matriz de iteracion de Gauss-Seidel es:

$$G_{GS} = \begin{pmatrix} 0 & -1/4 \\ 0 & 1/16 \end{pmatrix}$$

Su radio espectral es $1/16 = 0,0625$, mucho menor que el de Jacobi ($1/4 = 0,25$). Empezando con $x^{(0)} = (0, 0)$:

$$x^{(1)} = \left(\frac{5}{4}, \frac{5}{4} - \frac{1}{4} \cdot \frac{5}{4} \right)^T = (1,25, 0,9375)^T$$

$$x^{(2)} = \left(\frac{5}{4} - \frac{1}{4} \cdot 0,9375, \frac{5}{4} - \frac{1}{4} \cdot 1,25 \right)^T = (1,015625, 0,984375)^T$$

$$x^{(3)} = \left(\frac{5}{4} - \frac{1}{4} \cdot 0,984375, \frac{5}{4} - \frac{1}{4} \cdot 1,015625 \right)^T \approx (1,003906, 0,996094)^T$$

¡Ya en tres pasos estamos mucho mas cerca que Jacobi en cinco! Gauss-Seidel suele ser mas rapido que Jacobi, aunque no siempre (veremos un contraejemplo).

Comparacion Grafica de Jacobi vs Gauss-Seidel

Vamos a graficar el error en funcion de las iteraciones para ambos metodos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def jacobi_error(A, b, x0, n_iter=20):
5     x = x0.copy()
6     D_inv = np.diag(1.0 / np.diag(A))
7     L_plus_U = A - np.diag(np.diag(A))
8     errores = []
9     for _ in range(n_iter):
10         x = D_inv @ (b - L_plus_U @ x)
11         errores.append(np.linalg.norm(x - np.linalg.solve(A,b), np.inf))
12     return errores
13
14 def gauss_seidel_error(A, b, x0, n_iter=20):
15     x = x0.copy()
16     n = len(b)
17     errores = []
18     for _ in range(n_iter):
19         for i in range(n):
20             suma = 0.0
21             for j in range(n):
22                 if j != i:
23                     suma += A[i,j] * x[j]
24             x[i] = (b[i] - suma) / A[i,i]
25         errores.append(np.linalg.norm(x - np.linalg.solve(A,b), np.inf))
26     return errores
27
28 # Sistema 2x2
29 A = np.array([[4.0, 1.0], [1.0, 4.0]])
30 b = np.array([5.0, 5.0])
31 x0 = np.array([0.0, 0.0])
32
33 err_j = jacobi_error(A, b, x0, n_iter=20)
34 err_gs = gauss_seidel_error(A, b, x0, n_iter=20)
35
36 plt.figure(figsize=(10, 6))
37 plt.semilogy(range(1, 21), err_j, 'o-', label='Jacobi')
38 plt.semilogy(range(1, 21), err_gs, 's-', label='Gauss-Seidel')
39 plt.xlabel('Iteracion k', fontsize=14)
40 plt.ylabel('Error infinito ||x^{(k)} - x^*||_{\infty}', fontsize=14)
41 plt.title('Comparacion de convergencia: Jacobi vs Gauss-Seidel', fontsize=14)
42 plt.legend()
43 plt.grid(True, alpha=0.3)
44 plt.show()

```

Listing 41: Comparacion de errores: Jacobi vs Gauss-Seidel

El grafico muestra claramente que Gauss-Seidel converge mucho mas rapido. La pendiente de la curva en escala semilogaritmica esta relacionada con el radio espectral.

Teoria de Convergencia: El Radio Espectral es el Rey

La pregunta fundamental es: ¿bajo que condiciones $x^{(k)} \rightarrow x^*$? Como vimos, el error satisface:

$$e^{(k+1)} = Ge^{(k)} = G^k e^{(0)}$$

Por lo tanto, necesitamos que $G^k \rightarrow 0$ para todo $\epsilon^{(0)}$. El siguiente teorema es la clave de todo: Sea $G \in \mathbb{R}^{n \times n}$. Entonces $G^k \rightarrow 0$ si y solo si $\rho(G) < 1$, donde $\rho(G) = \max_i |\lambda_i|$ es el radio espectral de G (el máximo de los módulos de sus autovalores).

Traducción Matemática del Radio Espectral El radio espectral $\rho(G) = \max_i |\lambda_i|$ se lee: "el radio espectral de G es el máximo de los valores absolutos de sus autovalores". Un autovalor es un número λ tal que existe un vector no nulo v con $Gv = \lambda v$. Geométricamente, λ mide cuanto se estira o encoge el vector v al aplicar G . Si todos los autovalores tienen módulo menor que 1, entonces cualquier vector se encoge repetidamente y el error tiende a cero. Si algún autovalor tiene módulo mayor o igual a 1, entonces hay una dirección en la que el error no se achica (o crece), y el método no converge para todos los datos iniciales.

Demostración Intuitiva (y no tan formal) de $\rho(G) < 1 \iff G^k \rightarrow 0$

Si G es diagonalizable, $G = S\Lambda S^{-1}$, entonces $G^k = S\Lambda^k S^{-1}$. Si $|\lambda_i| < 1$ para todo i , entonces $\Lambda^k \rightarrow 0$ y por lo tanto $G^k \rightarrow 0$. Si algún $|\lambda_i| \geq 1$, entonces esa componente no tiende a cero, y G^k no tiende a cero. El caso no diagonalizable es más técnico, pero el resultado es el mismo: la condición necesaria y suficiente es $\rho(G) < 1$. Una prueba más general usa la forma de Jordan y el hecho de que se puede construir una norma tal que $\|G\| \leq \rho(G) + \epsilon$, lo que permite demostrar que si $\rho(G) < 1$, existe una norma con $\|G\| < 1$, y entonces $\|G^k\| \leq \|G\|^k \rightarrow 0$.

Reflexión: Cuando vi esto por primera vez, me costó entender por qué el radio espectral y no la norma. La clave es que el radio espectral es el "verdadero" factor de contracción asintótico. La norma puede ser grande aunque el radio sea pequeño, como en el ejemplo de la matriz $\begin{pmatrix} 1/2 & 1000 \\ 0 & 1/2 \end{pmatrix}$, que tiene radio 0.5 pero norma infinita 1000.5. Sin embargo, G^k decae como $\rho(G)^k$. Por eso, para comparar métodos, miramos el radio espectral.

Condición Suficiente: Diagonal Dominante Estricta

Ya vimos que si A es estrictamente diagonal dominante por filas, entonces $\|G_J\|_\infty < 1$, y por lo tanto $\rho(G_J) < 1$. Así que Jacobi converge. Lo mismo ocurre con Gauss-Seidel, aunque la demostración es un poco más complicada.

Si A es estrictamente diagonal dominante por filas, entonces tanto el método de Jacobi como el de Gauss-Seidel convergen para cualquier condición inicial.

La demostración para Gauss-Seidel usa el siguiente argumento: sea λ un autovalor de G_{GS} y x un autovector con $\|x\|_\infty = 1$. De la ecuación $\lambda(D + L)x = -Ux$, tomando la componente de mayor módulo, se obtiene:

$$|\lambda| \leq \frac{\sum_{j=i+1}^n |a_{ij}|}{|a_{ii}| - \sum_{j=1}^{i-1} |a_{ij}|} < 1$$

gracias a la diagonal dominancia. Así que $\rho(G_{GS}) < 1$.

Matrices Simétricas Definidas Positivas (SPD)

Otro caso importante es cuando A es simétrica y definida positiva (SPD). En ese caso, el método de Gauss-Seidel siempre converge. De hecho, se puede demostrar que $\rho(G_{GS}) < 1$. Sin embargo, Jacobi no necesariamente converge para SPD, como vimos en el ejemplo de la matriz con $a = 1/2$. Sea A SPD. Entonces el método de Gauss-Seidel converge para cualquier condición inicial.

La demostracion usa la forma $A = L + D + L^T$ y el hecho de que el radio espectral de G_{GS} es menor que 1. No la vamos a desarrollar en detalle, pero la idea es que la matriz de iteracion es una contraccion en la norma inducida por A .

Caso Tridiagonal: Una Relacion Hermosa

Si A es tridiagonal, entonces hay una relacion exacta entre los radios espectrales de Jacobi y Gauss-Seidel:

$$\rho(G_{GS}) = \rho(G_J)^2 \quad (11)$$

Esto significa que Gauss-Seidel converge el doble de rapido (en terminos asintoticos) que Jacobi. Es un resultado hermoso que se demuestra usando el lema de semejanza de matrices tridiagonales: para cualquier $\alpha \neq 0$, las matrices $D + L + U$ y $D + \alpha L + \alpha^{-1}U$ son semejantes. A partir de ahi, se relacionan los autovalores.

Ejemplo de Divergencia de Gauss-Seidel (y Jacobi converge)

Existen casos donde Jacobi converge pero Gauss-Seidel no. Un ejemplo es la matriz de Collatz modificada:

$$A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}$$

Como vimos, Jacobi tiene radio espectral 0, pero Gauss-Seidel tiene radio espectral 2. Asi que Gauss-Seidel diverge. Esto muestra que Gauss-Seidel no es universalmente mejor que Jacobi.

Metodo SOR (Successive Over-Relaxation): Acelerando la Convergencia

El metodo SOR es una generalizacion de Gauss-Seidel que introduce un parametro ω para acelerar la convergencia. La idea es tomar un "promedio ponderado" entre la solucion anterior y la que daria Gauss-Seidel:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega \left(\frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right) \right) \quad (12)$$

donde ω es el parametro de relajacion. Si $\omega = 1$, recuperamos Gauss-Seidel. Si $\omega > 1$, es sobre-relajacion (aceleras). Si $0 < \omega < 1$, es sub-relajacion (usado para problemas donde la sobre-relajacion desestabiliza).

En forma matricial:

$$(D + \omega L)x^{(k+1)} = ((1 - \omega)D - \omega U)x^{(k)} + \omega b$$

La matriz de iteracion es:

$$G_\omega = (D + \omega L)^{-1}((1 - \omega)D - \omega U) \quad (13)$$

Traduccion Matematica de SOR La formula de SOR se lee: "la nueva aproximacion $x_i^{(k+1)}$ es una combinacion lineal de la aproximacion anterior $x_i^{(k)}$ (con peso $1 - \omega$) y la que daría Gauss-Seidel (con peso ω)". Si $\omega > 1$, le das mas peso a la nueva informacion, lo que puede acelerar la convergencia, pero si te pasas, el metodo se vuelve inestable. El parametro optimo ω_{opt} minimiza el radio espectral y se puede calcular para ciertas matrices (por ejemplo, para matrices tridiagonales SPD).

Condicion Necesaria para Convergencia

Un teorema de Kahan dice que para que SOR converja, es necesario que:

$$0 < \omega < 2 \quad (14)$$

Esto se demuestra viendo que $\det(G_\omega) = (1 - \omega)^n$, y como $\rho(G_\omega) \geq |\det(G_\omega)|^{1/n} = |1 - \omega|$, para que $\rho(G_\omega) < 1$ se necesita $|1 - \omega| < 1$, es decir, $0 < \omega < 2$.

Si A es SPD, esta condicion tambien es suficiente: SOR converge para cualquier $0 < \omega < 2$.

Parametro Optimo para Matrices Tridiagonales

Para una matriz tridiagonal SPD, se puede demostrar que el radio espectral de G_ω esta dado por:

$$\rho(G_\omega) = \begin{cases} \frac{1}{4} \left(\omega\mu + \sqrt{\omega^2\mu^2 - 4(\omega - 1)} \right)^2 & \text{si } 0 < \omega \leq \omega_{\text{opt}} \\ \omega - 1 & \text{si } \omega_{\text{opt}} \leq \omega < 2 \end{cases}$$

donde $\mu = \rho(G_J)$ (el radio espectral de Jacobi) y

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \mu^2}} \quad (15)$$

Este es un resultado clasico que permite elegir el omega optimo para acelerar al maximo la convergencia. Para una matriz tridiagonal con 2 en la diagonal y -1 en las sub y superdiagonales (de tamaño n), se tiene $\mu = \cos\left(\frac{\pi}{n+1}\right)$, y entonces:

$$\omega_{\text{opt}} = \frac{2}{1 + \sin\left(\frac{\pi}{n+1}\right)}$$

Ejemplo Numerico de SOR

Vamos a implementar SOR y buscar el omega optimo para un caso concreto.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def radio_espectral(G):
5     return np.max(np.abs(np.linalg.eigvals(G)))
6
7 # Definir una matriz SPD tridiagonal
8 n = 10
9 A = np.zeros((n, n))
10 for i in range(n):
11     A[i, i] = 2.0
12     if i > 0: A[i, i-1] = -1.0
13     if i < n-1: A[i, i+1] = -1.0
```

```

14
15 D = np.diag(np.diag(A))
16 L = np.tril(A, -1)
17 U = np.triu(A, 1)
18
19 # Calcular rho de Jacobi
20 G_J = -np.linalg.inv(D) @ (L + U)
21 mu = radio_espectral(G_J)
22 print(f"Rho de Jacobi: {mu:.6f}")
23
24 # Omega optimo teorico
25 omega_opt = 2.0 / (1.0 + np.sqrt(1.0 - mu**2))
26 print(f"Omega optimo teorico: {omega_opt:.6f}")
27
28 # Barrido de omega
29 omegas = np.linspace(0.1, 1.9, 200)
30 rhos = []
31 for w in omegas:
32     G_w = np.linalg.inv(D + w * L) @ ((1 - w) * D - w * U)
33     rhos.append(radio_espectral(G_w))
34
35 plt.figure(figsize=(10, 6))
36 plt.plot(omegas, rhos, 'b-', linewidth=2)
37 plt.axvline(x=omega_opt, color='r', linestyle='--', label=f'Omega optimo teorico
    = {omega_opt:.4f}')
38 plt.axhline(y=1, color='k', linestyle=':', alpha=0.5)
39 plt.xlabel('Omega', fontsize=14)
40 plt.ylabel('Rho(G_omega)', fontsize=14)
41 plt.title('Radio espectral de SOR en funcion de omega', fontsize=14)
42 plt.legend()
43 plt.grid(True, alpha=0.3)
44 plt.show()

```

Listing 42: Busqueda del omega optimo para SOR

El grafico muestra una curva con un minimo pronunciado en ω_{opt} . Para $\omega > \omega_{\text{opt}}$, el radio espectral crece linealmente como $\omega - 1$, hasta llegar a 2, donde diverge. La eleccion del omega optimo puede acelerar enormemente la convergencia.

Relacion con el Condicionamiento y la Estabilidad

Hemos hablado del radio espectral como el factor que determina la convergencia de los metodos iterativos. Sin embargo, incluso si $\rho(G) < 1$, la convergencia puede ser muy lenta si $\rho(G)$ esta cerca de 1. Esto suele ocurrir cuando la matriz A esta mal condicionada, es decir, cuando su numero de condicion $\kappa(A) = \|A\| \|A^{-1}\|$ es grande. ¿Por que? Porque el condicionamiento de A se relaciona con la distribucion de sus autovalores, y eso afecta a los autovalores de G .

Por ejemplo, para la matriz de Hilbert, que es extremadamente mal condicionada, los metodos iterativos convergen muy lentamente. De hecho, para $n = 10$, $\kappa_2(H_{10}) \approx 1,6 \times 10^{13}$, y el radio espectral de Gauss-Seidel esta muy cerca de 1. Esto significa que se necesitan muchas iteraciones para obtener una precision aceptable.

Traduccion Matematica del Condicionamiento El numero de condicion $\kappa(A) = \|A\| \|A^{-1}\|$ mide la sensibilidad de la solucion de $Ax = b$ ante perturbaciones en b o en A . Si $\kappa(A)$ es grande, la matriz esta cerca de ser singular, y pequenos errores en los datos pueden causar grandes errores en la solucion. Para los metodos iterativos, un $\kappa(A)$ grande suele implicar que los autovalores

de A están muy separados, lo que hace que $\rho(G)$ se acerque a 1. Esto se ve claramente en el ejemplo de la matriz de Hilbert.

Reflexion: Una vez intente resolver un sistema de ecuaciones que venia de la discretizacion de una ecuacion diferencial eliptica en un dominio muy alargado. La matriz era muy mal condicionada, y Gauss-Seidel tardaba una eternidad. Ahi aprendi que para problemas mal condicionados, los metodos iterativos basicos no bastan; hay que usar preconditionadores o metodos mas avanzados como el gradiente conjugado.

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Radio espectral y norma:** Demostrar que para cualquier norma matricial inducida, $\rho(A) \leq \|A\|$. Usar esto para probar que si $\|G\| < 1$ para alguna norma, entonces el metodo iterativo converge.
- T2. Matriz de iteracion de Jacobi:** Escribir explicitamente la matriz de iteracion de Jacobi para una matriz A de 3×3 en funcion de los elementos a_{ij} . ¿Cual es la condicion sobre A para que Jacobi este bien definido?
- T3. Convergencia de Jacobi para diagonal dominante:** Sea A estrictamente diagonal dominante por filas. Probar que $\|G_J\|_\infty < 1$. Concluir que el metodo de Jacobi converge.
- T4. Convergencia de Gauss-Seidel para diagonal dominante:** Sea A estrictamente diagonal dominante por filas. Probar que $\rho(G_{GS}) < 1$. (Ayuda: usar un autovector con norma infinito = 1 y la ecuacion $\lambda(D + L)x = -Ux$).
- T5. SOR y el parametro omega:** Demostrar que para cualquier matriz A con $a_{ii} \neq 0$, $\rho(G_\omega) \geq |1 - \omega|$. Concluir que una condicion necesaria para la convergencia de SOR es $0 < \omega < 2$.
- T6. Gauss-Seidel para matrices SPD:** Demostrar que si A es simetrica definida positiva, entonces el metodo de Gauss-Seidel converge. (Ayuda: usar la forma $A = L + D + L^T$ y el producto interno definido por A).
- T7. Relacion entre Jacobi y Gauss-Seidel para tridiagonales:** Demostrar que si A es tridiagonal, entonces $\rho(G_{GS}) = \rho(G_J)^2$. (Ayuda: usar el lema de semejanza $D + L + U \sim D + \alpha L + \alpha^{-1}U$).
- T8. Contraejemplo de Jacobi para SPD:** Dar un ejemplo de una matriz A simetrica definida positiva para la cual el metodo de Jacobi NO converge. (Ayuda: considerar A con 1 en la diagonal y a en las posiciones (1,2), (1,3), (2,3), con a cercano a 1).
- T9. Contraejemplo inverso:** Dar un ejemplo de una matriz A para la cual Jacobi converge pero Gauss-Seidel NO. (Ayuda: usar el ejemplo de Collatz).
- T10. Estabilidad y condicionamiento:** Explicar por que una matriz mal condicionada (con $\kappa(A)$ grande) puede hacer que los metodos iterativos converjan lentamente o que los errores de redondeo se amplifiquen, incluso si $\rho(G) < 1$.

Parte Computacional (10 ejercicios)

- C1. Implementacion de Jacobi:** Escribe una funcion en Python que implemente el metodo de Jacobi para una matriz A y un vector b . La funcion debe aceptar un x_0 inicial, un numero maximo de iteraciones y una tolerancia. Debe devolver la solucion aproximada y el numero de iteraciones realizadas.
- C2. Implementacion de Gauss-Seidel:** Escribe una funcion en Python que implemente el metodo de Gauss-Seidel. Compara su desempeno con el de Jacobi en el sistema $4x_1 + x_2 = 5, x_1 + 4x_2 = 5$. Grafica el error vs iteraciones para ambos metodos.
- C3. Radio espectral y convergencia:** Para la matriz $A = \begin{bmatrix} 3 & 1 & 1 \\ 2 & 6 & 1 \\ 1 & 1 & 4 \end{bmatrix}$, calcula las matrices de iteracion de Jacobi y Gauss-Seidel y sus radios espectrales. ¿Cual metodo converge mas rapido? Verifica numericamente.
- C4. SOR y busqueda del omega optimo:** Implementa el metodo SOR. Para la matriz del ejercicio anterior, encuentra el valor de ω que minimiza el radio espectral (podes hacer una busqueda en $0 < \omega < 2$). Grafica $\rho(G_\omega)$ vs ω .
- C5. Matriz de Hilbert y condicionamiento:** Para $n = 5, 6, \dots, 10$, construye la matriz de Hilbert H_n (con $H_{ij} = 1/(i + j - 1)$). Calcula el numero de condicion $\kappa_2(H_n)$ y el radio espectral de la matriz de iteracion de Gauss-Seidel. ¿Que relacion ves entre el condicionamiento y la convergencia?
- C6. Sistema mal condicionado:** Resuelve el sistema $Ax = b$ donde $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 + 10^{-6} \end{bmatrix}$ y $b = [1, 2 + 10^{-6}]$ usando Jacobi y Gauss-Seidel. ¿Que observas? Calcula $\kappa_2(A)$.
- C7. Diagonal dominante:** Genera una matriz aleatoria A de tamaño 10×10 que sea estrictamente diagonal dominante (podes sumarle una matriz diagonal grande). Resuelve un sistema usando Jacobi y Gauss-Seidel. Verifica la convergencia y compara las velocidades.
- C8. Matriz no diagonal dominante pero SPD:** Genera una matriz A de tamaño 10×10 que NO sea diagonal dominante pero que sea simetrica definida positiva. Aplica Jacobi y Gauss-Seidel. ¿Que sucede? ¿Converge alguno?
- C9. Convergencia asintotica:** Para la matriz $A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$, calcula $\rho(G_J)$ y $\rho(G_{GS})$. Verifica numericamente que $\|G_J^k\|$ decae como $\rho(G_J)^k$ y $\|G_{GS}^k\|$ decae como $\rho(G_{GS})^k$. Grafica $\|G^k\|$ vs k en escala semilogaritmica.
- C10. SOR optimo para tridiagonal:** Para la matriz tridiagonal A con 2 en la diagonal y -1 en las sub y superdiagonales (de tamaño n), demuestra que el ω optimo para SOR es $\omega_{\text{opt}} = \frac{2}{1 + \sin(\frac{\pi}{n+1})}$. Implementa SOR con este ω y compara con Jacobi y Gauss-Seidel para $n = 10, 20, 50$. Grafica las tasas de convergencia.

Soluciones Completas

Soluciones Teoricas

T1. Sea λ un autovalor de A con autovector $v \neq 0$. Entonces $Av = \lambda v$. Tomando norma: $|\lambda| \|v\| = \|Av\| \leq \|A\| \|v\|$. Como $\|v\| \neq 0$, $|\lambda| \leq \|A\|$. El maximo sobre todos los autovalores da $\rho(A) \leq \|A\|$.

Si $\|G\| < 1$, entonces $\|G^k\| \leq \|G\|^k \rightarrow 0$. Por lo tanto, para cualquier $e^{(0)}$, $\|e^{(k)}\| = \|G^k e^{(0)}\| \leq \|G^k\| \|e^{(0)}\| \rightarrow 0$. Luego converge.

T2. La matriz de iteración de Jacobi es $G_J = -D^{-1}(L + U)$. Para 3×3 :

$$G_J = \begin{pmatrix} 0 & -a_{12}/a_{11} & -a_{13}/a_{11} \\ -a_{21}/a_{22} & 0 & -a_{23}/a_{22} \\ -a_{31}/a_{33} & -a_{32}/a_{33} & 0 \end{pmatrix}$$

El método está bien definido si $a_{ii} \neq 0$ para todo i .

T3. Sea $G_J = -D^{-1}(L + U)$. Entonces $\|G_J\|_\infty = \max_i \sum_{j \neq i} |a_{ij}|/|a_{ii}| < 1$ por la diagonal dominancia estricta. Luego $\rho(G_J) \leq \|G_J\|_\infty < 1$, y Jacobi converge.

T4. Sea λ autovalor de $G_{GS} = -(D + L)^{-1}U$ y x autovector con $\|x\|_\infty = 1$. Entonces $-(D + L)^{-1}Ux = \lambda x$, lo que implica $-Ux = \lambda(D + L)x$. Tomando la componente i donde $|x_i| = 1$:

$$-\sum_{j=i+1}^n a_{ij}x_j = \lambda \sum_{j=1}^i a_{ij}x_j$$

Entonces:

$$|\lambda||a_{ii}| \leq |\lambda| \sum_{j=1}^{i-1} |a_{ij}| + \sum_{j=i+1}^n |a_{ij}|$$

Reordenando:

$$|\lambda| \leq \frac{\sum_{j=i+1}^n |a_{ij}|}{|a_{ii}| - \sum_{j=1}^{i-1} |a_{ij}|} < 1$$

por la diagonal dominancia estricta. Luego $\rho(G_{GS}) < 1$.

T5. Sea $G_\omega = (D + \omega L)^{-1}((1 - \omega)D - \omega U)$. El determinante de G_ω es:

$$\det(G_\omega) = \det((D + \omega L)^{-1}) \det((1 - \omega)D - \omega U) = \det(D^{-1}) \det((1 - \omega)D) = (1 - \omega)^n$$

porque L y U tienen diagonal cero. Pero $\det(G_\omega) = \prod_i \lambda_i$, así que $\rho(G_\omega)^n \geq |\det(G_\omega)| = |1 - \omega|^n$, luego $\rho(G_\omega) \geq |1 - \omega|$. Para convergencia necesitamos $\rho(G_\omega) < 1$, lo que implica $|1 - \omega| < 1$, es decir, $0 < \omega < 2$.

T6. (Esquema) Sea $A = L + D + L^T$. Sea λ autovalor de $G_{GS} = -(D + L)^{-1}L^T$, con autovector z . Se puede mostrar que $\lambda \neq 1$. Multiplicando por z^* y sumando con su conjugado, se obtiene:

$$2\operatorname{Re}\left(\frac{1}{1 - \lambda}\right) = 1 + \frac{z^* D z}{z^* A z} > 1$$

porque A es definida positiva y D también (sus elementos diagonales son positivos). Esto implica que $|\lambda| < 1$. Luego $\rho(G_{GS}) < 1$.

T7. Para matrices tridiagonales, se puede mostrar que los autovalores de G_J son μ que satisfacen $\det(\mu D + L + U) = 0$. Los autovalores de G_{GS} son λ que satisfacen $\det(\lambda(D + L) + U) = 0$. Usando el lema de semejanza: $\det(\lambda D + \alpha \lambda L + \alpha^{-1} U) = 0$. Tomando $\alpha^2 = 1/\lambda$, se obtiene que $\lambda = \mu^2$. Luego $\rho(G_{GS}) = \rho(G_J)^2$.

T8. Considerar $A = \begin{pmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{pmatrix}$ con $1/2 \leq a < 1$. Esta matriz es simétrica definida positiva

(autovalores: $1 + 2a > 0$ y $1 - a > 0$). La matriz de Jacobi es $G_J = \begin{pmatrix} 0 & -a & -a \\ -a & 0 & -a \\ -a & -a & 0 \end{pmatrix}$. Sus

autovalores son a (doble) y $-2a$. Si $a \geq 1/2$, $\rho(G_J) = 2a \geq 1$, así que Jacobi no converge. Es un contraejemplo.

T9. Usar el ejemplo de Collatz: $A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}$. Jacobi converge (radio espectral 0), Gauss-Seidel diverge (radio espectral 2).

T10. Si $\kappa(A)$ es grande, la matriz está cerca de ser singular. Esto significa que hay una dirección en la que A aplasta el vector (valor singular pequeño). En esa dirección, la solución es muy sensible a perturbaciones. Aunque $\rho(G) < 1$, la tasa de convergencia puede ser muy lenta si los autovalores de G están cerca de 1. Además, los errores de redondeo pueden excitar la componente mal condicionada y arruinar la solución.

Soluciones Computacionales

C1. Implementación de Jacobi.

```

1 import numpy as np
2
3 def jacobi(A, b, x0, max_iter=1000, tol=1e-8):
4     """Resuelve Ax = b usando el metodo de Jacobi."""
5     n = len(b)
6     x = x0.copy()
7     D_inv = np.diag(1.0 / np.diag(A))
8     L_plus_U = A - np.diag(np.diag(A))
9     for k in range(max_iter):
10         x_nuevo = D_inv @ (b - L_plus_U @ x)
11         if np.linalg.norm(x_nuevo - x, np.inf) < tol:
12             return x_nuevo, k+1
13         x = x_nuevo
14     return x, max_iter
15
16 # Prueba
17 A = np.array([[4.0, 1.0], [1.0, 4.0]])
18 b = np.array([5.0, 5.0])
19 x0 = np.array([0.0, 0.0])
20 sol, iters = jacobi(A, b, x0)
21 print(f"Solucion: {sol}, Iteraciones: {iters}")
22 # Deberia dar [1.0, 1.0] en pocas iteraciones

```

Listing 43: Metodo de Jacobi

C2. Implementación de Gauss-Seidel.

```

1 def gauss_seidel(A, b, x0, max_iter=1000, tol=1e-8):
2     """Resuelve Ax = b usando el metodo de Gauss-Seidel."""
3     n = len(b)
4     x = x0.copy()
5     for k in range(max_iter):
6         x_nuevo = x.copy()
7         for i in range(n):
8             suma = 0.0
9             for j in range(n):
10                 if j != i:
11                     suma += A[i, j] * x_nuevo[j]
12             x_nuevo[i] = (b[i] - suma) / A[i, i]
13         if np.linalg.norm(x_nuevo - x, np.inf) < tol:
14             return x_nuevo, k+1
15         x = x_nuevo
16     return x, max_iter

```

```

17
18 # Prueba y comparacion con Jacobi
19 sol_gs, iters_gs = gauss_seidel(A, b, x0)
20 print(f"GS: Solucion: {sol_gs}, Iteraciones: {iters_gs}")

```

Listing 44: Metodo de Gauss-Seidel

C3. Radio espectral y convergencia.

```

1 A = np.array([[3.0, 1.0, 1.0], [2.0, 6.0, 1.0], [1.0, 1.0, 4.0]])
2 b = np.array([5.0, 9.0, 6.0])
3 x0 = np.zeros(3)
4
5 # Jacobi
6 D = np.diag(np.diag(A))
7 L_plus_U = A - D
8 G_J = -np.linalg.inv(D) @ L_plus_U
9 rho_J = np.max(np.abs(np.linalg.eigvals(G_J)))
10 print(f"Rho Jacobi: {rho_J:.6f}")
11
12 # Gauss-Seidel
13 L = np.tril(A, -1)
14 U = np.triu(A, 1)
15 G_GS = -np.linalg.inv(D + L) @ U
16 rho_GS = np.max(np.abs(np.linalg.eigvals(G_GS)))
17 print(f"Rho GS: {rho_GS:.6f}")
18
19 # Verificacion numerica
20 sol_j, it_j = jacobi(A, b, x0)
21 sol_gs, it_gs = gauss_seidel(A, b, x0)
22 print(f"Jacobi: {it_j} iteraciones, error: {np.linalg.norm(sol_j - np.linalg.solve(A,b))}")
23 print(f"GS: {it_gs} iteraciones, error: {np.linalg.norm(sol_gs - np.linalg.solve(A,b))}")

```

Listing 45: Calculo de radios espectrales

C4. SOR y busqueda del omega optimo.

```

1 def sor(A, b, x0, omega, max_iter=1000, tol=1e-8):
2     n = len(b)
3     x = x0.copy()
4     for k in range(max_iter):
5         x_nuevo = x.copy()
6         for i in range(n):
7             suma = 0.0
8             for j in range(n):
9                 if j != i:
10                     suma += A[i, j] * x_nuevo[j]
11             x_nuevo[i] = (1 - omega) * x[i] + omega * (b[i] - suma) / A[i, i]
12         if np.linalg.norm(x_nuevo - x, np.inf) < tol:
13             return x_nuevo, k+1
14     x = x_nuevo
15     return x, max_iter
16
17 # Busqueda del omega optimo
18 omegas = np.linspace(0.1, 1.9, 100)
19 rhos = []
20 for w in omegas:
21     G_w = np.linalg.inv(D + w * L) @ ((1 - w) * D - w * U)
22     rhos.append(np.max(np.abs(np.linalg.eigvals(G_w))))
23 plt.plot(omegas, rhos)

```

```

24 plt.xlabel('omega')
25 plt.ylabel('rho(G_omega)')
26 plt.title('Radio espectral de SOR en funcion de omega')
27 plt.grid(True)
28 plt.show()
29 # El minimo da el omega optimo aproximado
30 idx_min = np.argmin(rhos)
31 print(f"Omega optimo aproximado: {omegas[idx_min]:.4f}")

```

Listing 46: SOR y busqueda de omega

C5. Matriz de Hilbert y condicionamiento.

```

1 def hilbert(n):
2     H = np.zeros((n, n))
3     for i in range(n):
4         for j in range(n):
5             H[i, j] = 1.0 / (i + j + 1)
6     return H
7
8 ns = range(5, 11)
9 condiciones = []
10 rhos = []
11 for n in ns:
12     H = hilbert(n)
13     condiciones.append(np.linalg.cond(H, 2))
14     # Gauss-Seidel
15     D = np.diag(np.diag(H))
16     L = np.tril(H, -1)
17     U = np.triu(H, 1)
18     G_GS = -np.linalg.inv(D + L) @ U
19     rhos.append(np.max(np.abs(np.linalg.eigvals(G_GS))))
20 plt.figure()
21 plt.semilogy(ns, condiciones, 'o-', label='Condicionamiento')
22 plt.semilogy(ns, rhos, 's-', label='Rho GS')
23 plt.xlabel('n')
24 plt.ylabel('Valor')
25 plt.legend()
26 plt.grid(True)
27 plt.title('Condicionamiento y radio espectral para Hilbert')
28 plt.show()

```

Listing 47: Hilbert y condicionamiento

C6. Sistema mal condicionado.

```

1 eps = 1e-6
2 A = np.array([[1.0, 2.0], [2.0, 4.0 + eps]])
3 b = np.array([1.0, 2.0 + eps])
4 x0 = np.zeros(2)
5 print(f"Condicion: {np.linalg.cond(A, 2):.2e}")
6 sol_j, it_j = jacobi(A, b, x0, max_iter=1000, tol=1e-12)
7 sol_gs, it_gs = gauss_seidel(A, b, x0, max_iter=1000, tol=1e-12)
8 print(f"Jacobi: {it_j} iteraciones, sol: {sol_j}")
9 print(f"GS: {it_gs} iteraciones, sol: {sol_gs}")
10 print(f"Exacto (numpy): {np.linalg.solve(A,b)}")

```

Listing 48: Mal condicionamiento

C7. Diagonal dominante.

```

1 n = 10
2 A = np.random.randn(n, n)

```

```

3 # Hacerla diagonal dominante
4 A = A + (np.sum(np.abs(A), axis=1) + 0.1) * np.eye(n)
5 b = np.random.randn(n)
6 x0 = np.zeros(n)
7 sol_j, it_j = jacobi(A, b, x0)
8 sol_gs, it_gs = gauss_seidel(A, b, x0)
9 print(f"Jacobi: {it_j} iteraciones")
10 print(f"GS: {it_gs} iteraciones")

```

Listing 49: Generación y prueba de diagonal dominante

C8. No diagonal dominante pero SPD.

```

1 # Construir una matriz SPD (ej:  $A = M^T M + I$ )
2 M = np.random.randn(10, 10)
3 A = M.T @ M + 0.1 * np.eye(10)
4 # Verificar si es diagonal dominante (podría serlo, pero no importa)
5 # Aplicar Jacobi y GS
6 b = np.random.randn(10)
7 x0 = np.zeros(10)
8 try:
9     sol_j, it_j = jacobi(A, b, x0)
10    print(f"Jacobi: {it_j} iteraciones")
11 except:
12    print("Jacobi no convergio")
13 try:
14    sol_gs, it_gs = gauss_seidel(A, b, x0)
15    print(f"GS: {it_gs} iteraciones")
16 except:
17    print("GS no convergio")

```

Listing 50: SPD no diagonal dominante

C9. Convergencia asintótica.

```

1 n = 3
2 A = np.array([[2.0, -1.0, 0.0], [-1.0, 2.0, -1.0], [0.0, -1.0, 2.0]])
3 D = np.diag(np.diag(A))
4 L = np.tril(A, -1)
5 U = np.triu(A, 1)
6
7 G_J = -np.linalg.inv(D) @ (L + U)
8 G_GS = -np.linalg.inv(D + L) @ U
9
10 rho_J = np.max(np.abs(np.linalg.eigvals(G_J)))
11 rho_GS = np.max(np.abs(np.linalg.eigvals(G_GS)))
12
13 k_max = 50
14 normas_J = []
15 normas_GS = []
16 for k in range(k_max):
17     Gk_J = np.linalg.matrix_power(G_J, k)
18     Gk_GS = np.linalg.matrix_power(G_GS, k)
19     normas_J.append(np.linalg.norm(Gk_J, 2))
20     normas_GS.append(np.linalg.norm(Gk_GS, 2))
21
22 plt.figure()
23 plt.semilogy(range(k_max), normas_J, label='Jacobi')
24 plt.semilogy(range(k_max), normas_GS, label='GS')
25 plt.semilogy(range(k_max), [rho_J**k for k in range(k_max)], '--', label='rho_J^k')
26 plt.semilogy(range(k_max), [rho_GS**k for k in range(k_max)], '--', label='rho_GS^k')

```

```

27 plt.xlabel('k')
28 plt.ylabel('||G^k||_2')
29 plt.legend()
30 plt.grid(True)
31 plt.show()

```

Listing 51: Decaimiento de la norma de G^k **C10. SOR optimo para tridiagonal.**

```

1 def tridiagonal(n, diag=2, off=-1):
2     A = np.zeros((n, n))
3     for i in range(n):
4         A[i, i] = diag
5         if i > 0: A[i, i-1] = off
6         if i < n-1: A[i, i+1] = off
7     return A
8
9 def jacobi(A, b, x0, max_iter=1000, tol=1e-8):
10    x = x0.copy()
11    D_inv = np.diag(1.0 / np.diag(A))
12    L_plus_U = A - np.diag(np.diag(A))
13    for k in range(max_iter):
14        x_nuevo = D_inv @ (b - L_plus_U @ x)
15        if np.linalg.norm(x_nuevo - x, np.inf) < tol:
16            return x_nuevo, k+1
17    x = x_nuevo
18    return x, max_iter
19
20 def gauss_seidel(A, b, x0, max_iter=1000, tol=1e-8):
21    n = len(b)
22    x = x0.copy()
23    for k in range(max_iter):
24        x_nuevo = x.copy()
25        for i in range(n):
26            suma = 0.0
27            for j in range(n):
28                if j != i:
29                    suma += A[i, j] * x_nuevo[j]
30            x_nuevo[i] = (b[i] - suma) / A[i, i]
31        if np.linalg.norm(x_nuevo - x, np.inf) < tol:
32            return x_nuevo, k+1
33    x = x_nuevo
34    return x, max_iter
35
36 def sor(A, b, x0, omega, max_iter=1000, tol=1e-8):
37    n = len(b)
38    x = x0.copy()
39    for k in range(max_iter):
40        x_nuevo = x.copy()
41        for i in range(n):
42            suma = 0.0
43            for j in range(n):
44                if j != i:
45                    suma += A[i, j] * x_nuevo[j]
46            x_nuevo[i] = (1 - omega) * x[i] + omega * (b[i] - suma) / A[i, i]
47        if np.linalg.norm(x_nuevo - x, np.inf) < tol:
48            return x_nuevo, k+1
49    x = x_nuevo
50    return x, max_iter
51
52 for n in [10, 20, 50]:

```

```

53 A = tridiagonal(n)
54 b = np.ones(n)
55 x0 = np.zeros(n)
56 omega_opt = 2.0 / (1.0 + np.sin(np.pi / (n + 1)))
57 sol_j, it_j = jacobi(A, b, x0)
58 sol_gs, it_gs = gauss_seidel(A, b, x0)
59 sol_sor, it_sor = sor(A, b, x0, omega_opt)
60 print(f"n={n}: Jacobi={it_j}, GS={it_gs}, SOR={it_sor}")

```

Listing 52: SOR optimo para tridiagonal

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por los metodos iterativos para sistemas lineales. Viste que no eran tan complicados: solo hay que despejar, iterar, y rezar para que el radio espectral te acompañe. La posta es que estos metodos son la base de la simulacion numerica moderna. Cuando resuelvas ecuaciones diferenciales parciales en 2D o 3D, las matrices son tan grandes que no te queda otra que usar iterativos. Entender Jacobi, Gauss-Seidel y SOR te va a salvar de muchos dolores de cabeza.

Recordá siempre: el radio espectral es tu amigo. Si es menor que 1, estas joya. Si es cercano a 1, la convergencia va a ser lenta, y quizás necesites un preconditionador o un metodo mas avanzado (como gradiente conjugado). Y si es mayor que 1, salí corriendo y busca otro metodo. No te olvides de que la eleccion del metodo depende del problema. Si tu matriz es diagonal dominante, Jacobi o Gauss-Seidel van a andar bien. Si es SPD, Gauss-Seidel es seguro. Si quieres acelerar, proba con SOR y busca el omega optimo. Pero si la matriz es muy mal condicionada, vas a necesitar algo mas potente.

Ahora, a practicar con los ejercicios. No te frustres si al principio no convergen o si los parametros no son los optimos. A todos nos paso. La practica hace al maestro. ¡Dale para adelante!

Metodos Iterativos para Sistemas Lineales

De la teoria a la practica: resolviendo sistemas grandes sin volverte loco.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a desmenuzar los metodos iterativos para resolver sistemas lineales. Si llegaste hasta aca, ya sabes que resolver un sistema $Ax = b$ con eliminacion gaussiana te sale $O(n^3)$ operaciones, y para matrices grandes (pensá en $n = 10^4$ o mas) eso es inviable. Te juro que cuando curse calculo numerico por primera vez, me acuerdo de haber corrido un codigo en Matlab con una matriz de 1000x1000 y la compu se quedó pensando una eternidad. Despues entendí que para matrices ralas (con muchos ceros) los metodos iterativos son la posta. Y aca vamos a ver exactamente eso: como construir sucesiones $x^{(k)}$ que converjan a la solucion sin tener que factorizar una matriz enorme.

En este capitulo vamos a cubrir:

- El metodo de Jacobi: el mas simple de todos, pero que a veces no converge.
- El metodo de Gauss-Seidel: una mejora sobre Jacobi que usa la informacion recién calculada.
- La teoria de convergencia: cuando funciona y cuando no, con el concepto de radio espectral.
- Ejemplos concretos y graficos para que veas como se comportan estos metodos en la practica.

La idea es que vos puedas implementar estos metodos en Python, entender por que funcionan (o no), y saber elegir cual usar segun el problema. Asi que agarra un cafe, ponete comodo, y vamos a darle.

¿Por que metodos iterativos?

Antes de meterle a los algoritmos, hagamos una pausa y pensemos en el problema. Tenemos un sistema $Ax = b$, con $A \in \mathbb{R}^{n \times n}$ y $b \in \mathbb{R}^n$. La solucion exacta es $x = A^{-1}b$, pero calcular la inversa es carisimo. La eliminacion gaussiana (factorizacion LU) nos da la solucion en $O(n^3)$ operaciones. Para $n = 1000$, eso son mil millones de operaciones. Para $n = 10^5$, olvidate.

Pero hay un monton de problemas en fisica e ingenieria donde la matriz A es **rala** (sparse): tiene muy pocos elementos no nulos por fila. Por ejemplo, cuando discretizas ecuaciones diferenciales parciales con diferencias finitas, cada ecuacion solo involucra a la variable y a sus vecinos. En esos casos, los metodos iterativos aprovechan la estructura rala para hacer cada iteracion en $O(n)$ operaciones, y si convergen rapido, te ahorras una bocha de tiempo.

Reflexion: La primera vez que vi esto, pense "¿por que no usar eliminacion gaussiana y listo?". Pero despues de intentar resolver un sistema de 10000 ecuaciones con una matriz tridiagonal, me di cuenta de que los metodos iterativos no son una opcion, son una necesidad. Ademas, tienen la ventaja de que puedes detenerte cuando tengas una aproximacion suficientemente buena, sin tener que llegar al final exacto.

El metodo de Jacobi

El metodo de Jacobi es el mas intuitivo de todos. La idea es simple: de la i -esima ecuacion, despejas la variable x_i suponiendo que conoces todas las otras variables. Despues, usas las aproximaciones viejas para calcular las nuevas.

Idea fundamental

Supongamos que tenemos el sistema:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Si $a_{ii} \neq 0$ para todo i (y si no, reordenamos), podemos despejar x_i de la ecuacion i :

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}x_j \right)$$

Entonces, para obtener la iteracion $k+1$, usamos los valores de la iteracion k en el lado derecho:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

Traduccion Matematica de la formula de Jacobi La formula $x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)})$ se lee en criollo: "para calcular la nueva aproximacion de la variable x_i , agarra el termino independiente b_i , restale la contribucion de todas las otras variables usando sus valores viejos, y dividi por el coeficiente a_{ii} ". Pensalo como si estuvieras resolviendo cada ecuacion por separado, como si las otras variables fueran constantes conocidas. En la practica, lo que haces es actualizar cada variable usando la informacion disponible en el paso anterior.

Forma matricial

Para analizar la convergencia, conviene escribir Jacobi en forma matricial. Descomponemos A en tres partes:

$$A = D + L + U$$

donde:

- D es la matriz diagonal con los elementos a_{ii} .
- L es la matriz triangular inferior estricta (ceros en la diagonal).
- U es la matriz triangular superior estricta (ceros en la diagonal).

Entonces, la ecuacion $Ax = b$ se escribe como:

$$(D + L + U)x = b \quad \Rightarrow \quad Dx = -(L + U)x + b$$

Si multiplicamos por D^{-1} (que existe porque asumimos $a_{ii} \neq 0$):

$$x = -D^{-1}(L + U)x + D^{-1}b$$

La iteracion de Jacobi es entonces:

$$x^{(k+1)} = -D^{-1}(L + U)x^{(k)} + D^{-1}b$$

o, mas compacto:

$$x^{(k+1)} = B_J x^{(k)} + c$$

donde $B_J = -D^{-1}(L + U)$ es la **matriz de iteracion de Jacobi** y $c = D^{-1}b$.

Ejemplo concreto: 2x2

Para ver como funciona, agarremos el ejemplo clasico de Durán:

$$\begin{cases} 4x_1 + x_2 = 5 \\ x_1 + 4x_2 = 5 \end{cases}$$

Aca $A = \begin{pmatrix} 4 & 1 \\ 1 & 4 \end{pmatrix}$, $b = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$. La solucion exacta es $x = (1, 1)^T$.

Despejamos:

$$x_1^{(k+1)} = \frac{5 - x_2^{(k)}}{4}, \quad x_2^{(k+1)} = \frac{5 - x_1^{(k)}}{4}$$

Empezamos con $x^{(0)} = (0, 0)$:

$$\begin{aligned} x_1^{(1)} &= \frac{5 - 0}{4} = 1,25, & x_2^{(1)} &= \frac{5 - 0}{4} = 1,25 \\ x_1^{(2)} &= \frac{5 - 1,25}{4} = 0,9375, & x_2^{(2)} &= \frac{5 - 1,25}{4} = 0,9375 \\ x_1^{(3)} &= \frac{5 - 0,9375}{4} = 1,015625, & x_2^{(3)} &= \frac{5 - 0,9375}{4} = 1,015625 \end{aligned}$$

Ves como se va acercando a (1,1). En este caso, la matriz de iteracion es:

$$B_J = -D^{-1}(L + U) = \begin{pmatrix} 0 & -1/4 \\ -1/4 & 0 \end{pmatrix}$$

y su norma infinito es $1/4 < 1$, asi que converge para cualquier dato inicial.

Implementacion en Python

Ahora vamos a la practica. Te voy a mostrar como implementar Jacobi en Python, y ademas vamos a graficar la convergencia para que veas como se comporta.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def jacobi(A, b, x0, max_iter=100, tol=1e-6):
5     """
6     Resuelve Ax = b usando el metodo de Jacobi.
7
```

```

8 Parametros:
9 A: matriz (n x n)
10 b: vector (n)
11 x0: estimacion inicial
12 max_iter: maximo de iteraciones
13 tol: tolerancia para detenerse
14
15 Retorna:
16 x: solucion aproximada
17 historial: lista de errores relativos
18 """
19 n = len(b)
20 x = x0.copy()
21 D = np.diag(A)
22 # Para Jacobi, necesitamos la inversa de la diagonal
23 D_inv = 1.0 / D
24 historial = []
25
26 for k in range(max_iter):
27     x_new = np.zeros(n)
28     for i in range(n):
29         suma = np.dot(A[i, :], x) - A[i, i] * x[i]
30         x_new[i] = (b[i] - suma) / A[i, i]
31
32     # Calculamos el error relativo (norma infinita)
33     error = np.linalg.norm(x_new - x, np.inf) / (np.linalg.norm(x_new, np.
34 inf) + 1e-12)
35     historial.append(error)
36
37     x = x_new
38
39     if error < tol:
40         break
41
42     return x, historial
43
44 # Ejemplo del libro
45 A = np.array([[4.0, 1.0], [1.0, 4.0]])
46 b = np.array([5.0, 5.0])
47 x0 = np.array([0.0, 0.0])
48
49 x_sol, hist = jacobi(A, b, x0, max_iter=20)
50
51 print("Solucion aproximada:", x_sol)
52 print("Solucion exacta: (1, 1)")
53 print("Iteraciones realizadas:", len(hist))
54
55 # Graficamos la convergencia
56 plt.figure(figsize=(10, 6))
57 plt.semilogy(hist, 'o-', linewidth=2, markersize=8)
58 plt.xlabel('Iteracion k', fontsize=14)
59 plt.ylabel('Error relativo (norma infinito)', fontsize=14)
60 plt.title('Convergencia del metodo de Jacobi para el sistema 2x2', fontsize=16)
61 plt.grid(True, alpha=0.3)
62 plt.show()

```

Listing 53: Metodo de Jacobi para un sistema 2x2 con grafico de convergencia

Si corres este codigo, vas a ver que el error decrece mas o menos linealmente en escala logaritmica, lo que indica convergencia lineal. La tasa de convergencia esta determinada por el radio espectral de B_J , que en este caso es $1/4$.

Cuando falla Jacobi: el problema de la convergencia

No todas las matrices son aptas para Jacobi. La condicion suficiente (pero no necesaria) para que converja es que la matriz sea **estrictamente diagonal dominante**:

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i$$

Si se cumple esto, entonces $\|B_J\|_\infty < 1$ y el metodo converge para cualquier $x^{(0)}$. Pero hay matrices que no son diagonal dominantes y aun asi Jacobi converge, y otras que no.

El metodo de Gauss-Seidel

La idea de Gauss-Seidel es simple: cuando calculas $x_i^{(k+1)}$, ya tenes disponibles las componentes $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ que son mas nuevas que $x_1^{(k)}, \dots, x_{i-1}^{(k)}$. Entonces, **usalas** en lugar de las viejas.

Formula de iteracion

Para $i = 1, \dots, n$:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

Traduccion Matematica de Gauss-Seidel La diferencia con Jacobi es que en la sumatoria de $j = 1$ a $i - 1$ (los terminos que ya calculaste en la iteracion actual) usas los valores recién actualizados $x_j^{(k+1)}$, mientras que en la sumatoria de $j = i + 1$ a n (los que todavia no calculaste) usas los viejos $x_j^{(k)}$. Es como si fueras resolviendo el sistema fila por fila, actualizando cada variable apenas la calculas.

Forma matricial

Con la misma descomposicion $A = D + L + U$, la iteracion de Gauss-Seidel se escribe como:

$$(D + L)x^{(k+1)} = -Ux^{(k)} + b$$

y por lo tanto:

$$x^{(k+1)} = -(D + L)^{-1}Ux^{(k)} + (D + L)^{-1}b$$

La matriz de iteracion es $B_{GS} = -(D + L)^{-1}U$.

Ejemplo: el mismo sistema 2x2

Aplicando Gauss-Seidel al ejemplo anterior:

$$x_1^{(k+1)} = \frac{5 - x_2^{(k)}}{4}, \quad x_2^{(k+1)} = \frac{5 - x_1^{(k+1)}}{4}$$

Empezando con $x^{(0)} = (0, 0)$:

$$\begin{aligned}
 x_1^{(1)} &= \frac{5-0}{4} = 1,25, & x_2^{(1)} &= \frac{5-1,25}{4} = 0,9375 \\
 x_1^{(2)} &= \frac{5-0,9375}{4} = 1,015625, & x_2^{(2)} &= \frac{5-1,015625}{4} = 0,99609375 \\
 x_1^{(3)} &= \frac{5-0,99609375}{4} = 1,0009765625, & x_2^{(3)} &= \frac{5-1,0009765625}{4} = 0,999755859375
 \end{aligned}$$

Fijate que converge mas rapido que Jacobi. En este caso particular, la matriz de iteracion de Gauss-Seidel es:

$$B_{GS} = \begin{pmatrix} 0 & -1/4 \\ 0 & 1/16 \end{pmatrix}$$

cuyo radio espectral es $1/16$, mientras que el de Jacobi es $1/4$. ¡Gauss-Seidel converge cuatro veces mas rapido!

Implementacion en Python

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gauss_seidel(A, b, x0, max_iter=100, tol=1e-6):
5     """
6     Resuelve Ax = b usando el metodo de Gauss-Seidel.
7     """
8     n = len(b)
9     x = x0.copy()
10    historial = []
11
12    for k in range(max_iter):
13        x_old = x.copy()
14        for i in range(n):
15            # Suma de los terminos ya actualizados (j < i) y los viejos (j > i)
16            suma = np.dot(A[i, :i], x[:i]) + np.dot(A[i, i+1:], x_old[i+1:])
17            x[i] = (b[i] - suma) / A[i, i]
18
19        error = np.linalg.norm(x - x_old, np.inf) / (np.linalg.norm(x, np.inf) +
20        1e-12)
21        historial.append(error)
22
23        if error < tol:
24            break
25
26    return x, historial
27
28 # Probamos con el mismo sistema
29 A = np.array([[4.0, 1.0], [1.0, 4.0]])
30 b = np.array([5.0, 5.0])
31 x0 = np.array([0.0, 0.0])
32
33 x_jac, hist_jac = jacobi(A, b, x0, max_iter=20)
34 x_gs, hist_gs = gauss_seidel(A, b, x0, max_iter=20)
35
36 print("Jacobi:", x_jac)
37 print("Gauss-Seidel:", x_gs)
38
39 # Graficamos la comparacion

```

```

39 plt.figure(figsize=(10, 6))
40 plt.semilogy(hist_jac, 'o-', label='Jacobi', linewidth=2, markersize=8)
41 plt.semilogy(hist_gs, 's-', label='Gauss-Seidel', linewidth=2, markersize=8)
42 plt.xlabel('Iteracion k', fontsize=14)
43 plt.ylabel('Error relativo (norma infinito)', fontsize=14)
44 plt.title('Comparacion: Jacobi vs Gauss-Seidel', fontsize=16)
45 plt.legend(fontsize=12)
46 plt.grid(True, alpha=0.3)
47 plt.show()

```

Listing 54: Metodo de Gauss-Seidel para sistema 2x2 con comparacion con Jacobi

Teoria de convergencia: el radio espectral

Ahora viene la parte mas importante: ¿cuando converge un metodo iterativo? La respuesta esta en el **radio espectral** de la matriz de iteracion.

El error y la matriz de iteracion

Supongamos que tenemos un metodo iterativo de la forma:

$$x^{(k+1)} = Bx^{(k)} + c$$

y que la solucion exacta x^* satisface $x^* = Bx^* + c$. Definimos el error en el paso k como $e^{(k)} = x^{(k)} - x^*$. Restando las dos ecuaciones:

$$e^{(k+1)} = Be^{(k)}$$

Entonces, iterando:

$$e^{(k)} = B^k e^{(0)}$$

Por lo tanto, el metodo converge para cualquier $e^{(0)}$ si y solo si $B^k \rightarrow 0$ (la matriz cero) cuando $k \rightarrow \infty$.

El radio espectral y la convergencia

Definimos el **radio espectral** de una matriz B como:

$$\rho(B) = \max\{|\lambda| : \lambda \text{ es autovalor de } B\}$$

El resultado fundamental es:

$$B^k \rightarrow 0 \iff \rho(B) < 1$$

Traduccion Matematica del radio espectral El radio espectral $\rho(B)$ es el mayor de los modulos de los autovalores de B . Si todos los autovalores tienen modulo menor que 1, entonces al elevar B a potencias cada vez mas grandes, la matriz se achica y tiende a cero. Es como si tuvieras una maquina que en cada paso multiplica el error por un numero menor que 1 en modulo; despues de muchos pasos, el error se hace chiquito. Si algun autovalor tiene modulo mayor o igual a 1, el error no se va a cero.

Convergencia de Jacobi y Gauss-Seidel

Para Jacobi: $B_J = -D^{-1}(L + U)$. Converge $\iff \rho(B_J) < 1$.

Para Gauss-Seidel: $B_{GS} = -(D + L)^{-1}U$. Converge $\iff \rho(B_{GS}) < 1$.

Condiciones suficientes para convergencia

Hay varias condiciones que garantizan convergencia sin tener que calcular el radio espectral:

1. **Diagonal dominancia estricta:** Si $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ para todo i , entonces $\|B_J\|_\infty < 1$ y $\rho(B_{GS}) < 1$. Ambos metodos convergen.
2. **Matriz simetrica definida positiva:** Si A es simetrica y definida positiva ($x^T A x > 0$ para todo $x \neq 0$), entonces Gauss-Seidel converge siempre. Jacobi, en cambio, puede no converger aunque A sea SPD (eso lo vimos en el ejemplo con $a = 0,8$ en Durán).
3. **Matriz tridiagonal:** Para matrices tridiagonales, hay una relacion especial: $\rho(B_{GS}) = \rho(B_J)^2$. Esto significa que si converge Jacobi, Gauss-Seidel converge mas rapido (porque el radio espectral es el cuadrado).

Grafico: comparacion de convergencia para diferentes matrices

Vamos a hacer un experimento numerico para ver como varía el radio espectral de Jacobi y Gauss-Seidel para una familia de matrices.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def radio_espectral_jacobi_gs(A):
5     """Calcula los radios espectrales de Jacobi y Gauss-Seidel"""
6     n = A.shape[0]
7     D = np.diag(np.diag(A))
8     L = np.tril(A, -1)
9     U = np.triu(A, 1)
10
11     # Jacobi
12     B_J = -np.linalg.inv(D) @ (L + U)
13     rho_J = max(abs(np.linalg.eigvals(B_J)))
14
15     # Gauss-Seidel
16     B_GS = -np.linalg.inv(D + L) @ U
17     rho_GS = max(abs(np.linalg.eigvals(B_GS)))
18
19     return rho_J, rho_GS
20
21 # Caso 1: matriz diagonal dominante
22 A1 = np.array([[4, 1, 1], [1, 4, 1], [1, 1, 4]])
23 rho_J1, rho_GS1 = radio_espectral_jacobi_gs(A1)
24
25 # Caso 2: matriz SPD pero no diagonal dominante (la del ejemplo con a=0.8)
26 A2 = np.array([[1, 0.8, 0.8], [0.8, 1, 0.8], [0.8, 0.8, 1]])
27 rho_J2, rho_GS2 = radio_espectral_jacobi_gs(A2)
28
29 # Caso 3: matriz tridiagonal
30 A3 = np.array([[4, 1, 0], [1, 4, 1], [0, 1, 4]])
31 rho_J3, rho_GS3 = radio_espectral_jacobi_gs(A3)
32
33 print("Caso 1 (diagonal dominante):")

```

```

34 print(f"   Jacobi: rho = {rho_J1:.4f}")
35 print(f"   Gauss-Seidel: rho = {rho_GS1:.4f}")
36 print("\nCaso 2 (SPD, no diagonal dominante):")
37 print(f"   Jacobi: rho = {rho_J2:.4f}")
38 print(f"   Gauss-Seidel: rho = {rho_GS2:.4f}")
39 print("\nCaso 3 (tridiagonal):")
40 print(f"   Jacobi: rho = {rho_J3:.4f}")
41 print(f"   Gauss-Seidel: rho = {rho_GS3:.4f}")
42
43 # Graficamos los radios espectrales
44 metodos = ['Jacobi', 'Gauss-Seidel']
45 rhos = np.array([[rho_J1, rho_GS1], [rho_J2, rho_GS2], [rho_J3, rho_GS3]])
46
47 x = np.arange(len(metodos))
48 width = 0.25
49
50 fig, ax = plt.subplots(figsize=(10, 6))
51 ax.bar(x - width, rhos[0, :], width, label='Diagonal dominante', color='blue',
52       alpha=0.7)
52 ax.bar(x, rhos[1, :], width, label='SPD no DD', color='orange', alpha=0.7)
53 ax.bar(x + width, rhos[2, :], width, label='Tridiagonal', color='green', alpha
54       =0.7)
55
56 ax.set_xticks(x)
57 ax.set_xticklabels(metodos)
58 ax.set_ylabel('Radio espectral rho', fontsize=12)
59 ax.set_title('Comparacion de radios espectrales para diferentes matrices',
60             fontsize=14)
61 ax.axhline(y=1, color='red', linestyle='--', label='rho = 1 (limite de
62             convergencia)')
63 ax.legend()
64 ax.grid(True, alpha=0.3)
65 plt.show()

```

Listing 55: Calculo del radio espectral para diferentes matrices

Ejemplos interesantes y contraejemplos

Vamos a ver algunos casos particulares que muestran el comportamiento de estos metodos.

Matriz SPD donde Jacobi no converge

Recorda el ejemplo de Durán:

$$A = \begin{pmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{pmatrix}, \quad 1/2 \leq a < 1$$

Para $a = 0,8$, la matriz es SPD, pero Jacobi no converge porque $\rho(B_J) = 2a = 1,6 > 1$. Gauss-Seidel, en cambio, converge. Esto muestra que ser SPD no alcanza para Jacobi.

Matriz donde Jacobi converge y Gauss-Seidel no

El ejemplo de Collatz (1942):

$$A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}$$

Para esta matriz, B_J es nilpotente ($B_J^3 = 0$), así que $\rho(B_J) = 0$ y Jacobi converge en tres pasos. Pero $\rho(B_{GS}) = 2 > 1$, así que Gauss-Seidel diverge. Esto es raro, pero pasa.

Matrices tridiagonales

Para matrices tridiagonales, se cumple la relación:

$$\rho(B_{GS}) = \rho(B_J)^2$$

Esto significa que si Jacobi converge, Gauss-Seidel converge más rápido (porque el radio espectral es el cuadrado). Si Jacobi no converge, Gauss-Seidel tampoco.

Grafico de la velocidad de convergencia para diferentes radios espectrales

Vamos a simular como se comporta el error para diferentes radios espectrales.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def simular_convergencia(rho, k_max=50):
5     """
6     Simula la convergencia del error para un radio espectral rho.
7     Asume que el error se comporta como ||e_k|| <= C * rho^k.
8     """
9     k = np.arange(k_max)
10    error = rho**k
11    return k, error
12
13 # Diferentes valores de rho
14 rhos = [0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 0.99]
15 plt.figure(figsize=(10, 6))
16
17 for rho in rhos:
18     k, error = simular_convergencia(rho)
19     plt.semilogy(k, error, label=f'rho = {rho:.2f}', linewidth=2)
20
21 plt.xlabel('Iteracion k', fontsize=14)
22 plt.ylabel('Error relativo estimado', fontsize=14)
23 plt.title('Efecto del radio espectral en la velocidad de convergencia', fontsize=16)
24 plt.legend(fontsize=12)
25 plt.grid(True, alpha=0.3)
26 plt.show()

```

Listing 56: Efecto del radio espectral en la convergencia

El metodo SOR (Successive Over-Relaxation)

El metodo SOR es una extensión de Gauss-Seidel que introduce un parámetro ω para acelerar la convergencia. La idea es tomar un "promedio" entre el valor viejo y el nuevo:

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

Si $\omega = 1$, recuperamos Gauss-Seidel. Si $\omega < 1$, es una sub-relajacion. Si $\omega > 1$, es una sobre-relajacion (de ahí el nombre).

Condicion necesaria para convergencia

Se puede demostrar que si $\rho(B_\omega) < 1$, entonces $0 < \omega < 2$. Esta condicion es necesaria (Kahan, 1958). Si A es SPD, tambien es suficiente.

Parametro optimo

Para matrices SPD, el parametro optimo esta relacionado con el radio espectral de Jacobi:

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(B_J)^2}}$$

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Demostracion de la formula de Jacobi:** Partiendo del sistema $Ax = b$, demuestra que la iteracion de Jacobi se puede escribir como $x^{(k+1)} = B_J x^{(k)} + c$, especificando B_J y c .
- T2. Convergencia y radio espectral:** Demuestra que si $\rho(B) < 1$, entonces $B^k \rightarrow 0$ para todo vector inicial. Usa la forma de Jordan o la existencia de una norma subordinada.
- T3. Diagonal dominancia estricta:** Prueba que si A es estrictamente diagonal dominante, entonces $\|B_J\|_\infty < 1$ y por lo tanto Jacobi converge.
- T4. Gauss-Seidel para SPD:** Demuestra que si A es simetrica definida positiva, entonces $\rho(B_{GS}) < 1$. (Sugerencia: usa el argumento con el cociente de Rayleigh que vimos en clase).
- T5. Relacion entre Jacobi y Gauss-Seidel para tridiagonales:** Demuestra que para matrices tridiagonales, $\rho(B_{GS}) = \rho(B_J)^2$. ¿Que implica esto sobre la velocidad de convergencia?
- T6. Condicion necesaria para SOR:** Demuestra que si $\rho(B_\omega) < 1$, entonces $|\omega - 1| < 1$. En particular, para $\omega \in \mathbb{R}$, se cumple $0 < \omega < 2$.
- T7. Ejemplo donde Jacobi converge y Gauss-Seidel no:** Analiza la matriz del ejemplo de Collatz y muestra explicitamente que B_J es nilpotente pero $\rho(B_{GS}) > 1$.
- T8. Matriz SPD donde Jacobi no converge:** Para la matriz $A = \begin{pmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{pmatrix}$ con $1/2 \leq a < 1$, calcula $\rho(B_J)$ y muestra que es ≥ 1 .
- T9. Estimacion del error:** Si $\rho(B) = \rho < 1$, prueba que $\|e^{(k)}\| \leq C \rho^k \|e^{(0)}\|$ para alguna constante C . ¿Que pasa con C si la matriz no es diagonalizable?
- T10. Parametro optimo para SOR:** Para una matriz SPD con $\rho(B_J) = \mu$, demuestra que el parametro optimo de SOR es $\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \mu^2}}$.

Parte Computacional (10 ejercicios)

- C1. Implementacion basica de Jacobi:** Escribe una funcion en Python que implemente el metodo de Jacobi para una matriz general. Incluye un maximo de iteraciones y una tolerancia. Prueba con el sistema 2x2 del ejemplo.
- C2. Implementacion de Gauss-Seidel:** Escribe una funcion en Python que implemente Gauss-Seidel. Prueba con el mismo sistema 2x2 y compara los errores con Jacobi.
- C3. Grafico de convergencia:** Para una matriz A de tamaño $n = 10$ con elementos $A_{ii} = 4$ y $A_{ij} = 1$ para $j \neq i$, grafica la convergencia de Jacobi y Gauss-Seidel en el mismo grafico. Explica por que Gauss-Seidel converge mas rapido.
- C4. Matriz de Hilbert:** La matriz de Hilbert H_n con $H_{ij} = 1/(i + j - 1)$ esta muy mal condicionada. Para $n = 5$, resuelve $H_n x = b$ con $b = H_n \cdot \mathbf{1}$ (la solucion exacta es $\mathbf{1}$). Usa Jacobi y Gauss-Seidel y grafica el error. ¿Que observas? ¿Por que?
- C5. Matriz SPD no diagonal dominante:** Para la matriz $A = \begin{pmatrix} 1 & 0,8 & 0,8 \\ 0,8 & 1 & 0,8 \\ 0,8 & 0,8 & 1 \end{pmatrix}$, verifica numericamente que Jacobi no converge y Gauss-Seidel si. Grafica el error para ambos metodos.
- C6. Matriz tridiagonal:** Genera una matriz tridiagonal $A \in \mathbb{R}^{50 \times 50}$ con $A_{ii} = 4$, $A_{i,i+1} = A_{i+1,i} = 1$. Resuelve $Ax = b$ con $b = A \cdot \mathbf{1}$. Calcula los radios espectrales de Jacobi y Gauss-Seidel y verifica la relacion $\rho(B_{GS}) = \rho(B_J)^2$.
- C7. SOR para diferentes omega:** Para la matriz tridiagonal del ejercicio anterior, implementa SOR y prueba con $\omega = 0,5, 0,8, 1, 1,2, 1,5, 1,8$. Grafica el numero de iteraciones necesarias para alcanzar una tolerancia de 10^{-6} en funcion de ω . Encuentra el ω optimo.
- C8. Comparacion de costos:** Para una matriz $A \in \mathbb{R}^{n \times n}$ rala con m elementos no nulos por fila, estima el costo de una iteracion de Jacobi en terminos de n y m . Compara con el costo de una factorizacion LU ($O(n^3)$). ¿Para que valores de n y m conviene usar Jacobi?
- C9. Metodo de Jacobi por bloques:** Implementa el metodo de Jacobi por bloques para el sistema del Ejercicio 11 de Durán (pagina 62). Usa el bloque de 2×2 superior izquierdo y el 1×1 inferior derecho. Grafica la convergencia.
- C10. Estabilidad y condicionamiento:** Considera la matriz $A = \begin{pmatrix} 1 & 2 \\ 2 & 4 + \epsilon \end{pmatrix}$ con ϵ pequeno. Resuelve el sistema $Ax = b$ con b tal que la solucion exacta es $(1, 1)^T$. Usa Gauss-Seidel y observa la convergencia. ¿Que relacion tiene con el numero de condicion de A ?

Soluciones

Soluciones Teoricas

T1. Partimos de $Ax = b$. Escribimos $A = D + L + U$. Entonces:

$$(D + L + U)x = b \Rightarrow Dx = -(L + U)x + b$$

Multiplicando por D^{-1} :

$$x = -D^{-1}(L + U)x + D^{-1}b$$

Luego, la iteracion de Jacobi es $x^{(k+1)} = -D^{-1}(L+U)x^{(k)} + D^{-1}b$. Entonces $B_J = -D^{-1}(L+U)$ y $c = D^{-1}b$.

T2. Si $\rho(B) < 1$, por el Teorema 3.4 de Maas, existe una norma $\|\cdot\|$ tal que $\|B\| < 1$. Entonces $\|B^k\| \leq \|B\|^k \rightarrow 0$. Luego, para cualquier $x^{(0)}$, $e^{(k)} = B^k e^{(0)} \rightarrow 0$.

T3. Si A es diagonal dominante estricta, $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ para todo i . La matriz de Jacobi es $B_J = -D^{-1}(L+U)$, con elementos $b_{ij} = -a_{ij}/a_{ii}$ para $i \neq j$ y $b_{ii} = 0$. Entonces:

$$\|B_J\|_\infty = \max_i \sum_{j=1}^n |b_{ij}| = \max_i \sum_{j \neq i} \frac{|a_{ij}|}{|a_{ii}|} < 1$$

Como $\rho(B_J) \leq \|B_J\|_\infty < 1$, Jacobi converge.

T4. Ver la demostracion del Teorema 3.22 en Maas (pagina 58). La idea es que si A es SPD, entonces $z^*Az > 0$ y $z^*Dz > 0$, lo que lleva a $|\lambda| < 1$ para todo autovalor de B_{GS} .

T5. Para matrices tridiagonales, los autovalores no nulos de B_{GS} son los cuadrados de los autovalores no nulos de B_J . Entonces $\rho(B_{GS}) = \max |\lambda_{GS}| = \max |\lambda_J|^2 = \rho(B_J)^2$. Esto implica que si Jacobi converge ($\rho_J < 1$), entonces Gauss-Seidel converge mas rapido porque $\rho_{GS} = \rho_J^2 < \rho_J$.

T6. Ver el Teorema 3.23 de Kahan en Maas (pagina 59). Se demuestra que $\det(B_\omega) = (1-\omega)^n$, y como $\det(B_\omega) = \prod_i \lambda_i$, se tiene $\rho(B_\omega) \geq |1-\omega|$. Para convergencia, necesitamos $\rho(B_\omega) < 1$, asi que $|1-\omega| < 1$.

T7. Para la matriz de Collatz:

$$A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}$$

Se calcula $B_J = \begin{pmatrix} 0 & -2 & 2 \\ -1 & 0 & -1 \\ -2 & -2 & 0 \end{pmatrix}$, y $B_J^3 = 0$, asi que $\rho(B_J) = 0$. Para Gauss-Seidel, $B_{GS} =$

$\begin{pmatrix} 0 & -2 & 2 \\ 0 & 2 & -3 \\ 0 & 0 & 2 \end{pmatrix}$, cuyos autovalores son 0, 2, 2, asi que $\rho(B_{GS}) = 2 > 1$.

T8. Para la matriz $A = \begin{pmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{pmatrix}$, se calcula $B_J = \begin{pmatrix} 0 & -a & -a \\ -a & 0 & -a \\ -a & -a & 0 \end{pmatrix}$. Los autovalores son $a, a, -2a$. Si $a \geq 1/2$, entonces $\rho(B_J) = 2a \geq 1$, asi que Jacobi no converge.

T9. Si $\rho(B) = \rho < 1$, por el Teorema 3.6 de Maas, existe una constante C tal que $\|B^k\| \leq C\rho^k$. Entonces $\|e^{(k)}\| = \|B^k e^{(0)}\| \leq C\rho^k \|e^{(0)}\|$. Si B no es diagonalizable, la constante C puede crecer como k^{m-1} donde m es el tamaño del bloque de Jordan mas grande, pero aun asi el termino dominante es ρ^k .

T10. La demostracion del parametro optimo para SOR es tecnica. La idea es que para matrices SPD, el radio espectral de B_ω esta relacionado con el de B_J a traves de la ecuacion $(\rho(B_\omega) + \omega - 1)^2 = \omega^2 \rho(B_J)^2 \rho(B_\omega)$. Resolviendo para $\rho(B_\omega)$ y minimizando se obtiene $\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(B_J)^2}}$.

Soluciones Computacionales

C1. Ver el codigo de la Seccion . La funcion `jacobi` implementa el metodo.

C2. Ver el codigo de la Seccion . La funcion `gauss_seidel` implementa el metodo.

C3.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

```

3
4 def jacobi(A, b, x0, max_iter=100):
5     n = len(b)
6     x = x0.copy()
7     historial = []
8     for _ in range(max_iter):
9         x_new = np.zeros(n)
10        for i in range(n):
11            suma = np.dot(A[i, :], x) - A[i, i] * x[i]
12            x_new[i] = (b[i] - suma) / A[i, i]
13        error = np.linalg.norm(x_new - x, np.inf)
14        historial.append(error)
15        x = x_new
16    return x, historial
17
18 def gauss_seidel(A, b, x0, max_iter=100):
19     n = len(b)
20     x = x0.copy()
21     historial = []
22     for _ in range(max_iter):
23         x_old = x.copy()
24         for i in range(n):
25             suma = np.dot(A[i, :i], x[:i]) + np.dot(A[i, i+1:], x_old[i+1:])
26             x[i] = (b[i] - suma) / A[i, i]
27         error = np.linalg.norm(x - x_old, np.inf)
28         historial.append(error)
29    return x, historial
30
31 n = 10
32 A = np.eye(n) * 4 + (np.ones((n, n)) - np.eye(n))
33 b = np.ones(n) * (4 + (n-1))
34
35 x0 = np.zeros(n)
36
37 _, hist_jac = jacobi(A, b, x0, max_iter=50)
38 _, hist_gs = gauss_seidel(A, b, x0, max_iter=50)
39
40 plt.figure(figsize=(10, 6))
41 plt.semilogy(hist_jac, 'o-', label='Jacobi')
42 plt.semilogy(hist_gs, 's-', label='Gauss-Seidel')
43 plt.xlabel('Iteracion')
44 plt.ylabel('Error infinito')
45 plt.title('Convergencia para matriz con diagonal dominante (n=10)')
46 plt.legend()
47 plt.grid(True)
48 plt.show()

```

Listing 57: Comparacion de convergencia para matriz con diagonal dominante

C4.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def hilbert(n):
5     return np.array([[1/(i+j+1) for j in range(n)] for i in range(n)])
6
7 n = 5
8 H = hilbert(n)
9 b = H @ np.ones(n)
10
11 x0 = np.zeros(n)

```

```

12
13 # Jacobi (puede no converger)
14 _, hist_jac = jacobi(H, b, x0, max_iter=100)
15
16 # Gauss-Seidel
17 _, hist_gs = gauss_seidel(H, b, x0, max_iter=100)
18
19 plt.figure(figsize=(10, 6))
20 plt.semilogy(hist_jac, 'o-', label='Jacobi', color='red')
21 plt.semilogy(hist_gs, 's-', label='Gauss-Seidel', color='blue')
22 plt.xlabel('Iteracion')
23 plt.ylabel('Error infinito')
24 plt.title('Matriz de Hilbert n=5: ambos metodos convergen lentamente')
25 plt.legend()
26 plt.grid(True)
27 plt.show()

```

Listing 58: Matriz de Hilbert, resolucion con Jacobi y Gauss-Seidel

C5.

```

1 A = np.array([[1.0, 0.8, 0.8], [0.8, 1.0, 0.8], [0.8, 0.8, 1.0]])
2 b = np.array([2.6, 2.6, 2.6]) # Solucion exacta: (1,1,1)
3
4 x0 = np.zeros(3)
5
6 _, hist_jac = jacobi(A, b, x0, max_iter=100)
7 _, hist_gs = gauss_seidel(A, b, x0, max_iter=100)
8
9 plt.figure(figsize=(10, 6))
10 plt.semilogy(hist_jac, 'o-', label='Jacobi (diverge)', color='red')
11 plt.semilogy(hist_gs, 's-', label='Gauss-Seidel (converge)', color='blue')
12 plt.xlabel('Iteracion')
13 plt.ylabel('Error infinito')
14 plt.title('SPD no diagonal dominante: Jacobi diverge, GS converge')
15 plt.legend()
16 plt.grid(True)
17 plt.show()

```

Listing 59: SPD no diagonal dominante

C6.

```

1 def tridiagonal(n):
2     A = np.zeros((n, n))
3     for i in range(n):
4         A[i, i] = 4.0
5         if i > 0:
6             A[i, i-1] = 1.0
7         if i < n-1:
8             A[i, i+1] = 1.0
9     return A
10
11 n = 50
12 A = tridiagonal(n)
13 b = A @ np.ones(n)
14
15 # Calculamos los radios espectrales teoricos
16 # Para esta matriz, los autovalores de Jacobi son mu_j = cos(j*pi/(n+1)) / 2
17 # y rho_J = |cos(pi/(n+1))| / 2
18 import math
19 rho_J_teorico = abs(math.cos(math.pi / (n+1))) / 2

```

```

20 rho_GS_teorico = rho_J_teorico**2
21
22 print(f"Rho teorico de Jacobi: {rho_J_teorico:.6f}")
23 print(f"Rho teorico de Gauss-Seidel: {rho_GS_teorico:.6f}")
24
25 # Verificamos numericamente
26 def radio_espectral(B):
27     return max(abs(np.linalg.eigvals(B)))
28
29 D = np.diag(np.diag(A))
30 L = np.tril(A, -1)
31 U = np.triu(A, 1)
32
33 B_J = -np.linalg.inv(D) @ (L + U)
34 rho_J_num = radio_espectral(B_J)
35 B_GS = -np.linalg.inv(D + L) @ U
36 rho_GS_num = radio_espectral(B_GS)
37
38 print(f"Rho numerico de Jacobi: {rho_J_num:.6f}")
39 print(f"Rho numerico de Gauss-Seidel: {rho_GS_num:.6f}")
40 print(f"Relacion rho_GS / rho_J^2: {rho_GS_num / rho_J_num**2:.6f}")

```

Listing 60: Matriz tridiagonal y relacion de radios espectrales

C7.

```

1 def sor(A, b, x0, omega, max_iter=1000, tol=1e-6):
2     n = len(b)
3     x = x0.copy()
4     for k in range(max_iter):
5         x_old = x.copy()
6         for i in range(n):
7             suma1 = np.dot(A[i, :i], x[:i])
8             suma2 = np.dot(A[i, i+1:], x_old[i+1:])
9             x[i] = (1 - omega) * x_old[i] + omega * (b[i] - suma1 - suma2) / A[i, i]
10         if np.linalg.norm(x - x_old, np.inf) < tol:
11             return k+1, x
12     return max_iter, x
13
14 # Matriz tridiagonal
15 n = 50
16 A = tridiagonal(n)
17 b = A @ np.ones(n)
18 x0 = np.zeros(n)
19
20 omegas = np.linspace(0.1, 1.9, 30)
21 iteraciones = []
22
23 for omega in omegas:
24     k, _ = sor(A, b, x0, omega)
25     iteraciones.append(k)
26
27 plt.figure(figsize=(10, 6))
28 plt.plot(omegas, iteraciones, 'o-')
29 plt.xlabel('omega')
30 plt.ylabel('Iteraciones para converger')
31 plt.title('SOR: numero de iteraciones vs omega')
32 plt.grid(True)
33 plt.show()

```

Listing 61: SOR para diferentes omega

C8. El costo de una iteración de Jacobi es $O(nm)$, donde m es el número de no nulos por fila. Si la matriz es rala con $m \ll n$, el costo es $O(nm) \ll O(n^3)$. Para $n = 1000$ y $m = 5$, Jacobi cuesta $O(5000)$ por iteración, mientras que LU cuesta $O(10^9)$. Si Jacobi converge en menos de 10^5 iteraciones, es más rápido.

C9. Ver el código del Ejercicio 11 de Durán (página 62). Se implementa el método de Jacobi por bloques usando el bloque de 2×2 superior izquierdo y el 1×1 inferior derecho.

C10.

```

1 eps = 1e-6
2 A = np.array([[1.0, 2.0], [2.0, 4.0 + eps]])
3 b = np.array([3.0, 6.0 + eps]) # Solucion exacta: (1, 1)
4
5 x0 = np.zeros(2)
6 _, hist_gs = gauss_seidel(A, b, x0, max_iter=100)
7
8 # Numero de condicion
9 cond = np.linalg.cond(A, np.inf)
10 print(f"Numero de condicion: {cond:.2e}")
11
12 plt.figure(figsize=(10, 6))
13 plt.semilogy(hist_gs, 'o-')
14 plt.xlabel('Iteracion')
15 plt.ylabel('Error infinito')
16 plt.title(f'Gauss-Seidel con eps = {eps:.0e}, cond = {cond:.2e}')
17 plt.grid(True)
18 plt.show()

```

Listing 62: Efecto del condicionamiento en la convergencia

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por los métodos iterativos. Viste que no era tan terrible. Arrancamos con la idea simple de despejar variables y terminamos analizando radios espectrales y condiciones de convergencia. La clave es entender que estos métodos no son para cualquier sistema: brillan cuando la matriz es grande y rala, y cuando el radio espectral es chico. Los puntos clave para llevar:

- Jacobi es simple pero puede no converger. Gauss-Seidel es casi siempre mejor.
- La convergencia la dicta el radio espectral $\rho(B)$. Si es menor que 1, el método converge.
- La diagonal dominancia estricta es una condición suficiente (pero no necesaria) para que ambos métodos converjan.
- Para matrices SPD, Gauss-Seidel siempre converge. Jacobi, no necesariamente.
- SOR puede acelerar la convergencia si elegis bien ω , pero el parámetro óptimo depende de la matriz.

Ahora solo falta practicar, y para eso están los ejercicios. No te frustres si al principio no te salen todos; a mí me pasó lo mismo. Lo importante es que entiendas las ideas y sepas implementar los métodos. ¡Dale para adelante!

Ortogonalidad y Espacios con Producto Interno

De la geometria euclidiana a los espacios abstractos, con Python y mucha onda.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a explorar una de las ideas mas poderosas de la matematica aplicada: la ortogonalidad. Si llegaste hasta aca, seguro ya viste algo de algebra lineal, pero quizas te quedo la sensacion de que los vectores ortogonales eran solo una curiosidad geometrica. Nada mas lejos de la realidad. La ortogonalidad es la base de todo: desde la descomposicion QR hasta los polinomios ortogonales, pasando por los metodos de cuadrados minimos y las series de Fourier.

Cuando curse Analisis Numerico, recuerdo que el profesor nos dijo: "si entiendes ortogonalidad, entiendes la mitad del curso". Al principio pense que exageraba, pero despues de ver como Gram-Schmidt, QR, y las proyecciones ortogonales aparecian en todos lados, me di cuenta de que tenia razon. Asi que agarra un cafe, ponete comodo, y vamos a sumergirnos en el mundo de los espacios con producto interno.

En este capitulo vamos a cubrir:

- Que es un producto interno y por que es tan importante.
- La idea de ortogonalidad y proyeccion ortogonal.
- El proceso de Gram-Schmidt: como construir bases ortogonales a partir de cualquier base.
- La factorizacion QR y por que es mejor que LU para problemas mal condicionados.
- Espacios de Hilbert y bases ortonormales en dimension infinita (si, Fourier esta a la vuelta de la esquina).

La idea es que vos puedas implementar estos conceptos en Python, entender por que funcionan, y ver como se aplican en problemas reales. Asi que dale, metele.

¿Que es un producto interno?

Empecemos por el principio. En el mundo real, vos sabes lo que es el producto escalar de dos vectores en $\mathbb{R}^3$: $u \cdot v = u_1v_1 + u_2v_2 + u_3v_3$. Eso te da la longitud de un vector ($\|u\| = \sqrt{u \cdot u}$) y el angulo entre dos vectores ($\cos \theta = \frac{u \cdot v}{\|u\|\|v\|}$). Pero, ¿y si trabajamos con funciones? ¿O con matrices? ¿O con sucesiones infinitas? Ahí el producto escalar usual no sirve. Necesitamos una generalizacion.

Definicion formal

Un **producto interno** en un espacio vectorial V sobre $\mathbb{R}$ (o $\mathbb{C}$) es una funcion $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{R}$ (o $\mathbb{C}$) que cumple:

1. **Linealidad en el primer argumento:** $\langle \alpha u + \beta v, w \rangle = \alpha \langle u, w \rangle + \beta \langle v, w \rangle$.

2. **Simetria (o hermeticidad en el caso complejo):** $\langle u, v \rangle = \overline{\langle v, u \rangle}$.

3. **Definido positivo:** $\langle u, u \rangle \geq 0$, y $\langle u, u \rangle = 0$ si y solo si $u = 0$.

Traduccion Matematica del producto interno La definicion anterior se lee en criollo: "un producto interno es una regla que te permite multiplicar dos vectores para obtener un numero, y esta regla se porta bien con la suma de vectores y con los numeros reales (o complejos)". La propiedad de ser "definido positivo" es la que garantiza que la "longitud" de un vector (definida como $\|u\| = \sqrt{\langle u, u \rangle}$) sea siempre un numero real no negativo, y que el unico vector de longitud cero sea el vector nulo.

Pensa en el producto interno como una herramienta para medir: te da una manera de calcular distancias y angulos en cualquier espacio vectorial, no solo en el espacio fisico de tres dimensiones.

Ejemplos de productos internos

1. **El clasico en $\mathbb{R}^n$:** $\langle u, v \rangle = \sum_{i=1}^n u_i v_i$.

2. **Con pesos:** $\langle u, v \rangle = \sum_{i=1}^n w_i u_i v_i$ con $w_i > 0$.

3. **En el espacio de funciones continuas $C[a, b]$:** $\langle f, g \rangle = \int_a^b f(x)g(x) dx$. Este es el producto interno que usamos para las series de Fourier y los polinomios ortogonales.

4. **En el espacio de matrices:** $\langle A, B \rangle = \text{tr}(A^T B)$. Este es el producto interno de Frobenius.

La norma inducida y la desigualdad de Cauchy-Schwarz

Una vez que tenes un producto interno, podes definir una **norma**:

$$\|u\| = \sqrt{\langle u, u \rangle}$$

Esta norma tiene todas las propiedades que esperas: es no negativa, homogenea ($\|\alpha u\| = |\alpha| \|u\|$), y satisface la desigualdad triangular.

Ademas, se cumple la **desigualdad de Cauchy-Schwarz**, que es una de las herramientas mas importantes del algebra lineal:

$$|\langle u, v \rangle| \leq \|u\| \|v\|$$

Traduccion Matematica de Cauchy-Schwarz La desigualdad de Cauchy-Schwarz dice que el valor absoluto del producto interno de dos vectores nunca supera el producto de sus normas. En criollo: "el coseno de un angulo siempre esta entre -1 y 1". Esto es lo que permite definir el angulo entre dos vectores en un espacio abstracto. La igualdad se da cuando los vectores son linealmente dependientes (uno es multiplo del otro).

Ortogonalidad y proyecciones

Dos vectores u y v son **ortogonales** si $\langle u, v \rangle = 0$. En $\mathbb{R}^2$, eso significa que forman un angulo de 90 grados. En espacios abstractos, la idea es la misma: son "perpendiculares" en el sentido del producto interno.

Proyeccion ortogonal

La proyeccion ortogonal de un vector u sobre un subespacio S es el vector $p \in S$ que minimiza la distancia $\|u - p\|$. Es decir, p es la mejor aproximacion de u en S . La proyeccion se caracteriza por la condicion de ortogonalidad:

$$\langle u - p, s \rangle = 0 \quad \forall s \in S$$

Traduccion Matematica de la proyeccion La condicion $\langle u - p, s \rangle = 0$ dice que el error $u - p$ es ortogonal a todos los vectores de S . En criollo: "el error es perpendicular al subespacio donde estas proyectando". Esto garantiza que p es el vector de S que esta mas cerca de u .

Si S tiene una base ortonormal $\{e_1, \dots, e_k\}$, la proyeccion se calcula como:

$$p = \sum_{i=1}^k \langle u, e_i \rangle e_i$$

Grafico: proyeccion en 2D

Vamos a visualizar la proyeccion ortogonal en $\mathbb{R}^2$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Definimos un vector u y una direccion v
5 u = np.array([3.0, 1.0])
6 v = np.array([1.0, 2.0])
7
8 # Calculamos la proyeccion de u sobre la recta generada por v
9 # p = (u . v) / (v . v) * v
10 p = (np.dot(u, v) / np.dot(v, v)) * v
11
12 # Graficamos
13 fig, ax = plt.subplots(figsize=(8, 8))
14 ax.quiver(0, 0, u[0], u[1], angles='xy', scale_units='xy', scale=1, color='blue',
15          label='u')
16 ax.quiver(0, 0, p[0], p[1], angles='xy', scale_units='xy', scale=1, color='red',
17          label='p (proyeccion)')
18 ax.quiver(p[0], p[1], u[0]-p[0], u[1]-p[1], angles='xy', scale_units='xy', scale=1, color='green',
19          label='error (u - p)')
20
21 # Dibujamos la recta generada por v
22 t = np.linspace(-2, 4, 100)
23 ax.plot(t * v[0], t * v[1], 'k--', label='span(v)')
24
25 ax.set_xlim(-1, 4)
26 ax.set_ylim(-1, 3)
27 ax.axhline(0, color='black', linewidth=0.5)
28 ax.axvline(0, color='black', linewidth=0.5)
29 ax.grid(True, alpha=0.3)
30 ax.legend()
31 ax.set_aspect('equal')
32 plt.title('Proyeccion ortogonal de u sobre la recta generada por v')
33 plt.show()

```

Listing 63: Proyeccion ortogonal de un vector sobre una recta

En el grafico, ves el vector u (azul), su proyeccion p sobre la recta (rojo), y el error $u - p$ (verde), que es perpendicular a la recta. La proyeccion es el punto de la recta que minimiza la distancia a u .

El proceso de Gram-Schmidt

Uno de los problemas mas comunes en algebra lineal es: dada una base cualquiera de un espacio vectorial, ¿como podemos construir una base **ortonormal** (vectores ortogonales y de norma 1)? La respuesta es el proceso de Gram-Schmidt.

La idea

Supongamos que tenemos una base $\{v_1, v_2, \dots, v_n\}$. Queremos construir una base ortonormal $\{u_1, u_2, \dots, u_n\}$ tal que el subespacio generado por los primeros k vectores sea el mismo:

$$\text{span}\{v_1, \dots, v_k\} = \text{span}\{u_1, \dots, u_k\}$$

La idea es simple: para cada k , tomamos v_k y le restamos su proyeccion sobre el subespacio generado por los vectores anteriores. El resultado es ortogonal a todos los anteriores. Luego normalizamos.

El algoritmo

1. Para $k = 1$: $u_1 = \frac{v_1}{\|v_1\|}$.
2. Para $k = 2, \dots, n$:
 - a) $w_k = v_k - \sum_{i=1}^{k-1} \langle v_k, u_i \rangle u_i$
 - b) $u_k = \frac{w_k}{\|w_k\|}$

Traduccion Matematica de Gram-Schmidt El algoritmo de Gram-Schmidt se lee en criollo: "para cada vector de la base original, agarra el vector, restale todas las proyecciones sobre los vectores que ya construiste, y despues normaliza el resultado". Lo que haces es ir "limpiando" cada vector de las componentes que ya estan representadas por los vectores anteriores, asegurandote de que el nuevo vector sea perpendicular a todos los que ya tenias.

Ejemplo numerico

Vamos a aplicar Gram-Schmidt a la base $\{(1, 1, 1), (1, 0, 1), (0, 1, 1)\}$ en $\mathbb{R}^3$.

```

1 import numpy as np
2
3 def gram_schmidt(V):
4     """
5     Aplica el proceso de Gram-Schmidt a una lista de vectores V.
6     Retorna la base ortonormal.
7     """
8     U = []
9     for v in V:
10         # Restamos las proyecciones sobre los vectores ya calculados
11         w = v.copy()
12         for u in U:
13             w = w - np.dot(w, u) * u
14         # Normalizamos

```

```

15         if np.linalg.norm(w) > 1e-12:
16             u = w / np.linalg.norm(w)
17             U.append(u)
18         else:
19             # Si w es cero, el vector era linealmente dependiente
20             print(f"Vector {v} es linealmente dependiente, se descarta.")
21         return U
22
23 # Definimos la base original
24 v1 = np.array([1.0, 1.0, 1.0])
25 v2 = np.array([1.0, 0.0, 1.0])
26 v3 = np.array([0.0, 1.0, 1.0])
27
28 V = [v1, v2, v3]
29 U = gram_schmidt(V)
30
31 print("Base ortonormal:")
32 for i, u in enumerate(U):
33     print(f"u{i+1} = {u}")
34
35 # Verificamos ortogonalidad
36 print("\nVerificacion de ortogonalidad:")
37 for i in range(len(U)):
38     for j in range(i+1, len(U)):
39         print(f"<u{i+1}, u{j+1}> = {np.dot(U[i], U[j]):.2e}")

```

Listing 64: Gram-Schmidt paso a paso

Si corres este código, vas a ver que la base resultante es ortonormal (los productos internos entre vectores distintos son prácticamente cero). Fíjate que el algoritmo descarta vectores linealmente dependientes (si el w_k es cero, significa que el vector original era combinación lineal de los anteriores).

Gram-Schmidt clásico vs modificado

Hay dos versiones del algoritmo de Gram-Schmidt: el **clásico** (CGS) y el **modificado** (MGS). La diferencia es sutil pero importante.

En el CGS, para calcular w_k , restas *de una sola vez* todas las proyecciones sobre los vectores $u_1, \dots, u_{k-1}$:

$$w_k = v_k - \sum_{i=1}^{k-1} \langle v_k, u_i \rangle u_i$$

En el MGS, haces la resta *de forma iterativa*. Primero restas la proyección sobre u_1 , luego sobre u_2 , y así sucesivamente, usando el vector actualizado en cada paso:

1. $w_k = v_k$
2. Para $i = 1, \dots, k-1$:

$$a) \quad w_k = w_k - \langle w_k, u_i \rangle u_i$$

¿Por qué MGS es mejor? El MGS es numéricamente más estable que el CGS. ¿Por qué? Porque en el CGS, los errores de redondeo en los coeficientes $\langle v_k, u_i \rangle$ se acumulan, y los vectores resultantes pueden perder ortogonalidad. En el MGS, al ir actualizando w_k paso a paso, los errores no se acumulan tanto. En la práctica, siempre usa MGS.

Visualizacion de Gram-Schmidt en 2D

Vamos a graficar el proceso de Gram-Schmidt para dos vectores en $\mathbb{R}^2$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Vectores originales
5 v1 = np.array([2.0, 1.0])
6 v2 = np.array([1.0, 2.0])
7
8 # Gram-Schmidt
9 u1 = v1 / np.linalg.norm(v1)
10 w2 = v2 - np.dot(v2, u1) * u1
11 u2 = w2 / np.linalg.norm(w2)
12
13 # Graficamos
14 fig, ax = plt.subplots(figsize=(8, 8))
15
16 # Vectores originales
17 ax.quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1, color='
    blue', label='v1')
18 ax.quiver(0, 0, v2[0], v2[1], angles='xy', scale_units='xy', scale=1, color='
    orange', label='v2')
19
20 # Vectores ortonormales
21 ax.quiver(0, 0, u1[0], u1[1], angles='xy', scale_units='xy', scale=1, color='red
    ', label='u1')
22 ax.quiver(0, 0, u2[0], u2[1], angles='xy', scale_units='xy', scale=1, color='
    green', label='u2')
23
24 # Proyeccion de v2 sobre u1
25 proy_v2_u1 = np.dot(v2, u1) * u1
26 ax.quiver(0, 0, proy_v2_u1[0], proy_v2_u1[1], angles='xy', scale_units='xy',
    scale=1, color='purple', alpha=0.6, label='proyeccion de v2 sobre u1')
27
28 # Error
29 w2 = v2 - proy_v2_u1
30 ax.quiver(proy_v2_u1[0], proy_v2_u1[1], w2[0], w2[1], angles='xy', scale_units='
    xy', scale=1, color='gray', alpha=0.6, label='error (w2)')
31
32 ax.set_xlim(-1, 3)
33 ax.set_ylim(-1, 3)
34 ax.axhline(0, color='black', linewidth=0.5)
35 ax.axvline(0, color='black', linewidth=0.5)
36 ax.grid(True, alpha=0.3)
37 ax.legend()
38 ax.set_aspect('equal')
39 plt.title('Proceso de Gram-Schmidt en 2D')
40 plt.show()

```

Listing 65: Visualizacion de Gram-Schmidt en 2D

En el grafico, ves los vectores originales v_1 y v_2 . Primero se normaliza v_1 para obtener u_1 (rojo). Luego se calcula la proyeccion de v_2 sobre u_1 (morado), y se resta para obtener w_2 (gris), que es ortogonal a u_1 . Finalmente se normaliza w_2 para obtener u_2 (verde). El resultado es una base ortonormal.

Factorizacion QR

El proceso de Gram-Schmidt aplicado a las columnas de una matriz $A \in \mathbb{R}^{m \times n}$ (con $m \geq n$) produce la **factorizacion QR**:

$$A = QR$$

donde:

- $Q \in \mathbb{R}^{m \times n}$ tiene columnas ortonormales ($Q^T Q = I_n$).
- $R \in \mathbb{R}^{n \times n}$ es triangular superior.

¿Por que es importante QR?

La factorizacion QR es una de las herramientas mas poderosas del algebra lineal numerica. Tiene varias ventajas sobre la factorizacion LU:

1. **Estabilidad:** QR es mucho mas estable numericamente que LU, especialmente para matrices mal condicionadas.
2. **Cuadrados minimos:** QR es la forma preferida de resolver problemas de cuadrados minimos, porque evita las ecuaciones normales (que pueden estar mal condicionadas).
3. **Autovalores:** El metodo QR iterativo es el algoritmo estandar para calcular todos los autovalores de una matriz.

Construccion de QR via Gram-Schmidt

Si aplicas Gram-Schmidt a las columnas de A , obtienes las columnas de Q . Los coeficientes de la combinacion lineal que expresan cada columna de A en terminos de las columnas de Q forman la matriz R .

Mas precisamente, si $A = [a_1 | a_2 | \dots | a_n]$ y $Q = [q_1 | q_2 | \dots | q_n]$, entonces:

$$a_k = \sum_{i=1}^k r_{ik} q_i$$

donde $r_{ik} = \langle a_k, q_i \rangle$ para $i < k$, y $r_{kk} = \|w_k\|$ (la norma del vector antes de normalizar). En forma matricial:

$$A = QR$$

con R triangular superior.

Ejemplo: QR de una matriz 3x2

Vamos a calcular la factorizacion QR de una matriz $A \in \mathbb{R}^{3 \times 2}$.

```

1 import numpy as np
2
3 def qr_gram_schmidt(A):
4     """
5     Calcula la factorizacion QR de A usando Gram-Schmidt modificado.
6     A debe tener rango completo (columnas linealmente independientes).
7     """

```



```

8     m, n = A.shape
9     Q = np.zeros((m, n))
10    R = np.zeros((n, n))
11
12    for j in range(n):
13        # Tomamos la columna j de A
14        v = A[:, j].copy()
15
16        for i in range(j):
17            # Calculamos el coeficiente de proyeccion sobre q_i
18            r_ij = np.dot(v, Q[:, i])
19            R[i, j] = r_ij
20            # Restamos la proyeccion
21            v = v - r_ij * Q[:, i]
22
23        # Normalizamos
24        r_jj = np.linalg.norm(v)
25        R[j, j] = r_jj
26        Q[:, j] = v / r_jj
27
28    return Q, R
29
30 # Definimos una matriz 3x2
31 A = np.array([[1.0, 1.0],
32               [1.0, 0.0],
33               [0.0, 1.0]])
34
35 Q, R = qr_gram_schmidt(A)
36
37 print("Matriz A:")
38 print(A)
39 print("\nMatriz Q (columnas ortonormales):")
40 print(Q)
41 print("\nMatriz R (triangular superior):")
42 print(R)
43 print("\nVerificacion: Q R =")
44 print(Q @ R)
45 print("\nQ^T Q =")
46 print(Q.T @ Q)

```

Listing 66: Factorizacion QR via Gram-Schmidt

Si corres este codigo, vas a ver que $Q^T Q \approx I$ (ortogonalidad de las columnas) y que $QR \approx A$. La matriz R es triangular superior, como esperabamos.

QR vs LU: ¿cual usar?

Cuadro 3: Comparacion entre factorizacion LU y QR

Propiedad	LU	QR	Comentario
Costo computacional	$2/3n^3$	$2n^3$	LU es mas rapida
Estabilidad numerica	Buena (con pivoteo)	Excelente	QR es mas estable
Mal condicionamiento	Puede fallar	Robusto	QR es preferible
Estructura	Mantiene esparcidad	Rellena	LU mejor para matrices ral
Aplicacion principal	Sistemas lineales	Cuadrados minimos, autovalores	

Grafico: error de QR vs LU para matrices mal condicionadas

Vamos a comparar el error de resolver un sistema lineal usando LU y QR para matrices de Hilbert (que son famosas por estar mal condicionadas).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def hilbert(n):
5     return np.array([[1/(i+j+1) for j in range(n)] for i in range(n)])
6
7 def solve_lu(A, b):
8     """Resuelve Ax = b usando LU (con pivoteo)"""
9     return np.linalg.solve(A, b)
10
11 def solve_qr(A, b):
12     """Resuelve Ax = b usando QR"""
13     Q, R = np.linalg.qr(A)
14     # Rx = Q^T b
15     return np.linalg.solve(R, Q.T @ b)
16
17 n_values = range(2, 12)
18 errores_lu = []
19 errores_qr = []
20
21 for n in n_values:
22     H = hilbert(n)
23     x_exacto = np.ones(n)
24     b = H @ x_exacto
25
26     # LU
27     x_lu = solve_lu(H, b)
28     error_lu = np.linalg.norm(x_lu - x_exacto, np.inf) / np.linalg.norm(x_exacto, np.inf)
29     errores_lu.append(error_lu)
30
31     # QR
32     x_qr = solve_qr(H, b)
33     error_qr = np.linalg.norm(x_qr - x_exacto, np.inf) / np.linalg.norm(x_exacto, np.inf)
34     errores_qr.append(error_qr)
35
36 plt.figure(figsize=(10, 6))
37 plt.semilogy(n_values, errores_lu, 'o-', label='LU', linewidth=2, markersize=8)
38 plt.semilogy(n_values, errores_qr, 's-', label='QR', linewidth=2, markersize=8)
39 plt.xlabel('Tamaño de la matriz de Hilbert n', fontsize=14)
40 plt.ylabel('Error relativo en norma infinito', fontsize=14)
41 plt.title('Comparacion de estabilidad: LU vs QR para matrices de Hilbert',
42         fontsize=16)
43 plt.legend(fontsize=12)
44 plt.grid(True, alpha=0.3)
45 plt.show()

```

Listing 67: Comparacion de error: QR vs LU para matrices de Hilbert

En el grafico, vas a ver que para n grande (digamos $n \geq 8$), el error de LU explota, mientras que QR se mantiene mucho mas estable. Esto se debe a que la matriz de Hilbert es muy mal condicionada, y el pivoteo de LU no es suficiente para controlar los errores de redondeo. QR, al ser una factorizacion ortogonal, es mucho mas robusta.

Espacios de Hilbert y bases ortonormales

Hasta ahora trabajamos en espacios de dimension finita. Pero en muchas aplicaciones (como las series de Fourier), necesitamos trabajar en espacios de dimension infinita. Ahi entran los **espacios de Hilbert**.

Definicion

Un **espacio de Hilbert** es un espacio con producto interno que es **completo** con respecto a la norma inducida. En criollo: "es un espacio donde las sucesiones de Cauchy convergen a un elemento del espacio". Esto es importante porque garantiza que las series infinitas converjan.

Los ejemplos mas comunes son:

- ℓ^2 : el espacio de sucesiones de cuadrado sumable ($\sum |x_n|^2 < \infty$).
- $L^2[a, b]$: el espacio de funciones de cuadrado integrable ($\int_a^b |f(x)|^2 dx < \infty$).

Bases ortonormales en dimension infinita

En un espacio de Hilbert separable (que tiene una base numerable), una **base ortonormal** es un conjunto de vectores ortonormales $\{e_n\}_{n=1}^{\infty}$ tal que cualquier vector x se puede escribir como:

$$x = \sum_{n=1}^{\infty} \langle x, e_n \rangle e_n$$

Traduccion Matematica de una base ortonormal Una base ortonormal en dimension infinita es como un sistema de coordenadas: cada vector se puede expresar como una combinacion lineal (infinita) de los vectores de la base. Los coeficientes $\langle x, e_n \rangle$ son las "coordenadas" de x en esa base. La identidad de Parseval dice que la norma al cuadrado de x es la suma de los cuadrados de sus coordenadas:

$$\|x\|^2 = \sum_{n=1}^{\infty} |\langle x, e_n \rangle|^2$$

Ejemplo: las series de Fourier

El ejemplo mas clasico de espacio de Hilbert y base ortonormal es $L^2[0, 2\pi]$ con el producto interno $\langle f, g \rangle = \frac{1}{2\pi} \int_0^{2\pi} f(x) \overline{g(x)} dx$. Las funciones $\{e^{ikx}\}_{k \in \mathbb{Z}}$ forman una base ortonormal. Esto es exactamente lo que usamos para las series de Fourier.

Grafico: aproximacion de una funcion por su serie de Fourier

Vamos a visualizar como una funcion se aproxima por su serie de Fourier, que es un ejemplo de proyeccion ortogonal en un espacio de Hilbert.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def fourier_aproximacion(x, N):
5     """
6     Calcula la aproximacion de Fourier de la funcion cuadrada
7     f(x) = 1 para x en [0, pi), -1 para x en [pi, 2pi)
8     con N terminos.

```

```

9      """
10     f = np.zeros_like(x)
11     for k in range(1, N+1):
12         # Los coeficientes de Fourier de la funcion cuadrada son:
13         # a_k = 0, b_k = 2*(1 - (-1)^k) / (pi * k)
14         # Solo los terminos impares son no nulos
15         if k % 2 == 1:
16             f += (4 / (np.pi * k)) * np.sin(k * x)
17     return f
18
19 x = np.linspace(0, 2*np.pi, 1000)
20
21 plt.figure(figsize=(12, 8))
22 for N in [1, 3, 5, 10, 20, 50]:
23     f = fourier_aproximacion(x, N)
24     plt.plot(x, f, label=f'N = {N}', linewidth=2)
25
26 # Funcion exacta (cuadrada)
27 f_exacta = np.where((x >= 0) & (x < np.pi), 1, -1)
28 plt.plot(x, f_exacta, 'k--', label='Funcion exacta', linewidth=2)
29
30 plt.xlabel('x', fontsize=14)
31 plt.ylabel('f(x)', fontsize=14)
32 plt.title('Aproximacion de Fourier de una funcion cuadrada', fontsize=16)
33 plt.legend(fontsize=12)
34 plt.grid(True, alpha=0.3)
35 plt.show()

```

Listing 68: Aproximacion de Fourier de una funcion cuadrada

En el grafico, ves como la serie de Fourier (que es la proyeccion ortogonal de la funcion sobre el subespacio generado por los primeros N senos) va mejorando a medida que aumenta N . Pero fijate que cerca de las discontinuidades aparecen oscilaciones (el fenomeno de Gibbs). Esto pasa porque la funcion no es continua; en espacios de Hilbert, la convergencia es en norma L^2 , no puntual.

Polinomios ortogonales

Los polinomios ortogonales son una familia de polinomios que son ortogonales entre si con respecto a un producto interno con peso. Aparecen en todas partes: en cuadratura gaussiana, en ecuaciones diferenciales, en aproximacion de funciones, etc.

Definicion

Una familia de polinomios $\{p_n(x)\}_{n=0}^{\infty}$ es **ortogonal** con respecto al peso $w(x) > 0$ en el intervalo (a, b) si:

$$\int_a^b p_n(x)p_m(x)w(x)dx = 0 \quad \text{para } n \neq m$$

Ejemplos clasicos

- **Legendre:** $w(x) = 1$, $[-1, 1]$. Son los polinomios que aparecen en la cuadratura gaussiana.
- **Chebyshev:** $w(x) = \frac{1}{\sqrt{1-x^2}}$, $[-1, 1]$. Son los polinomios que minimizan el error de interpolacion.

- **Laguerre:** $w(x) = e^{-x}$, $[0, \infty)$.
- **Hermite:** $w(x) = e^{-x^2}$, $(-\infty, \infty)$.

Construccion via Gram-Schmidt

Los polinomios ortogonales se pueden construir aplicando Gram-Schmidt a la base $\{1, x, x^2, \dots\}$ con el producto interno $\langle f, g \rangle = \int_a^b f(x)g(x)w(x)dx$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def producto_interno_legendre(f, g):
5     """Producto interno de Legendre en [-1, 1]"""
6     # Integracion numerica con la regla del trapecio
7     x = np.linspace(-1, 1, 1000)
8     return np.trapz(f(x) * g(x), x)
9
10 def gram_schmidt_polinomios(n):
11     """
12     Construye los primeros n polinomios de Legendre usando Gram-Schmidt.
13     Retorna una lista de funciones.
14     """
15     # Base original: {1, x, x^2, ...}
16     def base_polinomio(k):
17         return lambda x: x**k
18
19     # Lista de polinomios ortonormales
20     p = []
21
22     for k in range(n):
23         # Vector w_k = x^k
24         def w_k(x, k=k):
25             return x**k
26
27         # Restamos proyecciones sobre los polinomios anteriores
28         def w_k_ort(x, k=k, p=p):
29             result = x**k
30             for pi in p:
31                 # Coeficiente de proyeccion: <x^k, pi> / <pi, pi>
32                 # Como pi es ortonormal, <pi, pi> = 1
33                 coef = producto_interno_legendre(lambda t: t**k, pi)
34                 result -= coef * pi(x)
35             return result
36
37         # Normalizamos
38         def p_k(x, k=k, p=p):
39             w = w_k_ort(x, k, p)
40             # Calculamos la norma
41             w_func = lambda t: w_k_ort(t, k, p)
42             norm = np.sqrt(producto_interno_legendre(w_func, w_func))
43             if norm > 1e-12:
44                 return w / norm
45             else:
46                 return np.zeros_like(x)
47
48         p.append(p_k)
49
50     return p
51

```

```

52 # Construimos los primeros 5 polinomios de Legendre
53 p_legendre = gram_schmidt_polinomios(5)
54
55 # Graficamos
56 x_plot = np.linspace(-1, 1, 200)
57 plt.figure(figsize=(10, 6))
58 for i, p in enumerate(p_legendre):
59     plt.plot(x_plot, p(x_plot), label=f'p_{i}(x)')
60
61 plt.xlabel('x', fontsize=14)
62 plt.ylabel('p_n(x)', fontsize=14)
63 plt.title('Polinomios de Legendre contruidos via Gram-Schmidt', fontsize=16)
64 plt.legend(fontsize=12)
65 plt.grid(True, alpha=0.3)
66 plt.axhline(0, color='black', linewidth=0.5)
67 plt.axvline(0, color='black', linewidth=0.5)
68 plt.show()

```

Listing 69: Construcción de polinomios de Legendre via Gram-Schmidt

Este código es una implementación didáctica de Gram-Schmidt para construir polinomios de Legendre. En la práctica, para calcular polinomios ortogonales de forma eficiente se usan recurrencias de tres términos, pero la idea conceptual es la misma.

Ejercicios

Parte Teórica (10 ejercicios)

- T1. Demostración de Cauchy-Schwarz:** Demuestra la desigualdad de Cauchy-Schwarz para un espacio con producto interno. Sugerencia: usa el hecho de que $\|u - \alpha v\|^2 \geq 0$ para todo $\alpha \in \mathbb{R}$ y elige α apropiadamente.
- T2. Propiedades de la proyección:** Demuestra que la proyección ortogonal P_S sobre un subespacio S es lineal, idempotente ($P_S^2 = P_S$) y autoadjunta ($\langle P_S u, v \rangle = \langle u, P_S v \rangle$).
- T3. Gram-Schmidt y bases:** Demuestra que si aplicas Gram-Schmidt a una base $\{v_1, \dots, v_n\}$, el subespacio generado por los primeros k vectores de la nueva base es el mismo que el generado por los primeros k vectores de la base original.
- T4. Factorización QR:** Demuestra que si $A = QR$ con Q de columnas ortonormales y R triangular superior, entonces las columnas de Q forman una base del espacio columna de A .
- T5. Identidad de Parseval:** Demuestra que si $\{e_n\}$ es una base ortonormal de un espacio de Hilbert H , entonces para todo $x \in H$, $\|x\|^2 = \sum_{n=1}^{\infty} |\langle x, e_n \rangle|^2$.
- T6. Polinomios de Legendre:** Demuestra que los polinomios de Legendre P_n son ortogonales en $[-1, 1]$ con peso $w(x) = 1$. Usa la fórmula de Rodrigues $P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n$.
- T7. Relación entre QR y cuadrados mínimos:** Demuestra que la solución del problema de cuadrados mínimos $\min_x \|Ax - b\|_2$ se puede obtener resolviendo $Rx = Q^T b$, donde $A = QR$.
- T8. Estabilidad de Gram-Schmidt:** Explica por qué el Gram-Schmidt modificado (MGS) es numéricamente más estable que el clásico (CGS). Da un ejemplo donde CGS falle y MGS no.

- T9. Descomposicion polar:** Demuestra que toda matriz $A \in \mathbb{R}^{n \times n}$ se puede escribir como $A = UP$, donde U es ortogonal y P es simetrica definida positiva. Sugerencia: usa la factorizacion QR.
- T10. Cociente de Rayleigh:** Demuestra que para una matriz simetrica A , el cociente de Rayleigh $\rho(x) = \frac{x^T A x}{x^T x}$ alcanza su minimo y maximo en los autovectores de A . ¿Que relacion tiene esto con la proyeccion ortogonal?

Parte Computacional (10 ejercicios)

- C1. Implementacion de Gram-Schmidt:** Escribe una funcion en Python que implemente el proceso de Gram-Schmidt modificado. La funcion debe recibir una lista de vectores y retornar una base ortonormal.
- C2. Factorizacion QR:** Implementa la factorizacion QR usando Gram-Schmidt modificado. Prueba con una matriz 4×3 y verifica que $Q^T Q = I$ y que $QR \approx A$.
- C3. Cuadrados minimos con QR:** Usa la factorizacion QR para resolver el problema de cuadrados minimos $\min_x \|Ax - b\|_2$ para $A \in \mathbb{R}^{10 \times 3}$ y b generado aleatoriamente. Compara con la solucion de las ecuaciones normales.
- C4. Proyeccion sobre un subespacio:** Dada una matriz $A \in \mathbb{R}^{m \times n}$, implementa una funcion que calcule la proyeccion ortogonal de un vector b sobre el espacio columna de A . Usa la factorizacion QR.
- C5. Grafico de convergencia de QR vs LU:** Para matrices de Hilbert de tamaño $n = 2, \dots, 15$, resuelve el sistema $H_n x = b$ con $b = H_n \mathbf{1}$ usando LU y QR. Grafica el error relativo en funcion de n en escala logaritmica.
- C6. Construccion de polinomios de Chebyshev:** Implementa Gram-Schmidt con el producto interno $\langle f, g \rangle = \int_{-1}^1 f(x)g(x) \frac{1}{\sqrt{1-x^2}} dx$ para construir los primeros 5 polinomios de Chebyshev. Graficalos.
- C7. Aproximacion por polinomios de Legendre:** Dada la funcion $f(x) = e^x$ en $[-1, 1]$, calcula su aproximacion por los primeros n polinomios de Legendre (proyeccion ortogonal). Grafica f y la aproximacion para $n = 2, 4, 6, 8$.
- C8. Estabilidad de Gram-Schmidt:** Genera una matriz $A \in \mathbb{R}^{10 \times 10}$ con columnas casi linealmente dependientes (por ejemplo, $a_i = v + \epsilon r_i$, con r_i aleatorio). Aplica Gram-Schmidt clasico y modificado. Compara la ortogonalidad de las columnas resultantes ($\|Q^T Q - I\|_F$).
- C9. Descomposicion QR y autovalores:** Implementa el metodo QR para calcular autovalores de una matriz simetrica $A \in \mathbb{R}^{5 \times 5}$. Genera A aleatoria simetrica y verifica que la diagonal de A_k converge a los autovalores.
- C10. Series de Fourier y Parseval:** Para la funcion $f(x) = x^2$ en $[-\pi, \pi]$, calcula sus coeficientes de Fourier y verifica numericamente la identidad de Parseval $\frac{1}{2\pi} \int_{-\pi}^{\pi} |f(x)|^2 dx = \sum_{k=-\infty}^{\infty} |\hat{f}_k|^2$ usando los primeros 20 terminos.

Soluciones

Soluciones Teóricas

T1. Para cualquier $\alpha \in \mathbb{R}$, tenemos:

$$0 \leq \|u - \alpha v\|^2 = \|u\|^2 - 2\alpha \langle u, v \rangle + \alpha^2 \|v\|^2$$

Esta es una cuadrática en α . El mínimo se alcanza en $\alpha = \frac{\langle u, v \rangle}{\|v\|^2}$. Sustituyendo:

$$0 \leq \|u\|^2 - \frac{\langle u, v \rangle^2}{\|v\|^2}$$

Despejando: $|\langle u, v \rangle| \leq \|u\| \|v\|$.

T2. Linealidad: $P_S(u + v) = P_S(u) + P_S(v)$ porque el subespacio es cerrado bajo suma. Idempotencia: $P_S(P_S(u)) = P_S(u)$ porque $P_S(u) \in S$. Autoadjunta: $\langle P_S u, v \rangle = \langle u, P_S v \rangle$ porque $u - P_S u \perp S$ y $v - P_S v \perp S$.

T3. Por construcción, cada u_k es combinación lineal de $v_1, \dots, v_k$. Además, como el proceso es reversible, cada v_k es combinación lineal de $u_1, \dots, u_k$. Por lo tanto, los subespacios generados coinciden.

T4. Las columnas de Q son ortonormales, así que son linealmente independientes. Como $A = QR$ y R es triangular superior con diagonal no nula (si A tiene rango completo), las columnas de A son combinaciones lineales de las columnas de Q . Por lo tanto, el espacio columna de A es el mismo que el de Q .

T5. Sea $x_N = \sum_{n=1}^N \langle x, e_n \rangle e_n$. Por el teorema de Pitágoras en espacios de Hilbert, $\|x\|^2 = \|x_N\|^2 + \|x - x_N\|^2$. Como $x_N \rightarrow x$ (por ser base ortonormal), $\|x - x_N\| \rightarrow 0$. Entonces $\|x\|^2 = \lim_{N \rightarrow \infty} \|x_N\|^2 = \lim_{N \rightarrow \infty} \sum_{n=1}^N |\langle x, e_n \rangle|^2$.

T6. La fórmula de Rodrigues da $P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n$. Integrando por partes n veces, se demuestra que $\int_{-1}^1 P_n(x) P_m(x) dx = 0$ para $n \neq m$.

T7. Si $A = QR$, entonces $\|Ax - b\|_2 = \|QRx - b\|_2 = \|Rx - Q^T b\|_2$ porque Q es ortogonal. El mínimo se alcanza cuando $Rx = Q^T b$, que es un sistema triangular superior.

T8. En CGS, los errores de redondeo se acumulan porque se calculan todas las proyecciones de una vez. En MGS, al ir restando paso a paso, los errores no se acumulan tanto. Un ejemplo numérico: la matriz de Hilbert es un caso donde CGS falla y MGS funciona bien.

T9. La descomposición polar se obtiene de la factorización QR: $A = QR$. Si R tiene autovalores positivos, podemos escribir $R = U\Lambda^{1/2}U^T$, y entonces $A = (QU)(U^T\Lambda^{1/2}U) = U_1P$.

T10. El cociente de Rayleigh $\rho(x) = \frac{x^T A x}{x^T x}$ es una media ponderada de los autovalores de A . El mínimo y el máximo se alcanzan en los autovectores correspondientes al autovalor mínimo y máximo, respectivamente. Esto es una consecuencia de la proyección ortogonal sobre los subespacios invariantes.

Soluciones Computacionales

C1. Ver el código de la Sección . La función `gram_schmidt` implementa el algoritmo modificado.

C2. Ver el código de la Sección . La función `qr_gram_schmidt` implementa QR via Gram-Schmidt.

C3.

```

1 import numpy as np
2
3 def qr_gram_schmidt(A):
4     # Implementación de QR via Gram-Schmidt modificado

```



```

5  m, n = A.shape
6  Q = np.zeros((m, n))
7  R = np.zeros((n, n))
8  for j in range(n):
9      v = A[:, j].copy()
10     for i in range(j):
11         r_ij = np.dot(v, Q[:, i])
12         R[i, j] = r_ij
13         v = v - r_ij * Q[:, i]
14     r_jj = np.linalg.norm(v)
15     R[j, j] = r_jj
16     Q[:, j] = v / r_jj
17     return Q, R
18
19 # Generamos datos
20 np.random.seed(42)
21 m, n = 10, 3
22 A = np.random.randn(m, n)
23 x_exacto = np.random.randn(n)
24 b = A @ x_exacto + 0.1 * np.random.randn(m) # con ruido
25
26 # QR
27 Q, R = qr_gram_schmidt(A)
28 x_qr = np.linalg.solve(R, Q.T @ b)
29
30 # Ecuaciones normales
31 x_norm = np.linalg.solve(A.T @ A, A.T @ b)
32
33 print("Solucion QR:", x_qr)
34 print("Solucion ecuaciones normales:", x_norm)
35 print("Error QR vs exacto:", np.linalg.norm(x_qr - x_exacto))
36 print("Error normales vs exacto:", np.linalg.norm(x_norm - x_exacto))

```

Listing 70: Cuadrados minimos con QR

C4.

```

1  def proyeccion_columna(A, b):
2      """Proyecta b sobre el espacio columna de A"""
3      Q, R = np.linalg.qr(A)
4      # La proyeccion es Q Q^T b
5      return Q @ Q.T @ b
6
7  # Ejemplo
8  A = np.array([[1, 2], [3, 4], [5, 6]])
9  b = np.array([1, 2, 3])
10 p = proyeccion_columna(A, b)
11 print("Proyeccion:", p)
12 print("Error (b - p) es ortogonal a columnas de A:")
13 print("A^T (b - p) =", A.T @ (b - p))

```

Listing 71: Proyeccion sobre espacio columna

C5. Ver el codigo de la Seccion .**C6.**

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def producto_interno_chebyshev(f, g):
5      """Producto interno de Chebyshev con peso 1/sqrt(1-x^2)"""
6      x = np.linspace(-0.999, 0.999, 1000)

```

```

7     w = 1 / np.sqrt(1 - x**2)
8     return np.trapz(f(x) * g(x) * w, x)
9
10 def gram_schmidt_chebyshev(n):
11     """Construye los primeros n polinomios de Chebyshev"""
12     p = []
13     for k in range(n):
14         # Vector original: x^k
15         # Restamos proyecciones
16         def w_k(x, k=k):
17             return x**k
18
19         # Implementacion simplificada: usamos la recurrencia para Chebyshev
20         # T_0 = 1, T_1 = x, T_{n+1} = 2x T_n - T_{n-1}
21         if k == 0:
22             p.append(lambda x: np.ones_like(x))
23         elif k == 1:
24             p.append(lambda x: x)
25         else:
26             # Recurrencia
27             def T_k(x, k=k):
28                 if k == 0:
29                     return 1
30                 elif k == 1:
31                     return x
32                 else:
33                     # Usamos la recurrencia para evaluar
34                     T_prev = 1
35                     T_curr = x
36                     for _ in range(2, k+1):
37                         T_next = 2*x*T_curr - T_prev
38                         T_prev, T_curr = T_curr, T_next
39                     return T_curr
40             p.append(T_k)
41     return p
42
43 # Construimos los primeros 5 polinomios de Chebyshev
44 p_cheb = gram_schmidt_chebyshev(5)
45
46 # Graficamos
47 x_plot = np.linspace(-1, 1, 200)
48 plt.figure(figsize=(10, 6))
49 for i, p in enumerate(p_cheb):
50     plt.plot(x_plot, p(x_plot), label=f'T_{i}(x)')
51
52 plt.xlabel('x')
53 plt.ylabel('T_n(x)')
54 plt.title('Polinomios de Chebyshev')
55 plt.legend()
56 plt.grid(True)
57 plt.show()

```

Listing 72: Polinomios de Chebyshev via Gram-Schmidt

C7.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.special import eval_legendre
4
5 def aproximacion_legendre(f, n, x):
6     """Calcula la aproximacion de f por los primeros n polinomios de Legendre"""

```

```

7   # Usamos scipy para los polinomios de Legendre
8   # Coeficientes:  $c_k = (2k+1)/2 \int_{-1}^1 f(x) P_k(x) dx$ 
9   # Integramos numericamente
10  x_int = np.linspace(-1, 1, 500)
11  f_int = f(x_int)
12  coefs = []
13  for k in range(n):
14      P_k = eval_legendre(k, x_int)
15      c_k = (2*k+1)/2 * np.trapz(f_int * P_k, x_int)
16      coefs.append(c_k)
17
18  # Evaluamos la aproximacion
19  aprox = np.zeros_like(x)
20  for k, c_k in enumerate(coefs):
21      aprox += c_k * eval_legendre(k, x)
22  return aprox
23
24 # Funcion a aproximar
25 def f(x):
26     return np.exp(x)
27
28 x = np.linspace(-1, 1, 200)
29 plt.figure(figsize=(10, 6))
30 plt.plot(x, f(x), 'k--', label='f(x) = e^x')
31
32 for n in [2, 4, 6, 8]:
33     aprox = aproximacion_legendre(f, n, x)
34     plt.plot(x, aprox, label=f'n = {n}')
35
36 plt.xlabel('x')
37 plt.ylabel('f(x)')
38 plt.title('Aproximacion de e^x por polinomios de Legendre')
39 plt.legend()
40 plt.grid(True)
41 plt.show()

```

Listing 73: Aproximacion por polinomios de Legendre

C8.

```

1  import numpy as np
2
3  def gram_schmidt_clasico(V):
4      U = []
5      for v in V:
6          w = v.copy()
7          for u in U:
8              w = w - np.dot(w, u) * u
9          if np.linalg.norm(w) > 1e-12:
10             u = w / np.linalg.norm(w)
11             U.append(u)
12     return np.array(U).T
13
14 def gram_schmidt_modificado(V):
15     U = []
16     for v in V:
17         w = v.copy()
18         for u in U:
19             w = w - np.dot(w, u) * u
20         if np.linalg.norm(w) > 1e-12:
21             u = w / np.linalg.norm(w)
22             U.append(u)

```

```

23     return np.array(U).T
24
25 # Generamos una matriz con columnas casi dependientes
26 np.random.seed(42)
27 n = 10
28 v = np.random.randn(n)
29 epsilon = 1e-6
30 A = np.column_stack([v + epsilon * np.random.randn(n) for _ in range(n)])
31
32 Q_clasico = gram_schmidt_clasico([A[:, i] for i in range(n)])
33 Q_modificado = gram_schmidt_modificado([A[:, i] for i in range(n)])
34
35 # Medimos la ortogonalidad
36 error_clasico = np.linalg.norm(Q_clasico.T @ Q_clasico - np.eye(n), 'fro')
37 error_modificado = np.linalg.norm(Q_modificado.T @ Q_modificado - np.eye(n), '
    fro')
38
39 print(f"Error de ortogonalidad (CGS): {error_clasico:.2e}")
40 print(f"Error de ortogonalidad (MGS): {error_modificado:.2e}")

```

Listing 74: Estabilidad de Gram-Schmidt

C9.

```

1 import numpy as np
2
3 def metodo_qr_autovalores(A, max_iter=100):
4     """Calcula autovalores de A usando el metodo QR"""
5     A_k = A.copy()
6     for _ in range(max_iter):
7         Q, R = np.linalg.qr(A_k)
8         A_k = R @ Q
9     return np.diag(A_k)
10
11 # Generamos una matriz simetrica aleatoria
12 n = 5
13 A = np.random.randn(n, n)
14 A = A + A.T # Simetrizamos
15
16 autovalores = metodo_qr_autovalores(A)
17 autovalores_exactos = np.sort(np.linalg.eigvalsh(A))
18
19 print("Autovalores calculados con QR:", autovalores)
20 print("Autovalores exactos:", autovalores_exactos)
21 print("Error:", np.linalg.norm(np.sort(autovalores) - autovalores_exactos))

```

Listing 75: Metodo QR para autovalores

C10.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def coeficientes_fourier(f, N):
5     """Calcula los coeficientes de Fourier de f en [-pi, pi]"""
6     # Integracion numerica
7     x = np.linspace(-np.pi, np.pi, 1000)
8     dx = x[1] - x[0]
9     f_vals = f(x)
10
11     coefs = []
12     for k in range(-N, N+1):

```

```

13     c_k = (1/(2*np.pi)) * np.trapz(f_vals * np.exp(-1j * k * x), x)
14     coefs.append(c_k)
15     return np.array(coefs)
16
17 def f(x):
18     return x**2
19
20 N = 20
21 coefs = coeficientes_fourier(f, N)
22
23 # Lado izquierdo de Parseval
24 x = np.linspace(-np.pi, np.pi, 1000)
25 lado_izq = (1/(2*np.pi)) * np.trapz(f(x)**2, x)
26
27 # Lado derecho de Parseval (con los primeros 2N+1 terminos)
28 lado_der = np.sum(np.abs(coefs)**2)
29
30 print(f"Lado izquierdo (norma al cuadrado): {lado_izq:.6f}")
31 print(f"Lado derecho (suma de |c_k|^2): {lado_der:.6f}")
32 print(f"Diferencia: {abs(lado_izq - lado_der):.2e}")
33
34 # Graficamos la convergencia de Parseval
35 Ns = range(1, 21)
36 errores = []
37 for N in Ns:
38     coefs = coeficientes_fourier(f, N)
39     lado_der = np.sum(np.abs(coefs)**2)
40     errores.append(abs(lado_izq - lado_der))
41
42 plt.figure(figsize=(10, 6))
43 plt.semilogy(Ns, errores, 'o-')
44 plt.xlabel('N (numero de terminos)')
45 plt.ylabel('Error en la identidad de Parseval')
46 plt.title('Convergencia de la identidad de Parseval para f(x) = x^2')
47 plt.grid(True)
48 plt.show()

```

Listing 76: Parseval para $f(x) = x^2$

Palabras Finales

Y bueno, loco, llegamos al final de este viaje por el mundo de la ortogonalidad. Viste que no era tan abstracto como parecía. Arrancamos con la idea de producto interno, pasamos por Gram-Schmidt y QR, y terminamos en espacios de Hilbert y series de Fourier. Todo esta conectado: la ortogonalidad es la llave que abre la puerta a la aproximacion, a la estabilidad numerica, y a la comprension profunda de los espacios de funciones.

Los puntos clave para llevar:

- El producto interno generaliza el producto escalar a espacios abstractos.
- La proyeccion ortogonal es la mejor aproximacion en un subespacio.
- Gram-Schmidt te permite construir bases ortonormales a partir de cualquier base.
- La factorizacion QR es mas estable que LU y es la base de muchos algoritmos numericos.
- Los espacios de Hilbert son el hogar natural de las series de Fourier y los polinomios ortogonales.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. ¡Dale para adelante, que vos podes!

Cuadrados Minimos y Descomposiciones

Como ajustar datos, resolver sistemas sobredeterminados y dominar la SVD.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a meterle a uno de los problemas mas frecuentes en ciencia e ingenieria: el ajuste de datos. Seguro que en algun momento tuviste que ajustar una recta a un monton de puntos experimentales, o resolver un sistema con mas ecuaciones que incognitas. Eso es exactamente el problema de cuadrados minimos.

Cuando curse Laboratorio 1, recuerdo que el profesor nos puso un ejercicio con datos de un experimento de laboratorio y nos pidio que ajustemos una curva. Yo, todo confiado, arme un sistema de ecuaciones y trate de resolverlo con eliminacion gaussiana. La compu me tiro un error de matriz singular y ahi me di cuenta de que el sistema era sobredeterminado. Fue ahi cuando aprendi que el problema no es resolver ^{el} sistema, sino ^{en}contrar la mejor solucion posible en el sentido de minimizar el error”. Y ese es exactamente el espiritu de los cuadrados minimos.

En este capitulo vamos a cubrir:

- El problema de cuadrados minimos y por que aparece en todas partes.
- La solucion via ecuaciones normales (y por que a veces no es una buena idea).
- La factorizacion QR como la forma estable de resolver cuadrados minimos.
- La descomposicion en valores singulares (SVD), la navaja suiza del algebra lineal numerica.
- La pseudoinversa de Moore-Penrose y como manejar sistemas mal condicionados.
- La regularizacion de Tikhonov para problemas inversos.
- La aproximacion de bajo rango con el teorema de Eckart-Young.

La idea es que vos puedas implementar todos estos conceptos en Python, entender por que funcionan, y saber cual herramienta usar en cada situacion. Asi que agarra un cafe, ponete comodo, y vamos a darle.

El problema de cuadrados minimos

Imaginate que tenes un monton de puntos (x_i, y_i) y queres encontrar una recta $y = mx + b$ que los aproxime lo mejor posible. Si tenes n puntos, te quedan n ecuaciones:

$$mx_i + b = y_i, \quad i = 1, \dots, n$$

Esto es un sistema $Ax = b$ donde:

$$A = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{pmatrix}, \quad x = \begin{pmatrix} m \\ b \end{pmatrix}, \quad b = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Si $n > 2$, el sistema está **sobredeterminado**: hay más ecuaciones que incógnitas. En general, no va a tener solución exacta. Entonces, ¿qué hacemos? Buscamos x que minimice el error:

$$\min_x \|Ax - b\|_2^2 = \min_x \sum_{i=1}^n (A_i x - b_i)^2$$

Este es el **problema de cuadrados mínimos** (least squares).

Traducción Matemática del problema de cuadrados mínimos La expresión $\|Ax - b\|_2^2 = \sum_{i=1}^n (A_i x - b_i)^2$ se lee en criollo: "queremos encontrar el vector x que hace que la suma de los cuadrados de las diferencias entre los valores predichos $A_i x$ y los valores observados b_i sea lo más chica posible". La norma 2 al cuadrado es simplemente una forma compacta de escribir esa suma de cuadrados. ¿Por qué usamos el cuadrado del error y no el error absoluto? Por varias razones: es diferenciable, penaliza más los errores grandes, y tiene una solución analítica simple. Pensa en el problema como si estuvieras tratando de dibujar la mejor línea recta a través de un montón de puntos. La "mejor" línea es la que minimiza la distancia vertical al cuadrado entre la línea y cada punto.

Ecuaciones normales

La solución del problema de cuadrados mínimos satisface las **ecuaciones normales**:

$$A^T A x = A^T b$$

Traducción Matemática de las ecuaciones normales La ecuación $A^T A x = A^T b$ se lee: "el vector x que minimiza el error es aquel que hace que el residuo $r = b - Ax$ sea ortogonal al espacio columna de A ". En otras palabras, el error es perpendicular a todas las columnas de A . Esto garantiza que no haya una dirección en el espacio columna de A en la que puedas reducir el error.

Si A tiene rango completo (columnas linealmente independientes), entonces $A^T A$ es invertible y la solución es:

$$x = (A^T A)^{-1} A^T b$$

¿Por qué las ecuaciones normales pueden ser problemáticas?

El problema con las ecuaciones normales es que $A^T A$ puede estar **mal condicionada**. Si las columnas de A son casi linealmente dependientes, el número de condición de $A^T A$ es el cuadrado del número de condición de A :

$$\kappa_2(A^T A) = \kappa_2(A)^2$$

Esto significa que si A ya está mal condicionada, las ecuaciones normales lo empeoran. Por eso, en la práctica, rara vez se usan las ecuaciones normales. En su lugar, se usa la factorización QR.

Grafico: ajuste de una recta por cuadrados mínimos

Vamos a visualizar el problema de ajustar una recta a datos ruidosos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generamos datos con ruido
5 np.random.seed(42)
6 n = 50
7 x = np.linspace(0, 10, n)
8 m_real = 2.5
9 b_real = -1.0
10 y_real = m_real * x + b_real
11 y = y_real + 2 * np.random.randn(n) # ruido
12
13 # Resolvemos cuadrados minimos para encontrar m y b
14 A = np.column_stack([x, np.ones(n)])
15 x_sol, _, _ = np.linalg.lstsq(A, y, rcond=None)
16 m, b = x_sol
17
18 # Graficamos
19 plt.figure(figsize=(10, 6))
20 plt.scatter(x, y, label='Datos con ruido', color='blue', alpha=0.6)
21 plt.plot(x, y_real, 'g--', label='Recta real (sin ruido)', linewidth=2)
22 plt.plot(x, m * x + b, 'r-', label=f'Recta ajustada: y = {m:.2f}x + {b:.2f}',
23         linewidth=2)
24 plt.xlabel('x', fontsize=14)
25 plt.ylabel('y', fontsize=14)
26 plt.title('Ajuste de una recta por cuadrados minimos', fontsize=16)
27 plt.legend(fontsize=12)
28 plt.grid(True, alpha=0.3)
29 plt.show()

```

Listing 77: Ajuste de una recta por cuadrados minimos

En el grafico, ves los datos con ruido (puntos azules), la recta real (verde punteada) y la recta ajustada por cuadrados minimos (rojo). La recta ajustada es la que minimiza la suma de los cuadrados de las distancias verticales entre los puntos y la recta.

Cuadrados minimos con factorizacion QR

La factorizacion QR es la forma mas estable de resolver problemas de cuadrados minimos. Si $A = QR$, donde Q tiene columnas ortonormales y R es triangular superior, entonces:

$$\|Ax - b\|_2 = \|QRx - b\|_2 = \|Rx - Q^T b\|_2$$

Porque la multiplicacion por Q^T preserva la norma (las matrices ortogonales son isometrias). El problema se reduce a resolver el sistema triangular:

$$Rx = Q^T b$$

Traduccion Matematica de QR para cuadrados minimos La idea es sencilla: en lugar de resolver el sistema original (que esta sobredeterminado), transformamos el problema a uno equivalente donde la matriz es triangular superior. Como Q es ortogonal, no cambia la norma del error. Entonces, minimizar $\|Ax - b\|_2$ es lo mismo que minimizar $\|Rx - Q^T b\|_2$. Y como R es triangular superior, podemos resolverlo por sustitucion hacia atras.

¿Por que QR es mejor que las ecuaciones normales?

1. **Estabilidad:** QR no requiere calcular $A^T A$, que puede estar mal condicionada.
2. **Exactitud:** QR produce errores relativos del orden de $\kappa(A)\epsilon_{\text{mach}}$, mientras que las ecuaciones normales producen errores del orden de $\kappa(A)^2\epsilon_{\text{mach}}$.
3. **Eficiencia:** Para matrices ralas, QR puede ser mas eficiente que las ecuaciones normales.

Ejemplo: QR para cuadrados minimos

Vamos a resolver un problema de cuadrados minimos usando QR.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def qr_lstsq(A, b):
5     """Resuelve  $Ax = b$  en el sentido de cuadrados minimos usando QR."""
6     Q, R = np.linalg.qr(A)
7     # Resolvemos  $Rx = Q^T b$ 
8     return np.linalg.solve(R, Q.T @ b)
9
10 # Generamos datos
11 np.random.seed(42)
12 n = 100
13 x = np.linspace(0, 10, n)
14 y = 3.0 * x + 2.0 + 5 * np.random.randn(n)
15
16 # Construimos la matriz A para el ajuste de una recta
17 A = np.column_stack([x, np.ones(n)])
18 b = y
19
20 # Solucion con QR
21 x_qr = qr_lstsq(A, b)
22
23 # Solucion con ecuaciones normales
24 x_norm = np.linalg.solve(A.T @ A, A.T @ b)
25
26 print("Solucion con QR: m = {:.4f}, b = {:.4f}".format(x_qr[0], x_qr[1]))
27 print("Solucion con ecuaciones normales: m = {:.4f}, b = {:.4f}".format(x_norm
    [0], x_norm[1]))
28
29 # Calculamos el error de la solucion de QR
30 residuo_qr = np.linalg.norm(A @ x_qr - b)
31 residuo_norm = np.linalg.norm(A @ x_norm - b)
32 print("Residuo QR:", residuo_qr)
33 print("Residuo ecuaciones normales:", residuo_norm)
34
35 # Graficamos
36 plt.figure(figsize=(10, 6))
37 plt.scatter(x, y, label='Datos', alpha=0.6)
38 plt.plot(x, x_qr[0] * x + x_qr[1], 'r-', label='Ajuste por QR', linewidth=2)
39 plt.plot(x, x_norm[0] * x + x_norm[1], 'b--', label='Ajuste por ecuaciones
    normales', linewidth=2)
40 plt.xlabel('x', fontsize=14)
41 plt.ylabel('y', fontsize=14)
42 plt.title('Comparacion: QR vs ecuaciones normales', fontsize=16)
43 plt.legend(fontsize=12)
44 plt.grid(True, alpha=0.3)
45 plt.show()

```

Listing 78: Cuadrados minimos con QR

En este ejemplo, ambas soluciones son practicamente iguales porque el problema no esta mal condicionado. Pero cuando la matriz A esta mal condicionada, la diferencia se hace evidente.

La Descomposicion en Valores Singulares (SVD)

La SVD es posiblemente la factorizacion mas importante del algebra lineal numerica. Descompone cualquier matriz $A \in \mathbb{R}^{m \times n}$ como:

$$A = U \Sigma V^T$$

donde:

- $U \in \mathbb{R}^{m \times m}$ es ortogonal ($U^T U = I_m$).
- $V \in \mathbb{R}^{n \times n}$ es ortogonal ($V^T V = I_n$).
- $\Sigma \in \mathbb{R}^{m \times n}$ es diagonal con elementos $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0$, donde $p = \min(m, n)$.

Los σ_i se llaman **valores singulares** de A . Las columnas de U son los **vectores singulares izquierdos**, y las columnas de V son los **vectores singulares derechos**.

Traduccion Matematica de la SVD La SVD se lee en criollo: cualquier matriz se puede descomponer como el producto de una rotacion (o reflexion) U , una matriz diagonal Σ que estira o comprime en cada direccion, y otra rotacion V^T . Pensa en A como una transformacion lineal: primero rota el espacio (con V^T), luego estira las direcciones principales (con Σ), y finalmente rota de nuevo (con U). Los valores singulares σ_i son los factores de estiramiento en cada direccion principal.

Propiedades de la SVD

1. Los valores singulares son unicos (salvo orden).
2. El rango de A es el numero de valores singulares no nulos.
3. $\|A\|_2 = \sigma_1$ (norma espectral).
4. $\|A\|_F = \sqrt{\sum_{i=1}^p \sigma_i^2}$ (norma de Frobenius).
5. Los autovalores de $A^T A$ son σ_i^2 , y los autovectores son las columnas de V .
6. Los autovalores de $A A^T$ son σ_i^2 , y los autovectores son las columnas de U .

La SVD y los cuadrados minimos

La SVD proporciona una forma elegante de resolver el problema de cuadrados minimos. Si $A = U \Sigma V^T$, entonces la solucion de norma minima del problema $\min_x \|Ax - b\|_2$ es:

$$x = V \Sigma^+ U^T b$$

donde Σ^+ es la **pseudoinversa** de Σ , que se obtiene tomando $1/\sigma_i$ para los valores singulares no nulos y 0 para los nulos.

Grafico: valores singulares de una matriz

Vamos a calcular los valores singulares de una matriz y graficarlos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generamos una matriz aleatoria
5 np.random.seed(42)
6 m, n = 20, 10
7 A = np.random.randn(m, n)
8
9 # Calculamos la SVD
10 U, s, Vt = np.linalg.svd(A)
11
12 # Graficamos los valores singulares
13 plt.figure(figsize=(10, 6))
14 plt.semilogy(s, 'o-', linewidth=2, markersize=8)
15 plt.xlabel('Indice i', fontsize=14)
16 plt.ylabel('Valor singular sigma_i', fontsize=14)
17 plt.title('Valores singulares de una matriz aleatoria (m=20, n=10)', fontsize
18           =16)
19 plt.grid(True, alpha=0.3)
20 plt.show()

```

Listing 79: Valores singulares de una matriz aleatoria

En el grafico, ves los valores singulares en escala logaritmica. Para una matriz aleatoria, los valores singulares suelen decaer gradualmente, sin un corte brusco. Cuando hay un corte (algunos valores singulares muy pequenos), la matriz esta cerca de ser singular.

La Pseudoinversa de Moore-Penrose

La pseudoinversa A^+ de una matriz $A \in \mathbb{R}^{m \times n}$ es una generalizacion de la inversa para matrices que no son cuadradas o no tienen rango completo. Se define a partir de la SVD:

$$A^+ = V \Sigma^+ U^T$$

donde Σ^+ es la matriz diagonal con $1/\sigma_i$ en la diagonal para los valores singulares no nulos, y 0 en el resto.

Propiedades de la pseudoinversa

La pseudoinversa satisface las siguientes propiedades (llamadas condiciones de Moore-Penrose):

1. $AA^+A = A$
2. $A^+AA^+ = A^+$
3. $(AA^+)^T = AA^+$
4. $(A^+A)^T = A^+A$

Traduccion Matematica de la pseudoinversa La pseudoinversa se lee en criollo: "es la matriz que, en el sentido de cuadrados minimos, actua como inversa". Si el sistema $Ax = b$ tiene solucion, $x = A^+b$ es la solucion de norma minima. Si no tiene solucion, $x = A^+b$ es la solucion de cuadrados minimos que ademas tiene norma minima.

Pensa en la pseudoinversa como una forma de invertir una matriz que no es invertible. Si la matriz tiene columnas linealmente dependientes, la pseudoinversa te da la solucion que minimiza el error y, entre todas las soluciones, la que tiene la norma mas chica.

Ejemplo: resolucion de un sistema con la pseudoinversa

```

1 import numpy as np
2
3 # Definimos una matriz singular
4 A = np.array([[1, 1], [2, 2], [3, 3]])
5 b = np.array([2, 4, 6])
6
7 # Calculamos la pseudoinversa
8 A_pinv = np.linalg.pinv(A)
9
10 # Resolvemos el sistema en el sentido de cuadrados minimos
11 x = A_pinv @ b
12
13 print("Matriz A:")
14 print(A)
15 print("Pseudoinversa A^+:")
16 print(A_pinv)
17 print("Solucion de cuadrados minimos x = A^+ b:")
18 print(x)
19 print("Error Ax - b:", A @ x - b)
20 print("Norma del error:", np.linalg.norm(A @ x - b))

```

Listing 80: Pseudoinversa de Moore-Penrose

Este ejemplo muestra una matriz con columnas linealmente dependientes (todas las columnas son multiples de la primera). La pseudoinversa encuentra la solucion de cuadrados minimos (que en este caso es exacta porque b esta en el espacio columna de A).

Regularizacion de Tikhonov

Cuando el problema de cuadrados minimos esta mal condicionado (por ejemplo, cuando la matriz tiene valores singulares muy pequenos), la solucion puede ser muy sensible a perturbaciones en los datos. La **regularizacion de Tikhonov** (o regularizacion L^2) consiste en agregar un termino de penalizacion a la funcion objetivo:

$$\min_x (\|Ax - b\|_2^2 + \lambda^2 \|x\|_2^2)$$

donde $\lambda > 0$ es un parametro de regularizacion.

Traduccion Matematica de Tikhonov La regularizacion de Tikhonov se lee en criollo: "queremos encontrar x que no solo ajuste bien los datos, sino que ademas tenga una norma no demasiado grande". El parametro λ controla el balance entre el ajuste a los datos (primer termino) y la suavidad o pequenez de la solucion (segundo termino). Si λ es pequeno, la solucion se parece a la de cuadrados minimos. Si λ es grande, la solucion tiende a cero.

Pensa en esto como si estuvieras tratando de dibujar una curva a traves de puntos con mucho ruido. Si permites que la curva sea muy complicada (norma grande), puede que siga el ruido. La regularizacion te fuerza a elegir una curva mas suave, aunque no pase exactamente por todos los puntos.

Solucion de Tikhonov via SVD

Si $A = U\Sigma V^T$, la solucion regularizada es:

$$x_\lambda = \sum_{i=1}^p \frac{\sigma_i}{\sigma_i^2 + \lambda^2} (u_i^T b) v_i$$

Los factores $\frac{\sigma_i}{\sigma_i^2 + \lambda^2}$ actuan como un filtro: para $\sigma_i \gg \lambda$, son aproximadamente $1/\sigma_i$; para $\sigma_i \ll \lambda$, son aproximadamente σ_i/λ^2 , que es muy pequeno.

Grafico: efecto de la regularizacion

Vamos a ver como la regularizacion afecta la solucion de un problema mal condicionado.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def tikhonov_svd(A, b, lambdas):
5     """
6     Calcula la solucion de Tikhonov para diferentes lambdas usando la SVD.
7     """
8     U, s, Vt = np.linalg.svd(A, full_matrices=False)
9     V = Vt.T
10    solutions = []
11    for lam in lambdas:
12        # Filtro de Tikhonov
13        filtro = s / (s**2 + lam**2)
14        # x = V * diag(filtro) * U^T * b
15        x = V @ (filtro * (U.T @ b))
16        solutions.append(x)
17    return solutions
18
19 # Generamos un problema mal condicionado
20 np.random.seed(42)
21 n = 5
22 A = np.diag([10.0, 1.0, 0.1, 0.01, 0.001])
23 b = np.array([1.0, 1.0, 1.0, 1.0, 1.0])
24
25 # Aplicamos regularizacion para diferentes lambdas
26 lambdas = [1e-6, 1e-4, 1e-2, 1.0]
27 solutions = tikhonov_svd(A, b, lambdas)
28
29 # Graficamos las soluciones
30 plt.figure(figsize=(10, 6))
31 for i, (lam, x) in enumerate(zip(lambdas, solutions)):
32     plt.plot(range(1, n+1), x, 'o-', label=f'lambda = {lam}', linewidth=2,
33             markersize=8)
34
35 plt.xlabel('Indice i', fontsize=14)
36 plt.ylabel('x_i', fontsize=14)
37 plt.title('Efecto de la regularizacion de Tikhonov en la solucion', fontsize=16)
38 plt.legend(fontsize=12)
39 plt.grid(True, alpha=0.3)

```

```
39 plt.show()
```

Listing 81: Efecto de la regularizacion de Tikhonov

En el grafico, ves como para λ pequeno, la solucion es muy sensible a los valores singulares pequenos (los ultimos componentes explotan). A medida que λ aumenta, la solucion se vuelve mas suave y estable.

Aproximacion de Bajo Rango

Una de las aplicaciones mas potentes de la SVD es la **aproximacion de bajo rango**. El **Teorema de Eckart-Young** dice que la mejor aproximacion de rango k de una matriz A (en la norma espectral o en la norma de Frobenius) se obtiene truncando la SVD:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$$

En otras palabras, para obtener la mejor aproximacion de rango k , te quedas con los k valores singulares mas grandes y descartas el resto.

Traduccion Matematica de la aproximacion de bajo rango La aproximacion de bajo rango se lee en criollo: "de todas las matrices de rango k , la que mas se parece a A (en el sentido de la norma de Frobenius) es la que se obtiene truncando la SVD". Es como si estuvieras comprimiendo una imagen: te quedas con las componentes principales (las que tienen mas informacion) y descartas el ruido o los detalles finos.

Pensa en la SVD como una forma de descomponer una matriz en capas de informacion, ordenadas por importancia. La primera capa (el primer valor singular) es la que mas contribuye a la matriz. La segunda capa es la siguiente mas importante, y asi sucesivamente. Si quieres una version simplificada de la matriz, te quedas con las primeras k capas.

Ejemplo: compresion de imagenes

Un ejemplo tipico de aproximacion de bajo rango es la compresion de imagenes. Una imagen en escala de grises es una matriz de pixeles. Si truncamos la SVD a un rango k , obtenemos una version comprimida de la imagen.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import misc
4
5 def comprimir_svd(imagen, k):
6     """
7     Comprime una imagen usando la SVD truncada a rango k.
8     """
9     U, s, Vt = np.linalg.svd(imagen, full_matrices=False)
10    # Truncamos
11    U_k = U[:, :k]
12    s_k = s[:k]
13    Vt_k = Vt[:k, :]
14    # Reconstruimos
15    imagen_comp = U_k @ np.diag(s_k) @ Vt_k
16    return imagen_comp
17
18 # Cargamos una imagen (usamos una imagen de ejemplo de scipy)
19 try:
```



```

20     imagen = misc.face(gray=True)
21 except:
22     # Si no tenemos scipy, generamos una imagen de prueba
23     imagen = np.random.rand(200, 200)
24
25 # Comprimos con diferentes rangos
26 rangos = [5, 20, 50, 100, 200]
27 fig, axes = plt.subplots(2, 3, figsize=(12, 8))
28 axes = axes.flatten()
29
30 # Imagen original
31 axes[0].imshow(imagen, cmap='gray')
32 axes[0].set_title('Original')
33
34 for i, k in enumerate(rangos):
35     imagen_comp = comprimir_svd(imagen, k)
36     axes[i+1].imshow(imagen_comp, cmap='gray')
37     axes[i+1].set_title(f'Rango {k}')
38
39 for ax in axes:
40     ax.axis('off')
41
42 plt.tight_layout()
43 plt.show()

```

Listing 82: Compresion de una imagen con SVD

En el grafico, ves como la imagen se vuelve mas borrosa a medida que reduces el rango. Pero con un rango relativamente pequeno (por ejemplo, $k = 50$), la imagen aun es reconocible, mientras que el espacio de almacenamiento se reduce drasticamente.

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Demostracion de las ecuaciones normales:** Demuestra que la solucion del problema de cuadrados minimos $\min_x \|Ax - b\|_2^2$ satisface las ecuaciones normales $A^T Ax = A^T b$. Sugerencia: usa el hecho de que el gradiente de $\|Ax - b\|_2^2$ es $2A^T(Ax - b)$.
- T2. Proyeccion ortogonal y cuadrados minimos:** Demuestra que la solucion de cuadrados minimos x satisface que el residuo $r = b - Ax$ es ortogonal al espacio columna de A .
- T3. Norma de Frobenius y SVD:** Demuestra que $\|A\|_F^2 = \sum_{i=1}^p \sigma_i^2$, donde σ_i son los valores singulares de A .
- T4. Numero de condicion y SVD:** Demuestra que $\kappa_2(A) = \frac{\sigma_1}{\sigma_n}$ para una matriz $A \in \mathbb{R}^{n \times n}$ invertible.
- T5. Pseudoinversa y cuadrados minimos:** Demuestra que $x = A^+b$ es la solucion de cuadrados minimos de norma minima.
- T6. Propiedades de la pseudoinversa:** Verifica que la pseudoinversa satisface las cuatro condiciones de Moore-Penrose.
- T7. Teorema de Eckart-Young:** Demuestra que la mejor aproximacion de rango k de A en la norma de Frobenius es la SVD truncada $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$.

- T8. Regularizacion de Tikhonov:** Demuestra que la solucion de $\min_x (\|Ax - b\|_2^2 + \lambda^2 \|x\|_2^2)$ es $x_\lambda = (A^T A + \lambda^2 I)^{-1} A^T b$.
- T9. Filtro de Tikhonov:** Usando la SVD, demuestra que $x_\lambda = \sum_{i=1}^p \frac{\sigma_i}{\sigma_i^2 + \lambda^2} (u_i^T b) v_i$.
- T10. QR y cuadrados minimos:** Demuestra que la solucion de cuadrados minimos se puede obtener resolviendo $Rx = Q^T b$, donde $A = QR$.

Parte Computacional (10 ejercicios)

- C1. Ajuste polinomial:** Dados puntos (x_i, y_i) , escribe una funcion que ajuste un polinomio de grado d usando cuadrados minimos. Prueba con datos generados a partir de un polinomio conocido con ruido.
- C2. Comparacion de metodos:** Resuelve el problema de cuadrados minimos para la matriz de Hilbert H_n con $b = H_n \mathbf{1}$ usando ecuaciones normales y QR. Grafica el error en funcion de n para $n = 2, \dots, 15$.
- C3. Pseudoinversa:** Implementa tu propia funcion para calcular la pseudoinversa de una matriz usando la SVD. Compara con `np.linalg.pinv`.
- C4. Regularizacion de Tikhonov:** Implementa la regularizacion de Tikhonov usando la SVD. Grafica la solucion para diferentes valores de λ para un problema mal condicionado.
- C5. Seleccion del parametro de regularizacion:** Implementa el metodo de la curva L (L-curve) para elegir el parametro λ en Tikhonov. Grafica la curva y encuentra el punto de maxima curvatura.
- C6. Compresion de imagenes:** Usa la SVD para comprimir una imagen en escala de grises. Grafica la imagen original y las versiones comprimidas para diferentes rangos. Calcula la tasa de compresion y el error relativo.
- C7. Ajuste de una funcion no lineal:** Dados datos (x_i, y_i) , ajusta una funcion de la forma $y = ae^{bx} + c$ usando cuadrados minimos. Sugerencia: linealiza el problema transformando las variables.
- C8. Descomposicion de bajo rango para datos faltantes:** Dada una matriz con algunos valores faltantes (NaN), implementa un algoritmo simple para completar la matriz usando la SVD (por ejemplo, algoritmo de soft-impute).
- C9. Analisis de componentes principales (PCA):** Usa la SVD para implementar PCA. Dado un conjunto de datos en $\mathbb{R}^d$, reduce la dimensionalidad a $k < d$ y grafica los datos proyectados en las primeras dos componentes principales.
- C10. Descomposicion en valores singulares de una imagen:** Calcula la SVD de una imagen y analiza la contribucion de cada valor singular. Grafica el error de aproximacion en funcion del rango.

Soluciones

Soluciones Teoricas

T1. La funcion a minimizar es $f(x) = \|Ax - b\|_2^2 = (Ax - b)^T (Ax - b)$. Desarrollando:

$$f(x) = x^T A^T A x - 2b^T A x + b^T b$$

El gradiente es $\nabla f(x) = 2A^T Ax - 2A^T b$. Igualando a cero: $A^T Ax = A^T b$.

T2. Si x es la solución de cuadrados mínimos, entonces $A^T(b - Ax) = 0$. Esto significa que el residuo $r = b - Ax$ es ortogonal a todas las columnas de A , es decir, $r \perp \text{Col}(A)$.

T3. Si $A = U\Sigma V^T$, entonces $\|A\|_F^2 = \text{tr}(A^T A) = \text{tr}(V\Sigma^T U^T U \Sigma V^T) = \text{tr}(V\Sigma^T \Sigma V^T) = \text{tr}(\Sigma^T \Sigma) = \sum_{i=1}^p \sigma_i^2$.

T4. Por definición, $\|A\|_2 = \sigma_1$ y $\|A^{-1}\|_2 = 1/\sigma_n$. Entonces $\kappa_2(A) = \sigma_1/\sigma_n$.

T5. La solución general de cuadrados mínimos es $x = A^+b + (I - A^+A)z$ para cualquier z . La norma mínima se obtiene cuando $z = 0$, porque $(I - A^+A)z$ está en el núcleo de A , y es ortogonal a A^+b . Por lo tanto, $x = A^+b$ es la solución de norma mínima.

T6. Las propiedades se verifican directamente usando $A = U\Sigma V^T$ y $A^+ = V\Sigma^+U^T$. Por ejemplo, $AA^+A = U\Sigma V^T V\Sigma^+U^T U \Sigma V^T = U\Sigma \Sigma^+ \Sigma V^T = U\Sigma V^T = A$.

T7. Para cualquier matriz B de rango $\leq k$, $\|A - B\|_F^2 = \sum_{i=1}^p \sigma_i^2 - 2 \sum_{i=1}^k \sigma_i^2 + \|B\|_F^2$. El mínimo se alcanza cuando $B = A_k$, y el error es $\sum_{i=k+1}^p \sigma_i^2$.

T8. La función a minimizar es $f(x) = \|Ax - b\|_2^2 + \lambda^2 \|x\|_2^2 = x^T A^T Ax - 2b^T Ax + b^T b + \lambda^2 x^T x$. El gradiente es $\nabla f(x) = 2(A^T A + \lambda^2 I)x - 2A^T b$. Igualando a cero: $(A^T A + \lambda^2 I)x = A^T b$.

T9. Usando la SVD, $A^T A + \lambda^2 I = V(\Sigma^T \Sigma + \lambda^2 I)V^T$. Entonces:

$$x_\lambda = V(\Sigma^T \Sigma + \lambda^2 I)^{-1} \Sigma^T U^T b = \sum_{i=1}^p \frac{\sigma_i}{\sigma_i^2 + \lambda^2} (u_i^T b) v_i$$

T10. Si $A = QR$, entonces $\|Ax - b\|_2^2 = \|QRx - b\|_2^2 = \|Rx - Q^T b\|_2^2$. El mínimo se alcanza cuando $Rx = Q^T b$.

Soluciones Computacionales

C1.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def ajuste_polinomial(x, y, grado):
5     """Ajusta un polinomio de grado d por cuadrados mínimos."""
6     # Construimos la matriz de Vandermonde
7     A = np.vander(x, grado+1, increasing=True)
8     # Resolvemos con QR
9     Q, R = np.linalg.qr(A)
10    coefs = np.linalg.solve(R, Q.T @ y)
11    return coefs
12
13 # Generamos datos
14 np.random.seed(42)
15 n = 50
16 x = np.linspace(0, 10, n)
17 y = 2.0 * x**3 - 1.5 * x**2 + 3.0 * x + 1.0 + 10 * np.random.randn(n)
18
19 # Ajustamos polinomios de diferentes grados
20 for grado in [1, 2, 3]:
21     coefs = ajuste_polinomial(x, y, grado)
22     y_pred = np.polyval(coefs[:-1], x)
23     error = np.linalg.norm(y - y_pred) / np.linalg.norm(y)
24     print(f"Grado {grado}: Error relativo = {error:.4f}")
25
26 # Graficamos
27 plt.figure(figsize=(10, 6))
28 plt.scatter(x, y, label='Datos', alpha=0.6)
29 for grado in [1, 2, 3]:

```

```

30     coefs = ajuste_polinomial(x, y, grado)
31     y_pred = np.polyval(coefs[::-1], x)
32     plt.plot(x, y_pred, label=f'Grado {grado}')
33
34 plt.xlabel('x')
35 plt.ylabel('y')
36 plt.title('Ajuste polinomial por cuadrados minimos')
37 plt.legend()
38 plt.grid(True)
39 plt.show()

```

Listing 83: Ajuste polinomial

C2.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def hilbert(n):
5     return np.array([[1/(i+j+1) for j in range(n)] for i in range(n)])
6
7 def error_relativo(A, b, x_exacto, metodo):
8     if metodo == 'normales':
9         x = np.linalg.solve(A.T @ A, A.T @ b)
10    elif metodo == 'qr':
11        Q, R = np.linalg.qr(A)
12        x = np.linalg.solve(R, Q.T @ b)
13    return np.linalg.norm(x - x_exacto) / np.linalg.norm(x_exacto)
14
15 n_values = range(2, 15)
16 errores_norm = []
17 errores_qr = []
18
19 for n in n_values:
20     H = hilbert(n)
21     x_exacto = np.ones(n)
22     b = H @ x_exacto
23     errores_norm.append(error_relativo(H, b, x_exacto, 'normales'))
24     errores_qr.append(error_relativo(H, b, x_exacto, 'qr'))
25
26 plt.figure(figsize=(10, 6))
27 plt.semilogy(n_values, errores_norm, 'o-', label='Ecuaciones normales')
28 plt.semilogy(n_values, errores_qr, 's-', label='QR')
29 plt.xlabel('n')
30 plt.ylabel('Error relativo')
31 plt.title('Comparacion: ecuaciones normales vs QR')
32 plt.legend()
33 plt.grid(True)
34 plt.show()

```

Listing 84: Ecuaciones normales vs QR para matriz de Hilbert

C3.

```

1 import numpy as np
2
3 def pseudo_inversa(A):
4     """Calcula la pseudoinversa de A usando SVD."""
5     U, s, Vt = np.linalg.svd(A, full_matrices=False)
6     # Invertimos los valores singulares no nulos
7     s_inv = np.zeros_like(s)
8     tol = 1e-12 * np.max(s)

```

```

9     s_inv[s > tol] = 1.0 / s[s > tol]
10    # Reconstruimos la pseudoinversa
11    return Vt.T @ np.diag(s_inv) @ U.T
12
13    # Test
14    A = np.array([[1, 1], [2, 2], [3, 3]])
15    A_pinv_mia = pseudo_inversa(A)
16    A_pinv_np = np.linalg.pinv(A)
17
18    print("Pseudoinversa (implementada):")
19    print(A_pinv_mia)
20    print("Pseudoinversa (np.linalg.pinv):")
21    print(A_pinv_np)
22    print("Diferencia:", np.linalg.norm(A_pinv_mia - A_pinv_np))

```

Listing 85: Pseudoinversa con SVD

C4.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def tikhonov_svd(A, b, lam):
5      """Solucion de Tikhonov usando SVD."""
6      U, s, Vt = np.linalg.svd(A, full_matrices=False)
7      V = Vt.T
8      filtro = s / (s**2 + lam**2)
9      return V @ (filtro * (U.T @ b))
10
11  # Problema mal condicionado
12  A = np.diag([1.0, 0.1, 0.01, 0.001])
13  b = np.ones(4)
14
15  lambdas = [0.0, 1e-4, 1e-2, 1.0]
16  plt.figure(figsize=(10, 6))
17  for lam in lambdas:
18      x = tikhonov_svd(A, b, lam)
19      plt.plot(range(1, 5), x, 'o-', label=f'lambda = {lam}')
20
21  plt.xlabel('Indice i')
22  plt.ylabel('x_i')
23  plt.title('Regularizacion de Tikhonov')
24  plt.legend()
25  plt.grid(True)
26  plt.show()

```

Listing 86: Regularizacion de Tikhonov

C5.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def curva_L(A, b, lambdas):
5      """Calcula la curva L para regularizacion de Tikhonov."""
6      U, s, Vt = np.linalg.svd(A, full_matrices=False)
7      V = Vt.T
8      norm_sol = []
9      norm_res = []
10     for lam in lambdas:
11         filtro = s / (s**2 + lam**2)
12         x = V @ (filtro * (U.T @ b))

```

```

13     norm_sol.append(np.linalg.norm(x))
14     norm_res.append(np.linalg.norm(A @ x - b))
15     return norm_sol, norm_res
16
17 # Problema de prueba
18 A = np.random.randn(20, 10)
19 b = np.random.randn(20)
20 lambdas = np.logspace(-6, 2, 50)
21 norm_sol, norm_res = curva_L(A, b, lambdas)
22
23 plt.figure(figsize=(8, 6))
24 plt.loglog(norm_res, norm_sol, 'o-')
25 plt.xlabel('Norma del residuo ||Ax - b||_2')
26 plt.ylabel('Norma de la solución ||x||_2')
27 plt.title('Curva L para regularización de Tikhonov')
28 plt.grid(True)
29 plt.show()

```

Listing 87: Curva L para Tikhonov

C6.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import misc
4
5 def comprimir_svd(imagen, k):
6     U, s, Vt = np.linalg.svd(imagen, full_matrices=False)
7     U_k = U[:, :k]
8     s_k = s[:k]
9     Vt_k = Vt[:, :k]
10    return U_k @ np.diag(s_k) @ Vt_k
11
12 # Cargamos una imagen
13 imagen = misc.face(gray=True)
14
15 # Comprimimos con diferentes rangos
16 rangos = [5, 20, 50, 100, 200]
17 fig, axes = plt.subplots(2, 3, figsize=(12, 8))
18 axes = axes.flatten()
19
20 axes[0].imshow(imagen, cmap='gray')
21 axes[0].set_title('Original')
22
23 for i, k in enumerate(rangos):
24     img_comp = comprimir_svd(imagen, k)
25     axes[i+1].imshow(img_comp, cmap='gray')
26     axes[i+1].set_title(f'Rango {k}')
27     # Calculamos la tasa de compresión y el error
28     m, n = imagen.shape
29     compresion = (m*k + k*n) / (m*n)
30     error = np.linalg.norm(imagen - img_comp, 'fro') / np.linalg.norm(imagen, 'fro')
31     print(f'Rango {k}: Compresion = {compresion:.2f}, Error = {error:.4f}")
32
33 for ax in axes:
34     ax.axis('off')
35 plt.tight_layout()
36 plt.show()

```

Listing 88: Compresión de imágenes con SVD

C7.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generamos datos de una exponencial con ruido
5 np.random.seed(42)
6 n = 100
7 x = np.linspace(0, 5, n)
8 a_real, b_real, c_real = 2.0, -0.5, 1.0
9 y_real = a_real * np.exp(b_real * x) + c_real
10 y = y_real + 0.5 * np.random.randn(n)
11
12 # Linealizamos:  $y - c = a * \exp(b * x)$ 
13 # Tomamos logaritmo:  $\log(y - c) = \log(a) + b * x$ 
14 # Pero c es desconocido... usamos un enfoque iterativo o minimizamos
    directamente
15 # Aqui usamos scipy.optimize para el ajuste no lineal
16 from scipy.optimize import curve_fit
17
18 def func(x, a, b, c):
19     return a * np.exp(b * x) + c
20
21 popt, pcov = curve_fit(func, x, y, p0=[1, -1, 1])
22 a_est, b_est, c_est = popt
23
24 print(f"Valores reales: a={a_real}, b={b_real}, c={c_real}")
25 print(f"Valores estimados: a={a_est:.4f}, b={b_est:.4f}, c={c_est:.4f}")
26
27 plt.figure(figsize=(10, 6))
28 plt.scatter(x, y, label='Datos', alpha=0.6)
29 plt.plot(x, y_real, 'g--', label='Funcion real')
30 plt.plot(x, func(x, *popt), 'r-', label='Ajuste')
31 plt.xlabel('x')
32 plt.ylabel('y')
33 plt.title('Ajuste de una funcion exponencial')
34 plt.legend()
35 plt.grid(True)
36 plt.show()

```

Listing 89: Ajuste de una funcion no lineal

C8.

```

1 import numpy as np
2
3 def completar_svd(A, k, max_iter=100):
4     """
5     Completa una matriz con datos faltantes (NaN) usando SVD.
6     """
7     # Rellenamos los NaN con el promedio de la columna
8     A_filled = A.copy()
9     for j in range(A.shape[1]):
10         col = A[:, j]
11         mean = np.nanmean(col)
12         A_filled[np.isnan(col), j] = mean
13
14     for _ in range(max_iter):
15         # SVD truncada
16         U, s, Vt = np.linalg.svd(A_filled, full_matrices=False)
17         U_k = U[:, :k]
18         s_k = s[:k]

```

```

19     Vt_k = Vt[:k, :]
20     A_approx = U_k @ np.diag(s_k) @ Vt_k
21
22     # Actualizamos solo los valores faltantes
23     A_filled[np.isnan(A)] = A_approx[np.isnan(A)]
24
25     return A_filled
26
27 # Ejemplo: matriz con datos faltantes
28 np.random.seed(42)
29 A = np.random.randn(10, 10)
30 # Ocultamos algunos valores
31 mask = np.random.rand(10, 10) < 0.3
32 A_missing = A.copy()
33 A_missing[mask] = np.nan
34
35 # Completamos
36 A_completa = completar_svd(A_missing, k=3)
37
38 print("Matriz original:")
39 print(A)
40 print("\nMatriz con datos faltantes:")
41 print(A_missing)
42 print("\nMatriz completada:")
43 print(A_completa)
44 print(f"Error de completado: {np.linalg.norm(A - A_completa, 'fro') / np.linalg.
    norm(A, 'fro'):.4f}")

```

Listing 90: Descomposicion de bajo rango para datos faltantes

C9.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_blobs
4
5 # Generamos datos en 3D
6 X, y = make_blobs(n_samples=300, n_features=3, centers=3, random_state=42)
7
8 # Centramos los datos
9 X_centered = X - np.mean(X, axis=0)
10
11 # PCA via SVD
12 U, s, Vt = np.linalg.svd(X_centered, full_matrices=False)
13 # Las componentes principales son las columnas de Vt.T
14 PC = Vt.T
15
16 # Proyectamos los datos en las primeras 2 componentes
17 X_pca = X_centered @ PC[:, :2]
18
19 # Graficamos
20 plt.figure(figsize=(10, 8))
21 scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', alpha=0.7)
22 plt.xlabel('Componente principal 1')
23 plt.ylabel('Componente principal 2')
24 plt.title('PCA: Datos proyectados en las dos primeras componentes')
25 plt.colorbar(scatter)
26 plt.grid(True)
27 plt.show()
28
29 # Varianza explicada
30 varianza_explicada = s**2 / np.sum(s**2)

```



```

31 print("Varianza explicada por cada componente principal:")
32 for i, v in enumerate(varianza_explicada):
33     print(f"PC{i+1}: {v:.4f} ({v*100:.2f}%)")
34 print(f"Varianza acumulada primeras 2 componentes: {np.sum(varianza_explicada
    [:2]):.4f}")

```

Listing 91: Analisis de componentes principales (PCA)

C10.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import misc
4
5 # Cargamos una imagen
6 imagen = misc.face(gray=True)
7
8 # Calculamos la SVD
9 U, s, Vt = np.linalg.svd(imagen, full_matrices=False)
10
11 # Graficamos los valores singulares
12 plt.figure(figsize=(12, 5))
13
14 plt.subplot(1, 2, 1)
15 plt.semilogy(s, 'o-')
16 plt.xlabel('Indice i')
17 plt.ylabel('Valor singular sigma_i')
18 plt.title('Valores singulares de la imagen')
19 plt.grid(True)
20
21 # Calculamos el error de aproximacion en funcion del rango
22 rangos = np.arange(1, len(s)+1)
23 errores = []
24 for k in rangos:
25     error = np.sqrt(np.sum(s[k:]**2)) / np.sqrt(np.sum(s**2))
26     errores.append(error)
27
28 plt.subplot(1, 2, 2)
29 plt.plot(rangos, errores)
30 plt.xlabel('Rango k')
31 plt.ylabel('Error relativo de aproximacion')
32 plt.title('Error de aproximacion en funcion del rango')
33 plt.grid(True)
34
35 plt.tight_layout()
36 plt.show()

```

Listing 92: Analisis de la SVD de una imagen

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por el mundo de los cuadrados minimos y las descomposiciones. Viste que no era tan complicado. Arrancamos con el problema de ajustar una recta a datos, pasamos por QR, la SVD, la pseudoinversa, la regularizacion y la compresion de imagenes. Todo esta conectado: la SVD es la herramienta que unifica todos estos conceptos.

Los puntos clave para llevar:

- El problema de cuadrados minimos aparece cuando tenes mas ecuaciones que incognitas.

- Las ecuaciones normales son la solución teórica, pero en la práctica conviene usar QR.
- La SVD es la descomposición más poderosa: te da los valores singulares, que son como las "fuerzas" de la matriz.
- La pseudoinversa te permite "invertir" matrices que no son invertibles.
- La regularización de Tikhonov es crucial para problemas mal condicionados.
- La aproximación de bajo rango con SVD es la base de la compresión de datos y del PCA.

Ahora solo falta practicar. Los ejercicios están ahí para que te ensucies las manos. No te frustres si al principio no te salen; a mí me pasó lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. Y acórdate: la SVD es tu amiga, úsala con confianza. ¡Dale para adelante, que vos podés!

Autovalores y Autovectores

Como encontrar los modos propios de una matriz sin perder la cabeza.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a meterle a uno de los temas mas importantes del algebra lineal numerica: el calculo de autovalores y autovectores. Si llegaste hasta aca, seguro ya viste la teoria en algebra lineal: una matriz A tiene autovalores λ y autovectores v que satisfacen $Av = \lambda v$. Pero en la practica, calcular autovalores para matrices grandes es un desafio enorme.

Cuando curse Algebra, recuerdo que el profesor nos dijo: “el problema de autovalores es el segundo problema mas importante del algebra lineal, despues de resolver sistemas”. Y tenia razon. Los autovalores aparecen en todas partes: en la estabilidad de sistemas dinamicos, en la mecanica cuantica, en el analisis de vibraciones, en el procesamiento de imagenes, en el machine learning... basicamente, si hay una matriz, los autovalores estan a la vuelta de la esquina.

En este capitulo vamos a cubrir:

- El teorema de Gershgorin: como localizar autovalores sin calcularlos.
- El metodo de la potencia: el algoritmo mas simple para encontrar el autovalor dominante.
- El metodo QR: el algoritmo estandar para calcular todos los autovalores.
- La matriz companera y como calcular raices de polinomios.

La idea es que vos puedas implementar estos metodos en Python, entender por que funcionan, y saber cual usar en cada situacion. Asi que agarra un cafe, ponete comodo, y vamos a darle.

El problema de autovalores

Dada una matriz $A \in \mathbb{R}^{n \times n}$, queremos encontrar escalares λ (autovalores) y vectores no nulos v (autovectores) tales que:

$$Av = \lambda v$$

Traduccion Matematica del problema de autovalores La ecuacion $Av = \lambda v$ se lee en criollo: quando aplico la transformacion lineal A al vector v , el resultado es simplemente el mismo vector multiplicado por un escalar λ . Pensa en una transformacion lineal como una maquina que estira, rota y comprime el espacio. Los autovectores son las direcciones que no cambian de direccion (solo se estiran o comprimen), y los autovalores son los factores de estiramiento.

En la practica, el problema de autovalores es equivalente a encontrar las raices del polinomio caracteristico:

$$\det(A - \lambda I) = 0$$

Pero para $n > 4$, no hay formulas cerradas para las raices. Ademias, el polinomio caracteristico puede estar muy mal condicionado. Por eso necesitamos metodos numericos.

Propiedades importantes

1. Los autovalores de una matriz simétrica son reales.
2. Los autovalores de una matriz definida positiva son positivos.
3. Si A es diagonalizable, $A = V\Lambda V^{-1}$, donde Λ es diagonal con los autovalores.
4. La traza de A es la suma de los autovalores: $\text{tr}(A) = \sum_i \lambda_i$.
5. El determinante de A es el producto de los autovalores: $\det(A) = \prod_i \lambda_i$.

El teorema de los círculos de Gershgorin

Antes de calcular autovalores, a veces es útil saber donde están. El teorema de Gershgorin te da una región en el plano complejo que contiene todos los autovalores, sin hacer ningún cálculo complicado.

Para una matriz $A \in \mathbb{C}^{n \times n}$, definimos los **círculos de Gershgorin**:

$$D_i = \left\{ z \in \mathbb{C} : |z - a_{ii}| \leq \sum_{j \neq i} |a_{ij}| \right\}$$

El teorema dice que todos los autovalores de A están en la unión de estos círculos:

$$\sigma(A) \subseteq \bigcup_{i=1}^n D_i$$

Traducción Matemática de Gershgorin El teorema de Gershgorin se lee en criollo: cada autovalor de A está a una distancia de a_{ii} no mayor que la suma de los valores absolutos de los elementos no diagonales de la fila i . Pensa en cada fila como un centro a_{ii} con un radio R_i . El autovalor debe estar dentro de al menos uno de estos círculos.

Esto es útil porque te da una idea de donde buscar los autovalores. Si los círculos están separados, cada círculo contiene exactamente un autovalor.

Grafico: círculos de Gershgorin

Vamos a visualizar los círculos de Gershgorin para una matriz aleatoria.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gershgorin(A):
5     """Grafica los círculos de Gershgorin de la matriz A."""
6     n = A.shape[0]
7     # Centros y radios
8     centros = np.diag(A)
9     radios = np.sum(np.abs(A), axis=1) - np.abs(centros)
10
11     # Calculamos los autovalores para comparar
12     autovalores = np.linalg.eigvals(A)
13
14     # Graficamos
15     fig, ax = plt.subplots(figsize=(8, 8))
16
17     # Dibujamos los círculos

```

```

18     for i in range(n):
19         circle = plt.Circle((centros[i].real, centros[i].imag), radios[i],
20                             fill=False, color='blue', alpha=0.5)
21         ax.add_patch(circle)
22         # Centro
23         ax.plot(centros[i].real, centros[i].imag, 'bo', markersize=5)
24
25     # Autovalores
26     ax.plot(autovalores.real, autovalores.imag, 'rx', markersize=8, label='
Autovalores')
27
28     ax.set_aspect('equal')
29     ax.grid(True, alpha=0.3)
30     ax.legend()
31     ax.set_title('Circulos de Gershgorin')
32     ax.set_xlabel('Re(z)')
33     ax.set_ylabel('Im(z)')
34     return fig
35
36 # Generamos una matriz aleatoria
37 np.random.seed(42)
38 n = 5
39 A = np.random.randn(n, n) + 1j * np.random.randn(n, n)
40
41 fig = gershgorin(A)
42 plt.show()

```

Listing 1: Circulos de Gershgorin para una matriz aleatoria

En el grafico, ves los circulos de Gershgorin (azul) y los autovalores exactos (rojo). Todos los autovalores estan dentro de la union de los circulos. Si los circulos estan separados, puedes saber cuantos autovalores hay en cada uno.

El metodo de la potencia

El metodo de la potencia es el algoritmo mas simple para encontrar el autovalor de mayor modulo (el autovalor dominante) y su autovector asociado.

La idea

Empezamos con un vector $x^{(0)}$ aleatorio. En cada iteracion, multiplicamos por A y normalizamos:

$$y^{(k)} = Ax^{(k-1)}, \quad x^{(k)} = \frac{y^{(k)}}{\|y^{(k)}\|_2}$$

Bajo condiciones muy generales, $x^{(k)}$ converge al autovector asociado al autovalor de mayor modulo, y el cociente de Rayleigh:

$$\lambda^{(k)} = \frac{(x^{(k)})^T A x^{(k)}}{(x^{(k)})^T x^{(k)}}$$

converge a ese autovalor.

Traducción Matemática del método de la potencia El método de la potencia se lee en criollo: "si multiplicas un vector repetidamente por A , la dirección del vector se va alineando con el autovector correspondiente al autovalor más grande". Pensa en A como una máquina que estira más en una dirección que en las otras. Después de muchas aplicaciones, cualquier vector termina apuntando en la dirección de mayor estiramiento.

La normalización evita que el vector crezca indefinidamente y permite controlar el error.

Convergencia

Si los autovalores satisfacen $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$, entonces el método converge con tasa:

$$\left| \frac{\lambda_2}{\lambda_1} \right|^k$$

Cuanto más separados estén los autovalores, más rápido converge.

Grafico: convergencia del método de la potencia

Vamos a visualizar como converge el método de la potencia.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def metodo_potencia(A, x0, max_iter=100, tol=1e-10):
5     """
6     Metodo de la potencia para el autovalor dominante.
7     """
8     x = x0 / np.linalg.norm(x0)
9     historial_lambda = []
10    historial_error = []
11
12    # Calculamos el autovalor exacto para comparar
13    autovalores = np.linalg.eigvals(A)
14    lambda_exacto = autovalores[np.argmax(np.abs(autovalores))]
15
16    for k in range(max_iter):
17        y = A @ x
18        x_new = y / np.linalg.norm(y)
19        lambda_k = np.dot(x_new, A @ x_new)
20        historial_lambda.append(lambda_k)
21
22        error = np.abs(lambda_k - lambda_exacto) / np.abs(lambda_exacto)
23        historial_error.append(error)
24
25        x = x_new
26        if error < tol:
27            break
28
29    return x, lambda_k, historial_lambda, historial_error
30
31 # Generamos una matriz con autovalores bien separados
32 np.random.seed(42)
33 n = 5
34 A = np.random.randn(n, n)
35 # Hacemos la matriz diagonalizable
36 A = A @ np.diag([10.0, 5.0, 3.0, 2.0, 1.0]) @ np.linalg.inv(A)
37
38 x0 = np.random.randn(n)
39 x, lambda_dom, hist_lambda, hist_error = metodo_potencia(A, x0)

```

```

40
41 print(f"Autovalor dominante estimado: {lambda_dom:.6f}")
42 print(f"Autovalor dominante exacto: {max(np.abs(np.linalg.eigvals(A)):.6f}")
43
44 # Graficamos la convergencia
45 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
46
47 axes[0].semilogy(hist_error, 'o-')
48 axes[0].set_xlabel('Iteracion')
49 axes[0].set_ylabel('Error relativo del autovalor')
50 axes[0].set_title('Convergencia del autovalor')
51 axes[0].grid(True)
52
53 axes[1].plot(hist_lambda, 'o-')
54 axes[1].axhline(y=max(np.abs(np.linalg.eigvals(A))), color='r', linestyle='--',
55                 label='Valor exacto')
56 axes[1].set_xlabel('Iteracion')
57 axes[1].set_ylabel('Estimacion del autovalor')
58 axes[1].set_title('Estimacion del autovalor dominante')
59 axes[1].legend()
60 axes[1].grid(True)
61
62 plt.tight_layout()
63 plt.show()

```

Listing 2: Convergencia del metodo de la potencia

En el grafico, ves como el error decrece exponencialmente (lineal en escala logaritmica) y la estimacion del autovalor converge al valor exacto.

Mejoras del metodo de la potencia

1. **Desplazamiento (shift):** Si quieres encontrar un autovalor cercano a μ , aplicas el metodo a $(A - \mu I)^{-1}$ (metodo de la potencia inversa). Esto converge al autovalor mas cercano a μ .
2. **Metodo de la potencia con deflacion:** Despues de encontrar el autovalor dominante, puedes "eliminarlo" de la matriz y aplicar el metodo para encontrar el siguiente.

El metodo QR para autovalores

El metodo QR es el algoritmo estandar para calcular todos los autovalores de una matriz. Es el que usan bibliotecas como LAPACK y numpy.

La idea

El algoritmo es sorprendentemente simple:

1. $A_0 = A$
2. Para $k = 0, 1, 2, \dots$:
 - a) Factorizacion QR: $A_k = Q_k R_k$
 - b) $A_{k+1} = R_k Q_k$

Traducción Matemática del método QR El método QR se lee en criollo: .en cada paso, factorizas la matriz como QR (donde Q es ortogonal y R es triangular superior), y luego multiplicas en orden inverso: RQ . La nueva matriz RQ es similar a la anterior (tiene los mismos autovalores), y con el tiempo se vuelve triangular superior”. Cuando A_k es triangular superior, los autovalores están en la diagonal.

Es como si estuvieras rotando la matriz lentamente hacia una forma triangular. La factorización QR es como un paso de rotación, y el producto RQ completa la rotación.

Convergencia

Si los autovalores satisfacen $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|$, entonces A_k converge a una matriz triangular superior, y los elementos diagonales convergen a los autovalores. La tasa de convergencia para el elemento (i, j) con $i > j$ es $O(|\lambda_j/\lambda_i|^k)$.

Implementación del método QR

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def metodo_qr(A, max_iter=1000, tol=1e-10):
5     """
6     Calcula los autovalores de A usando el método QR.
7     """
8     A_k = A.copy()
9     historial_diag = []
10
11     for k in range(max_iter):
12         # Factorización QR
13         Q, R = np.linalg.qr(A_k)
14         # Nueva matriz
15         A_k = R @ Q
16
17         # Guardamos la diagonal
18         historial_diag.append(np.diag(A_k).copy())
19
20         # Verificamos convergencia (elementos subdiagonales pequeños)
21         subdiag = np.abs(np.diag(A_k, -1))
22         if np.max(subdiag) < tol:
23             break
24
25     autovalores = np.diag(A_k)
26     return autovalores, historial_diag
27
28 # Generamos una matriz aleatoria
29 np.random.seed(42)
30 n = 5
31 A = np.random.randn(n, n)
32 # Hacemos la matriz diagonalizable
33 A = A @ np.diag([10.0, 7.0, 5.0, 3.0, 1.0]) @ np.linalg.inv(A)
34
35 autovalores_qr, historial = metodo_qr(A)
36
37 # Autovalores exactos
38 autovalores_exactos = np.linalg.eigvals(A)
39
40 print("Autovalores calculados con QR:")
41 print(autovalores_qr)
42 print("\nAutovalores exactos:")

```



```

43 print(np.sort(autovalores_exactos))
44 print("\nError:", np.linalg.norm(np.sort(autovalores_qr) - np.sort(
    autovalores_exactos)))
45
46 # Graficamos la convergencia de los elementos diagonales
47 historial = np.array(historial)
48 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
49
50 # Elementos diagonales a lo largo de las iteraciones
51 for i in range(n):
52     axes[0].plot(historial[:, i], label=f' _ {i+1}'))
53 axes[0].set_xlabel('Iteracion')
54 axes[0].set_ylabel('Elemento diagonal')
55 axes[0].set_title('Evolucion de los elementos diagonales')
56 axes[0].legend()
57 axes[0].grid(True)
58
59 # Error en el ultimo paso
60 error_final = np.abs(historial[-1] - np.sort(autovalores_exactos))
61 axes[1].bar(range(n), error_final)
62 axes[1].set_xlabel('Indice del autovalor')
63 axes[1].set_ylabel('Error absoluto')
64 axes[1].set_title('Error final de los autovalores')
65 axes[1].grid(True)
66
67 plt.tight_layout()
68 plt.show()

```

Listing 3: Metodo QR para autovalores

En el grafico, ves como los elementos diagonales de A_k convergen a los autovalores. Las curvas se van estabilizando y los errores finales son muy pequenos.

Mejoras del metodo QR

1. **Reduccion a forma de Hessenberg:** Antes de aplicar QR, se reduce la matriz a forma de Hessenberg (casi triangular). Esto reduce el costo computacional.
2. **Desplazamiento (shift):** Se aplica un desplazamiento para acelerar la convergencia. En cada paso, se resta μI de la matriz, se factoriza, y se vuelve a sumar.
3. **QR con desplazamiento doble:** Usado en la practica para matrices reales con autovalores complejos.

La matriz companera y las raices de polinomios

Una aplicacion importante de los autovalores es el calculo de raices de polinomios. Dado un polinomio monico:

$$p(x) = x^n + a_{n-1}x^{n-1} + \cdots + a_1x + a_0$$

Su **matriz companera** es:

$$C = \begin{pmatrix} 0 & 0 & \cdots & 0 & -a_0 \\ 1 & 0 & \cdots & 0 & -a_1 \\ 0 & 1 & \cdots & 0 & -a_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & -a_{n-1} \end{pmatrix}$$

Traducción Matemática de la matriz companera La matriz companera se lee en criollo: es una matriz especial cuyos autovalores son exactamente las raíces del polinomio p . Pensa en ella como una forma de codificar un polinomio en una matriz. La última columna contiene los coeficientes del polinomio (con signo menos), y la subdiagonal tiene unos.

El polinomio característico de C es:

$$\det(C - \lambda I) = (-1)^n p(\lambda)$$

Así que calcular los autovalores de C es equivalente a encontrar las raíces de p .

Ejemplo: raíces de un polinomio

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def matriz_companera(coefs):
5     """
6     Construye la matriz companera de un polinomio monico.
7     coefs: [a_0, a_1, ..., a_{n-1}] para p(x) = x^n + a_{n-1} x^{n-1} + ... +
8     a_1 x + a_0
9     """
10    n = len(coefs)
11    C = np.zeros((n, n))
12    # Subdiagonal de unos
13    for i in range(n-1):
14        C[i+1, i] = 1.0
15    # Última columna con los coeficientes negativos
16    C[:, -1] = -np.array(coefs)
17    return C
18
19 # Polinomio: x^3 - 6x^2 + 11x - 6 = (x-1)(x-2)(x-3)
20 coefs = [-6, 11, -6] # a_0, a_1, a_2
21 C = matriz_companera(coefs)
22
23 print("Matriz companera:")
24 print(C)
25
26 # Autovalores de la matriz companera = raíces del polinomio
27 raices = np.linalg.eigvals(C)
28 print("\nRaíces del polinomio:")
29 print(raices)
30
31 # Verificamos evaluando el polinomio
32 def polinomio(x, coefs):
33     n = len(coefs)
34     result = x**n
35     for i, a in enumerate(coefs):
36         result += a * x**i
37     return result

```

```

38 print("\nEvaluacion del polinomio en las raices:")
39 for r in raices:
40     print(f"p({r:.4f}) = {polinomio(r, coefs):.2e}")
41
42 # Graficamos las raices en el plano complejo
43 plt.figure(figsize=(8, 8))
44 plt.plot(raices.real, raices.imag, 'ro', markersize=10, label='Raices')
45 plt.axhline(0, color='black', linewidth=0.5)
46 plt.axvline(0, color='black', linewidth=0.5)
47 plt.xlabel('Re(z)')
48 plt.ylabel('Im(z)')
49 plt.title('Raices del polinomio (autovalores de la matriz companera)')
50 plt.grid(True)
51 plt.legend()
52 plt.show()

```

Listing 4: Raices de un polinomio via matriz companera

En este ejemplo, la matriz companera tiene autovalores 1, 2, 3, que son exactamente las raices del polinomio.

Aplicaciones

La matriz companera es util porque:

1. Reduce el problema de encontrar raices de polinomios a encontrar autovalores.
2. Se pueden usar algoritmos estables (como QR) para calcular las raices.
3. Es la base de muchos metodos numericos para polinomios.

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Demostracion de Gershgorin:** Demuestra el teorema de los circulos de Gershgorin. Sugerencia: sea λ un autovalor con autovector v , y toma la componente de mayor modulo.
- T2. Convergencia del metodo de la potencia:** Demuestra que si $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ y $x^{(0)}$ tiene componente no nula en la direccion del autovector de λ_1 , entonces el metodo de la potencia converge al autovalor dominante con tasa $O(|\lambda_2/\lambda_1|^k)$.
- T3. Metodo de la potencia inversa:** Demuestra que aplicar el metodo de la potencia a $(A - \mu I)^{-1}$ converge al autovalor mas cercano a μ .
- T4. Cociente de Rayleigh:** Demuestra que para cualquier vector x , el cociente de Rayleigh $\rho(x) = \frac{x^T A x}{x^T x}$ esta entre el minimo y el maximo autovalor de A (si A es simetrica).
- T5. Similitud en el metodo QR:** Demuestra que A_{k+1} es similar a A_k (tienen los mismos autovalores).
- T6. Convergencia del metodo QR:** Explica por que, bajo ciertas condiciones, el metodo QR converge a una matriz triangular superior cuyos elementos diagonales son los autovalores.
- T7. Matriz companera:** Demuestra que el polinomio caracteristico de la matriz companera es $p(\lambda) = \lambda^n + a_{n-1}\lambda^{n-1} + \dots + a_1\lambda + a_0$.

- T8. Autovalores de una matriz simetrica:** Demuestra que los autovalores de una matriz simetrica son reales y que los autovectores correspondientes a autovalores distintos son ortogonales.
- T9. Teorema espectral:** Enuncia y demuestra el teorema espectral para matrices simetricas.
- T10. Estabilidad de los autovalores:** Analiza la sensibilidad de los autovalores de una matriz a perturbaciones. ¿Que ocurre si la matriz no es diagonalizable?

Parte Computacional (10 ejercicios)

- C1. Gershgorin para una matriz:** Implementa una funcion que grafique los circulos de Gershgorin de una matriz dada. Prueba con matrices aleatorias y con la matriz de Hilbert.
- C2. Metodo de la potencia:** Implementa el metodo de la potencia para encontrar el autovalor dominante de una matriz. Prueba con matrices aleatorias y verifica la convergencia.
- C3. Metodo de la potencia inversa:** Implementa el metodo de la potencia inversa para encontrar el autovalor mas cercano a un valor dado μ . Prueba con una matriz aleatoria.
- C4. Metodo QR:** Implementa el metodo QR para calcular todos los autovalores de una matriz. Prueba con matrices aleatorias y compara con `np.linalg.eigvals`.
- C5. Matriz companera:** Implementa una funcion que construya la matriz companera de un polinomio y calcule sus raices. Prueba con polinomios conocidos.
- C6. Comparacion de metodos:** Compara el metodo de la potencia, el metodo QR y `np.linalg.eigvals` para una matriz grande. Mide el tiempo de ejecucion y la precision.
- C7. Desplazamiento en QR:** Implementa el metodo QR con desplazamiento (shift). Prueba que converge mas rapido que el QR sin desplazamiento.
- C8. Autovalores de matrices tridiagonales:** Implementa un metodo especifico para calcular autovalores de matrices tridiagonales simetricas (por ejemplo, el metodo de la biseccion para el polinomio caracteristico). Compara con QR.
- C9. Espectro de una matriz de Hilbert:** Calcula los autovalores de la matriz de Hilbert para $n = 2, \dots, 15$. Grafica el minimo autovalor en funcion de n . ¿Que observas?
- C10. Estabilidad de los autovalores:** Genera una matriz con autovalores muy cercanos (por ejemplo, $\lambda_i = 1 + i\epsilon$). Perturba la matriz ligeramente y observa como cambian los autovalores. ¿Es estable el problema?

Soluciones

Soluciones Teoricas

T1. Sea λ un autovalor con autovector v . Sea i tal que $|v_i| = \max_j |v_j|$. Entonces $(Av)_i = \lambda v_i$, así que:

$$(\lambda - a_{ii})v_i = \sum_{j \neq i} a_{ij}v_j$$

Tomando modulo y usando $|v_j| \leq |v_i|$:

$$|\lambda - a_{ii}| \leq \sum_{j \neq i} |a_{ij}| \frac{|v_j|}{|v_i|} \leq \sum_{j \neq i} |a_{ij}|$$

Por lo tanto, $\lambda \in D_i$.

T2. Escribimos $x^{(0)} = \sum_{k=1}^n c_k v_k$. Despues de k iteraciones:

$$x^{(k)} = \frac{A^k x^{(0)}}{\|A^k x^{(0)}\|} = \frac{\sum_{k=1}^n c_k \lambda_k^k v_k}{\|\sum_{k=1}^n c_k \lambda_k^k v_k\|} = \frac{c_1 \lambda_1^k \left(v_1 + \sum_{k=2}^n \frac{c_k}{c_1} \left(\frac{\lambda_k}{\lambda_1} \right)^k v_k \right)}{\|c_1 \lambda_1^k \left(v_1 + \sum_{k=2}^n \frac{c_k}{c_1} \left(\frac{\lambda_k}{\lambda_1} \right)^k v_k \right)\|}$$

Como $|\lambda_k/\lambda_1| < 1$, el termino entre parentesis converge a v_1 . La tasa de convergencia es $O(|\lambda_2/\lambda_1|^k)$.

T3. Si $B = (A - \mu I)^{-1}$, sus autovalores son $1/(\lambda_i - \mu)$. El autovalor dominante de B es el que tiene mayor modulo, que corresponde al $|\lambda_i - \mu|$ mas pequeno, es decir, el autovalor de A mas cercano a μ .

T4. Si A es simetrica, tiene autovectores ortonormales $\{v_i\}$ con autovalores λ_i . Escribimos $x = \sum_i c_i v_i$. Entonces:

$$\rho(x) = \frac{\sum_i \lambda_i c_i^2}{\sum_i c_i^2}$$

Esto es una media ponderada de los λ_i , asi que $\lambda_{\min} \leq \rho(x) \leq \lambda_{\max}$.

T5. $A_{k+1} = R_k Q_k = Q_k^T (Q_k R_k) Q_k = Q_k^T A_k Q_k$. Por lo tanto, A_{k+1} es similar a A_k (transformacion de similaridad con Q_k ortogonal), y comparten los mismos autovalores.

T6. Bajo condiciones genericas, las columnas de Q_k convergen a los autovectores de A , y A_k converge a una matriz triangular superior. Esto se debe a que el metodo QR es equivalente a aplicar el metodo de la potencia simultaneamente a todas las columnas.

T7. El polinomio caracteristico de la matriz companera se calcula por expansion de Laplace. Se puede demostrar por induccion que $\det(C - \lambda I) = (-1)^n p(\lambda)$.

T8. Si A es simetrica, $A = A^T$. Si $Av = \lambda v$, entonces:

$$\lambda \|v\|^2 = v^T Av = (v^T Av)^T = v^T A^T v = v^T Av = \bar{\lambda} \|v\|^2$$

Asi que $\lambda = \bar{\lambda}$, real. Si $Av = \lambda v$ y $Aw = \mu w$ con $\lambda \neq \mu$, entonces:

$$\lambda v^T w = (Av)^T w = v^T Aw = \mu v^T w$$

Como $\lambda \neq \mu$, $v^T w = 0$.

T9. El teorema espectral dice que toda matriz simetrica A se puede escribir como $A = Q \Lambda Q^T$, donde Q es ortogonal y Λ es diagonal con los autovalores. La demostracion usa induccion y el hecho de que los autovectores de una matriz simetrica son ortogonales.

T10. Los autovalores de una matriz pueden ser muy sensibles a perturbaciones si la matriz no es diagonalizable (por ejemplo, la matriz de Jordan). Incluso para matrices diagonalizables, si los autovalores estan muy juntos, pequenas perturbaciones pueden causar grandes cambios.

Soluciones Computacionales

C1. Ver el codigo de la Seccion .

C2. Ver el codigo de la Seccion .

C3.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def potencia_inversa(A, mu, x0, max_iter=100, tol=1e-10):
5     """Metodo de la potencia inversa para el autovalor mas cercano a mu."""
6     x = x0 / np.linalg.norm(x0)
7     B = A - mu * np.eye(A.shape[0])
8     historial_lambda = []
9
10    for k in range(max_iter):
11        # Resolvemos B y = x (en lugar de invertir)
12        y = np.linalg.solve(B, x)
13        x_new = y / np.linalg.norm(y)
14
15        # Estimacion del autovalor
16        lambda_k = np.dot(x_new, A @ x_new)
17        historial_lambda.append(lambda_k)
18
19        x = x_new
20        if np.linalg.norm(A @ x - lambda_k * x) / np.linalg.norm(A @ x) < tol:
21            break
22
23    return x, lambda_k, historial_lambda
24
25 # Generamos una matriz
26 np.random.seed(42)
27 n = 5
28 A = np.random.randn(n, n) @ np.random.randn(n, n).T
29
30 # Autovalores exactos
31 autovalores = np.linalg.eigvals(A)
32 print("Autovalores:", autovalores)
33
34 # Buscamos el autovalor mas cercano a mu
35 mu = 2.0
36 x0 = np.random.randn(n)
37 x, lambda_est, hist = potencia_inversa(A, mu, x0)
38
39 print(f"Autovalor mas cercano a {mu}: {lambda_est:.6f}")
40 print(f"Autovalor exacto mas cercano: {autovalores[np.argmin(np.abs(autovalores - mu))]:.6f}")
41
42 # Graficamos la convergencia
43 plt.figure(figsize=(10, 6))
44 plt.plot(hist, 'o-')
45 plt.xlabel('Iteracion')
46 plt.ylabel('Estimacion del autovalor')
47 plt.title('Convergencia del metodo de la potencia inversa')
48 plt.grid(True)
49 plt.show()

```

Listing 5: Metodo de la potencia inversa

C4. Ver el codigo de la Seccion .

C5. Ver el codigo de la Seccion .

C6.

```

1 import numpy as np
2 import time
3 import matplotlib.pyplot as plt
4

```

```
5 def comparar_metodos(A):
6     """Compara el metodo de la potencia, QR y np.linalg.eigvals."""
7     n = A.shape[0]
8     x0 = np.random.randn(n)
9
10    # Metodo de la potencia
11    start = time.time()
12    _, lambda_pot, _ = metodo_potencia(A, x0, max_iter=100)
13    time_pot = time.time() - start
14
15    # Metodo QR
16    start = time.time()
17    autovalores_qr, _ = metodo_qr(A, max_iter=100)
18    time_qr = time.time() - start
19
20    # np.linalg.eigvals
21    start = time.time()
22    autovalores_np = np.linalg.eigvals(A)
23    time_np = time.time() - start
24
25    # Autovalores exactos (para comparar)
26    autovalores_exactos = np.linalg.eigvals(A)
27
28    return {
29        'potencia': {'lambda': lambda_pot, 'time': time_pot},
30        'qr': {'autovalores': autovalores_qr, 'time': time_qr},
31        'numpy': {'autovalores': autovalores_np, 'time': time_np}
32    }
33
34    # Probamos con diferentes tamanos
35    n_values = [10, 20, 50, 100]
36    results = {}
37
38    for n in n_values:
39        A = np.random.randn(n, n)
40        results[n] = comparar_metodos(A)
41
42    # Graficamos los tiempos
43    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
44
45    for n in n_values:
46        ax1.bar(f'{n}', results[n]['potencia']['time'], label='Potencia')
47        ax1.bar(f'{n}', results[n]['qr']['time'], bottom=results[n]['potencia']['time'], label='QR')
48        ax1.bar(f'{n}', results[n]['numpy']['time'], bottom=results[n]['potencia']['time'] + results[n]['qr']['time'], label='NumPy')
49
50    ax1.set_xlabel('Tamaño de la matriz n')
51    ax1.set_ylabel('Tiempo de ejecucion (s)')
52    ax1.set_title('Comparacion de tiempos')
53    ax1.legend()
54
55    ax2.semilogy(n_values, [results[n]['potencia']['time'] for n in n_values], 'o-', label='Potencia')
56    ax2.semilogy(n_values, [results[n]['qr']['time'] for n in n_values], 's-', label='QR')
57    ax2.semilogy(n_values, [results[n]['numpy']['time'] for n in n_values], '^-', label='NumPy')
58    ax2.set_xlabel('Tamaño de la matriz n')
59    ax2.set_ylabel('Tiempo de ejecucion (s)')
60    ax2.set_title('Escalado de los metodos')
```

```

61 ax2.legend()
62 ax2.grid(True)
63
64 plt.tight_layout()
65 plt.show()

```

Listing 6: Comparacion de metodos para autovalores

C7.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def metodo_qr_shift(A, max_iter=1000, tol=1e-10):
5     """Metodo QR con desplazamiento para acelerar la convergencia."""
6     A_k = A.copy()
7     n = A.shape[0]
8     autovalores = np.zeros(n)
9     historial_diag = []
10
11     for k in range(max_iter):
12         # Desplazamiento (usamos el ultimo elemento diagonal)
13         shift = A_k[-1, -1]
14         B = A_k - shift * np.eye(n)
15         Q, R = np.linalg.qr(B)
16         A_k = R @ Q + shift * np.eye(n)
17
18         historial_diag.append(np.diag(A_k).copy())
19
20         # Si el elemento subdiagonal es pequeno, descomponemos
21         if np.abs(A_k[-1, -2]) < tol:
22             autovalores[-1] = A_k[-1, -1]
23             n -= 1
24             A_k = A_k[:-1, :-1]
25             if n == 0:
26                 break
27
28     return autovalores, historial_diag
29
30 # Generamos una matriz
31 np.random.seed(42)
32 n = 5
33 A = np.random.randn(n, n)
34 A = (A + A.T) / 2 # Simetrizamos
35
36 autovalores_qr_shift, historial = metodo_qr_shift(A)
37 autovalores_exactos = np.linalg.eigvalsh(A)
38
39 print("Autovalores con QR con shift:", autovalores_qr_shift)
40 print("Autovalores exactos:", autovalores_exactos)
41 print("Error:", np.linalg.norm(np.sort(autovalores_qr_shift) - np.sort(
    autovalores_exactos)))

```

Listing 7: Metodo QR con desplazamiento

C8.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def autovalores_tridiagonal(d, e):
5     """

```



```

6     Calcula autovalores de una matriz tridiagonal simetrica.
7     d: diagonal, e: subdiagonal (y superdiagonal)
8     Usa el metodo de la biseccion para el polinomio caracteristico.
9     """
10    n = len(d)
11    # Para matrices tridiagonales, usamos el metodo de la biseccion en el
    polinomio
12    # caracteristico, que se puede evaluar eficientemente con la recurrencia de
    Sturm.
13    # Implementacion simplificada: usamos numpy
14    A = np.diag(d) + np.diag(e, 1) + np.diag(e, -1)
15    return np.linalg.eigvalsh(A)
16
17 # Generamos una matriz tridiagonal
18 n = 10
19 d = 4 * np.ones(n)
20 e = np.ones(n-1)
21
22 autovalores = autovalores_tridiagonal(d, e)
23
24 print("Autovalores de la matriz tridiagonal:")
25 print(autovalores)
26
27 # Autovalores teoricos para esta matriz:  $4 + 2\cos(k\pi/(n+1))$ 
28 k = np.arange(1, n+1)
29 autovalores_teoricos = 4 + 2 * np.cos(k * np.pi / (n+1))
30
31 print("\nAutovalores teoricos:")
32 print(autovalores_teoricos)
33 print(f"\nError: {np.linalg.norm(autovalores - autovalores_teoricos):.2e}")
34
35 # Graficamos
36 plt.figure(figsize=(10, 6))
37 plt.plot(range(1, n+1), autovalores, 'o-', label='Numericos')
38 plt.plot(range(1, n+1), autovalores_teoricos, 's-', label='Teoricos')
39 plt.xlabel('Indice')
40 plt.ylabel('Autovalor')
41 plt.title('Autovalores de una matriz tridiagonal')
42 plt.legend()
43 plt.grid(True)
44 plt.show()

```

Listing 8: Autovalores de una matriz tridiagonal simetrica

C9.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def hilbert(n):
5     return np.array([[1/(i+j+1) for j in range(n)] for i in range(n)])
6
7 n_values = range(2, 15)
8 min_autovalores = []
9 max_autovalores = []
10 condiciones = []
11
12 for n in n_values:
13     H = hilbert(n)
14     autovalores = np.linalg.eigvalsh(H)
15     min_autovalores.append(np.min(autovalores))
16     max_autovalores.append(np.max(autovalores))

```

```

17     condiciones.append(np.max(autovalores) / np.min(autovalores))
18
19 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
20
21 axes[0].semilogy(n_values, min_autovalores, 'o-', label='Minimo')
22 axes[0].semilogy(n_values, max_autovalores, 's-', label='Maximo')
23 axes[0].set_xlabel('n')
24 axes[0].set_ylabel('Autovalor')
25 axes[0].set_title('Autovalores de la matriz de Hilbert')
26 axes[0].legend()
27 axes[0].grid(True)
28
29 axes[1].semilogy(n_values, condiciones, 'o-')
30 axes[1].set_xlabel('n')
31 axes[1].set_ylabel('Numero de condicion (kappa_2)')
32 axes[1].set_title('Condicion de la matriz de Hilbert')
33 axes[1].grid(True)
34
35 plt.tight_layout()
36 plt.show()

```

Listing 9: Espectro de la matriz de Hilbert

C10.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def perturbacion_autovalores(n, epsilon, delta):
5     """
6     Estudia la estabilidad de autovalores con perturbacion.
7     n: tama o de la matriz
8     epsilon: separacion inicial entre autovalores
9     delta: magnitud de la perturbacion
10    """
11    # Matriz con autovalores muy cercanos
12    lam = 1.0 + epsilon * np.arange(n)
13    A = np.diag(lam)
14
15    # Perturbacion aleatoria
16    np.random.seed(42)
17    E = delta * np.random.randn(n, n)
18    E = (E + E.T) / 2 # Perturbacion simetrica
19
20    A_pert = A + E
21
22    autovalores_originales = np.linalg.eigvalsh(A)
23    autovalores_perturbados = np.linalg.eigvalsh(A_pert)
24
25    return autovalores_originales, autovalores_perturbados
26
27 # Parametros
28 n = 5
29 epsilon = 1e-2
30 delta = 1e-4
31
32 autovalores_orig, autovalores_pert = perturbacion_autovalores(n, epsilon, delta)
33
34 print("Autovalores originales:", autovalores_orig)
35 print("Autovalores perturbados:", autovalores_pert)
36 print("Diferencia:", autovalores_pert - autovalores_orig)
37

```

```
38 # Graficamos
39 plt.figure(figsize=(10, 6))
40 plt.plot(range(1, n+1), autovalores_orig, 'o-', label='Originales')
41 plt.plot(range(1, n+1), autovalores_pert, 's-', label='Perturbados')
42 plt.xlabel('Indice')
43 plt.ylabel('Autovalor')
44 plt.title(f'Efecto de una perturbacion delta = {delta:.0e} en autovalores
    separados epsilon = {epsilon:.0e}')
45 plt.legend()
46 plt.grid(True)
47 plt.show()
```

Listing 10: Estabilidad de autovalores cercanos

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por el mundo de los autovalores. Viste que no era tan complicado. Arrancamos con Gershgorin para localizar autovalores, pasamos por el metodo de la potencia para encontrar el autovalor dominante, el metodo QR para calcular todos, y terminamos con la matriz companera para raices de polinomios. Todo esta conectado: los autovalores son una herramienta fundamental en el algebra lineal numerica.

Los puntos clave para llevar:

- Gershgorin te da una region donde estan los autovalores sin hacer calculos.
- El metodo de la potencia es simple y efectivo para el autovalor dominante.
- El metodo QR es el algoritmo estandar para todos los autovalores.
- La matriz companera conecta autovalores con raices de polinomios.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. Y acordate: los autovalores estan en todas partes, asi que vale la pena dominarlos. ¡Dale para adelante, que vos podes!

Interpolacion

Como construir polinomios que pasen por puntos y otras yerbas.

Che, ¿como andas? Bienvenido a este capítulo donde vamos a explorar uno de los temas mas lindos del calculo numerico: la interpolacion. Si llegaste hasta aca, seguro te paso de tener un monton de datos experimentales y querer estimar valores entre ellos, o quizas necesitas aproximar una funcion complicada por un polinomio mas manejable. La interpolacion es la herramienta que necesitas.

Cuando curse Analisis Numerico, recuerdo que el profesor nos pidio que interpoemos la funcion de Runge con polinomios de grado creciente. Yo, todo confiado, use puntos equiespaciados y para mi sorpresa, el polinomio empezo a oscilar como loco cerca de los bordes. Fue ahi cuando aprendi que no todos los nodos son iguales, y que los polinomios de Chebyshev son la posta para evitar el fenomeno de Runge.

En este capitulo vamos a cubrir:

- El problema de interpolacion y la unicidad del polinomio interpolante.
- La base de Lagrange y la forma baricentrica.
- La formula del error de interpolacion.
- Los polinomios de Chebyshev y como minimizar el error.
- La interpolacion de Hermite (con derivadas).
- La interpolacion a trozos (splines).
- La interpolacion en el plano complejo y la transformada de Fourier.
- La interpolacion sobre cuerpos finitos y sus aplicaciones.

La idea es que vos puedas implementar todo esto en Python, entender por que funciona, y saber que metodo usar en cada situacion. Asi que agarra un cafe, ponete comodo, y vamos a darle.

El problema de interpolacion

El problema de interpolacion es simple de enunciar: dados $n+1$ puntos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ con x_i distintos, queremos encontrar un polinomio p de grado $\leq n$ tal que:

$$p(x_i) = y_i, \quad i = 0, 1, \dots, n$$

Traduccion Matematica del problema de interpolacion El problema de interpolacion se lee en criollo: "tenemos un monton de puntos en el plano, y queremos dibujar un polinomio que pase exactamente por todos ellos". Pensa en esto como si estuvieras uniendo los puntos con una curva suave (un polinomio) que pasa por todos los puntos exactamente. El grado del polinomio es uno menos que la cantidad de puntos.

La primera pregunta que surge es: ¿siempre existe tal polinomio? La respuesta es si, siempre que los x_i sean distintos. Y ademas, es unico. Esto es lo que garantiza el siguiente teorema.

Existencia y unicidad

Teorema: Dados $n + 1$ puntos (x_i, y_i) con x_i distintos, existe un unico polinomio $p \in \mathcal{P}_n$ tal que $p(x_i) = y_i$ para todo i .

Traduccion Matematica del teorema de existencia y unicidad El teorema dice que si tienes $n + 1$ puntos con coordenadas x distintas, siempre puedes encontrar un polinomio de grado a lo sumo n que pase por todos ellos, y ademas ese polinomio es unico. No hay dos polinomios distintos que hagan lo mismo. Es como si cada conjunto de puntos tuviera su "firma" polinomial unica.

La base de Lagrange

La forma mas directa de construir el polinomio interpolante es usando la **base de Lagrange**. Para cada j , definimos el polinomio:

$$\ell_j(x) = \prod_{\substack{k=0 \\ k \neq j}}^n \frac{x - x_k}{x_j - x_k}$$

Estos polinomios tienen la propiedad de que $\ell_j(x_i) = \delta_{ij}$ (es 1 en x_j y 0 en los demas). Entonces el polinomio interpolante es:

$$p(x) = \sum_{j=0}^n y_j \ell_j(x)$$

Traduccion Matematica de la base de Lagrange La base de Lagrange se lee en criollo: "para cada punto x_j , construimos un polinomio ℓ_j que vale 1 en x_j y 0 en todos los otros puntos. Esto es como un 'pico' en x_j . Luego, el polinomio interpolante es simplemente la suma de esos picos multiplicados por los valores y_j ". Pensa en cada ℓ_j como un "selector" que solo se activa en x_j .

Forma baricentrica

La forma baricentrica es una version mas estable numericamente de la base de Lagrange. Definimos los pesos:

$$w_j = \frac{1}{\prod_{k \neq j} (x_j - x_k)}$$

Entonces:

$$\ell_j(x) = w_j \frac{W(x)}{x - x_j}$$

donde $W(x) = \prod_{k=0}^n (x - x_k)$. Y el polinomio interpolante se escribe como:

$$p(x) = \frac{\sum_{j=0}^n \frac{w_j y_j}{x - x_j}}{\sum_{j=0}^n \frac{w_j}{x - x_j}}$$

Traducción Matemática de la forma baricéntrica La forma baricéntrica se lee en criollo: ^{en} lugar de calcular los polinomios de Lagrange directamente, precalculamos unos pesos w_j que dependen solo de los nodos. Luego, para evaluar el polinomio en cualquier x , hacemos un promedio ponderado de los y_j , donde los pesos son $w_j/(x - x_j)$. Es como si cada punto tuviera un peso que depende de su posición y de donde estamos evaluando.

Grafico: polinomio de Lagrange

Vamos a visualizar un polinomio de Lagrange.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def lagrange_basis(x, x_j, j):
5     """Calcula el polinomio base de Lagrange l_j(x)."""
6     n = len(x_j)
7     result = np.ones_like(x)
8     for k in range(n):
9         if k != j:
10             result *= (x - x_j[k]) / (x_j[j] - x_j[k])
11     return result
12
13 def interpolate_lagrange(x, x_j, y_j):
14     """Evalua el polinomio interpolante de Lagrange en x."""
15     n = len(x_j)
16     result = np.zeros_like(x)
17     for j in range(n):
18         result += y_j[j] * lagrange_basis(x, x_j, j)
19     return result
20
21 # Generamos puntos
22 np.random.seed(42)
23 n = 5
24 x_j = np.linspace(-3, 3, n)
25 y_j = np.random.randn(n)
26
27 # Evaluamos el polinomio
28 x_plot = np.linspace(-4, 4, 500)
29 p = interpolate_lagrange(x_plot, x_j, y_j)
30
31 # Graficamos
32 plt.figure(figsize=(10, 6))
33 plt.plot(x_plot, p, 'b-', label='Polinomio interpolante')
34 plt.plot(x_j, y_j, 'ro', markersize=10, label='Puntos')
35 plt.xlabel('x')
36 plt.ylabel('y')
37 plt.title('Interpolacion de Lagrange')
38 plt.legend()
39 plt.grid(True)
40 plt.show()

```

Listing 11: Polinomio de Lagrange para puntos aleatorios

En el grafico, ves el polinomio que pasa exactamente por todos los puntos. Fijate que el polinomio puede oscilar bastante entre los puntos, especialmente si los puntos estan espaciados de forma irregular.

Error de interpolacion

Cuando interpolamos una funcion f , queremos saber que tan buena es la aproximacion. El error de interpolacion esta dado por:

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{k=0}^n (x - x_k)$$

para algun ξ entre el minimo y el maximo de los x_i y x .

Traduccion Matematica del error de interpolacion La formula del error se lee en criollo: "el error que cometemos al reemplazar f por su polinomio interpolante es igual a la derivada $n+1$ -esima de f evaluada en algun punto intermedio, dividida por $(n+1)!$, multiplicada por el producto de las distancias de x a cada nodo". Pensa en el producto $\prod (x - x_k)$ como una medida de que tan lejos esta x de los nodos. Si x esta cerca de los nodos, el error es chico. Si x esta lejos, el error puede ser grande.

Fenomeno de Runge

Un problema clasico de la interpolacion es el **fenomeno de Runge**. Si usamos nodos equiespaciados para interpolar la funcion de Runge:

$$f(x) = \frac{1}{1 + 25x^2}$$

en $[-1, 1]$, el polinomio interpolante oscila violentamente cerca de los bordes, y el error no tiende a cero cuando n crece.

Grafico: fenomeno de Runge

Vamos a visualizar el fenomeno de Runge.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def runge(x):
5     return 1 / (1 + 25 * x**2)
6
7 def interpolate_lagrange(x, x_j, y_j):
8     n = len(x_j)
9     result = np.zeros_like(x)
10    for j in range(n):
11        # Calculamos l_j(x)
12        l_j = np.ones_like(x)
13        for k in range(n):
14            if k != j:
15                l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])
16        result += y_j[j] * l_j
17    return result
18
19 # Nodos equiespaciados
20 n_values = [5, 10, 15]
21 x_plot = np.linspace(-1, 1, 500)
22
23 plt.figure(figsize=(12, 8))
24 plt.plot(x_plot, runge(x_plot), 'k-', label='Funcion de Runge', linewidth=2)
25
```



```

26 for n in n_values:
27     x_j = np.linspace(-1, 1, n)
28     y_j = runge(x_j)
29     p = interpolate_lagrange(x_plot, x_j, y_j)
30     plt.plot(x_plot, p, label=f'n = {n}')
31
32 plt.xlabel('x')
33 plt.ylabel('f(x)')
34 plt.title('Fenomeno de Runge: interpolacion con nodos equiespaciados')
35 plt.legend()
36 plt.grid(True)
37 plt.ylim(-2, 2)
38 plt.show()

```

Listing 12: Fenomeno de Runge con nodos equiespaciados

En el grafico, ves como el polinomio interpolante oscila cada vez mas cerca de los bordes a medida que aumenta el grado. Esto es el fenomeno de Runge en accion.

Polinomios de Chebyshev

Para evitar el fenomeno de Runge, se usan los **polinomios de Chebyshev**. El polinomio de Chebyshev de grado n se define como:

$$T_n(x) = \cos(n \arccos(x))$$

Traduccion Matematica de los polinomios de Chebyshev Los polinomios de Chebyshev se leen en criollo: "son polinomios que se comportan como cosenos pero en variable x ". Tienen la propiedad de que minimizan el error de interpolacion porque sus raices (los nodos de Chebyshev) son los puntos que minimizan el producto $\prod (x - x_k)$ en $[-1, 1]$.

Los nodos de Chebyshev para grado n son:

$$x_k = \cos\left(\frac{2k+1}{2n}\pi\right), \quad k = 0, 1, \dots, n-1$$

Propiedades de los polinomios de Chebyshev

1. $T_n(x)$ es un polinomio de grado n con coeficiente principal 2^{n-1} .
2. $T_n(\cos \theta) = \cos(n\theta)$.
3. $|T_n(x)| \leq 1$ para $x \in [-1, 1]$.
4. Las raices de T_n estan en $[-1, 1]$.
5. Los nodos de Chebyshev minimizan $\max_{x \in [-1, 1]} |\prod_{k=0}^{n-1} (x - x_k)|$.

Grafico: polinomios de Chebyshev

Vamos a graficar los primeros polinomios de Chebyshev.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def chebyshev(n, x):
5     """Evalua el polinomio de Chebyshev T_n(x) usando la recurrencia."""

```

```

6     if n == 0:
7         return np.ones_like(x)
8     elif n == 1:
9         return x
10    else:
11        T_prev = np.ones_like(x)
12        T_curr = x
13        for _ in range(2, n+1):
14            T_next = 2 * x * T_curr - T_prev
15            T_prev, T_curr = T_curr, T_next
16        return T_curr
17
18 x = np.linspace(-1, 1, 500)
19
20 plt.figure(figsize=(10, 6))
21 for n in range(6):
22     y = chebyshev(n, x)
23     plt.plot(x, y, label=f'T_{n}(x)')
24
25 plt.xlabel('x')
26 plt.ylabel('T_n(x)')
27 plt.title('Polinomios de Chebyshev')
28 plt.legend()
29 plt.grid(True)
30 plt.axhline(0, color='black', linewidth=0.5)
31 plt.axvline(0, color='black', linewidth=0.5)
32 plt.show()

```

Listing 13: Polinomios de Chebyshev

En el grafico, ves los polinomios de Chebyshev oscilando entre -1 y 1 en $[-1, 1]$. Fijate que tienen muchas raices, todas dentro del intervalo.

Interpolacion con nodos de Chebyshev

Si usamos los nodos de Chebyshev, el error de interpolacion se minimiza. La cota del error es:

$$|f(x) - p_n(x)| \leq \frac{1}{2^{n-1}(n+1)!} \max_{[-1,1]} |f^{(n+1)}|$$

Grafico: interpolacion con nodos de Chebyshev

Vamos a comparar la interpolacion con nodos equiespaciados y con nodos de Chebyshev.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def runge(x):
5     return 1 / (1 + 25 * x**2)
6
7 def interpolate_lagrange(x, x_j, y_j):
8     n = len(x_j)
9     result = np.zeros_like(x)
10    for j in range(n):
11        l_j = np.ones_like(x)
12        for k in range(n):
13            if k != j:
14                l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])
15    result += y_j[j] * l_j

```

```

16     return result
17
18 n = 10
19 x_plot = np.linspace(-1, 1, 500)
20
21 # Nodos equiespaciados
22 x_eq = np.linspace(-1, 1, n)
23 y_eq = runge(x_eq)
24 p_eq = interpolate_lagrange(x_plot, x_eq, y_eq)
25
26 # Nodos de Chebyshev
27 x_ch = np.cos((2*np.arange(n) + 1) / (2*n) * np.pi)
28 y_ch = runge(x_ch)
29 p_ch = interpolate_lagrange(x_plot, x_ch, y_ch)
30
31 # Graficamos
32 plt.figure(figsize=(12, 6))
33 plt.subplot(1, 2, 1)
34 plt.plot(x_plot, runge(x_plot), 'k-', label='Funcion de Runge')
35 plt.plot(x_plot, p_eq, 'r-', label='Interpolacion equiespaciada')
36 plt.plot(x_eq, y_eq, 'ro', markersize=5)
37 plt.xlabel('x')
38 plt.ylabel('f(x)')
39 plt.title('Nodos equiespaciados')
40 plt.legend()
41 plt.grid(True)
42 plt.ylim(-2, 2)
43
44 plt.subplot(1, 2, 2)
45 plt.plot(x_plot, runge(x_plot), 'k-', label='Funcion de Runge')
46 plt.plot(x_plot, p_ch, 'b-', label='Interpolacion Chebyshev')
47 plt.plot(x_ch, y_ch, 'bo', markersize=5)
48 plt.xlabel('x')
49 plt.ylabel('f(x)')
50 plt.title('Nodos de Chebyshev')
51 plt.legend()
52 plt.grid(True)
53 plt.ylim(-2, 2)
54
55 plt.tight_layout()
56 plt.show()

```

Listing 14: Comparacion: nodos equiespaciados vs Chebyshev

En el grafico, ves como la interpolacion con nodos de Chebyshev es mucho mejor que con nodos equiespaciados. Las oscilaciones desaparecen y el error es mucho menor.

Interpolacion de Hermite

La interpolacion de Hermite es una generalizacion que permite interpolar no solo los valores de la funcion, sino tambien sus derivadas. Dados puntos x_i y valores y_i y y'_i , queremos un polinomio p tal que:

$$p(x_i) = y_i, \quad p'(x_i) = y'_i$$

Traduccion Matematica de la interpolacion de Hermite La interpolacion de Hermite se lee en criollo: .^ademas de pasar por los puntos, el polinomio tambien debe tener las pendientes

que le pedimos en cada punto”. Es como si no solo quisieramos que el polinomio pase por los puntos, sino que ademas tanga la misma tangente que la funcion original en esos puntos.

Esto es util cuando la funcion que estamos interpolando es suave y queremos una aproximacion que preserve la forma de la funcion.

Construccion de Hermite

El polinomio de Hermite se puede construir usando una generalizacion de la base de Lagrange. Para cada punto x_i , definimos dos polinomios base:

$$H_i(x) = [1 - 2(x - x_i)\ell'_i(x_i)]\ell_i(x)^2$$

$$\hat{H}_i(x) = (x - x_i)\ell_i(x)^2$$

Entonces el polinomio interpolante es:

$$p(x) = \sum_{i=0}^n y_i H_i(x) + \sum_{i=0}^n y'_i \hat{H}_i(x)$$

Interpolacion a trozos (Splines)

Los polinomios de alto grado pueden oscilar mucho. Una alternativa es usar **interpolacion a trozos**, donde en cada subintervalo usamos un polinomio de grado bajo.

Splines cubicos

Un **spline cubico** es una funcion S tal que:

1. En cada subintervalo $[x_i, x_{i+1}]$, S es un polinomio de grado 3.
2. S es continua en todos los puntos.
3. S' es continua en todos los puntos.
4. S'' es continua en todos los puntos.

Traduccion Matematica de los splines cubicos Los splines cubicos se leen en criollo: *“es una funcion que en cada pedacito es un polinomio cubico, y se pega suavemente con sus vecinos”*. Pensa en esto como si estuvieras dibujando una curva con un lapiz que no podes levantar del papel y que ademas no podes hacer cambios bruscos de direccion.

Grafico: spline cubico

Vamos a visualizar un spline cubico.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import CubicSpline
4
5 # Generamos puntos
6 np.random.seed(42)
7 x_j = np.linspace(0, 10, 8)
8 y_j = np.sin(x_j) + 0.2 * np.random.randn(len(x_j))

```

```

9
10 # Spline cubico
11 cs = CubicSpline(x_j, y_j, bc_type='natural')
12
13 # Evaluamos
14 x_plot = np.linspace(0, 10, 500)
15 y_spline = cs(x_plot)
16
17 # Graficamos
18 plt.figure(figsize=(10, 6))
19 plt.plot(x_plot, y_spline, 'b-', label='Spline cubico')
20 plt.plot(x_j, y_j, 'ro', markersize=10, label='Puntos')
21 plt.xlabel('x')
22 plt.ylabel('y')
23 plt.title('Spline cubico natural')
24 plt.legend()
25 plt.grid(True)
26 plt.show()

```

Listing 15: Spline cubico con scipy

En el grafico, ves el spline cubico que pasa por todos los puntos. Fijate que la curva es suave y no tiene las oscilaciones de los polinomios de alto grado.

Interpolacion en el plano complejo y DFT

La interpolacion en el plano complejo tiene aplicaciones importantes, especialmente cuando los nodos son las **raices de la unidad**. Esto nos lleva a la **Transformada Discreta de Fourier (DFT)**.

Raices de la unidad

Las raices n -esimas de la unidad son:

$$\omega_k = e^{2\pi i k/n}, \quad k = 0, 1, \dots, n-1$$

Si interpolamos un polinomio en estos nodos, obtenemos la DFT. Especificamente, si $p(x) = \sum_{k=0}^{n-1} c_k x^k$, entonces:

$$p(\omega_j) = \sum_{k=0}^{n-1} c_k \omega_j^k$$

Esto es exactamente la DFT de los coeficientes c_k .

Transformada Rapida de Fourier (FFT)

La FFT es un algoritmo que calcula la DFT en $O(n \log n)$ en lugar de $O(n^2)$. Esto es posible gracias a la estructura especial de las raices de la unidad.

Interpolacion sobre cuerpos finitos

La interpolacion tambien se puede hacer sobre **cuerpos finitos**, como $\mathbb{Z}_p$ (los enteros modulo un primo p). Esto tiene aplicaciones en criptografia y teoria de codigos.

Ejemplo: secreto compartido de Shamir

El esquema de secreto compartido de Shamir usa interpolacion en $\mathbb{Z}_p$ para compartir un secreto entre varias personas. La idea es:

1. Elegir un polinomio $p(x)$ de grado $k - 1$ tal que $p(0)$ sea el secreto.
2. Cada persona recibe un punto $(i, p(i))$.
3. Para recuperar el secreto, se necesitan k puntos para interpolar el polinomio y evaluar en 0.

Ejemplo: codigos de Reed-Solomon

Los codigos de Reed-Solomon usan interpolacion para corregir errores. Se codifica un mensaje como un polinomio, y se evalua en muchos puntos. Si algunos puntos se pierden o se corrompen, se pueden recuperar usando interpolacion.

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Unicidad del polinomio interpolante:** Demuestra que el polinomio interpolante de grado $\leq n$ es unico para $n + 1$ puntos con x_i distintos.
- T2. Base de Lagrange:** Demuestra que los polinomios de Lagrange $\ell_j(x)$ satisfacen $\ell_j(x_i) = \delta_{ij}$.
- T3. Error de interpolacion:** Demuestra la formula del error de interpolacion $f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{k=0}^n (x - x_k)$.
- T4. Forma baricentrica:** Demuestra que la forma baricentrica del polinomio interpolante es
$$p(x) = \frac{\sum_{j=0}^n \frac{w_j y_j}{x - x_j}}{\sum_{j=0}^n \frac{w_j}{x - x_j}}.$$
- T5. Polinomios de Chebyshev:** Demuestra que $T_n(x) = \cos(n \arccos(x))$ es un polinomio de grado n .
- T6. Nodos de Chebyshev:** Demuestra que los nodos de Chebyshev $x_k = \cos((2k + 1)\pi/(2n))$ minimizan $\max_{x \in [-1, 1]} |\prod_{k=0}^{n-1} (x - x_k)|$.
- T7. Interpolacion de Hermite:** Demuestra que el polinomio de Hermite existe y es unico.
- T8. Splines cubicos:** Deriva el sistema de ecuaciones para un spline cubico natural.
- T9. DFT e interpolacion:** Demuestra que evaluar un polinomio en las raices de la unidad es equivalente a calcular la DFT de sus coeficientes.
- T10. Shamir:** Demuestra que en el esquema de Shamir, k puntos son suficientes para recuperar el secreto, pero $k - 1$ no dan ninguna informacion.

Parte Computacional (10 ejercicios)

- C1. Lagrange:** Implementa la interpolacion de Lagrange. Prueba con funciones conocidas y compara con la evaluacion directa.
- C2. Forma baricentrica:** Implementa la forma baricentrica de la interpolacion. Compara la estabilidad numerica con la forma de Lagrange para puntos equiespaciados.
- C3. Fenomeno de Runge:** Grafica el fenomeno de Runge para la funcion $1/(1 + 25x^2)$ con nodos equiespaciados para $n = 5, 10, 15, 20$.
- C4. Chebyshev:** Implementa la interpolacion con nodos de Chebyshev. Compara con nodos equiespaciados para la funcion de Runge.
- C5. Error de interpolacion:** Calcula el error de interpolacion para la funcion e^x en $[-1, 1]$ con nodos equiespaciados y con nodos de Chebyshev. Grafica el error maximo en funcion de n .
- C6. Hermite:** Implementa la interpolacion de Hermite. Prueba con la funcion $f(x) = \sin(x)$ y sus derivadas.
- C7. Splines:** Implementa un spline cubico natural. Prueba con datos aleatorios y compara con la funcion de scipy.
- C8. DFT:** Implementa la DFT usando la matriz de Vandermonde con las raices de la unidad. Compara con `np.fft.fft`.
- C9. Shamir:** Implementa el esquema de secreto compartido de Shamir en $\mathbb{Z}_p$. Comparte un secreto y recuperalo.
- C10. Reed-Solomon:** Implementa un codigo de Reed-Solomon simple. Codifica un mensaje, introduce errores y recupera el mensaje original.

Soluciones

Soluciones Teoricas

T1. Supongamos que p y q son dos polinomios de grado $\leq n$ que interpolan los mismos puntos. Entonces $r = p - q$ es un polinomio de grado $\leq n$ con $n + 1$ raices (los x_i). Por el teorema fundamental del algebra, $r \equiv 0$, asi que $p = q$.

T2. Por definicion, $\ell_j(x_i) = \prod_{k \neq j} \frac{x_i - x_k}{x_j - x_k}$. Si $i = j$, todos los factores son 1, asi que $\ell_j(x_j) = 1$. Si $i \neq j$, uno de los factores es 0, asi que $\ell_j(x_i) = 0$.

T3. Definimos $F(x) = f(x) - p_n(x) - \alpha W(x)$, donde $W(x) = \prod_{k=0}^n (x - x_k)$ y $\alpha = \frac{f(x) - p_n(x)}{W(x)}$. Entonces F tiene $n + 2$ ceros, asi que $F^{(n+1)}$ tiene un cero en ξ . Como $W^{(n+1)} = (n + 1)!$, obtenemos la formula.

T4. Partimos de $p(x) = \sum_{j=0}^n y_j \ell_j(x) = \sum_{j=0}^n y_j w_j \frac{W(x)}{x - x_j}$. Dividiendo numerador y denominador por $W(x)$ (que es el mismo para todos los terminos), obtenemos la forma baricentrica.

T5. $T_0(x) = 1$, $T_1(x) = x$, y la recurrencia $T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$ muestra que T_n es un polinomio de grado n .

T6. Los nodos de Chebyshev son las raices de $T_n(x)$. El polinomio $W(x) = \prod (x - x_k)$ es precisamente $T_n(x)/2^{n-1}$. Como $|T_n(x)| \leq 1$, $\|W\|_\infty = 1/2^{n-1}$. Por el teorema de minimax de Chebyshev, esto es el minimo posible.

T7. La demostracion es similar a la de Lagrange, pero con $2n + 2$ condiciones (valores y derivadas). Se puede mostrar que el sistema de ecuaciones es no singular usando la formula de Hermite.

T8. Para un spline cubico natural, las condiciones son $S'''(x_0) = S'''(x_n) = 0$. El sistema resultante es tridiagonal y se puede resolver eficientemente.

T9. La DFT de un vector c es $\hat{c}_j = \sum_{k=0}^{n-1} c_k \omega_j^k$. Evaluar el polinomio $p(x) = \sum_{k=0}^{n-1} c_k x^k$ en $x = \omega_j$ da exactamente $\hat{c}_j$.

T10. Con k puntos, el polinomio de grado $k - 1$ esta univocamente determinado, asi que se puede evaluar en 0. Con $k - 1$ puntos, cualquier valor en 0 es posible (hay infinitos polinomios que pasan por esos puntos), asi que no se obtiene informacion.

Soluciones Computacionales

C1. Ver el codigo de la Seccion .

C2.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def barycentric_weights(x_j):
5     """Calcula los pesos baricentricos."""
6     n = len(x_j)
7     w = np.ones(n)
8     for j in range(n):
9         for k in range(n):
10             if k != j:
11                 w[j] /= (x_j[j] - x_j[k])
12     return w
13
14 def interpolate_barycentric(x, x_j, y_j):
15     """Interpolacion usando la forma baricentrica."""
16     w = barycentric_weights(x_j)
17     n = len(x_j)
18     num = np.zeros_like(x)
19     den = np.zeros_like(x)
20
21     for j in range(n):
22         # Manejar el caso x == x_j
23         mask = np.abs(x - x_j[j]) < 1e-12
24         if np.any(mask):
25             return y_j[j] * np.ones_like(x)
26         term = w[j] / (x - x_j[j])
27         num += y_j[j] * term
28         den += term
29
30     return num / den
31
32 # Prueba
33 x_j = np.array([0, 1, 2, 3, 4])
34 y_j = np.array([0, 1, 4, 9, 16]) # x^2
35 x_plot = np.linspace(-0.5, 4.5, 100)
36
37 p = interpolate_barycentric(x_plot, x_j, y_j)
38
39 plt.figure(figsize=(10, 6))
40 plt.plot(x_plot, p, 'b-', label='Interpolacion baricentrica')
41 plt.plot(x_j, y_j, 'ro', label='Puntos')
42 plt.xlabel('x')

```



```

43 plt.ylabel('y')
44 plt.title('Forma baricentrica')
45 plt.legend()
46 plt.grid(True)
47 plt.show()

```

Listing 16: Forma baricentrica

C3. Ver el código de la Sección .

C4. Ver el código de la Sección .

C5.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def interpolate_lagrange(x, x_j, y_j):
5     n = len(x_j)
6     result = np.zeros_like(x)
7     for j in range(n):
8         l_j = np.ones_like(x)
9         for k in range(n):
10             if k != j:
11                 l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])
12         result += y_j[j] * l_j
13     return result
14
15 def f(x):
16     return np.exp(x)
17
18 # Nodos equiespaciados y Chebyshev
19 n_values = range(2, 15)
20 errores_eq = []
21 errores_ch = []
22
23 for n in n_values:
24     # Nodos equiespaciados
25     x_eq = np.linspace(-1, 1, n)
26     y_eq = f(x_eq)
27     x_plot = np.linspace(-1, 1, 200)
28     p_eq = interpolate_lagrange(x_plot, x_eq, y_eq)
29     error_eq = np.max(np.abs(f(x_plot) - p_eq))
30     errores_eq.append(error_eq)
31
32     # Nodos de Chebyshev
33     x_ch = np.cos((2*np.arange(n) + 1) / (2*n) * np.pi)
34     y_ch = f(x_ch)
35     p_ch = interpolate_lagrange(x_plot, x_ch, y_ch)
36     error_ch = np.max(np.abs(f(x_plot) - p_ch))
37     errores_ch.append(error_ch)
38
39 plt.figure(figsize=(10, 6))
40 plt.semilogy(n_values, errores_eq, 'o-', label='Equiespaciados')
41 plt.semilogy(n_values, errores_ch, 's-', label='Chebyshev')
42 plt.xlabel('n (numero de puntos)')
43 plt.ylabel('Error maximo')
44 plt.title('Error de interpolacion para f(x) = e^x')
45 plt.legend()
46 plt.grid(True)
47 plt.show()

```

Listing 17: Error de interpolacion

C6.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def hermite_basis(x, x_j, j):
5     """Calcula los polinomios base de Hermite."""
6     n = len(x_j)
7     # Polinomio de Lagrange l_j
8     l_j = np.ones_like(x)
9     for k in range(n):
10         if k != j:
11             l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])
12
13     # Derivada de l_j en x_j (aproximada numericamente)
14     eps = 1e-8
15     l_j_der = (interpolate_lagrange(x_j[j] + eps, x_j, np.eye(n)[j]) -
16                interpolate_lagrange(x_j[j] - eps, x_j, np.eye(n)[j])) / (2*eps)
17
18     # Polinomios base de Hermite
19     H_j = (1 - 2 * (x - x_j[j]) * l_j_der) * l_j**2
20     Hhat_j = (x - x_j[j]) * l_j**2
21
22     return H_j, Hhat_j
23
24 def interpolate_hermite(x, x_j, y_j, yp_j):
25     """Interpolacion de Hermite."""
26     n = len(x_j)
27     result = np.zeros_like(x)
28     for j in range(n):
29         H_j, Hhat_j = hermite_basis(x, x_j, j)
30         result += y_j[j] * H_j + yp_j[j] * Hhat_j
31     return result
32
33 # Ejemplo: f(x) = sin(x)
34 x_j = np.array([0, np.pi/2, np.pi])
35 y_j = np.sin(x_j)
36 yp_j = np.cos(x_j)
37
38 x_plot = np.linspace(0, np.pi, 200)
39 p = interpolate_hermite(x_plot, x_j, y_j, yp_j)
40
41 plt.figure(figsize=(10, 6))
42 plt.plot(x_plot, np.sin(x_plot), 'k--', label='sin(x) (exacto)')
43 plt.plot(x_plot, p, 'b-', label='Hermite')
44 plt.plot(x_j, y_j, 'ro', label='Puntos')
45 plt.xlabel('x')
46 plt.ylabel('y')
47 plt.title('Interpolacion de Hermite para sin(x)')
48 plt.legend()
49 plt.grid(True)
50 plt.show()

```

Listing 18: Interpolacion de Hermite

C7. Ver el codigo de la Seccion .**C8.**

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def dft_vandermonde(x):

```

```

5     """DFT usando la matriz de Vandermonde con raices de la unidad."""
6     n = len(x)
7     omega = np.exp(2j * np.pi / n)
8     V = np.zeros((n, n), dtype=complex)
9     for j in range(n):
10        for k in range(n):
11            V[j, k] = omega**(j*k)
12    return V @ x
13
14 # Prueba
15 n = 8
16 x = np.random.randn(n)
17 dft_mia = dft_vandermonde(x)
18 dft_np = np.fft.fft(x)
19
20 print("DFT (implementada):", dft_mia)
21 print("DFT (numpy):", dft_np)
22 print("Diferencia:", np.linalg.norm(dft_mia - dft_np))

```

Listing 19: DFT con matriz de Vandermonde

C9.

```

1 import numpy as np
2
3 def shamir_generate_shares(secret, n, k, p):
4     """
5     Genera n shares para un secreto en Z_p con umbral k.
6     """
7     # Polinomio de grado k-1: p(x) = secret + a1*x + a2*x^2 + ... + a_{k-1}*x^{k-1}
8     coefs = np.random.randint(0, p, k-1)
9     coefs = np.append([secret], coefs)
10
11    shares = []
12    for i in range(1, n+1):
13        x = i
14        y = 0
15        for j, coef in enumerate(coefs):
16            y = (y + coef * (x**j)) % p
17        shares.append((x, y))
18
19    return shares, coefs
20
21 def shamir_reconstruct(shares, p):
22     """
23     Reconstruye el secreto a partir de k shares usando interpolacion de Lagrange.
24     """
25    k = len(shares)
26    x_j = np.array([s[0] for s in shares])
27    y_j = np.array([s[1] for s in shares])
28
29    # Evaluamos el polinomio interpolante en x=0 (el secreto)
30    secret = 0
31    for j in range(k):
32        l_j = 1
33        for m in range(k):
34            if m != j:
35                l_j = (l_j * (0 - x_j[m]) * pow(x_j[j] - x_j[m], -1, p)) % p
36        secret = (secret + y_j[j] * l_j) % p
37

```

```

38     return secret
39
40 # Ejemplo
41 p = 97 # Primo
42 secret = 42
43 n = 5 # Numero de shares
44 k = 3 # Umbral
45
46 shares, coefs = shamir_generate_shares(secret, n, k, p)
47 print("Shares:", shares)
48 print("Coeficientes del polinomio:", coefs)
49
50 # Reconstruimos con k shares
51 secret_reconstruido = shamir_reconstruct(shares[:k], p)
52 print("Secreto reconstruido:", secret_reconstruido)
53 print("Secreto original:", secret)

```

Listing 20: Secreto compartido de Shamir

C10.

```

1 import numpy as np
2
3 def reed_solomon_encode(message, n, p):
4     """
5     Codifica un mensaje usando Reed-Solomon.
6     message: lista de simbolos en Z_p (longitud k)
7     n: longitud total del codigo
8     p: primo
9     """
10    k = len(message)
11    # Polinomio:  $m(x) = \text{message}[0] + \text{message}[1]*x + \dots + \text{message}[k-1]*x^{k-1}$ 
12    # Evaluamos en n puntos
13    codeword = []
14    for i in range(n):
15        x = i
16        y = 0
17        for j, m in enumerate(message):
18            y = (y + m * (x**j)) % p
19        codeword.append(y)
20    return codeword
21
22 def reed_solomon_decode(codeword, k, p):
23     """
24     Decodifica un codeword de Reed-Solomon (asume que hay suficientes simbolos
25     correctos).
26     """
27    # Tomamos los primeros k simbolos (si hay errores, necesitamos un algoritmo
28    # mas complejo)
29    # Aqui implementamos un decodificador simple que asume que los primeros k
30    # son correctos
31    n = len(codeword)
32    x_j = np.arange(k)
33    y_j = codeword[:k]
34
35    # Interpolamos para recuperar el mensaje
36    message = []
37    for j in range(k):
38        l_j = 1
39        for m in range(k):
40            if m != j:
41                l_j = (l_j * (0 - x_j[m]) * pow(x_j[j] - x_j[m], -1, p)) % p

```

```
39     message.append((y_j[j] * l_j) % p)
40
41     return message
42
43 # Ejemplo
44 p = 97
45 k = 3 # Longitud del mensaje
46 n = 7 # Longitud del codigo
47
48 message = [5, 10, 15]
49 codeword = reed_solomon_encode(message, n, p)
50 print("Codeword:", codeword)
51
52 # Introducimos un error
53 codeword[2] = (codeword[2] + 1) % p
54 print("Codeword con error:", codeword)
55
56 # Decodificamos (con el decodificador simple)
57 message_recuperado = reed_solomon_decode(codeword, k, p)
58 print("Mensaje recuperado:", message_recuperado)
59 print("Mensaje original:", message)
```

Listing 21: Codigo de Reed-Solomon

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por el mundo de la interpolacion. Viste que no era tan complicado. Arrancamos con la base de Lagrange, pasamos por el error de interpolacion, los polinomios de Chebyshev, los splines, y terminamos con aplicaciones en el plano complejo y en cuerpos finitos. Todo esta conectado: la interpolacion es una herramienta fundamental para aproximar funciones y tiene aplicaciones en todas las areas de la ciencia y la ingenieria.

Los puntos clave para llevar:

- El polinomio interpolante existe y es unico.
- La base de Lagrange es la forma mas directa de construirlo.
- El error de interpolacion depende de la derivada $n + 1$ -esima y de la posicion de los nodos.
- Los nodos de Chebyshev minimizan el error y evitan el fenomeno de Runge.
- Los splines son mejores para aproximar funciones con muchos puntos.
- La interpolacion en el plano complejo da lugar a la DFT y la FFT.
- La interpolacion sobre cuerpos finitos tiene aplicaciones en criptografia y teoria de codigos.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. ¡Dale para adelante, que vos podes!

Aproximacion Numerica: De Fourier a los Polinomios Ortogonales

Como representar funciones de forma eficiente y precisa con Python.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a meterle a uno de los temas mas profundos y utiles del analisis numerico: la aproximacion de funciones. Si llegaste hasta aca, seguro te preguntaste alguna vez como hace tu compu para calcular un seno, o como se representa una imagen en formato JPEG, o como se resuelven ecuaciones diferenciales con metodos espectrales. La respuesta a todo eso esta en la aproximacion de funciones.

Cuando curse Analisis Numerico, recuerdo que el profesor nos dijo: "el objetivo de este curso es que entiendan como aproximar funciones de la mejor manera posible". Y tenia razon. Casi todo lo que hacemos en computacion cientifica es aproximar funciones: desde resolver ecuaciones diferenciales hasta procesar señales o hacer machine learning. La pregunta fundamental es: ¿como elegimos la mejor representacion de una funcion?

En este capitulo vamos a cubrir:

- El problema de aproximacion: ¿que significa "mejor" aproximacion?
- Los polinomios ortogonales y por que son tan especiales.
- Las series de Fourier y su convergencia espectral.
- Los polinomios de Chebyshev y la aproximacion en $[-1, 1]$.
- La constante de Lebesgue y el fenomeno de Runge.
- Los teoremas de Jackson y Bernstein: relacion entre suavidad y aproximabilidad.
- Aplicaciones en metodos espectrales y compresion de datos.

La idea es que vos puedas implementar todo esto en Python, entender por que funciona, y saber que metodo usar en cada situacion. Asi que agarra un cafe, ponete comodo, y vamos a darle.

El problema de aproximacion

El problema de aproximacion es simple de enunciar: tenemos una funcion f (que puede ser complicada, o solo conocida en algunos puntos) y queremos encontrar una funcion g de una familia mas simple (polinomios, funciones trigonometricas, etc.) que se parezca lo mas posible a f .

Traduccion Matematica del problema de aproximacion El problema de aproximacion se lee en criollo: "tenemos una funcion complicada y queremos reemplazarla por una funcion mas simple que haga el mismo trabajo, pero con un error controlado". Pensa en esto como si estuvieras describiendo una montania con una curva suave: no necesitas conocer cada piedra, solo la forma general.

Hay dos enfoques principales:

1. **Interpolacion:** g coincide con f en algunos puntos.
2. **Aproximacion por minimos cuadrados:** minimizamos $\|f - g\|_2$ (el error cuadratico medio).

Normas y medicion del error

Para medir que tan buena es una aproximacion, necesitamos una **norma**. Las mas comunes son:

- Norma infinito: $\|f\|_\infty = \max_{x \in [a,b]} |f(x)|$
- Norma 2 (con peso): $\|f\|_2 = \sqrt{\int_a^b |f(x)|^2 w(x) dx}$
- Norma 1: $\|f\|_1 = \int_a^b |f(x)| dx$

Traduccion Matematica de las normas Las normas se leen en criollo: "la norma infinito mide el error maximo en todo el intervalo; la norma 2 mide el error cuadratico medio; la norma 1 mide el error absoluto promedio". La eleccion depende del problema: si necesitas garantizar que el error nunca exceda un valor, usas la norma infinito. Si te interesa el comportamiento promedio, usas la norma 2.

Espacios con producto interno

Para el enfoque de minimos cuadrados, necesitamos un **producto interno**. Esto nos permite definir ortogonalidad y proyeccion.

Producto interno con peso

Dado un intervalo (a, b) y un peso $w(x) > 0$, definimos:

$$\langle f, g \rangle = \int_a^b f(x)g(x)w(x)dx$$

Traduccion Matematica del producto interno con peso El producto interno con peso se lee en criollo: ".es una forma de multiplicar dos funciones para obtener un numero, donde le damos mas importancia a algunas regiones del intervalo (las que tienen peso mas grande)". Pensa en el peso como una "importancia" que le asignas a cada punto.

Polinomios ortogonales

Una familia de polinomios $\{p_n\}$ es **ortogonal** si:

$$\langle p_n, p_m \rangle = 0 \quad \text{para } n \neq m$$

Traduccion Matematica de la ortogonalidad Los polinomios ortogonales se leen en criollo: "son polinomios que son perpendiculares entre si en el sentido del producto interno". Esto es como tener una base de vectores ortogonales en un espacio de funciones.

Los ejemplos clasicos son:

- **Legendre:** $w(x) = 1$, $(a, b) = (-1, 1)$

- **Chebyshev:** $w(x) = 1/\sqrt{1-x^2}$, $(a, b) = (-1, 1)$
- **Laguerre:** $w(x) = e^{-x}$, $(a, b) = (0, \infty)$
- **Hermite:** $w(x) = e^{-x^2}$, $(a, b) = (-\infty, \infty)$

Proyeccion ortogonal

La **proyeccion ortogonal** de f sobre el subespacio generado por $\{p_0, \dots, p_n\}$ es:

$$P_n f = \sum_{k=0}^n \langle f, p_k \rangle p_k$$

Si los p_k estan normalizados ($\|p_k\| = 1$), esta es la mejor aproximacion en norma 2.

Traduccion Matematica de la proyeccion La proyeccion se lee en criollo: "la mejor aproximacion de f en el subespacio de polinomios de grado $\leq n$ es simplemente la suma de las proyecciones de f sobre cada polinomio base". Es como descomponer un vector en sus componentes a lo largo de una base ortogonal.

Series de Fourier

Las series de Fourier son el ejemplo mas importante de aproximacion con funciones ortogonales. En $L^2[0, 2\pi]$, las funciones $\{e^{ikx}\}$ forman una base ortonormal.

Coeficientes de Fourier

Para una funcion f 2π -periodica, los coeficientes de Fourier son:

$$\hat{f}_k = \frac{1}{2\pi} \int_0^{2\pi} f(x) e^{-ikx} dx$$

Y la serie de Fourier es:

$$f(x) \sim \sum_{k=-\infty}^{\infty} \hat{f}_k e^{ikx}$$

Traduccion Matematica de los coeficientes de Fourier Los coeficientes de Fourier se leen en criollo: cada coeficiente $\hat{f}_k$ mide cuanto de la frecuencia k tiene la funcion f . Pensa en esto como descomponer una señal en sus frecuencias fundamentales, como un acorde de guitarra descompuesto en las notas que lo componen.

Convergencia espectral

Si f es m veces diferenciable y periodica, entonces:

$$|\hat{f}_k| = O(|k|^{-m})$$

Traduccion Matematica de la convergencia espectral La convergencia espectral se lee en criollo: ¿quanto mas suave es la funcion, mas rapidamente decaen sus coeficientes de Fourier”. Si la funcion es infinitamente diferenciable, los coeficientes decaen mas rapido que cualquier potencia de k (convergencia super-algebraica). Esto es lo que hace que las series de Fourier sean tan efectivas para funciones suaves.

Grafico: convergencia de Fourier

Vamos a visualizar la convergencia de la serie de Fourier para una funcion suave.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def fourier_aproximacion(x, N, f):
5     """Calcula la aproximacion de Fourier de orden N."""
6     # Coeficientes de Fourier (integrados numericamente)
7     n_pts = len(x)
8     dx = x[1] - x[0]
9     f_vals = f(x)
10
11     # Calculamos los coeficientes
12     coefs = []
13     for k in range(-N, N+1):
14         c_k = (1/(2*np.pi)) * np.trapz(f_vals * np.exp(-1j * k * x), x)
15         coefs.append(c_k)
16
17     # Reconstruimos
18     aprox = np.zeros_like(x, dtype=complex)
19     for k, c in zip(range(-N, N+1), coefs):
20         aprox += c * np.exp(1j * k * x)
21
22     return aprox.real
23
24 # Funcion suave: f(x) = cos(x) + 0.5*sin(2x) + 0.2*cos(3x)
25 def f(x):
26     return np.cos(x) + 0.5*np.sin(2*x) + 0.2*np.cos(3*x)
27
28 x = np.linspace(0, 2*np.pi, 500)
29
30 plt.figure(figsize=(10, 6))
31 plt.plot(x, f(x), 'k-', label='f(x) exacta', linewidth=2)
32
33 for N in [1, 2, 3, 5, 10]:
34     aprox = fourier_aproximacion(x, N, f)
35     plt.plot(x, aprox, label=f'N = {N}')
36
37 plt.xlabel('x')
38 plt.ylabel('f(x)')
39 plt.title('Convergencia de la serie de Fourier')
40 plt.legend()
41 plt.grid(True)
42 plt.show()

```

Listing 22: Convergencia de la serie de Fourier

En el grafico, ves como la serie de Fourier converge rapidamente a la funcion a medida que aumentamos N . Para $N = 10$, la aproximacion es practicamente indistinguible de la funcion exacta.

Polinomios de Chebyshev

Los polinomios de Chebyshev son la herramienta preferida para aproximar funciones en $[-1, 1]$. Se definen como:

$$T_n(x) = \cos(n \arccos(x))$$

Traducción Matemática de los polinomios de Chebyshev Los polinomios de Chebyshev se leen en criollo: "son polinomios que se comportan como cosenos en el intervalo $[-1, 1]$ ". Tienen la propiedad de que minimizan el error de interpolación y tienen una convergencia espectral similar a la de Fourier.

Propiedades

1. $T_n(\cos \theta) = \cos(n\theta)$
2. $|T_n(x)| \leq 1$ para $x \in [-1, 1]$
3. Coeficiente principal: 2^{n-1} (para $n \geq 1$)
4. Relación de recurrencia: $T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$
5. Ortogonalidad: $\int_{-1}^1 \frac{T_n(x)T_m(x)}{\sqrt{1-x^2}} dx = \begin{cases} \pi & n = m = 0 \\ \pi/2 & n = m \neq 0 \\ 0 & n \neq m \end{cases}$

Serie de Chebyshev

Cualquier función $f \in C[-1, 1]$ se puede expandir en serie de Chebyshev:

$$f(x) = \sum_{n=0}^{\infty} a_n T_n(x)$$

donde:

$$a_n = \frac{2}{\pi} \int_{-1}^1 \frac{f(x)T_n(x)}{\sqrt{1-x^2}} dx \quad (n > 0), \quad a_0 = \frac{1}{\pi} \int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx$$

Grafico: polinomios de Chebyshev

Vamos a graficar los primeros polinomios de Chebyshev.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def chebyshev(n, x):
5     """Evalua el polinomio de Chebyshev T_n(x) usando la recurrencia."""
6     if n == 0:
7         return np.ones_like(x)
8     elif n == 1:
9         return x
10    else:
11        T_prev = np.ones_like(x)
12        T_curr = x
13        for _ in range(2, n+1):

```

```

14         T_next = 2 * x * T_curr - T_prev
15         T_prev, T_curr = T_curr, T_next
16         return T_curr
17
18 x = np.linspace(-1, 1, 500)
19
20 plt.figure(figsize=(10, 6))
21 for n in range(6):
22     y = chebyshev(n, x)
23     plt.plot(x, y, label=f'T_{n}(x)')
24
25 plt.xlabel('x')
26 plt.ylabel('T_n(x)')
27 plt.title('Polinomios de Chebyshev')
28 plt.legend()
29 plt.grid(True)
30 plt.axhline(0, color='black', linewidth=0.5)
31 plt.axvline(0, color='black', linewidth=0.5)
32 plt.show()

```

Listing 23: Polinomios de Chebyshev

Convergencia de Chebyshev

Si f es analítica en una vecindad de $[-1, 1]$, los coeficientes de Chebyshev decaen exponencialmente:

$$|a_n| = O(\rho^{-n})$$

para algún $\rho > 1$. Esto es la **convergencia exponencial**, incluso mejor que la convergencia algebraica de Fourier para funciones C^∞ .

El error de interpolacion y la constante de Lebesgue

El error de interpolacion polinomial se puede descomponer en dos partes: el error de la mejor aproximacion y el factor de amplificacion dado por la **constante de Lebesgue**.

Constante de Lebesgue

Dados nodos $\{x_0, \dots, x_n\}$, la funcion de Lebesgue es:

$$\Lambda(x) = \sum_{k=0}^n |\ell_k(x)|$$

donde ℓ_k son los polinomios de Lagrange. La constante de Lebesgue es:

$$\Lambda_n = \max_{x \in [a, b]} \Lambda(x)$$

Traduccion Matematica de la constante de Lebesgue La constante de Lebesgue se lee en criollo: "mide cuanto puede amplificarse el error de la mejor aproximacion al pasar a la interpolacion". Si Λ_n es grande, la interpolacion puede ser mucho peor que la mejor aproximacion posible. Si es pequena, la interpolacion es casi optima.

Teorema de la constante de Lebesgue

Si p_n es el polinomio que interpola a f en los nodos, y p_n^* es la mejor aproximacion de grado n , entonces:

$$\|f - p_n\|_\infty \leq (1 + \Lambda_n) \|f - p_n^*\|_\infty$$

Valores de la constante de Lebesgue

- Nodos equiespaciados: $\Lambda_n \sim \frac{2^{n+1}}{en \log n}$ (crece exponencialmente).
- Nodos de Chebyshev: $\Lambda_n \sim \frac{2}{\pi} \log n$ (crece logarítmicamente).

Traducción Matemática de los valores de Lebesgue Los valores de la constante de Lebesgue se leen en criollo: con nodos equiespaciados, la constante de Lebesgue crece exponencialmente, lo que explica el fenómeno de Runge. Con nodos de Chebyshev, crece solo logarítmicamente, lo que la hace mucho más estable”.

El fenómeno de Runge

El **fenómeno de Runge** es un ejemplo clásico de cómo la interpolación con nodos equiespaciados puede fallar espectacularmente.

La función de Runge

$$f(x) = \frac{1}{1 + 25x^2}$$

en $[-1, 1]$. Esta función es infinitamente diferenciable, pero la interpolación con nodos equiespaciados diverge en los bordes.

¿Por qué ocurre?

La constante de Lebesgue para nodos equiespaciados crece exponencialmente, lo que amplifica los errores de la mejor aproximación. Además, los nodos están muy separados cerca de los bordes, lo que hace que el polinomio tenga que oscilar violentamente para pasar por todos los puntos.

Grafico: fenómeno de Runge

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def runge(x):
5     return 1 / (1 + 25 * x**2)
6
7 def interpolate_lagrange(x, x_j, y_j):
8     n = len(x_j)
9     result = np.zeros_like(x)
10    for j in range(n):
11        l_j = np.ones_like(x)
12        for k in range(n):
13            if k != j:
14                l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])

```

```

15         result += y_j[j] * l_j
16     return result
17
18 n_values = [5, 10, 15, 20]
19 x_plot = np.linspace(-1, 1, 500)
20
21 plt.figure(figsize=(12, 8))
22 plt.plot(x_plot, runge(x_plot), 'k-', label='Funcion de Runge', linewidth=2)
23
24 for n in n_values:
25     x_j = np.linspace(-1, 1, n)
26     y_j = runge(x_j)
27     p = interpolate_lagrange(x_plot, x_j, y_j)
28     plt.plot(x_plot, p, label=f'n = {n}')
29
30 plt.xlabel('x')
31 plt.ylabel('f(x)')
32 plt.title('Fenomeno de Runge: interpolacion con nodos equiespaciados')
33 plt.legend()
34 plt.grid(True)
35 plt.ylim(-2, 2)
36 plt.show()

```

Listing 24: Fenomeno de Runge

En el grafico, ves como el polinomio interpolante oscila cada vez mas cerca de los bordes a medida que aumenta el grado. Para $n = 20$, las oscilaciones son enormes.

Teoremas de Jackson y Bernstein

Estos dos teoremas caracterizan la relacion entre la suavidad de una funcion y la velocidad de su mejor aproximacion polinomial.

Teorema de Jackson

Si $f \in C^m[a, b]$, entonces:

$$E_n(f) \leq \frac{C_m}{n^m} \|f^{(m)}\|_\infty$$

donde $E_n(f) = \inf_{p \in \mathcal{P}_n} \|f - p\|_\infty$ es el error de la mejor aproximacion.

Traduccion Matematica del teorema de Jackson El teorema de Jackson se lee en criollo: "si una funcion tiene m derivadas continuas, entonces puede ser aproximada por polinomios de grado n con un error que decae como $1/n^m$ ". Es decir, la suavidad de la funcion determina la velocidad de convergencia.

Teorema de Bernstein (reciproco)

Si $E_n(f) = O(n^{-m})$, entonces $f \in C^{m-1}$ y su derivada $(m-1)$ -esima es Holder continua.

Traduccion Matematica del teorema de Bernstein El teorema de Bernstein se lee en criollo: "si una funcion puede ser aproximada por polinomios con error $O(n^{-m})$, entonces la funcion tiene $m-1$ derivadas continuas". Es el reciproco del teorema de Jackson: la velocidad de aproximacion determina la suavidad de la funcion.

Aplicaciones: metodos espectrales

Los metodos espectrales usan aproximaciones de Fourier o Chebyshev para resolver ecuaciones diferenciales. La idea es:

1. Representar la solucion como una serie de Fourier o Chebyshev.
2. Sustituir en la ecuacion diferencial.
3. Obtener un sistema de ecuaciones para los coeficientes.
4. Resolver el sistema.

Ventajas de los metodos espectrales

1. **Convergencia exponencial:** para funciones suaves, el error decae exponencialmente con el numero de modos.
2. **Precision:** se puede alcanzar precision de maquina con pocos modos.
3. **Eficiencia:** la FFT permite calcular transformadas en $O(n \log n)$.

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Producto interno:** Demuestra que $\langle f, g \rangle = \int_a^b f(x)g(x)w(x)dx$ con $w(x) > 0$ satisface las propiedades de un producto interno.
- T2. Ortogonalidad de Legendre:** Demuestra que los polinomios de Legendre son ortogonales en $[-1, 1]$ con peso $w(x) = 1$.
- T3. Coeficientes de Fourier:** Demuestra que si f es real y par, sus coeficientes de Fourier son reales.
- T4. Convergencia de Fourier:** Demuestra que si $f \in C^m(\mathbb{R})$ es 2π -periodica, entonces $|\hat{f}_k| = O(|k|^{-m})$.
- T5. Polinomios de Chebyshev:** Demuestra que $T_n(\cos \theta) = \cos(n\theta)$.
- T6. Ortogonalidad de Chebyshev:** Demuestra que $\int_{-1}^1 \frac{T_n(x)T_m(x)}{\sqrt{1-x^2}}dx = 0$ para $n \neq m$.
- T7. Constante de Lebesgue:** Demuestra que $\|f - p_n\|_\infty \leq (1 + \Lambda_n)\|f - p_n^*\|_\infty$.
- T8. Teorema de Jackson:** Demuestra que $E_n(f) \leq \frac{C}{n}\|f'\|_\infty$ para $f \in C^1$.
- T9. Teorema de Bernstein:** Demuestra que si $E_n(f) = O(n^{-m})$, entonces f es $(m-1)$ veces diferenciable.
- T10. Fenomeno de Runge:** Explica por que ocurre el fenomeno de Runge y como los nodos de Chebyshev lo evitan.

Parte Computacional (10 ejercicios)

- C1. Polinomios de Legendre:** Implementa una funcion que calcule los polinomios de Legendre usando la recurrencia. Grafica los primeros 6.
- C2. Serie de Fourier:** Implementa una funcion que calcule los coeficientes de Fourier de una funcion dada y reconstruya la aproximacion. Prueba con funciones suaves y con funciones con discontinuidades.
- C3. Serie de Chebyshev:** Implementa una funcion que calcule los coeficientes de Chebyshev usando la transformada coseno discreta (DCT). Prueba con varias funciones.
- C4. Constante de Lebesgue:** Calcula la constante de Lebesgue para nodos equiespaciados y para nodos de Chebyshev para $n = 2, \dots, 20$. Grafica los resultados.
- C5. Fenomeno de Runge:** Reproduce el fenomeno de Runge para la funcion $1/(1 + 25x^2)$ con nodos equiespaciados para $n = 5, 10, 15, 20$. Compara con nodos de Chebyshev.
- C6. Error de interpolacion:** Para $f(x) = e^x$ en $[-1, 1]$, calcula el error maximo de interpolacion con nodos equiespaciados y con nodos de Chebyshev para $n = 2, \dots, 15$. Grafica los errores.
- C7. Metodo espectral:** Resuelve la ecuacion diferencial $-u''(x) = f(x)$ en $[-1, 1]$ con condiciones de borde usando un metodo espectral de Chebyshev. Compara con la solucion exacta.
- C8. Aproximacion de funciones discontinuas:** Aproxima la funcion escalon $f(x) = \text{sign}(x)$ en $[-1, 1]$ usando series de Fourier y de Chebyshev. Compara las aproximaciones.
- C9. Compresion de datos:** Usa la SVD y la aproximacion de bajo rango para comprimir una imagen en escala de grises. Grafica la imagen original y las versiones comprimidas para diferentes rangos.
- C10. Comparacion de metodos:** Compara la interpolacion, la aproximacion por minimos cuadrados y la aproximacion por series de Chebyshev para una funcion ruidosa. ¿Cual metodo da mejores resultados?

Soluciones

Soluciones Teoricas

T1. Las propiedades de producto interno son: linealidad en el primer argumento, simetria, y positividad. La linealidad viene de la linealidad de la integral. La simetria es obvia porque el producto es conmutativo. La positividad: $\langle f, f \rangle = \int_a^b |f(x)|^2 w(x) dx \geq 0$, y es cero solo si $f \equiv 0$ (porque $w > 0$ y f es continua).

T2. Los polinomios de Legendre se definen como $P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n$. Usando integracion por partes n veces, se demuestra que $\int_{-1}^1 P_n(x) P_m(x) dx = 0$ para $n \neq m$.

T3. Si f es real y par, entonces $\hat{f}_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) \cos(kx) dx - i \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) \sin(kx) dx$. La integral del seno es 0 porque el integrando es impar. Asi que $\hat{f}_k$ es real.

T4. Integrando por partes m veces: $\hat{f}_k = \frac{1}{(ik)^m} \hat{f}_k^{(m)}$. Como $f^{(m)}$ es acotada, $|\hat{f}_k^{(m)}| \leq C$, asi que $|\hat{f}_k| \leq C/|k|^m$.

T5. Por definicion, $T_n(x) = \cos(n \arccos(x))$. Si $x = \cos \theta$, entonces $T_n(\cos \theta) = \cos(n\theta)$.

T6. Usando el cambio de variable $x = \cos \theta$: $\int_{-1}^1 \frac{T_n(x)T_m(x)}{\sqrt{1-x^2}} dx = \int_0^\pi \cos(n\theta) \cos(m\theta) d\theta = 0$ para $n \neq m$.

T7. Sea p_n^* la mejor aproximación. Entonces $\|f - p_n\|_\infty \leq \|f - p_n^*\|_\infty + \|p_n^* - p_n\|_\infty$. Pero $p_n^* - p_n = \sum_{k=0}^n (p_n^*(x_k) - f(x_k)) \ell_k(x)$, así que $\|p_n^* - p_n\|_\infty \leq \Lambda_n \|f - p_n^*\|_\infty$.

T8. El teorema de Jackson se demuestra usando los polinomios de Bernstein. Para $f \in C^1$, $E_n(f) \leq \frac{C}{n} \|f'\|_\infty$.

T9. Si $E_n(f) = O(n^{-m})$, entonces para cualquier $h > 0$, se puede mostrar que $\omega(f, h) = O(h^{m-1})$. Esto implica que f es $(m-1)$ veces diferenciable.

T10. El fenómeno de Runge ocurre porque la constante de Lebesgue para nodos equiespaciados crece exponencialmente, amplificando los errores. Los nodos de Chebyshev minimizan la constante de Lebesgue, evitando el fenómeno.

Soluciones Computacionales

C1.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def legendre(n, x):
5     """Evalua el polinomio de Legendre P_n(x) usando la recurrencia."""
6     if n == 0:
7         return np.ones_like(x)
8     elif n == 1:
9         return x
10    else:
11        P_prev = np.ones_like(x)
12        P_curr = x
13        for k in range(1, n):
14            P_next = ((2*k + 1) * x * P_curr - k * P_prev) / (k + 1)
15            P_prev, P_curr = P_curr, P_next
16        return P_curr
17
18 x = np.linspace(-1, 1, 500)
19 plt.figure(figsize=(10, 6))
20 for n in range(6):
21     y = legendre(n, x)
22     plt.plot(x, y, label=f'P_{n}(x)')
23 plt.xlabel('x')
24 plt.ylabel('P_n(x)')
25 plt.title('Polinomios de Legendre')
26 plt.legend()
27 plt.grid(True)
28 plt.axhline(0, color='black', linewidth=0.5)
29 plt.axvline(0, color='black', linewidth=0.5)
30 plt.show()

```

Listing 25: Polinomios de Legendre

C2.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def fourier_coefs(f, x, N):
5     """Calcula los coeficientes de Fourier de f en [-pi, pi]."""
6     dx = x[1] - x[0]
7     f_vals = f(x)
8     coefs = []

```

```

9     for k in range(-N, N+1):
10         c = (1/(2*np.pi)) * np.trapz(f_vals * np.exp(-1j * k * x), x)
11         coefs.append(c)
12     return coefs
13
14 def fourier_reconstruct(x, coefs):
15     """Reconstruye la funcion a partir de los coeficientes de Fourier."""
16     N = (len(coefs) - 1) // 2
17     result = np.zeros_like(x, dtype=complex)
18     for k, c in zip(range(-N, N+1), coefs):
19         result += c * np.exp(1j * k * x)
20     return result.real
21
22 # Funcion con discontinuidad
23 def f(x):
24     return np.where(x < 0, -1, 1)
25
26 x = np.linspace(-np.pi, np.pi, 500)
27 coefs = fourier_coefs(f, x, 20)
28 aprox = fourier_reconstruct(x, coefs)
29
30 plt.figure(figsize=(10, 6))
31 plt.plot(x, f(x), 'k-', label='Funcion exacta')
32 plt.plot(x, aprox, 'r-', label='Aproximacion Fourier N=20')
33 plt.xlabel('x')
34 plt.ylabel('f(x)')
35 plt.title('Serie de Fourier de una funcion escalon')
36 plt.legend()
37 plt.grid(True)
38 plt.show()

```

Listing 26: Serie de Fourier

C3.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def chebyshev_coefs(f, N):
5     """Calcula los coeficientes de Chebyshev de f en [-1, 1]."""
6     # Usamos los nodos de Chebyshev para la cuadratura
7     n_pts = 500
8     x = np.cos(np.linspace(0, np.pi, n_pts))
9     f_vals = f(x)
10    dx = np.diff(x)
11
12    coefs = np.zeros(N+1)
13    coefs[0] = (1/np.pi) * np.trapz(f_vals / np.sqrt(1 - x**2), x)
14    for n in range(1, N+1):
15        T_n = np.cos(n * np.arccos(x))
16        coefs[n] = (2/np.pi) * np.trapz(f_vals * T_n / np.sqrt(1 - x**2), x)
17    return coefs
18
19 def chebyshev_reconstruct(x, coefs):
20     """Reconstruye la funcion a partir de los coeficientes de Chebyshev."""
21     result = np.zeros_like(x)
22     for n, c in enumerate(coefs):
23         result += c * np.cos(n * np.arccos(x))
24     return result
25
26 # Funcion de prueba
27 def f(x):

```

```

28     return np.exp(x)
29
30 x = np.linspace(-1, 1, 200)
31 coefs = chebyshev_coefs(f, 10)
32 aprox = chebyshev_reconstruct(x, coefs)
33
34 plt.figure(figsize=(10, 6))
35 plt.plot(x, f(x), 'k-', label='f(x) = e^x')
36 plt.plot(x, aprox, 'r-', label='Aproximacion Chebyshev N=10')
37 plt.xlabel('x')
38 plt.ylabel('f(x)')
39 plt.title('Serie de Chebyshev de e^x')
40 plt.legend()
41 plt.grid(True)
42 plt.show()

```

Listing 27: Serie de Chebyshev

C4.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def lebesgue_constant(x_j):
5     """Calcula la constante de Lebesgue para nodos x_j."""
6     n = len(x_j)
7     x_plot = np.linspace(-1, 1, 500)
8     Lambda = np.zeros_like(x_plot)
9
10    for i, x in enumerate(x_plot):
11        for j in range(n):
12            l_j = 1
13            for k in range(n):
14                if k != j:
15                    l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])
16            Lambda[i] += np.abs(l_j)
17
18    return np.max(Lambda)
19
20 n_values = range(2, 20)
21 L_eq = []
22 L_ch = []
23
24 for n in n_values:
25     # Nodos equiespaciados
26     x_eq = np.linspace(-1, 1, n)
27     L_eq.append(lebesgue_constant(x_eq))
28
29     # Nodos de Chebyshev
30     x_ch = np.cos((2*np.arange(n) + 1) / (2*n) * np.pi)
31     L_ch.append(lebesgue_constant(x_ch))
32
33 plt.figure(figsize=(10, 6))
34 plt.semilogy(n_values, L_eq, 'o-', label='Equiespaciados')
35 plt.semilogy(n_values, L_ch, 's-', label='Chebyshev')
36 plt.xlabel('n (numero de nodos)')
37 plt.ylabel('Constante de Lebesgue')
38 plt.title('Constante de Lebesgue para diferentes nodos')
39 plt.legend()
40 plt.grid(True)
41 plt.show()

```

Listing 28: Constante de Lebesgue

C5.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def runge(x):
5     return 1 / (1 + 25 * x**2)
6
7 def interpolate_lagrange(x, x_j, y_j):
8     n = len(x_j)
9     result = np.zeros_like(x)
10    for j in range(n):
11        l_j = np.ones_like(x)
12        for k in range(n):
13            if k != j:
14                l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])
15        result += y_j[j] * l_j
16    return result
17
18 n = 15
19 x_plot = np.linspace(-1, 1, 500)
20
21 # Nodos equiespaciados
22 x_eq = np.linspace(-1, 1, n)
23 y_eq = runge(x_eq)
24 p_eq = interpolate_lagrange(x_plot, x_eq, y_eq)
25
26 # Nodos de Chebyshev
27 x_ch = np.cos((2*np.arange(n) + 1) / (2*n) * np.pi)
28 y_ch = runge(x_ch)
29 p_ch = interpolate_lagrange(x_plot, x_ch, y_ch)
30
31 plt.figure(figsize=(12, 6))
32 plt.subplot(1, 2, 1)
33 plt.plot(x_plot, runge(x_plot), 'k-', label='Funcion de Runge')
34 plt.plot(x_plot, p_eq, 'r-', label='Equiespaciados')
35 plt.plot(x_eq, y_eq, 'ro', markersize=5)
36 plt.xlabel('x')
37 plt.ylabel('f(x)')
38 plt.title('Nodos equiespaciados (n=15)')
39 plt.legend()
40 plt.grid(True)
41 plt.ylim(-2, 2)
42
43 plt.subplot(1, 2, 2)
44 plt.plot(x_plot, runge(x_plot), 'k-', label='Funcion de Runge')
45 plt.plot(x_plot, p_ch, 'b-', label='Chebyshev')
46 plt.plot(x_ch, y_ch, 'bo', markersize=5)
47 plt.xlabel('x')
48 plt.ylabel('f(x)')
49 plt.title('Nodos de Chebyshev (n=15)')
50 plt.legend()
51 plt.grid(True)
52 plt.ylim(-2, 2)
53
54 plt.tight_layout()
55 plt.show()

```

Listing 29: Fenomeno de Runge con Chebyshev

C6.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def interpolate_lagrange(x, x_j, y_j):
5     n = len(x_j)
6     result = np.zeros_like(x)
7     for j in range(n):
8         l_j = np.ones_like(x)
9         for k in range(n):
10             if k != j:
11                 l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])
12         result += y_j[j] * l_j
13     return result
14
15 def f(x):
16     return np.exp(x)
17
18 n_values = range(2, 16)
19 errores_eq = []
20 errores_ch = []
21
22 for n in n_values:
23     x_plot = np.linspace(-1, 1, 200)
24
25     # Nodos equiespaciados
26     x_eq = np.linspace(-1, 1, n)
27     y_eq = f(x_eq)
28     p_eq = interpolate_lagrange(x_plot, x_eq, y_eq)
29     error_eq = np.max(np.abs(f(x_plot) - p_eq))
30     errores_eq.append(error_eq)
31
32     # Nodos de Chebyshev
33     x_ch = np.cos((2*np.arange(n) + 1) / (2*n) * np.pi)
34     y_ch = f(x_ch)
35     p_ch = interpolate_lagrange(x_plot, x_ch, y_ch)
36     error_ch = np.max(np.abs(f(x_plot) - p_ch))
37     errores_ch.append(error_ch)
38
39 plt.figure(figsize=(10, 6))
40 plt.semilogy(n_values, errores_eq, 'o-', label='Equiespaciados')
41 plt.semilogy(n_values, errores_ch, 's-', label='Chebyshev')
42 plt.xlabel('n (numero de puntos)')
43 plt.ylabel('Error maximo')
44 plt.title('Error de interpolacion para f(x) = e^x')
45 plt.legend()
46 plt.grid(True)
47 plt.show()

```

Listing 30: Error de interpolacion

C7.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def chebyshev_spectral(N):

```

```

5  """Resuelve -u''(x) = f(x) en [-1, 1] con u(-1) = u(1) = 0 usando metodo
    espectral de Chebyshev."""
6  # Nodos de Chebyshev (incluyendo los bordes)
7  x = np.cos(np.linspace(0, np.pi, N+1))
8
9  # Matriz de derivacion espectral de Chebyshev
10 D = np.zeros((N+1, N+1))
11 for i in range(N+1):
12     for j in range(N+1):
13         if i != j:
14             D[i, j] = (-1)**(i+j) / (x[i] - x[j])
15         D[i, i] = 0
16
17 # Corregimos la diagonal
18 for i in range(N+1):
19     suma = 0
20     for j in range(N+1):
21         if i != j:
22             suma += 1 / (x[i] - x[j])
23     D[i, i] = -suma
24
25 # Derivada segunda
26 D2 = D @ D
27
28 # Condiciones de borde
29 D2 = D2[1:-1, 1:-1]
30 x_int = x[1:-1]
31
32 # Funcion f
33 def f(x):
34     return np.exp(x) * (1 - x**2) * (-2) # Solucion exacta: u(x) = exp(x)
    *(1-x^2)
35
36 b = f(x_int)
37
38 # Resolvemos
39 u_int = np.linalg.solve(D2, b)
40
41 # Reconstruimos la solucion
42 u = np.zeros(N+1)
43 u[1:-1] = u_int
44
45 return x, u
46
47 N = 20
48 x, u = chebyshev_spectral(N)
49 x_exact = np.linspace(-1, 1, 200)
50 u_exact = np.exp(x_exact) * (1 - x_exact**2)
51
52 plt.figure(figsize=(10, 6))
53 plt.plot(x, u, 'ro', label='Solucion espectral')
54 plt.plot(x_exact, u_exact, 'b-', label='Solucion exacta')
55 plt.xlabel('x')
56 plt.ylabel('u(x)')
57 plt.title('Metodo espectral de Chebyshev para -u''''(x) = f(x)')
58 plt.legend()
59 plt.grid(True)
60 plt.show()

```

Listing 31: Metodo espectral de Chebyshev

C8.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return np.sign(x)
6
7 def fourier_aprox(x, N):
8     """Aproximacion de Fourier de la funcion signo."""
9     result = np.zeros_like(x)
10    for k in range(1, N+1):
11        if k % 2 == 1:
12            result += (4 / (np.pi * k)) * np.sin(k * x)
13    return result
14
15 def chebyshev_aprox(x, N):
16     """Aproximacion de Chebyshev de la funcion signo."""
17     # Coeficientes de Chebyshev para sign(x)
18     # a_n = 0 para n par, a_n = 2/(n*pi) * (1 - (-1)^n) para n impar
19     result = np.zeros_like(x)
20     for n in range(1, N+1):
21         if n % 2 == 1:
22             coef = 2 / (n * np.pi) * (1 - (-1)**n)
23             result += coef * np.cos(n * np.arccos(x))
24    return result
25
26 x = np.linspace(-1, 1, 500)
27
28 plt.figure(figsize=(12, 5))
29 plt.subplot(1, 2, 1)
30 plt.plot(x, f(x), 'k-', label='Funcion exacta')
31 plt.plot(x, fourier_aprox(x, 20), 'r-', label='Fourier N=20')
32 plt.xlabel('x')
33 plt.ylabel('f(x)')
34 plt.title('Serie de Fourier')
35 plt.legend()
36 plt.grid(True)
37 plt.ylim(-1.5, 1.5)
38
39 plt.subplot(1, 2, 2)
40 plt.plot(x, f(x), 'k-', label='Funcion exacta')
41 plt.plot(x, chebyshev_aprox(x, 20), 'b-', label='Chebyshev N=20')
42 plt.xlabel('x')
43 plt.ylabel('f(x)')
44 plt.title('Serie de Chebyshev')
45 plt.legend()
46 plt.grid(True)
47 plt.ylim(-1.5, 1.5)
48
49 plt.tight_layout()
50 plt.show()

```

Listing 32: Aproximacion de funciones discontinuas

C9.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def comprimir_svd(imagen, k):
5     """Comprime una imagen usando SVD truncada a rango k."""

```

```

6     U, s, Vt = np.linalg.svd(imagen, full_matrices=False)
7     U_k = U[:, :k]
8     s_k = s[:k]
9     Vt_k = Vt[:, k, :]
10    return U_k @ np.diag(s_k) @ Vt_k
11
12    # Generamos una imagen de prueba (patron de rayas)
13    n = 128
14    imagen = np.zeros((n, n))
15    for i in range(n):
16        imagen[i, :] = np.sin(2 * np.pi * i / 8) + 0.5 * np.sin(4 * np.pi * i / 8)
17
18    rangos = [2, 5, 10, 20, 50]
19    plt.figure(figsize=(15, 10))
20
21    plt.subplot(2, 3, 1)
22    plt.imshow(imagen, cmap='gray')
23    plt.title('Original')
24    plt.axis('off')
25
26    for i, k in enumerate(rangos):
27        img_comp = comprimir_svd(imagen, k)
28        plt.subplot(2, 3, i+2)
29        plt.imshow(img_comp, cmap='gray')
30        plt.title(f'Rango {k}')
31        plt.axis('off')
32        error = np.linalg.norm(imagen - img_comp, 'fro') / np.linalg.norm(imagen, 'fro')
33        print(f"Rango {k}: Error = {error:.4f}")
34
35    plt.tight_layout()
36    plt.show()

```

Listing 33: Compresion de imagenes

C10.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def f(x):
5      return np.exp(x) + 0.5 * np.random.randn(len(x))
6
7  def interpolate_lagrange(x, x_j, y_j):
8      n = len(x_j)
9      result = np.zeros_like(x)
10     for j in range(n):
11         l_j = np.ones_like(x)
12         for k in range(n):
13             if k != j:
14                 l_j *= (x - x_j[k]) / (x_j[j] - x_j[k])
15         result += y_j[j] * l_j
16     return result
17
18  def least_squares(x, y, degree):
19      """Aproximacion por cuadrados minimos con polinomios de grado degree."""
20      A = np.vander(x, degree+1, increasing=True)
21      coefs = np.linalg.lstsq(A, y, rcond=None)[0]
22      return A @ coefs
23
24  def chebyshev_aprox(x, f_vals, N):
25      """Aproximacion de Chebyshev."""

```



```
26     # Usamos nodos de Chebyshev para la cuadratura
27     x_ch = np.cos((2*np.arange(N) + 1) / (2*N) * np.pi)
28     f_ch = f(x_ch)
29
30     # Coeficientes de Chebyshev
31     coefs = np.zeros(N+1)
32     coefs[0] = (1/N) * np.sum(f_ch)
33     for n in range(1, N+1):
34         coefs[n] = (2/N) * np.sum(f_ch * np.cos(n * np.arccos(x_ch)))
35
36     # Reconstruccion
37     result = np.zeros_like(x)
38     for n, c in enumerate(coefs):
39         result += c * np.cos(n * np.arccos(x))
40     return result
41
42 # Generamos datos ruidosos
43 x_data = np.linspace(-1, 1, 30)
44 y_data = f(x_data)
45
46 x_plot = np.linspace(-1, 1, 200)
47 y_exact = np.exp(x_plot)
48
49 # Interpolacion
50 x_int = x_data
51 y_int = y_data
52 p_int = interpolate_lagrange(x_plot, x_int, y_int)
53
54 # Cuadrados minimos
55 p_ls = least_squares(x_data, y_data, 5)
56
57 # Chebyshev
58 p_ch = chebyshev_aprox(x_plot, y_data, 15)
59
60 plt.figure(figsize=(12, 6))
61 plt.subplot(1, 3, 1)
62 plt.plot(x_plot, y_exact, 'k-', label='Exacta')
63 plt.plot(x_plot, p_int, 'r-', label='Interpolacion')
64 plt.plot(x_data, y_data, 'ro', markersize=3)
65 plt.xlabel('x')
66 plt.ylabel('f(x)')
67 plt.title('Interpolacion')
68 plt.legend()
69 plt.grid(True)
70
71 plt.subplot(1, 3, 2)
72 plt.plot(x_plot, y_exact, 'k-', label='Exacta')
73 plt.plot(x_plot, p_ls, 'g-', label='Cuadrados minimos')
74 plt.plot(x_data, y_data, 'go', markersize=3)
75 plt.xlabel('x')
76 plt.ylabel('f(x)')
77 plt.title('Cuadrados minimos')
78 plt.legend()
79 plt.grid(True)
80
81 plt.subplot(1, 3, 3)
82 plt.plot(x_plot, y_exact, 'k-', label='Exacta')
83 plt.plot(x_plot, p_ch, 'b-', label='Chebyshev')
84 plt.plot(x_data, y_data, 'bo', markersize=3)
85 plt.xlabel('x')
86 plt.ylabel('f(x)')
```

```
87 plt.title('Chebyshev')
88 plt.legend()
89 plt.grid(True)
90
91 plt.tight_layout()
92 plt.show()
```

Listing 34: Comparacion de metodos de aproximacion

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por el mundo de la aproximacion numerica. Viste que no era tan complicado. Arrancamos con el problema de aproximar funciones, pasamos por los polinomios ortogonales, las series de Fourier y Chebyshev, la constante de Lebesgue y el fenomeno de Runge, y terminamos con los teoremas de Jackson y Bernstein. Todo esta conectado: la aproximacion de funciones es la base de casi todo el analisis numerico.

Los puntos clave para llevar:

- La aproximacion se puede hacer por interpolacion o por minimos cuadrados.
- Los polinomios ortogonales son la herramienta fundamental para la aproximacion en norma 2.
- Las series de Fourier convergen espectralmente para funciones suaves.
- Los polinomios de Chebyshev son la mejor opcion para aproximar en $[-1, 1]$.
- La constante de Lebesgue mide la estabilidad de la interpolacion.
- El fenomeno de Runge se evita con nodos de Chebyshev.
- Los teoremas de Jackson y Bernstein relacionan suavidad y aproximabilidad.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. La aproximacion de funciones es una herramienta fundamental que vas a usar en practicamente todos los problemas numericos que encuentres. ¡Dale para adelante, que vos podes!

Integracion Numerica

Como calcular areas sin hacer la integral a mano (y que el error no te arruine la vida).

Che, ¿como andas? Bienvenido a este capitulo donde vamos a meterle a uno de los temas mas practicos del calculo numerico: la integracion. Seguro que en algun momento te paso de tener que calcular una integral que no tenia primitiva elemental, o de tener solo datos experimentales y querer estimar el area bajo la curva. La integracion numerica es la herramienta que necesitas para esos casos.

Cuando curse Analisis Numerico, recuerdo que el profesor nos pidio que calculemos numericamente la integral de la funcion de Runge. Yo, todo confiado, use la regla de Simpson con 100 puntos y pense que tenia alta precision. Para mi sorpresa, el error era enorme. Fue ahi cuando aprendi que la eleccion del metodo y los nodos es crucial, y que la cuadratura gaussiana es la posta para funciones suaves.

En este capitulo vamos a cubrir:

- La idea basica: reemplazar la funcion por un polinomio e integrarlo.
- Las formulas de Newton-Cotes: trapecio, Simpson y sus generalizaciones.
- El error de integracion y como estimarlo.
- Las formulas compuestas: partir el intervalo para mejorar la precision.
- La cuadratura gaussiana: elegir los nodos optimos para maximizar la precision.
- La convergencia de los metodos de cuadratura.

La idea es que vos puedas implementar todo esto en Python, entender por que funciona, y saber que metodo usar en cada situacion. Asi que agarra un cafe, ponete comodo, y vamos a darle.

El problema de la integracion numerica

El problema es simple: queremos aproximar:

$$I(f) = \int_a^b f(x)dx$$

donde f es una funcion continua (o al menos integrable). La idea basica de todos los metodos es reemplazar f por un polinomio p que la aproxime, y luego integrar el polinomio:

$$\int_a^b f(x)dx \approx \int_a^b p(x)dx$$

Traduccion Matematica del problema El problema de integracion numerica se lee en criollo: "tenemos una funcion y queremos calcular el area bajo su curva, pero sin usar el teorema fundamental del calculo". En lugar de buscar una primitiva, lo que hacemos es aproximar la funcion por algo mas simple (un polinomio) que si sabemos integrar.

Cuadratura interpolatoria

La forma más natural de construir un método de integración es usar el polinomio interpolante. Dados $n + 1$ nodos $x_0, \dots, x_n$, el polinomio de Lagrange es:

$$p_n(x) = \sum_{k=0}^n f(x_k) \ell_k(x)$$

Entonces:

$$\int_a^b p_n(x) dx = \sum_{k=0}^n \left(\int_a^b \ell_k(x) dx \right) f(x_k) = \sum_{k=0}^n w_k f(x_k)$$

donde $w_k = \int_a^b \ell_k(x) dx$ son los **pesos** de la cuadratura.

Traducción Matemática de la cuadratura interpolatoria La cuadratura interpolatoria se lee en criollo: "para integrar una función, la reemplazamos por un polinomio que pasa por algunos puntos, integramos ese polinomio (que es fácil), y el resultado es una combinación lineal de los valores de la función en esos puntos". Los pesos w_k son las integrales de los polinomios de Lagrange, y solo dependen de los nodos, no de la función.

Formulas de Newton-Cotes

Las fórmulas de Newton-Cotes son aquellas donde los nodos están **equiespaciados**. Son las más simples de implementar y las que se usan en los cursos elementales.

Regla del Trapecio (n=1)

Con dos nodos $x_0 = a$, $x_1 = b$, el polinomio interpolante es la recta que une los puntos. La fórmula es:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} (f(a) + f(b))$$

Traducción Matemática del Trapecio La regla del trapecio se lee en criollo: ".el área bajo la curva se aproxima por el área de un trapecio que tiene como bases las alturas de la función en los extremos y como altura la longitud del intervalo". Es la fórmula más simple, pero también la menos precisa.

Regla de Simpson (n=2)

Con tres nodos $x_0 = a$, $x_1 = (a+b)/2$, $x_2 = b$, el polinomio interpolante es una parábola. La fórmula es:

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right)$$

Traducción Matemática de Simpson La regla de Simpson se lee en criollo: ".el área bajo la curva se aproxima por el área bajo una parábola que pasa por los tres puntos: los extremos y el punto medio". Es mucho más precisa que el trapecio para funciones suaves.

Error de las formulas de Newton-Cotes

El error de la regla del trapecio es:

$$E_T = -\frac{(b-a)^3}{12} f''(\xi)$$

El error de la regla de Simpson es:

$$E_S = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$$

Traduccion Matematica del error El error se lee en criollo: "el error que cometemos al usar la formula es proporcional a una derivada de la funcion evaluada en algun punto del intervalo, multiplicada por una potencia de la longitud del intervalo". Esto muestra que la regla de Simpson es mucho mas precisa que el trapecio para funciones suaves (el error depende de $f^{(4)}$ en lugar de f'' , y el factor es mucho mas chico).

Grafico: comparacion trapecio vs Simpson

Vamos a visualizar como se comportan el trapecio y Simpson para una funcion.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return np.sin(x) + 0.5 * np.cos(2*x)
6
7 # Intervalo
8 a, b = 0, np.pi
9
10 # Trapecio
11 T = (b-a)/2 * (f(a) + f(b))
12
13 # Simpson
14 c = (a+b)/2
15 S = (b-a)/6 * (f(a) + 4*f(c) + f(b))
16
17 # Integral exacta (calculada analiticamente)
18 # sin(x) dx = -cos(x), 0.5*cos(2x) dx = 0.25*sin(2x)
19 exacta = -np.cos(b) + np.cos(a) + 0.25*(np.sin(2*b) - np.sin(2*a))
20
21 print(f"Trapezio: {T:.6f}")
22 print(f"Simpson: {S:.6f}")
23 print(f"Exacta: {exacta:.6f}")
24 print(f"Error trapecio: {abs(T - exacta):.6f}")
25 print(f"Error Simpson: {abs(S - exacta):.6f}")
26
27 # Graficamos la funcion y las aproximaciones
28 x = np.linspace(a, b, 100)
29 y = f(x)
30
31 plt.figure(figsize=(10, 6))
32 plt.plot(x, y, 'k-', label='f(x)', linewidth=2)
33 plt.fill_between(x, y, alpha=0.2)
34
35 # Trapecio
36 plt.plot([a, b], [f(a), f(b)], 'r--', label='Trapezio', linewidth=2)
37

```

```

38 # Simpson (parabola)
39 x_simpson = np.linspace(a, b, 30)
40 y_simpson = f(a) * (x_simpson - b)*(x_simpson - c) / ((a-b)*(a-c)) + \
41             f(c) * (x_simpson - a)*(x_simpson - b) / ((c-a)*(c-b)) + \
42             f(b) * (x_simpson - a)*(x_simpson - c) / ((b-a)*(b-c))
43 plt.plot(x_simpson, y_simpson, 'b--', label='Simpson', linewidth=2)
44
45 plt.xlabel('x')
46 plt.ylabel('f(x)')
47 plt.title('Comparacion: Trapecio vs Simpson')
48 plt.legend()
49 plt.grid(True)
50 plt.show()

```

Listing 35: Comparación de trapecio y Simpson

En el gráfico, ves como la parábola de Simpson se ajusta mucho mejor a la función que la recta del trapecio.

Formulas de Newton-Cotes de orden superior

Podemos generalizar a n puntos equiespaciados. Las formulas se llaman **reglas de Newton-Cotes cerradas** (incluyen los extremos) o **abiertas** (no los incluyen).

Regla de Boole (n=4)

Con 5 puntos, la formula es:

$$\int_a^b f(x)dx \approx \frac{2h}{45}(7f_0 + 32f_1 + 12f_2 + 32f_3 + 7f_4)$$

donde $h = (b - a)/4$.

Problemas con las formulas de orden superior

Las formulas de Newton-Cotes de orden alto pueden tener **pesos negativos**, lo que las hace inestables numericamente. Por eso, en la practica se usan las formulas compuestas en lugar de aumentar el grado.

Formulas compuestas

La idea es simple: en lugar de usar un polinomio de grado alto en todo el intervalo, partimos el intervalo en subintervalos pequenos y aplicamos una formula de bajo orden en cada uno.

Trapecio compuesto

Si partimos $[a, b]$ en n subintervalos de longitud $h = (b - a)/n$, y aplicamos el trapecio en cada uno:

$$\int_a^b f(x)dx \approx T_n = h \left(\frac{f(a)}{2} + \sum_{i=1}^{n-1} f(a + ih) + \frac{f(b)}{2} \right)$$

Traduccion Matematica del Trapecio compuesto El trapecio compuesto se lee en criollo: "partimos el intervalo en pedacitos, en cada pedacito aproximamos el area por un trapecio, y sumamos todas las areas". Esto es como sumar areas de rectangulos, pero con trapecios que se ajustan mejor.

Simpson compuesto

Para Simpson compuesto, necesitamos que n sea par:

$$\int_a^b f(x)dx \approx S_n = \frac{h}{3} \left(f(a) + 4 \sum_{i \text{ impar}} f(a + ih) + 2 \sum_{i \text{ par}} f(a + ih) + f(b) \right)$$

donde $h = (b - a)/n$.

Error de las formulas compuestas

El error del trapecio compuesto es:

$$E_T = -\frac{(b-a)h^2}{12} f''(\xi)$$

El error de Simpson compuesto es:

$$E_S = -\frac{(b-a)h^4}{180} f^{(4)}(\xi)$$

Traduccion Matematica del error compuesto El error se lee en criollo: "si usamos n subintervalos, el error es proporcional a h^2 (para trapecio) o h^4 (para Simpson), donde $h = (b - a)/n$ ". Esto significa que si duplicamos el numero de puntos, el error se reduce a la cuarta parte (trapecio) o a la decimosexta parte (Simpson).

Grafico: convergencia de las formulas compuestas

Vamos a visualizar como converge el error cuando aumentamos el numero de puntos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return np.exp(x)
6
7 a, b = 0, 1
8 exacta = np.exp(1) - 1
9
10 n_values = np.array([2, 4, 8, 16, 32, 64, 128])
11 errores_t = []
12 errores_s = []
13
14 for n in n_values:
15     h = (b - a) / n
16     x = np.linspace(a, b, n+1)
17     y = f(x)
18
19     # Trapecio compuesto
20     T = h * (0.5*y[0] + np.sum(y[1:-1]) + 0.5*y[-1])
21     errores_t.append(abs(T - exacta))

```

```

22
23     # Simpson compuesto (n debe ser par)
24     if n % 2 == 0:
25         S = h/3 * (y[0] + y[-1] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-2:2]))
26         errores_s.append(abs(S - exacta))
27     else:
28         errores_s.append(np.nan)
29
30 plt.figure(figsize=(10, 6))
31 plt.loglog(n_values, errores_t, 'o-', label='Trapezio')
32 plt.loglog(n_values[::2], errores_s[::2], 's-', label='Simpson')
33 plt.xlabel('n (numero de subintervalos)')
34 plt.ylabel('Error absoluto')
35 plt.title('Convergencia de las formulas compuestas para e^x')
36 plt.grid(True)
37 plt.legend()
38 plt.show()

```

Listing 36: Convergencia de trapezio y Simpson compuestos

En el gráfico, ves como el error de ambos métodos decae al aumentar n . Pero Simpson decae mucho más rápido (pendiente -4 en escala log-log) que trapezio (pendiente -2).

Cuadratura Gaussiana

La cuadratura gaussiana es la técnica más poderosa para integración numérica. La idea es elegir tanto los nodos como los pesos para maximizar la precisión.

La idea fundamental

Con n puntos, una fórmula de cuadratura tiene $2n$ parámetros (los n nodos y los n pesos). La cuadratura gaussiana elige estos parámetros para que la fórmula sea exacta para polinomios de grado hasta $2n - 1$.

Traducción Matemática de la cuadratura gaussiana La cuadratura gaussiana se lee en criollo: “en lugar de fijar los nodos (como en Newton-Cotes), los elegimos de forma inteligente para maximizar la precisión”. Con n puntos, podemos integrar exactamente polinomios de grado $2n - 1$. Esto es el doble de lo que logra Newton-Cotes con n puntos.

Nodos de Legendre

Los nodos de la cuadratura gaussiana en $[-1, 1]$ son las **raíces del polinomio de Legendre** $P_n(x)$. Por ejemplo:

- $n=1$: $x_0 = 0$
- $n=2$: $x_0 = -1/\sqrt{3}, x_1 = 1/\sqrt{3}$
- $n=3$: $x_0 = -\sqrt{3/5}, x_1 = 0, x_2 = \sqrt{3/5}$

Pesos de Legendre

Los pesos correspondientes son:

- $n=1$: $w_0 = 2$

- n=2: $w_0 = w_1 = 1$
- n=3: $w_0 = w_2 = 5/9, w_1 = 8/9$

Formula general

Para una funcion f en $[-1, 1]$:

$$\int_{-1}^1 f(x)dx \approx \sum_{i=1}^n w_i f(x_i)$$

Para un intervalo $[a, b]$, se hace el cambio de variable:

$$x = \frac{b-a}{2}t + \frac{a+b}{2}$$

y entonces:

$$\int_a^b f(x)dx \approx \frac{b-a}{2} \sum_{i=1}^n w_i f\left(\frac{b-a}{2}t_i + \frac{a+b}{2}\right)$$

Error de la cuadratura gaussiana

El error de la cuadratura gaussiana con n puntos es:

$$E_G = \frac{f^{(2n)}(\xi)}{(2n)!} \int_{-1}^1 [P_n(x)]^2 dx$$

donde $\xi \in (-1, 1)$.

Traduccion Matematica del error gaussiano El error se lee en criollo: "el error de la cuadratura gaussiana depende de la derivada $2n$ -esima de la funcion, lo que la hace extremadamente precisa para funciones suaves". Para funciones analiticas, el error decae exponencialmente con n .

Grafico: comparacion de metodos

Vamos a comparar la cuadratura gaussiana con trapecio y Simpson.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.special import roots_legendre
4
5 def gauss_legendre(f, a, b, n):
6     """Cuadratura gaussiana de Legendre con n puntos en [a, b]."""
7     x, w = roots_legendre(n)
8     # Cambio de variable al intervalo [a, b]
9     t = (b - a) / 2 * x + (a + b) / 2
10    return (b - a) / 2 * np.sum(w * f(t))
11
12 def trapecio_compuesto(f, a, b, n):
13     h = (b - a) / n
14     x = np.linspace(a, b, n+1)
15     y = f(x)
16     return h * (0.5*y[0] + np.sum(y[1:-1]) + 0.5*y[-1])
17
18 def simpson_compuesto(f, a, b, n):

```

```

19     if n % 2 == 1:
20         n += 1
21     h = (b - a) / n
22     x = np.linspace(a, b, n+1)
23     y = f(x)
24     return h/3 * (y[0] + y[-1] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-2:2]))
25
26 def f(x):
27     return np.exp(x)
28
29 a, b = 0, 1
30 exacta = np.exp(1) - 1
31
32 # Probamos con diferentes numeros de puntos
33 n_values = np.arange(2, 15)
34 errores_t = []
35 errores_s = []
36 errores_g = []
37
38 for n in n_values:
39     errores_t.append(abs(trapezio_compuesto(f, a, b, n) - exacta))
40     errores_s.append(abs(simpson_compuesto(f, a, b, n) - exacta))
41     errores_g.append(abs(gauss_legendre(f, a, b, n) - exacta))
42
43 plt.figure(figsize=(10, 6))
44 plt.semilogy(n_values, errores_t, 'o-', label='Trapezio compuesto')
45 plt.semilogy(n_values, errores_s, 's-', label='Simpson compuesto')
46 plt.semilogy(n_values, errores_g, '^-', label='Gauss-Legendre')
47 plt.xlabel('Numero de puntos')
48 plt.ylabel('Error absoluto')
49 plt.title('Comparacion de metodos de integracion para e^x')
50 plt.grid(True)
51 plt.legend()
52 plt.show()

```

Listing 37: Comparación de métodos de integración

En el gráfico, ves que la cuadratura gaussiana es mucho más precisa que los otros métodos, especialmente para un número pequeño de puntos. Para 10 puntos, Gauss-Legendre alcanza una precisión que trapecio y Simpson no logran ni con 100 puntos.

Convergencia de los métodos de cuadratura

Una pregunta fundamental es: ¿cuando los métodos de cuadratura convergen a la integral exacta cuando el número de puntos tiende a infinito?

Teorema de convergencia

Si los pesos de la cuadratura son **positivos** y la suma de los pesos converge a la integral de 1, entonces la cuadratura converge para toda función continua.

Traducción Matemática de la convergencia El teorema se lee en criollo: "si los pesos son todos positivos, el método es estable y converge para cualquier función continua". La cuadratura gaussiana tiene pesos positivos, por lo que es convergente. Las fórmulas de Newton-Cotes de orden alto pueden tener pesos negativos y no converger.

Convergencia de la cuadratura gaussiana

La cuadratura gaussiana converge para toda función continua en $[-1, 1]$. Además, para funciones suaves, la convergencia es **exponencial**.

Ejercicios

Parte Teórica (10 ejercicios)

- T1. Regla del trapecio:** Deriva la regla del trapecio a partir del polinomio de Lagrange de grado 1.
- T2. Regla de Simpson:** Deriva la regla de Simpson a partir del polinomio de Lagrange de grado 2.
- T3. Error del trapecio:** Demuestra que el error de la regla del trapecio es $E_T = -\frac{(b-a)^3}{12} f''(\xi)$.
- T4. Error de Simpson:** Demuestra que el error de la regla de Simpson es $E_S = -\frac{(b-a)^5}{2880} f^{(4)}(\xi)$.
- T5. Trapecio compuesto:** Deriva la fórmula del trapecio compuesto y su error.
- T6. Simpson compuesto:** Deriva la fórmula de Simpson compuesto y su error.
- T7. Cuadratura gaussiana:** Explica por qué la cuadratura gaussiana con n puntos es exacta para polinomios de grado $2n - 1$.
- T8. Nodos de Legendre:** Demuestra que los nodos de la cuadratura gaussiana son las raíces de los polinomios de Legendre.
- T9. Convergencia:** Demuestra que si los pesos de una cuadratura son positivos y suman $b - a$, entonces la cuadratura converge para toda función continua.
- T10. Comparación de métodos:** Compara el costo computacional y la precisión de la regla de Simpson compuesta y la cuadratura gaussiana para una función suave.

Parte Computacional (10 ejercicios)

- C1. Trapecio:** Implementa la regla del trapecio simple. Prueba con funciones conocidas y compara con el valor exacto.
- C2. Simpson:** Implementa la regla de Simpson simple. Prueba con funciones conocidas.
- C3. Trapecio compuesto:** Implementa el trapecio compuesto. Grafica el error en función de n para $f(x) = e^x$.
- C4. Simpson compuesto:** Implementa Simpson compuesto. Grafica el error en función de n para $f(x) = e^x$.
- C5. Cuadratura gaussiana:** Implementa la cuadratura gaussiana usando los nodos y pesos de Legendre (puedes usar `scipy.special.roots_legendre`). Prueba con $f(x) = e^x$.
- C6. Comparación de métodos:** Compara trapecio, Simpson y Gauss-Legendre para $f(x) = \sin(x)$ en $[0, \pi]$. Grafica los errores en función de n .

- C7. Integrales con singularidades:** Calcula $\int_0^1 \frac{1}{\sqrt{x}} dx$ numericamente. ¿Cual metodo funciona mejor? ¿Por que?
- C8. Convergencia de la cuadratura gaussiana:** Para $f(x) = e^x$, grafica el error de la cuadratura gaussiana en funcion de n en escala semilogaritmica. ¿Que observas?
- C9. Integración adaptativa:** Implementa un metodo de integración adaptativa que subdivida el intervalo donde el error estimado es grande. Prueba con funciones que tienen variaciones rapidas.
- C10. Aplicación: calculo de pi:** Usa la integración numerica para calcular π a partir de $\int_0^1 \frac{4}{1+x^2} dx$. Compara la precision de diferentes metodos.

Soluciones

Soluciones Teóricas

T1. El polinomio de Lagrange de grado 1 es $p(x) = \frac{x-b}{a-b}f(a) + \frac{x-a}{b-a}f(b)$. Integrando:

$$\int_a^b p(x)dx = f(a) \int_a^b \frac{x-b}{a-b} dx + f(b) \int_a^b \frac{x-a}{b-a} dx = \frac{b-a}{2}(f(a) + f(b))$$

T2. Similar al T1, pero con el polinomio de grado 2. Los pesos resultan $w_0 = (b-a)/6$, $w_1 = 4(b-a)/6$, $w_2 = (b-a)/6$.

T3. Usando el error de interpolación: $f(x) - p_1(x) = \frac{f''(\xi)}{2}(x-a)(x-b)$. Integrando y usando el teorema del valor medio para integrales.

T4. Similar al T3, pero con el error de interpolación de grado 2.

T5. Sumando la regla del trapecio en cada subintervalo. El error es la suma de los errores locales, que da $-\frac{(b-a)h^2}{12}f''(\xi)$.

T6. Similar al T5, pero con la regla de Simpson en cada par de subintervalos.

T7. La cuadratura gaussiana tiene $2n$ parametros libres. Para que sea exacta para polinomios de grado $2n-1$, se resuelve un sistema de $2n$ ecuaciones. Los nodos resultan ser las raices de P_n .

T8. Los polinomios de Legendre son ortogonales en $[-1, 1]$. Los nodos de la cuadratura gaussiana son los puntos donde el error de interpolación se anula, lo que coincide con las raices de P_n .

T9. Por el teorema de Weierstrass, toda función continua puede ser aproximada uniformemente por polinomios. Como la cuadratura es exacta para polinomios y los pesos son positivos, el error tiende a cero.

T10. Simpson compuesto cuesta $O(n)$ operaciones y tiene error $O(n^{-4})$. Gauss-Legendre cuesta $O(n^2)$ (para calcular los nodos) y tiene error exponencial para funciones suaves. Para alta precision, Gauss-Legendre es mejor.

Soluciones Computacionales

C1. Ver el código de la Sección .

C2. Ver el código de la Sección .

C3.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
```

```

4 def trapecio_compuesto(f, a, b, n):
5     h = (b - a) / n
6     x = np.linspace(a, b, n+1)
7     y = f(x)
8     return h * (0.5*y[0] + np.sum(y[1:-1]) + 0.5*y[-1])
9
10 def f(x):
11     return np.exp(x)
12
13 a, b = 0, 1
14 exacta = np.exp(1) - 1
15
16 n_values = np.array([2, 4, 8, 16, 32, 64, 128, 256])
17 errores = []
18
19 for n in n_values:
20     aprox = trapecio_compuesto(f, a, b, n)
21     errores.append(abs(aprox - exacta))
22
23 plt.figure(figsize=(10, 6))
24 plt.loglog(n_values, errores, 'o-')
25 plt.loglog(n_values, 1/n_values**2, 'r--', label='O(n^{-2})')
26 plt.xlabel('n (numero de subintervalos)')
27 plt.ylabel('Error absoluto')
28 plt.title('Convergencia del trapecio compuesto')
29 plt.legend()
30 plt.grid(True)
31 plt.show()

```

Listing 38: Trapecio compuesto

C4.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def simpson_compuesto(f, a, b, n):
5     if n % 2 == 1:
6         n += 1
7     h = (b - a) / n
8     x = np.linspace(a, b, n+1)
9     y = f(x)
10    return h/3 * (y[0] + y[-1] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-2:2]))
11
12 def f(x):
13     return np.exp(x)
14
15 a, b = 0, 1
16 exacta = np.exp(1) - 1
17
18 n_values = np.array([2, 4, 8, 16, 32, 64, 128])
19 errores = []
20
21 for n in n_values:
22     aprox = simpson_compuesto(f, a, b, n)
23     errores.append(abs(aprox - exacta))
24
25 plt.figure(figsize=(10, 6))
26 plt.loglog(n_values, errores, 'o-')
27 plt.loglog(n_values, 1/n_values**4, 'r--', label='O(n^{-4})')
28 plt.xlabel('n (numero de subintervalos)')
29 plt.ylabel('Error absoluto')

```

```

30 plt.title('Convergencia de Simpson compuesto')
31 plt.legend()
32 plt.grid(True)
33 plt.show()

```

Listing 39: Simpson compuesto

C5.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.special import roots_legendre
4
5 def gauss_legendre(f, a, b, n):
6     x, w = roots_legendre(n)
7     t = (b - a) / 2 * x + (a + b) / 2
8     return (b - a) / 2 * np.sum(w * f(t))
9
10 def f(x):
11     return np.exp(x)
12
13 a, b = 0, 1
14 exacta = np.exp(1) - 1
15
16 n_values = np.arange(2, 15)
17 errores = []
18
19 for n in n_values:
20     aprox = gauss_legendre(f, a, b, n)
21     errores.append(abs(aprox - exacta))
22
23 plt.figure(figsize=(10, 6))
24 plt.semilogy(n_values, errores, 'o-')
25 plt.xlabel('n (numero de puntos)')
26 plt.ylabel('Error absoluto')
27 plt.title('Convergencia de la cuadratura gaussiana')
28 plt.grid(True)
29 plt.show()

```

Listing 40: Cuadratura gaussiana

C6.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.special import roots_legendre
4
5 def trapecio_compuesto(f, a, b, n):
6     h = (b - a) / n
7     x = np.linspace(a, b, n+1)
8     y = f(x)
9     return h * (0.5*y[0] + np.sum(y[1:-1]) + 0.5*y[-1])
10
11 def simpson_compuesto(f, a, b, n):
12     if n % 2 == 1:
13         n += 1
14     h = (b - a) / n
15     x = np.linspace(a, b, n+1)
16     y = f(x)
17     return h/3 * (y[0] + y[-1] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-2:2]))
18
19 def gauss_legendre(f, a, b, n):

```

```

20     x, w = roots_legendre(n)
21     t = (b - a) / 2 * x + (a + b) / 2
22     return (b - a) / 2 * np.sum(w * f(t))
23
24 def f(x):
25     return np.sin(x)
26
27 a, b = 0, np.pi
28 exacta = 2 #  $\int_0^\pi \sin(x) dx = 2$ 
29
30 n_values = np.arange(2, 15)
31 errores_t = []
32 errores_s = []
33 errores_g = []
34
35 for n in n_values:
36     errores_t.append(abs(trapecio_compuesto(f, a, b, n) - exacta))
37     errores_s.append(abs(simpson_compuesto(f, a, b, n) - exacta))
38     errores_g.append(abs(gauss_legendre(f, a, b, n) - exacta))
39
40 plt.figure(figsize=(10, 6))
41 plt.semilogy(n_values, errores_t, 'o-', label='Trapezio')
42 plt.semilogy(n_values, errores_s, 's-', label='Simpson')
43 plt.semilogy(n_values, errores_g, '^-', label='Gauss-Legendre')
44 plt.xlabel('Numero de puntos')
45 plt.ylabel('Error absoluto')
46 plt.title('Comparacion de metodos para sin(x) en [0,  $\pi$ ]')
47 plt.legend()
48 plt.grid(True)
49 plt.show()

```

Listing 41: Comparacion de metodos

C7.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.special import roots_legendre
4
5 def f(x):
6     return 1 / np.sqrt(x)
7
8 def trapezio_compuesto(f, a, b, n):
9     h = (b - a) / n
10    x = np.linspace(a, b, n+1)
11    y = f(x)
12    # La singularidad en x=0 da problemas, usamos un punto muy cercano
13    y[0] = f(1e-12)
14    return h * (0.5*y[0] + np.sum(y[1:-1]) + 0.5*y[-1])
15
16 def gauss_legendre(f, a, b, n):
17     x, w = roots_legendre(n)
18     t = (b - a) / 2 * x + (a + b) / 2
19     # Evitamos la singularidad en t=0
20     epsilon = 1e-12
21     t = np.where(np.abs(t) < epsilon, epsilon, t)
22     return (b - a) / 2 * np.sum(w * f(t))
23
24 a, b = 0, 1
25 exacta = 2 #  $\int_0^1 1/\sqrt{x} dx = 2$ 
26
27 n_values = np.arange(2, 15)

```

```

28 errores_t = []
29 errores_g = []
30
31 for n in n_values:
32     errores_t.append(abs(trapecio_compuesto(f, a, b, n) - exacta))
33     errores_g.append(abs(gauss_legendre(f, a, b, n) - exacta))
34
35 plt.figure(figsize=(10, 6))
36 plt.semilogy(n_values, errores_t, 'o-', label='Trapezio')
37 plt.semilogy(n_values, errores_g, 's-', label='Gauss-Legendre')
38 plt.xlabel('Numero de puntos')
39 plt.ylabel('Error absoluto')
40 plt.title('Integral con singularidad:  $\int_0^1 \frac{1}{x} dx$ ')
41 plt.legend()
42 plt.grid(True)
43 plt.show()

```

Listing 42: Integral con singularidad

C8.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.special import roots_legendre
4
5 def gauss_legendre(f, a, b, n):
6     x, w = roots_legendre(n)
7     t = (b - a) / 2 * x + (a + b) / 2
8     return (b - a) / 2 * np.sum(w * f(t))
9
10 def f(x):
11     return np.exp(x)
12
13 a, b = 0, 1
14 exacta = np.exp(1) - 1
15
16 n_values = np.arange(2, 20)
17 errores = []
18
19 for n in n_values:
20     aprox = gauss_legendre(f, a, b, n)
21     errores.append(abs(aprox - exacta))
22
23 plt.figure(figsize=(10, 6))
24 plt.semilogy(n_values, errores, 'o-')
25 plt.xlabel('n (numero de puntos)')
26 plt.ylabel('Error absoluto')
27 plt.title('Convergencia exponencial de la cuadratura gaussiana')
28 plt.grid(True)
29 plt.show()

```

Listing 43: Convergencia exponencial de Gauss-Legendre

C9.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def adaptativa(f, a, b, tol, max_depth=10):
5     """Integración adaptativa usando Simpson."""
6     def _adaptativa(f, a, b, tol, depth):
7         if depth > max_depth:

```



```

8         return simpson(f, a, b)
9
10        c = (a + b) / 2
11        S_ab = simpson(f, a, b)
12        S_ac = simpson(f, a, c)
13        S_cb = simpson(f, c, b)
14        error = np.abs(S_ab - S_ac - S_cb)
15
16        if error < 15 * tol:
17            return S_ab + (S_ab - S_ac - S_cb) / 15
18
19        return _adaptativa(f, a, c, tol/2, depth+1) + _adaptativa(f, c, b, tol
20        /2, depth+1)
21
22    def simpson(f, a, b):
23        return (b-a)/6 * (f(a) + 4*f((a+b)/2) + f(b))
24
25    return _adaptativa(f, a, b, tol, 0)
26
27def f(x):
28    return np.sin(10*x) + np.exp(x)
29
30a, b = 0, 1
31exacta = (1 - np.cos(10))/10 + np.exp(1) - 1
32
33aprox = adaptativa(f, a, b, 1e-6)
34print(f"Aproximacion adaptativa: {aprox:.10f}")
35print(f"Exacta: {exacta:.10f}")
36print(f"Error: {abs(aprox - exacta):.10f}")

```

Listing 44: Integracion adaptativa

C10.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.special import roots_legendre
4
5 def f(x):
6     return 4 / (1 + x**2)
7
8 def trapecio_compuesto(f, a, b, n):
9     h = (b - a) / n
10    x = np.linspace(a, b, n+1)
11    y = f(x)
12    return h * (0.5*y[0] + np.sum(y[1:-1]) + 0.5*y[-1])
13
14 def simpson_compuesto(f, a, b, n):
15     if n % 2 == 1:
16         n += 1
17     h = (b - a) / n
18     x = np.linspace(a, b, n+1)
19     y = f(x)
20     return h/3 * (y[0] + y[-1] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-2:2]))
21
22 def gauss_legendre(f, a, b, n):
23     x, w = roots_legendre(n)
24     t = (b - a) / 2 * x + (a + b) / 2
25     return (b - a) / 2 * np.sum(w * f(t))
26
27a, b = 0, 1
28pi_exacto = np.pi

```

```
29
30 n_values = np.arange(2, 15)
31 errores_t = []
32 errores_s = []
33 errores_g = []
34
35 for n in n_values:
36     errores_t.append(abs(trapezio_compuesto(f, a, b, n) - pi_exacto))
37     errores_s.append(abs(simpson_compuesto(f, a, b, n) - pi_exacto))
38     errores_g.append(abs(gauss_legendre(f, a, b, n) - pi_exacto))
39
40 plt.figure(figsize=(10, 6))
41 plt.semilogy(n_values, errores_t, 'o-', label='Trapezio')
42 plt.semilogy(n_values, errores_s, 's-', label='Simpson')
43 plt.semilogy(n_values, errores_g, '^-', label='Gauss-Legendre')
44 plt.xlabel('Numero de puntos')
45 plt.ylabel('Error absoluto')
46 plt.title('Calculo de pi con integracion numerica')
47 plt.legend()
48 plt.grid(True)
49 plt.show()
```

Listing 45: Calculo de pi

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por el mundo de la integración numérica. Viste que no era tan complicado. Arrancamos con la idea de reemplazar la función por un polinomio, pasamos por las fórmulas de Newton-Cotes, las fórmulas compuestas, y terminamos con la cuadratura gaussiana. Todo está conectado: la integración numérica es una herramienta fundamental en el cálculo científico.

Los puntos clave para llevar:

- La cuadratura interpolatoria reemplaza la función por un polinomio e integra.
- Las fórmulas de Newton-Cotes usan nodos equiespaciados.
- Las fórmulas compuestas mejoran la precisión partiendo el intervalo.
- La cuadratura gaussiana es la más precisa para funciones suaves.
- La elección del método depende de la suavidad de la función y la precisión requerida.

Ahora solo falta practicar. Los ejercicios están ahí para que te ensucies las manos. No te frustres si al principio no te salen; a mí me pasó lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. La integración numérica es una herramienta que vas a usar toda tu vida como científico o ingeniero. ¡Dale para adelante, que vos podés!

Transformadas Discretas

De la interpolacion a la compresion de senales, con Python y la FFT.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a meterle a uno de los temas mas revolucionarios de la matematica aplicada: las transformadas discretas. Si llegaste hasta aca, seguro ya escuchaste hablar de la FFT, el JPEG, el MP3, o el procesamiento de señales. Todo eso esta basado en la Transformada Discreta de Fourier (DFT). La DFT es como tener un oido superpoderoso que te permite escuchar las frecuencias individuales que componen una señal, incluso si la señal es digital.

Cuando curse Fisica2, recuerdo que el profesor nos mostro como se puede descomponer una grabacion de audio en sus frecuencias usando la FFT. Ver como el grafico de frecuencias cambiaba en tiempo real mientras sonaba una cancion fue como ver magia. Despues entendi que detras de esa magia hay una historia hermosa que conecta la interpolacion polinomial, las raices de la unidad, y un algoritmo increíblemente ingenioso: la Transformada Rapida de Fourier (FFT).

En este capitulo vamos a cubrir:

- La DFT como interpolacion en las raices de la unidad.
- La matriz de Fourier y su propiedad de unitariedad.
- La transformada inversa (IDFT).
- La FFT: el algoritmo que cambio el mundo (Cooley-Tukey).
- El teorema de convolucion y sus aplicaciones.
- Filtrado digital de señales.
- Multiplicacion rapida de polinomios.
- El principio de incertidumbre de Donoho-Stark.

La idea es que vos puedas implementar todo esto en Python, entender por que funciona, y ver como la DFT es la herramienta fundamental para el procesamiento digital de señales. Asi que agarra un cafe, ponete comodo, y vamos a darle.

La Transformada Discreta de Fourier (DFT)

La DFT es una transformacion lineal que toma una secuencia de N numeros complejos $x_0, x_1, \dots, x_{N-1}$ y produce otra secuencia de N numeros complejos $X_0, X_1, \dots, X_{N-1}$ definida por:

$$X_k = \sum_{n=0}^{N-1} x_n \omega_N^{-kn}, \quad k = 0, 1, \dots, N-1$$

donde $\omega_N = e^{2\pi i/N}$ es una raiz N -esima primitiva de la unidad.

Traducción Matemática de la DFT La fórmula de la DFT se lee en criollo: "para cada frecuencia k , multiplicamos la señal x_n por una onda compleja $e^{-2\pi i k n / N}$ y sumamos todo". Pensa en esto como si estuvieras buscando cuanto de cada frecuencia hay en la señal. La frecuencia k corresponde a k ciclos completos en N muestras. Si la señal tiene esa frecuencia, el producto con la onda se acumula y da un número grande; si no la tiene, las oscilaciones se cancelan.

La DFT como interpolación polinomial

La DFT tiene una interpretación geométrica muy linda: es evaluar un polinomio en las raíces de la unidad. Definimos el polinomio:

$$p(z) = x_0 + x_1 z + x_2 z^2 + \cdots + x_{N-1} z^{N-1}$$

Entonces, evaluar p en la raíz ω_N^{-k} da exactamente:

$$p(\omega_N^{-k}) = \sum_{n=0}^{N-1} x_n (\omega_N^{-k})^n = \sum_{n=0}^{N-1} x_n \omega_N^{-kn} = X_k$$

Traducción Matemática de la interpretación polinomial La interpretación polinomial se lee en criollo: "los coeficientes de Fourier de la señal son simplemente los valores del polinomio que tiene a la señal como coeficientes, evaluado en las raíces de la unidad". Esto conecta la DFT con la interpolación polinomial que vimos en el capítulo anterior.

La matriz de Fourier

La DFT se puede escribir como una multiplicación matricial:

$$\mathbf{X} = F_N \mathbf{x}$$

donde F_N es la matriz de Fourier de $N \times N$:

$$F_N = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_N^{-1} & \omega_N^{-2} & \cdots & \omega_N^{-(N-1)} \\ 1 & \omega_N^{-2} & \omega_N^{-4} & \cdots & \omega_N^{-2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{-(N-1)} & \omega_N^{-2(N-1)} & \cdots & \omega_N^{-(N-1)(N-1)} \end{pmatrix}$$

Traducción Matemática de la matriz de Fourier La matriz de Fourier se lee en criollo: "es una matriz especial donde cada fila es una onda compleja de diferente frecuencia". La propiedad mágica es que sus columnas (y filas) son ortogonales. Esto hace que la transformada sea invertible y que la matriz sea (casi) unitaria.

Propiedad de unitariedad

La matriz de Fourier satisface:

$$F_N^* F_N = N I_N$$

donde F_N^* es la transpuesta conjugada (hermitica). En criollo: "las columnas de F_N son ortogonales y cada una tiene norma $\sqrt{N}$ ".

Esto implica que:

$$F_N^{-1} = \frac{1}{N} F_N^*$$

La transformada inversa (IDFT)

La transformada inversa es:

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \omega_N^{kn}$$

Traducción Matemática de la IDFT La IDFT se lee en criollo: "la señal original se puede reconstruir sumando todas las ondas de frecuencia k multiplicadas por sus coeficientes X_k ". Es como armar un acorde sumando las notas que lo componen.

La Transformada Rapida de Fourier (FFT)

La FFT es un algoritmo que calcula la DFT en $O(N \log N)$ operaciones, en lugar de las $O(N^2)$ que costaría la multiplicación matricial directa. Fue descubierta por Cooley y Tukey en 1965, pero Gauss ya la conocía en 1805.

La idea de Cooley-Tukey

La idea es simple pero brillante: dividir el problema en dos partes (par e impar) y aplicar recursivamente.

Supongamos que $N = 2M$ es par. Separamos la señal en índices pares e impares:

$$X_k = \sum_{n=0}^{N-1} x_n \omega_N^{-kn}$$

Separando en pares ($n = 2m$) e impares ($n = 2m + 1$):

$$X_k = \sum_{m=0}^{M-1} x_{2m} \omega_N^{-2km} + \omega_N^{-k} \sum_{m=0}^{M-1} x_{2m+1} \omega_N^{-2km}$$

Pero $\omega_N^2 = \omega_M$, así que:

$$X_k = \sum_{m=0}^{M-1} x_{2m} \omega_M^{-km} + \omega_N^{-k} \sum_{m=0}^{M-1} x_{2m+1} \omega_M^{-km}$$

Es decir:

$$X_k = E_k + \omega_N^{-k} O_k$$

donde E_k es la DFT de tamaño M de la parte par, y O_k es la DFT de tamaño M de la parte impar.

Traducción Matemática de la FFT La FFT se lee en criollo: "para calcular la transformada de tamaño N , calculas la transformada de los elementos pares, la transformada de los elementos impares, y las combinas con un factor de fase". Es como si dividieras el problema en dos mitades, resolvieras cada una por separado (recursivamente), y luego las juntaras. Esto reduce el costo de N^2 a $N \log N$.

Implementacion de la FFT

Vamos a implementar la FFT de Cooley-Tukey.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import time
4
5 def fft_cooley_tukey(x):
6     """
7     Implementacion recursiva de la FFT de Cooley-Tukey.
8     Asume que len(x) es potencia de 2.
9     """
10    N = len(x)
11    if N == 1:
12        return x
13
14    # Separar en pares e impares
15    even = fft_cooley_tukey(x[0::2])
16    odd = fft_cooley_tukey(x[1::2])
17
18    # Combinar
19    X = np.zeros(N, dtype=complex)
20    for k in range(N//2):
21        factor = np.exp(-2j * np.pi * k / N)
22        X[k] = even[k] + factor * odd[k]
23        X[k + N//2] = even[k] - factor * odd[k]
24
25    return X
26
27 # Probamos con una senal senoidal
28 N = 64
29 t = np.linspace(0, 1, N)
30 x = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 12 * t)
31
32 # FFT manual
33 X_manual = fft_cooley_tukey(x)
34
35 # FFT de numpy
36 X_numpy = np.fft.fft(x)
37
38 # Comparacion
39 print("Diferencia entre FFT manual y numpy:", np.linalg.norm(X_manual - X_numpy)
40 )
41
42 # Graficamos el espectro de frecuencias
43 freqs = np.fft.fftfreq(N, d=1/N)
44 plt.figure(figsize=(10, 6))
45 plt.stem(freqs, np.abs(X_manual))
46 plt.xlabel('Frecuencia (ciclos por muestra)')
47 plt.ylabel('Magnitud')
48 plt.title('Espectro de frecuencias de la senal')
49 plt.grid(True)
50 plt.show()

```

Listing 46: FFT de Cooley-Tukey

En el grafico, ves dos picos en las frecuencias 5 y 12, que son exactamente las frecuencias de los senos que usamos. La FFT encuentra estas frecuencias perfectamente.

Comparacion de tiempos: FFT vs DFT directa

Vamos a comparar el tiempo que tarda la FFT vs la DFT directa (multiplicacion matricial).

```

1 import numpy as np
2 import time
3 import matplotlib.pyplot as plt
4
5 def dft_directa(x):
6     """DFT directa usando la matriz de Fourier."""
7     N = len(x)
8     omega = np.exp(-2j * np.pi / N)
9     F = np.array([[omega**(k*n) for n in range(N)] for k in range(N)])
10    return F @ x
11
12 N_values = [2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]
13 tiempos_fft = []
14 tiempos_dft = []
15
16 for N in N_values:
17     x = np.random.randn(N) + 1j * np.random.randn(N)
18
19     # FFT
20     start = time.time()
21     X_fft = np.fft.fft(x)
22     tiempos_fft.append(time.time() - start)
23
24     # DFT directa
25     start = time.time()
26     X_dft = dft_directa(x)
27     tiempos_dft.append(time.time() - start)
28
29     # Verificamos que den lo mismo
30     assert np.linalg.norm(X_fft - X_dft) < 1e-10
31
32 # Graficamos
33 plt.figure(figsize=(10, 6))
34 plt.loglog(N_values, tiempos_fft, 'o-', label='FFT (O(N log N))')
35 plt.loglog(N_values, tiempos_dft, 's-', label='DFT directa (O(N^2))')
36 plt.xlabel('N (tamaño de la señal)')
37 plt.ylabel('Tiempo de ejecución (s)')
38 plt.title('Comparación: FFT vs DFT directa')
39 plt.legend()
40 plt.grid(True)
41 plt.show()

```

Listing 47: Comparacion de tiempos: FFT vs DFT directa

En el grafico, ves que para N chico ambos metodos son similares, pero para N grande la FFT es mucho mas rapida. La pendiente de la DFT es 2 en escala log-log ($O(N^2)$), mientras que la FFT tiene pendiente 1.

El Teorema de Convolucion

El teorema de convolucion es uno de los resultados mas importantes del procesamiento de señales. Dice que la convolucion en el dominio del tiempo es multiplicacion en el dominio de la frecuencia.

Convolucion discreta

La convolucion circular de dos secuencias x e y de longitud N es:

$$(x * y)_n = \sum_{m=0}^{N-1} x_m y_{(n-m) \bmod N}$$

Traducción Matemática de la convolución La convolución se lee en criollo: cada elemento de la salida es una suma ponderada de elementos de la entrada, donde los pesos son los elementos de y . Pensa en la convolución como un "deslizamiento" de una secuencia sobre la otra, multiplicando y sumando.

El teorema

$$\text{DFT}(x * y) = \text{DFT}(x) \cdot \text{DFT}(y)$$

donde el producto es punto a punto.

Traducción Matemática del teorema de convolución El teorema de convolución se lee en criollo: "si convolucionas dos señales, la DFT de la convolución es simplemente el producto de las DFTs de cada señal". Esto es increíblemente útil porque la convolución directa cuesta $O(N^2)$, pero con la FFT podemos hacerlo en $O(N \log N)$: transformar ambas señales, multiplicar, y transformar de vuelta.

Aplicación: filtrado digital

Para filtrar una señal, elegimos un filtro h (por ejemplo, un pasa-bajos) y hacemos:

$$y = x * h$$

Usando la FFT:

$$Y_k = X_k \cdot H_k$$

donde H_k es la DFT del filtro. Luego aplicamos la IDFT para obtener la señal filtrada.

Grafico: filtrado de una señal con ruido

Vamos a filtrar una señal con ruido usando un filtro pasa-bajos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generamos una se al con ruido
5 N = 1024
6 t = np.linspace(0, 1, N)
7 senal_limpia = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 10 * t)
8 ruido = 0.5 * np.random.randn(N)
9 senal = senal_limpia + ruido
10
11 # FFT de la se al
12 X = np.fft.fft(senal)
13 freqs = np.fft.fftfreq(N, d=1/N)
14
15 # Filtro pasa-bajos: eliminar frecuencias > 15
16 filtro = np.zeros(N)
17 filtro[np.abs(freqs) <= 15] = 1

```



```

18
19 # Aplicamos el filtro
20 X_filtrada = X * filtro
21
22 # IDFT
23 senal_filtrada = np.fft.ifft(X_filtrada).real
24
25 # Graficamos
26 plt.figure(figsize=(12, 8))
27 plt.subplot(3, 1, 1)
28 plt.plot(t, senal)
29 plt.title('Señal original con ruido')
30 plt.grid(True)
31
32 plt.subplot(3, 1, 2)
33 plt.plot(freqs, np.abs(X))
34 plt.xlim(-50, 50)
35 plt.title('Espectro de frecuencias')
36 plt.grid(True)
37
38 plt.subplot(3, 1, 3)
39 plt.plot(t, senal_filtrada)
40 plt.title('Señal filtrada (pasa-bajos)')
41 plt.grid(True)
42
43 plt.tight_layout()
44 plt.show()

```

Listing 48: Filtrado de una señal con ruido usando FFT

En el gráfico, ves la señal original con ruido, su espectro de frecuencias (donde se ven los picos en 5 y 10 Hz, y el ruido en todas las frecuencias), y la señal filtrada donde el ruido de alta frecuencia fue eliminado.

Multiplicación rápida de polinomios

La FFT también se usa para multiplicar polinomios rápidamente. Si tenemos dos polinomios de grado N , su producto tiene grado $2N$. La convolución de los coeficientes es exactamente la multiplicación de polinomios.

Algoritmo

1. Extender los polinomios a longitud $2N$ (rellenando con ceros).
2. Calcular la FFT de ambos polinomios.
3. Multiplicar punto a punto.
4. Calcular la IFFT para obtener los coeficientes del producto.

```

1 import numpy as np
2 import time
3 import matplotlib.pyplot as plt
4
5 def multiplicacion_polinomios_fft(p, q):
6     """Multiplica dos polinomios usando FFT."""
7     n = len(p) + len(q) - 1
8     # Extender a potencia de 2

```

```

9     N = 1
10    while N < n:
11        N *= 2
12
13    P = np.fft.fft(p, N)
14    Q = np.fft.fft(q, N)
15    R = P * Q
16    return np.fft.ifft(R)[:n].real
17
18 def multiplicacion_polinomios_directa(p, q):
19     """Multiplica dos polinomios de forma directa ( $O(n^2)$ )."""
20     n = len(p) + len(q) - 1
21     r = np.zeros(n)
22     for i, pi in enumerate(p):
23         for j, qj in enumerate(q):
24             r[i+j] += pi * qj
25     return r
26
27 # Polinomios aleatorios
28 np.random.seed(42)
29 n = 100
30 p = np.random.randn(n)
31 q = np.random.randn(n)
32
33 # Multiplicacion directa
34 start = time.time()
35 r_directa = multiplicacion_polinomios_directa(p, q)
36 tiempo_directa = time.time() - start
37
38 # Multiplicacion con FFT
39 start = time.time()
40 r_fft = multiplicacion_polinomios_fft(p, q)
41 tiempo_fft = time.time() - start
42
43 print(f"Tiempo directa: {tiempo_directa:.6f} s")
44 print(f"Tiempo FFT: {tiempo_fft:.6f} s")
45 print(f"Diferencia entre los resultados: {np.linalg.norm(r_directa - r_fft):.2e}")
46
47 # Comparacion de tiempos para diferentes tamanos
48 n_values = np.arange(10, 200, 10)
49 tiempos_directa = []
50 tiempos_fft = []
51
52 for n in n_values:
53     p = np.random.randn(n)
54     q = np.random.randn(n)
55
56     start = time.time()
57     r_directa = multiplicacion_polinomios_directa(p, q)
58     tiempos_directa.append(time.time() - start)
59
60     start = time.time()
61     r_fft = multiplicacion_polinomios_fft(p, q)
62     tiempos_fft.append(time.time() - start)
63
64 plt.figure(figsize=(10, 6))
65 plt.plot(n_values, tiempos_directa, 'o-', label='Directa ( $O(n^2)$ )')
66 plt.plot(n_values, tiempos_fft, 's-', label='FFT ( $O(n \log n)$ )')
67 plt.xlabel('Grado del polinomio')
68 plt.ylabel('Tiempo de ejecucion (s)')

```

```

69 plt.title('Multiplicacion de polinomios: Directa vs FFT')
70 plt.legend()
71 plt.grid(True)
72 plt.show()

```

Listing 49: Multiplicacion rapida de polinomios con FFT

En el grafico, ves que para n chico la multiplicacion directa es mas rapida, pero para n grande la FFT es mucho mas eficiente.

El principio de incertidumbre de Donoho-Stark

El principio de incertidumbre de Donoho-Stark dice que una señal no puede estar concentrada simultaneamente en tiempo y en frecuencia.

Enunciado

Sea $x \in \mathbb{C}^N$ no nulo. Sean $S = \{n : x_n \neq 0\}$ el soporte en tiempo y $T = \{k : X_k \neq 0\}$ el soporte en frecuencia. Entonces:

$$|S| \cdot |T| \geq N$$

Traduccion Matematica del principio de incertidumbre El principio de incertidumbre se lee en criollo: "si una señal tiene pocos elementos no nulos en el tiempo (está concentrada), entonces su transformada de Fourier debe tener muchos elementos no nulos (está dispersa en frecuencia), y viceversa". Esto es el analogo discreto del principio de incertidumbre de Heisenberg: no puedes saber exactamente la posicion y el momento de una partícula al mismo tiempo.

Grafico: senal concentrada en tiempo vs frecuencia

Vamos a visualizar el principio de incertidumbre.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Senal concentrada en tiempo (delta de Kronecker)
5 N = 64
6 x_tiempo = np.zeros(N)
7 x_tiempo[0] = 1
8 X_tiempo = np.fft.fft(x_tiempo)
9
10 # Senal concentrada en frecuencia (una sola frecuencia)
11 x_frecuencia = np.exp(2j * np.pi * 3 * np.arange(N) / N)
12 X_frecuencia = np.fft.fft(x_frecuencia)
13
14 # Senal intermedia
15 x_intermedia = np.zeros(N)
16 x_intermedia[:4] = 1
17 X_intermedia = np.fft.fft(x_intermedia)
18
19 plt.figure(figsize=(15, 10))
20
21 plt.subplot(3, 2, 1)
22 plt.stem(np.abs(x_tiempo))
23 plt.title('Se al concentrada en tiempo')
24 plt.grid(True)

```

```

25
26 plt.subplot(3, 2, 2)
27 plt.stem(np.abs(X_tiempo))
28 plt.title('FFT (dispersa en frecuencia)')
29 plt.grid(True)
30
31 plt.subplot(3, 2, 3)
32 plt.stem(np.abs(x_frecuencia))
33 plt.title('Se al dispersa en tiempo')
34 plt.grid(True)
35
36 plt.subplot(3, 2, 4)
37 plt.stem(np.abs(X_frecuencia))
38 plt.title('FFT (concentrada en frecuencia)')
39 plt.grid(True)
40
41 plt.subplot(3, 2, 5)
42 plt.stem(np.abs(x_intermedia))
43 plt.title('Se al intermedia')
44 plt.grid(True)
45
46 plt.subplot(3, 2, 6)
47 plt.stem(np.abs(X_intermedia))
48 plt.title('FFT (intermedia)')
49 plt.grid(True)
50
51 plt.tight_layout()
52 plt.show()
53
54 # Verificamos el principio de incertidumbre
55 def soporte(x, tol=1e-10):
56     return np.sum(np.abs(x) > tol)
57
58 s_tiempo = soporte(x_tiempo)
59 t_tiempo = soporte(X_tiempo)
60 print(f"Delta: |S|={s_tiempo}, |T|={t_tiempo}, |S|*|T|={s_tiempo*t_tiempo} >= {N
    }")
61
62 s_frecuencia = soporte(x_frecuencia)
63 t_frecuencia = soporte(X_frecuencia)
64 print(f"Exponencial: |S|={s_frecuencia}, |T|={t_frecuencia}, |S|*|T|={
    s_frecuencia*t_frecuencia} >= {N}")
65
66 s_intermedia = soporte(x_intermedia)
67 t_intermedia = soporte(X_intermedia)
68 print(f"Intermedia: |S|={s_intermedia}, |T|={t_intermedia}, |S|*|T|={
    s_intermedia*t_intermedia} >= {N}")

```

Listing 50: Principio de incertidumbre de Donoho-Stark

En el grafico, ves como la delta (concentrada en tiempo) tiene una FFT completamente dispersa, y la exponencial (concentrada en frecuencia) tiene una FFT concentrada. La senal intermedia tiene un soporte intermedio en ambos dominios.

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Demostracion de la IDFT:** Demuestra que la IDFT $x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \omega_N^{kn}$ es la inversa de la DFT.
- T2. Unitariedad de la matriz de Fourier:** Demuestra que $F_N^* F_N = N I_N$.
- T3. Teorema de convolucion:** Demuestra que $DFT(x * y) = DFT(x) \cdot DFT(y)$.
- T4. FFT de Cooley-Tukey:** Deduce el algoritmo de Cooley-Tukey para $N = 2M$.
- T5. Complejidad de la FFT:** Demuestra que la FFT de Cooley-Tukey tiene complejidad $O(N \log N)$.
- T6. Principio de incertidumbre:** Demuestra el principio de incertidumbre de Donoho-Stark: $|S| \cdot |T| \geq N$.
- T7. DFT de una senal real:** Demuestra que si x es real, entonces $X_k = \overline{X_{N-k}}$.
- T8. Parseval para la DFT:** Demuestra que $\sum_{n=0}^{N-1} |x_n|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X_k|^2$.
- T9. Convolucion lineal vs circular:** Explica la diferencia entre convolucion lineal y circular, y como usar la FFT para calcular convolucion lineal.
- T10. Aliasing:** Explica que es el aliasing en el contexto de la DFT y como evitarlo.

Parte Computacional (10 ejercicios)

- C1. DFT directa:** Implementa la DFT directa usando la matriz de Fourier. Prueba con una senal senoidal.
- C2. FFT recursiva:** Implementa la FFT de Cooley-Tukey recursiva. Compara con `np.fft.fft`.
- C3. FFT iterativa:** Implementa una version iterativa de la FFT (sin recursion).
- C4. Convolucion con FFT:** Implementa la convolucion circular usando FFT. Prueba con dos secuencias aleatorias.
- C5. Filtrado de audio:** Genera una senal de audio sintetica (suma de senos) y filtra las frecuencias no deseadas usando FFT.
- C6. Multiplicacion de polinomios:** Implementa la multiplicacion rapida de polinomios con FFT. Compara con el metodo directo.
- C7. Principio de incertidumbre:** Genera una senal aleatoria y calcula $|S| \cdot |T|$. Verifica que se cumple la desigualdad.
- C8. Compresion de senales:** Usa la FFT para comprimir una senal: guarda solo los coeficientes mas grandes y reconstruye. Grafica el error en funcion del numero de coeficientes.
- C9. Derivada numerica con FFT:** Usa la FFT para calcular la derivada de una funcion. Compara con diferencias finitas.
- C10. Interpolacion con FFT:** Usa la FFT para interpolar una senal a una frecuencia de muestreo mas alta (upsampling). Implementa el metodo de zero-padding en frecuencia.

Soluciones

Soluciones Teoricas

T1. Queremos demostrar que $\frac{1}{N} \sum_{k=0}^{N-1} X_k \omega_N^{kn} = x_n$. Sustituyendo $X_k = \sum_{m=0}^{N-1} x_m \omega_N^{-km}$:

$$\frac{1}{N} \sum_{k=0}^{N-1} \sum_{m=0}^{N-1} x_m \omega_N^{-km} \omega_N^{kn} = \frac{1}{N} \sum_{m=0}^{N-1} x_m \sum_{k=0}^{N-1} \omega_N^{k(n-m)}$$

La suma interior es N si $n = m$ y 0 si $n \neq m$ (serie geometrica). Por lo tanto, el resultado es x_n .

T2. El elemento (j, k) de $F_N^* F_N$ es $\sum_{l=0}^{N-1} \overline{F_{lj}} F_{lk} = \sum_{l=0}^{N-1} \omega_N^{lj} \omega_N^{-lk} = \sum_{l=0}^{N-1} \omega_N^{l(j-k)}$. Esta suma es N si $j = k$ y 0 si $j \neq k$. Por lo tanto $F_N^* F_N = N I_N$.

T3. $\text{DFT}(x * y)_k = \sum_{n=0}^{N-1} (x * y)_n \omega_N^{-kn} = \sum_{n=0}^{N-1} \sum_{m=0}^{N-1} x_m y_{n-m} \omega_N^{-kn}$. Haciendo cambio de variable $l = n - m$: $\sum_{m=0}^{N-1} x_m \omega_N^{-km} \sum_{l=0}^{N-1} y_l \omega_N^{-kl} = X_k Y_k$.

T4. $X_k = \sum_{n=0}^{N-1} x_n \omega_N^{-kn} = \sum_{m=0}^{M-1} x_{2m} \omega_N^{-2km} + \omega_N^{-k} \sum_{m=0}^{M-1} x_{2m+1} \omega_N^{-2km} = E_k + \omega_N^{-k} O_k$, donde E_k y O_k son DFT de tamaño M .

T5. La recurrencia es $T(N) = 2T(N/2) + O(N)$. Por el teorema maestro, $T(N) = O(N \log N)$.

T6. Por Cauchy-Schwarz en la IDFT: $x_n = \frac{1}{N} \sum_{k \in T} X_k \omega_N^{kn}$. Tomando modulo: $|x_n| \leq \frac{1}{N} \sum_{k \in T} |X_k| \leq \frac{\sqrt{|T|}}{N} \|X\|_2$. Elevando al cuadrado y sumando sobre $n \in S$: $\|x\|_2^2 \leq \frac{|S||T|}{N^2} \|X\|_2^2$. Pero $\|X\|_2^2 = N \|x\|_2^2$ (Parseval), así que $|S||T| \geq N$.

T7. $X_{N-k} = \sum_{n=0}^{N-1} x_n \omega_N^{-(N-k)n} = \sum_{n=0}^{N-1} x_n \omega_N^{kn} = \overline{\sum_{n=0}^{N-1} x_n \omega_N^{-kn}} = \overline{X_k}$.

T8. Por la unitariedad de $F_N / \sqrt{N}$, se tiene $\|x\|_2^2 = \|\frac{1}{\sqrt{N}} X\|_2^2 = \frac{1}{N} \|X\|_2^2$.

T9. La convolucion lineal tiene longitud $2N - 1$. Para calcularla con FFT, se extienden ambas secuencias a longitud $2N$ (o potencia de 2 superior) con ceros, se multiplican en frecuencia, y se aplica la IDFT.

T10. El aliasing ocurre cuando se muestrea una señal a una frecuencia menor que el doble de la frecuencia maxima (teorema de Nyquist). Las frecuencias altas se "disfrazan" como frecuencias bajas. Se evita filtrando la señal antes del muestreo.

Soluciones Computacionales

C1.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def dft_directa(x):
5     N = len(x)
6     omega = np.exp(-2j * np.pi / N)
7     F = np.array([[omega**(k*n) for n in range(N)] for k in range(N)])
8     return F @ x
9
10 N = 32
11 t = np.linspace(0, 1, N)
12 x = np.sin(2 * np.pi * 5 * t)
13
14 X_directa = dft_directa(x)
15 X_numpy = np.fft.fft(x)
16
17 print("Diferencia:", np.linalg.norm(X_directa - X_numpy))

```

Listing 51: DFT directa

C2. Ver el código de la Sección .

C3.

```

1 import numpy as np
2
3 def fft_iterativa(x):
4     N = len(x)
5     # Bit-reversal permutation
6     x = np.array(x)
7     j = 0
8     for i in range(1, N):
9         bit = N >> 1
10        while j & bit:
11            j ^= bit
12            bit >>= 1
13        j ^= bit
14        if i < j:
15            x[i], x[j] = x[j], x[i]
16
17    # Iterative butterfly
18    length = 2
19    while length <= N:
20        omega = np.exp(-2j * np.pi / length)
21        for i in range(0, N, length):
22            w = 1
23            for j in range(length // 2):
24                u = x[i + j]
25                v = x[i + j + length // 2] * w
26                x[i + j] = u + v
27                x[i + j + length // 2] = u - v
28                w *= omega
29            length *= 2
30
31    return x
32
33 N = 16
34 x = np.random.randn(N) + 1j * np.random.randn(N)
35 X_fft = fft_iterativa(x)
36 X_numpy = np.fft.fft(x)
37 print("Diferencia:", np.linalg.norm(X_fft - X_numpy))

```

Listing 52: FFT iterativa

C4.

```

1 import numpy as np
2
3 def convolucion_circular_fft(x, y):
4     N = len(x)
5     X = np.fft.fft(x)
6     Y = np.fft.fft(y)
7     return np.fft.ifft(X * Y).real
8
9 # Prueba
10 x = np.array([1, 2, 3])
11 y = np.array([4, 5, 6])
12 z = convolucion_circular_fft(x, y)
13 print("Convolucion circular:", z)
14
15 # Convolucion lineal con padding
16 def convolucion_lineal_fft(x, y):
17     N = len(x) + len(y) - 1

```

```

18     N_fft = 1
19     while N_fft < N:
20         N_fft *= 2
21     X = np.fft.fft(x, N_fft)
22     Y = np.fft.fft(y, N_fft)
23     return np.fft.ifft(X * Y)[:N].real
24
25 z_lineal = convolucion_lineal_fft(x, y)
26 print("Convolucion lineal:", z_lineal)

```

Listing 53: Convolucion circular con FFT

C5.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def filtro_pasa_bajos(frecs, f_corte):
5     return np.where(np.abs(frecs) <= f_corte, 1, 0)
6
7 N = 1024
8 t = np.linspace(0, 1, N)
9senal = np.sin(2 * np.pi * 5 * t) + 0.3 * np.sin(2 * np.pi * 20 * t) + 0.1 * np.
    sin(2 * np.pi * 50 * t)
10
11 X = np.fft.fft(senal)
12 frecs = np.fft.fftfreq(N, d=1/N)
13
14 filtro = filtro_pasa_bajos(frecs, 15)
15 X_filtrada = X * filtro
16senal_filtrada = np.fft.ifft(X_filtrada).real
17
18 plt.figure(figsize=(12, 8))
19 plt.subplot(3, 1, 1)
20 plt.plot(t, senal)
21 plt.title('Se al original (5 + 20 + 50 Hz)')
22 plt.grid(True)
23
24 plt.subplot(3, 1, 2)
25 plt.plot(frecs, np.abs(X))
26 plt.xlim(-60, 60)
27 plt.title('Espectro')
28 plt.grid(True)
29
30 plt.subplot(3, 1, 3)
31 plt.plot(t, senal_filtrada)
32 plt.title('Se al filtrada (solo 5 Hz)')
33 plt.grid(True)
34
35 plt.tight_layout()
36 plt.show()

```

Listing 54: Filtrado de una senal sintetica

C6. Ver el codigo de la Seccion .**C7.**

```

1 import numpy as np
2
3 def soporte(x, tol=1e-10):
4     return np.sum(np.abs(x) > tol)
5

```



```

6 N = 64
7 np.random.seed(42)
8
9 for _ in range(10):
10     x = np.random.randn(N) + 1j * np.random.randn(N)
11     X = np.fft.fft(x)
12     s = soporte(x)
13     t = soporte(X)
14     print(f"|S|={s}, |T|={t}, |S|*|T|={s*t}, N={N}, cumple={s*t >= N}")

```

Listing 55: Principio de incertidumbre

C8.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 N = 256
5 t = np.linspace(0, 1, N)
6 senal = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 12 * t) + 0.3 * np.
    sin(2 * np.pi * 25 * t)
7
8 X = np.fft.fft(senal)
9
10 # Ordenamos los coeficientes por magnitud
11 indices = np.argsort(np.abs(X))[:, :-1]
12 errores = []
13 coefs_guardados = []
14
15 for k in range(1, N+1):
16     X_comp = np.zeros(N, dtype=complex)
17     X_comp[indices[:k]] = X[indices[:k]]
18     senal_comp = np.fft.ifft(X_comp).real
19     error = np.linalg.norm(senal - senal_comp) / np.linalg.norm(senal)
20     errores.append(error)
21     coefs_guardados.append(k)
22
23 plt.figure(figsize=(10, 6))
24 plt.semilogy(coefs_guardados, errores)
25 plt.xlabel('Numero de coeficientes guardados')
26 plt.ylabel('Error relativo')
27 plt.title('Compresion de senal con FFT')
28 plt.grid(True)
29 plt.show()

```

Listing 56: Compresion de senales con FFT

C9.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def derivada_fft(f, x):
5     N = len(f)
6     F = np.fft.fft(f)
7     # Frecuencias
8     k = np.fft.fftfreq(N, d=x[1]-x[0])
9     # Derivada en frecuencia: F' = i * 2*pi * k * F
10    F_der = 1j * 2 * np.pi * k * F
11    return np.fft.ifft(F_der).real
12
13 x = np.linspace(0, 2*np.pi, 64)

```

```

14 f = np.sin(x)
15 f_exacta = np.cos(x)
16
17 f_der = derivada_fft(f, x)
18
19 plt.figure(figsize=(10, 6))
20 plt.plot(x, f_exacta, 'k-', label='Derivada exacta')
21 plt.plot(x, f_der, 'ro', label='Derivada con FFT')
22 plt.xlabel('x')
23 plt.ylabel('f\'(x)')
24 plt.title('Derivada numerica con FFT')
25 plt.legend()
26 plt.grid(True)
27 plt.show()
28
29 print("Error:", np.linalg.norm(f_der - f_exacta) / np.linalg.norm(f_exacta))

```

Listing 57: Derivada numerica con FFT

C10.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def upsampling_fft(x, factor):
5     N = len(x)
6     X = np.fft.fft(x)
7     N_new = N * factor
8     X_new = np.zeros(N_new, dtype=complex)
9     # Colocar las frecuencias bajas en el centro
10    half = N // 2
11    X_new[:half] = X[:half]
12    X_new[-half:] = X[-half:]
13    return np.fft.ifft(X_new).real * factor
14
15 # Senal original
16 N = 16
17 t = np.linspace(0, 1, N)
18 x = np.sin(2 * np.pi * 3 * t) + 0.5 * np.sin(2 * np.pi * 5 * t)
19
20 # Upsampling
21 factor = 4
22 x_up = upsampling_fft(x, factor)
23
24 t_up = np.linspace(0, 1, len(x_up))
25
26 plt.figure(figsize=(10, 6))
27 plt.plot(t, x, 'ro', label='Original (muestras)')
28 plt.plot(t_up, x_up, 'b-', label='Interpolada (FFT)')
29 plt.xlabel('t')
30 plt.ylabel('x(t)')
31 plt.title('Upsampling con FFT (zero-padding en frecuencia)')
32 plt.legend()
33 plt.grid(True)
34 plt.show()

```

Listing 58: Upsampling con FFT (zero-padding en frecuencia)

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por las transformadas discretas. Viste que no era tan complicado. Arrancamos con la DFT como interpolacion en las raices de la unidad, pasamos por la FFT, el teorema de convolucion, el filtrado digital, la multiplicacion rapida de polinomios, y el principio de incertidumbre. Todo esta conectado: la DFT es la herramienta fundamental para el procesamiento digital de señales.

Los puntos clave para llevar:

- La DFT transforma una señal del dominio del tiempo al dominio de la frecuencia.
- La FFT calcula la DFT en $O(N \log N)$.
- La convolucion en tiempo es multiplicacion en frecuencia (teorema de convolucion).
- La FFT permite filtrar señales eficientemente.
- La multiplicacion de polinomios se puede hacer rapidamente con FFT.
- Existe un principio de incertidumbre: no puedes concentrar una señal en tiempo y frecuencia simultaneamente.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. La DFT y la FFT son herramientas fundamentales que vas a usar en practicamente todos los problemas de procesamiento de señales y datos. ¡Dale para adelante, que vos podes!

Resolucion de Ecuaciones No Lineales

Como encontrar raices cuando la vida no es lineal, con Python y mucha paciencia.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a meterle a uno de los problemas mas viejos de la matematica: encontrar el valor de x que hace que $f(x) = 0$. Si llegaste hasta aca, seguro ya sabes que la mayoría de las ecuaciones que importan en fisica no son lineales. Pensa en la ley de gravitacion, en las ecuaciones de Navier-Stokes, en la mecanica cuantica no lineal... en todos lados aparecen ecuaciones no lineales.

Cuando curse Analisis Numerico, recuerdo que el profesor nos puso un ejemplo con la ecuacion $x = \cos(x)$ y nos dijo: "intenten resolverla analiticamente". Despues de media hora de intentos, todos nos dimos cuenta de que no se puede despejar. Ahi entendi que los metodos numericos no son una opcion, son una necesidad. Y desde ese dia, cada vez que veo una ecuacion no lineal, pienso en los metodos que vamos a ver hoy.

En este capitulo vamos a cubrir:

- El metodo de biseccion: simple, robusto, pero lento.
- El metodo de Regula Falsi: una mejora sobre biseccion.
- El metodo de Newton-Raphson: el rey de la convergencia cuadratica.
- El metodo de punto fijo: una vision general de la iteracion.
- El metodo de la secante: cuando no tenes derivada.
- Newton en varias variables: porque la vida es multidimensional.

La idea es que vos puedas implementar todo esto en Python, entender por que funciona, y saber que metodo usar en cada situacion. Asi que agarra un cafe, ponete comodo, y vamos a darle.

¿Por que ecuaciones no lineales?

El problema es simple: dada una funcion $f : \mathbb{R} \rightarrow \mathbb{R}$ (o $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$), queremos encontrar un punto x^* tal que $f(x^*) = 0$. En una variable, esto es encontrar las raices de f . En varias variables, es resolver un sistema de ecuaciones.

Traduccion Matematica del problema El problema se lee en criollo: "tenemos una funcion que transforma numeros en numeros (o vectores en vectores), y queremos saber donde se anula". Pensa en esto como si estuvieras buscando el punto donde la grafica de f cruza el eje x . En el caso de varias variables, es como buscar el punto donde varias superficies se intersectan.

La razon por la que necesitamos metodos numericos es simple: salvo casos muy especiales, las ecuaciones no lineales no se pueden resolver analiticamente. No hay una "formula cuadratica" para ecuaciones de grado 5 o mas (eso lo demostro Abel y Galois). Asi que tenemos que aproximar.

El metodo de biseccion

El metodo de biseccion es el mas simple y robusto de todos. La idea es usar el teorema de Bolzano: si f es continua y $f(a)f(b) < 0$, entonces hay una raiz en (a, b) . El metodo consiste en ir partiendo el intervalo a la mitad y quedandose con la mitad que contiene la raiz.

El algoritmo

1. Dado un intervalo $[a, b]$ con $f(a)f(b) < 0$.
2. Calcular $c = (a + b)/2$.
3. Si $f(c) = 0$, listo.
4. Si $f(a)f(c) < 0$, la raiz esta en $[a, c]$, entonces $b = c$.
5. Si no, la raiz esta en $[c, b]$, entonces $a = c$.
6. Repetir hasta que $|b - a| < \text{tol}$.

Traduccion Matematica de la biseccion La biseccion se lee en criollo: ¡imaginate que tenes un intervalo donde la funcion cambia de signo. Agarras el punto medio, ves que signo tiene, y te quedas con la mitad que sigue teniendo cambio de signo. Repetis esto hasta que el intervalo sea tan chico como quieras”. Es como buscar una llave en una habitacion oscura: vas descartando mitades del espacio hasta encontrar lo que buscas.

Convergencia

El error despues de n iteraciones es:

$$|e_n| \leq \frac{b - a}{2^n}$$

Traduccion Matematica del error El error de la biseccion se lee en criollo: “el error se reduce a la mitad en cada iteracion”. Esto es convergencia **lineal** con tasa $1/2$. Para obtener 1 digito de precision extra, necesitas unas 3-4 iteraciones. Es lento pero seguro.

Grafico: convergencia de la biseccion

Vamos a visualizar como converge la biseccion para $f(x) = x^2 - 2$ (la raiz es $\sqrt{2}$).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def biseccion(f, a, b, tol=1e-6, max_iter=100):
5     """Metodo de biseccion para encontrar una raiz."""
6     if f(a) * f(b) >= 0:
7         raise ValueError("f(a) y f(b) deben tener signos opuestos")
8
9     historial = []
10    for i in range(max_iter):
11        c = (a + b) / 2
12        historial.append(c)
13
14        if f(c) == 0 or (b - a) / 2 < tol:
15            break

```

```

16         if f(a) * f(c) < 0:
17             b = c
18         else:
19             a = c
20
21     return c, historial
22
23
24 def f(x):
25     return x**2 - 2
26
27 raiz_exacta = np.sqrt(2)
28 raiz, historial = biseccion(f, 1, 2, tol=1e-10)
29
30 print(f"Raiz aproximada: {raiz:.10f}")
31 print(f"Raiz exacta: {raiz_exacta:.10f}")
32 print(f"Error: {abs(raiz - raiz_exacta):.2e}")
33 print(f"Iteraciones: {len(historial)}")
34
35 # Graficamos la convergencia
36 iteraciones = np.arange(len(historial))
37 errores = np.abs(np.array(historial) - raiz_exacta)
38
39 plt.figure(figsize=(10, 6))
40 plt.semilogy(iteraciones, errores, 'o-')
41 plt.xlabel('Iteracion')
42 plt.ylabel('Error absoluto')
43 plt.title('Convergencia del metodo de biseccion para sqrt(2)')
44 plt.grid(True)
45 plt.show()

```

Listing 59: Metodo de biseccion para sqrt(2)

En el grafico, ves que el error decrece exponencialmente (lineal en escala logaritmica). Cada iteracion reduce el error a la mitad.

El metodo de Regula Falsi

El metodo de Regula Falsi (o falsa posicion) es una mejora sobre la biseccion. En lugar de tomar el punto medio, toma la interseccion de la recta que une $(a, f(a))$ y $(b, f(b))$ con el eje x .

La formula

$$c = \frac{af(b) - bf(a)}{f(b) - f(a)}$$

Traduccion Matematica de Regula Falsi Regula Falsi se lee en criollo: ^{en} lugar de partir el intervalo por la mitad, dibujas una recta entre los dos puntos extremos y ves donde esa recta cruza el eje x . Es como si estuvieras haciendo una interpolacion lineal para aproximar la raiz. Esto suele converger mas rapido que la biseccion, pero puede tener problemas si la funcion es muy curva.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def regula_falsi(f, a, b, tol=1e-6, max_iter=100):

```

```

5  """Metodo de Regula Falsi."""
6  if f(a) * f(b) >= 0:
7      raise ValueError("f(a) y f(b) deben tener signos opuestos")
8
9  historial = []
10 for i in range(max_iter):
11     c = (a * f(b) - b * f(a)) / (f(b) - f(a))
12     historial.append(c)
13
14     if abs(f(c)) < tol or abs(b - a) < tol:
15         break
16
17     if f(a) * f(c) < 0:
18         b = c
19     else:
20         a = c
21
22     return c, historial
23
24 def f(x):
25     return x**2 - 2
26
27 raiz_exacta = np.sqrt(2)
28 raiz, historial_rf = regula_falsi(f, 1, 2, tol=1e-10)
29
30 print(f"Regula Falsi: {raiz:.10f}, Error: {abs(raiz - raiz_exacta):.2e}")
31
32 # Comparacion con biseccion
33 raiz_b, historial_b = biseccion(f, 1, 2, tol=1e-10)
34
35 plt.figure(figsize=(10, 6))
36 plt.semilogy(np.arange(len(historial_b)), np.abs(np.array(historial_b) -
37     raiz_exacta), 'o-', label='Biseccion')
38 plt.semilogy(np.arange(len(historial_rf)), np.abs(np.array(historial_rf) -
39     raiz_exacta), 's-', label='Regula Falsi')
40 plt.xlabel('Iteracion')
41 plt.ylabel('Error absoluto')
42 plt.title('Comparacion: Biseccion vs Regula Falsi')
43 plt.legend()
44 plt.grid(True)
45 plt.show()

```

Listing 60: Regula Falsi para sqrt(2)

El metodo de Newton-Raphson

El metodo de Newton es el mas famoso y el mas poderoso de todos. La idea es simple: si tienes una aproximacion x_n , la recta tangente a f en x_n te da una mejor aproximacion.

La formula

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Traduccion Matematica de Newton Newton se lee en criollo: "dibujas la recta tangente a la funcion en el punto donde estas, y ves donde esa recta cruza el eje x . Ese nuevo punto

es tu siguiente aproximacion". Es como si estuvieras "deslizandote" por la funcion hacia la raiz siguiendo la pendiente.

Convergencia cuadratica

Si x_n esta cerca de una raiz simple ($f'(x^*) \neq 0$), entonces:

$$|e_{n+1}| \leq C|e_n|^2$$

Traduccion Matematica de la convergencia cuadratica La convergencia cuadratica se lee en criollo: "el error en cada paso es aproximadamente el cuadrado del error anterior". Esto significa que el numero de digitos correctos se **duplica** en cada iteracion. Si tienes 1 dígito correcto, en la siguiente iteracion tienes 2, luego 4, luego 8, etc. Es increíblemente rapido.

Grafico: convergencia de Newton

Vamos a ver la convergencia cuadratica de Newton para $f(x) = x^2 - 2$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def newton(f, df, x0, tol=1e-10, max_iter=100):
5     """Metodo de Newton-Raphson."""
6     x = x0
7     historial = [x]
8     for i in range(max_iter):
9         x_new = x - f(x) / df(x)
10        historial.append(x_new)
11        if abs(x_new - x) < tol:
12            break
13        x = x_new
14    return x, historial
15
16 def f(x):
17     return x**2 - 2
18
19 def df(x):
20     return 2*x
21
22 raiz_exacta = np.sqrt(2)
23 raiz, historial_n = newton(f, df, 3, tol=1e-12)
24
25 print(f"Newton: {raiz:.12f}")
26 print(f"Iteraciones: {len(historial_n)}")
27 print("Historial:")
28 for i, x in enumerate(historial_n[:7]):
29     print(f"x_{i} = {x:.10f}, error = {abs(x - raiz_exacta):.2e}")
30
31 # Graficamos la convergencia
32 iteraciones = np.arange(len(historial_n))
33 errores = np.abs(np.array(historial_n) - raiz_exacta)
34
35 plt.figure(figsize=(10, 6))
36 plt.semilogy(iteraciones, errores, 'o-')
37 plt.xlabel('Iteracion')
38 plt.ylabel('Error absoluto')
39 plt.title('Convergencia cuadratica de Newton para sqrt(2)')
40 plt.grid(True)

```

```
41 plt.show()
```

Listing 61: Metodo de Newton para sqrt(2)

En el grafico, ves que el error decrece mas rapido que exponencial: en cada paso se cuadruplica el numero de digitos correctos. Con 6 iteraciones ya tenes precision de maquina.

Problemas de Newton

Newton no es perfecto. Tiene algunos problemas:

1. **Necesita la derivada:** Si no tenes una formula para f' , tenes que aproximarla numericamente.
2. **Convergencia local:** Solo converge si empezas cerca de la raiz. Si empezas lejos, puede diverger.
3. **Raices multiples:** Si $f'(x^*) = 0$, la convergencia pasa a ser lineal en lugar de cuadratica.
4. **Puntos de inflexion:** Si $f'(x_n) = 0$, el metodo se rompe (division por cero).

Newton para raices multiples

Si f tiene una raiz multiple de multiplicidad m , la convergencia de Newton es lineal. Pero podemos modificar el metodo:

$$x_{n+1} = x_n - m \frac{f(x_n)}{f'(x_n)}$$

Si conocemos m , recuperamos la convergencia cuadratica.

El metodo de punto fijo

El metodo de punto fijo es una idea mas general: en lugar de resolver $f(x) = 0$, reescribimos la ecuacion como $x = g(x)$ y iteramos $x_{n+1} = g(x_n)$.

Traduccion Matematica del punto fijo El metodo de punto fijo se lee en criollo: .^{en} lugar de buscar donde la funcion se anula, buscamos donde la funcion es igual a x ". Esto es mas general porque g puede ser cualquier funcion que tenga la misma raiz. El truco esta en elegir g de forma que la iteracion converja.

Teorema de convergencia

Si $|g'(x)| < 1$ en un entorno de la raiz, entonces la iteracion converge. Ademas, el error satisface:

$$|e_{n+1}| \leq L|e_n|$$

donde $L = \max |g'(x)| < 1$. Esto es convergencia lineal.

Ejemplo: $x = \cos(x)$

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def punto_fijo(g, x0, tol=1e-10, max_iter=100):
5     """Metodo de punto fijo."""
6     x = x0
7     historial = [x]
8     for i in range(max_iter):
9         x_new = g(x)
10        historial.append(x_new)
11        if abs(x_new - x) < tol:
12            break
13        x = x_new
14    return x, historial
15
16 def g(x):
17     return np.cos(x)
18
19 raiz, historial = punto_fijo(g, 0.5, tol=1e-10)
20
21 print(f"Raiz de x = cos(x): {raiz:.10f}")
22 print(f"Iteraciones: {len(historial)}")
23
24 # Graficamos la convergencia
25 iteraciones = np.arange(len(historial))
26 errores = np.abs(np.array(historial) - raiz)
27
28 plt.figure(figsize=(10, 6))
29 plt.semilogy(iteraciones, errores, 'o-')
30 plt.xlabel('Iteracion')
31 plt.ylabel('Error absoluto')
32 plt.title('Convergencia de punto fijo para x = cos(x)')
33 plt.grid(True)
34 plt.show()

```

Listing 62: Punto fijo para $x = \cos(x)$ **El metodo de la secante**

El metodo de la secante es como Newton, pero sin necesidad de calcular la derivada. En lugar de la tangente, usa la secante que pasa por dos puntos anteriores.

La formula

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Traduccion Matematica de la secante La secante se lee en criollo: ".en lugar de usar la tangente (que requiere la derivada), usas la recta que pasa por los dos ultimos puntos que calculaste". Es una aproximacion de la derivada usando diferencias finitas.

Convergencia

El orden de convergencia del metodo de la secante es:

$$p = \frac{1 + \sqrt{5}}{2} \approx 1,618$$

Es decir, es mas rapido que lineal ($p=1$) pero mas lento que cuadratico ($p=2$). Es una buena opcion cuando no tienes la derivada.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def secante(f, x0, x1, tol=1e-10, max_iter=100):
5     """Metodo de la secante."""
6     historial = [x0, x1]
7     for i in range(max_iter):
8         x_new = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
9         historial.append(x_new)
10        if abs(x_new - x1) < tol:
11            break
12        x0, x1 = x1, x_new
13    return x_new, historial
14
15 def f(x):
16     return x**2 - 2
17
18 raiz_exacta = np.sqrt(2)
19 raiz, historial_s = secante(f, 1, 3, tol=1e-12)
20
21 print(f"Secante: {raiz:.12f}")
22 print(f"Iteraciones: {len(historial_s)}")
23
24 # Comparacion con Newton
25 _, historial_n = newton(f, lambda x: 2*x, 3, tol=1e-12)
26
27 plt.figure(figsize=(10, 6))
28 plt.semilogy(np.arange(len(historial_n)), np.abs(np.array(historial_n) -
29     raiz_exacta), 'o-', label='Newton')
30 plt.semilogy(np.arange(len(historial_s)), np.abs(np.array(historial_s) -
31     raiz_exacta), 's-', label='Secante')
32 plt.xlabel('Iteracion')
33 plt.ylabel('Error absoluto')
34 plt.title('Comparacion: Newton vs Secante')
35 plt.legend()
36 plt.grid(True)
37 plt.show()

```

Listing 63: Metodo de la secante

Newton en varias variables

En varias variables, el metodo de Newton se generaliza usando la matriz Jacobiana.

La formula

Dado $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$, queremos resolver $F(x) = 0$. La iteracion de Newton es:

$$x_{k+1} = x_k - J_F(x_k)^{-1} F(x_k)$$

donde J_F es la matriz Jacobiana.

Traducción Matemática de Newton multidimensional Newton multidimensional se lee en criollo: “en lugar de la derivada, usamos la matriz de todas las derivadas parciales (la Jacobiana). Resolvemos un sistema lineal en cada paso para encontrar la corrección”. Es como Newton en una variable, pero con matrices en lugar de escalares.

Ejemplo: sistema 2x2

Vamos a resolver el sistema:

$$\begin{cases} x^2 + y^2 = 4 \\ x^2 - y^2 = 1 \end{cases}$$

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def newton_multidim(F, JF, x0, tol=1e-10, max_iter=100):
5     """Metodo de Newton para sistemas no lineales."""
6     x = np.array(x0, dtype=float)
7     historial = [x.copy()]
8     for i in range(max_iter):
9         # Resolvemos JF(x) * dx = -F(x)
10        try:
11            dx = np.linalg.solve(JF(x), -F(x))
12        except np.linalg.LinAlgError:
13            break
14        x = x + dx
15        historial.append(x.copy())
16        if np.linalg.norm(dx) < tol:
17            break
18    return x, historial
19
20 # Definimos el sistema
21 def F(x):
22     return np.array([x[0]**2 + x[1]**2 - 4, x[0]**2 - x[1]**2 - 1])
23
24 def JF(x):
25     return np.array([[2*x[0], 2*x[1]], [2*x[0], -2*x[1]]])
26
27 x0 = np.array([1.5, 1.0])
28 solucion, historial = newton_multidim(F, JF, x0)
29
30 print(f"Solucion: x = {solucion[0]:.6f}, y = {solucion[1]:.6f}")
31 print(f"Verificacion: x^2+y^2 = {solucion[0]**2 + solucion[1]**2:.6f}")
32 print(f"Verificacion: x^2-y^2 = {solucion[0]**2 - solucion[1]**2:.6f}")
33
34 # Graficamos la trayectoria
35 historial = np.array(historial)
36 plt.figure(figsize=(8, 8))
37 plt.plot(historial[:, 0], historial[:, 1], 'o-', label='Trayectoria de Newton')
38 plt.plot(solucion[0], solucion[1], 'r*', markersize=15, label='Solucion')
39 # Circulo y hipérbola
40 theta = np.linspace(0, 2*np.pi, 100)
41 plt.plot(2*np.cos(theta), 2*np.sin(theta), 'k--', label='x^2+y^2=4')
42 plt.plot(np.linspace(-2, 2, 100), np.sqrt(np.maximum(np.linspace(-2, 2, 100)**2
43     - 1, 0)), 'k--')
44 plt.xlabel('x')
45 plt.ylabel('y')
46 plt.title('Newton multidimensional para el sistema')
47 plt.legend()
48 plt.grid(True)

```

```

48 plt.axis('equal')
49 plt.show()

```

Listing 64: Newton multidimensional

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Teorema de Bolzano:** Enuncia y demuestra el teorema de Bolzano. ¿Como se usa en el metodo de biseccion?
- T2. Convergencia de la biseccion:** Demuestra que el error de la biseccion satisface $|e_n| \leq (b - a)/2^n$.
- T3. Convergencia cuadratica de Newton:** Demuestra que si $f'(x^*) \neq 0$, el metodo de Newton tiene convergencia cuadratica.
- T4. Newton con raices multiples:** Demuestra que si f tiene una raiz de multiplicidad m , el metodo de Newton converge linealmente con tasa $(m - 1)/m$.
- T5. Newton modificado:** Demuestra que si f tiene una raiz de multiplicidad m , la iteracion $x_{n+1} = x_n - mf(x_n)/f'(x_n)$ recupera la convergencia cuadratica.
- T6. Metodo de punto fijo:** Demuestra que si $|g'(x)| \leq L < 1$ en un entorno de la raiz, la iteracion de punto fijo converge.
- T7. Orden de la secante:** Demuestra que el metodo de la secante tiene orden de convergencia $p = (1 + \sqrt{5})/2$.
- T8. Newton en varias variables:** Demuestra que el metodo de Newton en varias variables tiene convergencia cuadratica si la Jacobiana es invertible en la raiz.
- T9. Regula Falsi vs Biseccion:** Analiza las ventajas y desventajas de Regula Falsi comparado con biseccion. ¿En que casos uno es mejor que el otro?
- T10. Estabilidad de Newton:** Explica por que el metodo de Newton puede diverger si el punto inicial no esta cerca de la raiz. Da un ejemplo.

Parte Computacional (10 ejercicios)

- C1. Biseccion para cos(x):** Implementa la biseccion para encontrar la raiz de $\cos(x) = 0$ en $[1, 2]$. Grafica la convergencia.
- C2. Regula Falsi para exp(x):** Usa Regula Falsi para encontrar la raiz de $e^x - 3x = 0$ en $[0, 1]$. Grafica el error.
- C3. Newton para una funcion polinomial:** Encuentra todas las raices de $x^3 - 6x^2 + 11x - 6 = 0$ usando Newton con diferentes puntos iniciales.
- C4. Newton vs Secante:** Compara Newton y la secante para $f(x) = \tan(x) - x$ en $[4, 5]$. ¿Cual converge mas rapido?
- C5. Punto fijo para $x = \sin(x)$:** Encuentra la raiz de $x = \sin(x)$ usando punto fijo. Analiza la convergencia.

- C6. Newton con raiz multiple:** Para $f(x) = (x-1)^2$, compara Newton estandar con Newton modificado (con $m = 2$). Grafica los errores.
- C7. Newton multidimensional:** Resuelve el sistema $\begin{cases} x^2 + y^2 = 1 \\ x - y = 0,5 \end{cases}$ usando Newton multidimensional.
- C8. Metodo de la secante vs Newton con derivada numerica:** Compara la secante con Newton usando una derivada numerica (diferencias finitas) para $f(x) = e^{-x} - x$.
- C9. Convergencia de Newton para diferentes puntos iniciales:** Para $f(x) = x^3 - x$, estudia la convergencia de Newton para diferentes puntos iniciales. ¿Cuándo converge a cada raíz?
- C10. Implementacion de un solver general:** Implementa una funcion que, dada una funcion f , un metodo (biseccion, Newton, secante, punto fijo) y los parametros necesarios, encuentre la raiz. Grafica la convergencia de todos los metodos para una misma funcion.

Soluciones

Soluciones Teoricas

T1. El teorema de Bolzano dice: si f es continua en $[a, b]$ y $f(a)f(b) < 0$, entonces existe $c \in (a, b)$ tal que $f(c) = 0$. La demostracion usa biseccion: se construyen dos sucesiones a_n y b_n que convergen al mismo punto c por el teorema de los intervalos encajados, y por continuidad $f(c)^2 \leq 0$, así que $f(c) = 0$.

T2. En cada iteracion, el intervalo se reduce a la mitad: $b_n - a_n = (b - a)/2^n$. Como la raiz esta en el intervalo, el error es $|e_n| \leq (b_n - a_n) = (b - a)/2^n$.

T3. Usando Taylor: $0 = f(x^*) = f(x_n) + f'(x_n)(x^* - x_n) + \frac{1}{2}f''(\xi)(x^* - x_n)^2$. Dividiendo por $f'(x_n)$ y usando $x_{n+1} = x_n - f(x_n)/f'(x_n)$: $e_{n+1} = \frac{f''(\xi)}{2f'(x_n)}e_n^2$. Como $f'(x_n) \rightarrow f'(x^*) \neq 0$, $|e_{n+1}| \leq C|e_n|^2$.

T4. Si $f(x) = (x - x^*)^m g(x)$ con $g(x^*) \neq 0$, entonces $f'(x) = m(x - x^*)^{m-1}g(x) + (x - x^*)^m g'(x)$. Sustituyendo en Newton: $e_{n+1} = e_n - \frac{(x_n - x^*)^m g(x_n)}{m(x_n - x^*)^{m-1}g(x_n) + (x_n - x^*)^m g'(x_n)} = \frac{m-1}{m}e_n + O(e_n^2)$. La tasa es $(m - 1)/m$.

T5. Si usamos $x_{n+1} = x_n - mf(x_n)/f'(x_n)$, entonces $e_{n+1} = e_n - m \frac{e_n^m g(x_n)}{me_n^{m-1}g(x_n) + e_n^m g'(x_n)} = O(e_n^2)$. Recuperamos la convergencia cuadratica.

T6. Si $|g'(x)| \leq L < 1$, entonces por el teorema del valor medio: $|x_{n+1} - x^*| = |g(x_n) - g(x^*)| \leq L|x_n - x^*|$. Iterando: $|e_n| \leq L^n|e_0| \rightarrow 0$.

T7. El error de la secante satisface $e_{n+1} = \frac{f''(\xi)}{2f'(\eta)}e_n e_{n-1}$. Suponiendo $|e_{n+1}| \sim C|e_n|^p$, tenemos $C|e_n|^p \sim C^2|e_{n-1}|^{p^2}|e_{n-1}| = C^2|e_{n-1}|^{p^2+1}$. Así que $p = p^2 + 1$, o $p^2 - p - 1 = 0$, $p = (1 + \sqrt{5})/2$.

T8. En varias variables, el desarrollo de Taylor da: $F(x^*) = F(x_k) + J_F(x_k)(x^* - x_k) + O(\|x^* - x_k\|^2)$. Multiplicando por $J_F(x_k)^{-1}$ y restando: $x_{k+1} - x^* = O(\|x_k - x^*\|^2)$.

T9. Regula Falsi converge mas rapido que biseccion cuando la funcion es suave y la raiz no esta cerca de un extremo. Pero si la funcion es muy curva, Regula Falsi puede estancarse (un extremo del intervalo no se actualiza). Biseccion siempre garantiza convergencia, aunque sea lenta.

T10. Newton puede diverger si $f'(x_n)$ es muy pequenio (cerca de un punto de inflexion) o si el punto inicial esta en el "lado equivocado" de una raiz. Ejemplo: $f(x) = x^3 - x$ con $x_0 = 0,1$

converge a 0, pero con $x_0 = 0,5$ converge a 1. Con $x_0 = 1/\sqrt{3}$ (punto de inflexion), $f'(x_0) = 0$ y el metodo explota.

Soluciones Computacionales

C1.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def biseccion(f, a, b, tol=1e-10):
5     historial = []
6     while (b - a) / 2 > tol:
7         c = (a + b) / 2
8         historial.append(c)
9         if f(c) == 0:
10             break
11         if f(a) * f(c) < 0:
12             b = c
13         else:
14             a = c
15     return c, historial
16
17 def f(x):
18     return np.cos(x)
19
20 raiz, historial = biseccion(f, 1, 2, tol=1e-10)
21 print(f"Raiz de cos(x)=0 en [1,2]: {raiz:.10f}")
22
23 plt.figure(figsize=(10, 6))
24 plt.semilogy(np.arange(len(historial)), np.abs(np.array(historial) - np.pi/2), 'o-')
25 plt.xlabel('Iteracion')
26 plt.ylabel('Error absoluto')
27 plt.title('Biseccion para cos(x)')
28 plt.grid(True)
29 plt.show()

```

Listing 65: Biseccion para cos(x)

C2.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def regula_falsi(f, a, b, tol=1e-10):
5     historial = []
6     for i in range(100):
7         c = (a * f(b) - b * f(a)) / (f(b) - f(a))
8         historial.append(c)
9         if abs(f(c)) < tol:
10             break
11         if f(a) * f(c) < 0:
12             b = c
13         else:
14             a = c
15     return c, historial
16
17 def f(x):
18     return np.exp(x) - 3*x
19

```



```

20 raiz, historial = regula_falsi(f, 0, 1, tol=1e-10)
21 print(f"Raiz de  $e^x - 3x = 0$ : {raiz:.10f}")
22
23 plt.figure(figsize=(10, 6))
24 plt.semilogy(np.arange(len(historial)), np.abs(np.array(historial) - raiz), 'o
    -')
25 plt.xlabel('Iteracion')
26 plt.ylabel('Error absoluto')
27 plt.title('Regula Falsi para  $e^x - 3x$ ')
28 plt.grid(True)
29 plt.show()

```

Listing 66: Regula Falsi para $\exp(x)$ **C3.**

```

1 import numpy as np
2
3 def newton(f, df, x0, tol=1e-10):
4     x = x0
5     for i in range(100):
6         x_new = x - f(x) / df(x)
7         if abs(x_new - x) < tol:
8             break
9         x = x_new
10    return x
11
12 def f(x):
13     return x**3 - 6*x**2 + 11*x - 6
14
15 def df(x):
16     return 3*x**2 - 12*x + 11
17
18 # Raices: 1, 2, 3
19 for x0 in [0, 1.5, 2.5, 4]:
20     raiz = newton(f, df, x0)
21     print(f"x0 = {x0:.1f} -> raiz = {raiz:.6f}")

```

Listing 67: Newton para polinomio

C4.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def newton(f, df, x0, tol=1e-10):
5     x = x0
6     hist = [x]
7     for i in range(100):
8         x_new = x - f(x) / df(x)
9         hist.append(x_new)
10        if abs(x_new - x) < tol:
11            break
12        x = x_new
13    return x, hist
14
15 def secante(f, x0, x1, tol=1e-10):
16     hist = [x0, x1]
17     for i in range(100):
18         x_new = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
19         hist.append(x_new)
20         if abs(x_new - x1) < tol:

```

```

21         break
22         x0, x1 = x1, x_new
23         return x_new, hist
24
25 def f(x):
26     return np.tan(x) - x
27
28 def df(x):
29     return 1/np.cos(x)**2 - 1
30
31 raiz_n, hist_n = newton(f, df, 4.5, tol=1e-12)
32 raiz_s, hist_s = secante(f, 4, 5, tol=1e-12)
33 raiz_exacta = 4.493409457909064
34
35 plt.figure(figsize=(10, 6))
36 plt.semilogy(np.arange(len(hist_n)), np.abs(np.array(hist_n) - raiz_exacta), 'o-
    -', label='Newton')
37 plt.semilogy(np.arange(len(hist_s)), np.abs(np.array(hist_s) - raiz_exacta), 's
    -', label='Secante')
38 plt.xlabel('Iteracion')
39 plt.ylabel('Error absoluto')
40 plt.title('Newton vs Secante para tan(x) = x')
41 plt.legend()
42 plt.grid(True)
43 plt.show()

```

Listing 68: Newton vs Secante para $\tan(x)$ **C5.**

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def punto_fijo(g, x0, tol=1e-10):
5     x = x0
6     hist = [x]
7     for i in range(100):
8         x_new = g(x)
9         hist.append(x_new)
10        if abs(x_new - x) < tol:
11            break
12        x = x_new
13    return x, hist
14
15 def g(x):
16     return np.sin(x)
17
18 raiz, hist = punto_fijo(g, 0.5, tol=1e-10)
19 print(f"Raiz de x = sin(x): {raiz:.10f}")
20
21 plt.figure(figsize=(10, 6))
22 plt.semilogy(np.arange(len(hist)), np.abs(np.array(hist) - raiz), 'o-')
23 plt.xlabel('Iteracion')
24 plt.ylabel('Error absoluto')
25 plt.title('Punto fijo para x = sin(x)')
26 plt.grid(True)
27 plt.show()

```

Listing 69: Punto fijo para $x = \sin(x)$ **C6.**

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return (x - 1)**2
6
7 def df(x):
8     return 2*(x - 1)
9
10 def newton_estandar(f, df, x0, tol=1e-12):
11     x = x0
12     hist = [x]
13     for i in range(100):
14         x_new = x - f(x) / df(x)
15         hist.append(x_new)
16         if abs(x_new - x) < tol:
17             break
18         x = x_new
19     return hist
20
21 def newton_modificado(f, df, x0, m, tol=1e-12):
22     x = x0
23     hist = [x]
24     for i in range(100):
25         x_new = x - m * f(x) / df(x)
26         hist.append(x_new)
27         if abs(x_new - x) < tol:
28             break
29         x = x_new
30     return hist
31
32 hist_est = newton_estandar(f, df, 2, tol=1e-12)
33 hist_mod = newton_modificado(f, df, 2, 2, tol=1e-12)
34
35 plt.figure(figsize=(10, 6))
36 plt.semilogy(np.arange(len(hist_est)), np.abs(np.array(hist_est) - 1), 'o-',
37              label='Newton estandar')
38 plt.semilogy(np.arange(len(hist_mod)), np.abs(np.array(hist_mod) - 1), 's-',
39              label='Newton modificado (m=2)')
40 plt.xlabel('Iteracion')
41 plt.ylabel('Error absoluto')
42 plt.title('Newton para raiz multiple (x-1)^2')
43 plt.legend()
44 plt.grid(True)
45 plt.show()

```

Listing 70: Newton con raiz multiple

C7. Ver el codigo de la Seccion .

C8.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def newton_numerico(f, x0, h=1e-8, tol=1e-10):
5     x = x0
6     hist = [x]
7     for i in range(100):
8         df = (f(x + h) - f(x)) / h
9         x_new = x - f(x) / df
10        hist.append(x_new)

```

```

11         if abs(x_new - x) < tol:
12             break
13         x = x_new
14     return x, hist
15
16 def secante(f, x0, x1, tol=1e-10):
17     hist = [x0, x1]
18     for i in range(100):
19         x_new = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
20         hist.append(x_new)
21         if abs(x_new - x1) < tol:
22             break
23         x0, x1 = x1, x_new
24     return x1, hist
25
26 def f(x):
27     return np.exp(-x) - x
28
29 _, hist_n = newton_numerico(f, 0.5, tol=1e-12)
30 _, hist_s = secante(f, 0, 1, tol=1e-12)
31 raiz_exacta = 0.5671432904097838
32
33 plt.figure(figsize=(10, 6))
34 plt.semilogy(np.arange(len(hist_n)), np.abs(np.array(hist_n) - raiz_exacta), 'o',
35              label='Newton con derivada numerica')
36 plt.semilogy(np.arange(len(hist_s)), np.abs(np.array(hist_s) - raiz_exacta), 's',
37              label='Secante')
38 plt.xlabel('Iteracion')
39 plt.ylabel('Error absoluto')
40 plt.title('Newton numerico vs Secante para  $e^{-x} - x$ ')
41 plt.legend()
42 plt.grid(True)
43 plt.show()

```

Listing 71: Secante vs Newton con derivada numerica

C9.

```

1 import numpy as np
2
3 def f(x):
4     return x**3 - x
5
6 def df(x):
7     return 3*x**2 - 1
8
9 def newton(f, df, x0, tol=1e-10):
10     x = x0
11     for i in range(100):
12         x_new = x - f(x) / df(x)
13         if abs(x_new - x) < tol:
14             break
15         x = x_new
16     return x
17
18 puntos = np.linspace(-2, 2, 100)
19 raices = []
20 for x0 in puntos:
21     try:
22         raiz = newton(f, df, x0)
23         raices.append(raiz)
24     except:

```

```

25     raices.append(np.nan)
26
27 # Raices esperadas: -1, 0, 1
28 print("Raices encontradas:", np.unique(np.round(raices[~np.isnan(raices)], 6)))

```

Listing 72: Newton para diferentes puntos iniciales

C10.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def biseccion(f, a, b, tol=1e-10):
5     hist = []
6     while (b - a) / 2 > tol:
7         c = (a + b) / 2
8         hist.append(c)
9         if f(c) == 0:
10             break
11         if f(a) * f(c) < 0:
12             b = c
13         else:
14             a = c
15     return c, hist
16
17 def newton(f, df, x0, tol=1e-10):
18     x = x0
19     hist = [x]
20     for i in range(100):
21         x_new = x - f(x) / df(x)
22         hist.append(x_new)
23         if abs(x_new - x) < tol:
24             break
25         x = x_new
26     return x, hist
27
28 def secante(f, x0, x1, tol=1e-10):
29     hist = [x0, x1]
30     for i in range(100):
31         x_new = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
32         hist.append(x_new)
33         if abs(x_new - x1) < tol:
34             break
35         x0, x1 = x1, x_new
36     return x1, hist
37
38 def punto_fijo(g, x0, tol=1e-10):
39     x = x0
40     hist = [x]
41     for i in range(100):
42         x_new = g(x)
43         hist.append(x_new)
44         if abs(x_new - x) < tol:
45             break
46         x = x_new
47     return x, hist
48
49 def f(x):
50     return x**3 - x - 1
51
52 def df(x):
53     return 3*x**2 - 1

```

```

54
55 def g(x):
56     return (x + 1)**(1/3)
57
58 raiz_exacta = 1.324717957244746
59
60 # Todos los metodos
61 raiz_b, hist_b = biseccion(f, 1, 2, tol=1e-12)
62 raiz_n, hist_n = newton(f, df, 1.5, tol=1e-12)
63 raiz_s, hist_s = secante(f, 1, 2, tol=1e-12)
64 raiz_pf, hist_pf = punto_fijo(g, 1.5, tol=1e-12)
65
66 plt.figure(figsize=(10, 6))
67 plt.semilogy(np.arange(len(hist_b)), np.abs(np.array(hist_b) - raiz_exacta), 'o',
68              label='Biseccion')
69 plt.semilogy(np.arange(len(hist_n)), np.abs(np.array(hist_n) - raiz_exacta), 's',
70              label='Newton')
71 plt.semilogy(np.arange(len(hist_s)), np.abs(np.array(hist_s) - raiz_exacta),
72              label='Secante')
73 plt.semilogy(np.arange(len(hist_pf)), np.abs(np.array(hist_pf) - raiz_exacta),
74              label='Punto fijo')
75 plt.xlabel('Iteracion')
76 plt.ylabel('Error absoluto')
77 plt.title('Comparacion de metodos para x^3 - x - 1 = 0')
78 plt.legend()
79 plt.grid(True)
80 plt.show()

```

Listing 73: Solver general con comparacion de metodos

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por la resolucion de ecuaciones no lineales. Viste que no era tan complicado. Arrancamos con la biseccion que es lenta pero segura, pasamos por Regula Falsi, Newton que es el rey de la convergencia, el punto fijo que es una vision general, y la secante para cuando no tenes derivada. Tambien vimos Newton en varias variables para cuando la vida se pone multidimensional.

Los puntos clave para llevar:

- La biseccion es simple, robusta, pero lenta (convergencia lineal).
- Newton es increiblemente rapido (convergencia cuadratica) pero requiere la derivada y un buen punto inicial.
- La secante es un buen compromiso: no necesita derivada y converge con orden $\approx 1,618$.
- El punto fijo es una idea general que unifica muchos metodos.
- Newton en varias variables es la extension natural, pero requiere resolver un sistema lineal en cada paso.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. La resolucion de ecuaciones no lineales es una herramienta fundamental que vas a usar en todos los problemas de fisica y matematica aplicada. ¡Dale para adelante, que vos podes!

Ecuaciones Diferenciales Ordinarias

Como simular el movimiento del mundo, paso a paso, con Python.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a meterle a uno de los temas mas lindos y utiles del calculo numerico: la resolucion de ecuaciones diferenciales ordinarias (EDOs). Si llegaste hasta aca, seguro ya sabes que las EDOs estan en todos lados: la mecanica de Newton, el crecimiento de poblaciones, los circuitos electricos, la difusion del calor... practicamente toda la fisica se describe con ecuaciones diferenciales.

Cuando curse Matematica 3, recuerdo que el profesor nos dijo: “el 90 porciento de las ecuaciones diferenciales que aparecen en la naturaleza no se pueden resolver analiticamente”. En ese momento me quede pensando en todas las veces que en fisica habiamos resuelto el oscilador armonico o el problema de Kepler, y me di cuenta de que esos eran casos especiales. La mayoria de los problemas reales requieren simulaciones numericas.

En este capitulo vamos a cubrir:

- El metodo de Euler: el mas simple de todos, pero que ya te da una idea de como funciona.
- Los metodos de Taylor: cuando queres mas precision y te animas a calcular derivadas.
- Los metodos de Runge-Kutta: la familia de metodos mas usada, con RK4 como el rey.
- El analisis de error: ¿cuanto nos equivocamos y por que?
- Los metodos multipaso: cuando queres usar informacion de pasos anteriores.
- La estabilidad: porque algunos metodos explotan y otros no.
- Los problemas de contorno: cuando las condiciones estan en los bordes.
- Los metodos espectrales: cuando queres precision exponencial.

La idea es que vos puedas implementar todo esto en Python, entender por que funciona, y saber que metodo usar en cada situacion. Asi que agarra un cafe, ponete comodo, y vamos a darle.

¿Que es una ecuacion diferencial ordinaria?

Una EDO es una ecuacion que relaciona una funcion desconocida $x(t)$ con sus derivadas. La forma mas general es:

$$F(t, x, x', x'', \dots, x^{(n)}) = 0$$

Pero nosotros vamos a trabajar con la forma normal:

$$x'(t) = f(t, x(t))$$

con una condicion inicial $x(t_0) = x_0$. Esto se llama **problema de valores iniciales** (PVI).

Traducción Matemática de la EDO La EDO se lee en criollo: "la velocidad de cambio de x en el instante t depende del tiempo y del valor actual de x ". Pensa en esto como si estuvieras describiendo el movimiento de una partícula: la aceleración (o la velocidad) depende de donde está y cuando. Si sabes donde empieza y como se mueve, puedes predecir donde va a estar después.

¿Por que necesitamos metodos numericos?

La respuesta es simple: la mayoría de las EDOs no tienen soluciones analíticas expresables en términos de funciones elementales. Pensa en $x' = x^2 + \sin(x)$. No hay una fórmula cerrada para eso. Así que tenemos que aproximar.

El metodo de Euler

El metodo de Euler es el mas simple de todos. La idea es aproximar la derivada por un cociente incremental:

$$x'(t_n) \approx \frac{x(t_{n+1}) - x(t_n)}{h}$$

donde h es el paso de tiempo. Entonces:

$$x_{n+1} = x_n + hf(t_n, x_n)$$

Traducción Matemática de Euler Euler se lee en criollo: "si sabes donde estas y la velocidad a la que te moves, puedes estimar donde vas a estar después de un pequeño intervalo de tiempo". Es como si estuvieras caminando en línea recta siguiendo la dirección que tienes en cada momento. Es simple, pero si el paso es muy grande, te desvías.

Grafico: metodo de Euler

Vamos a resolver $x' = x$ con $x(0) = 1$ usando Euler.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def euler(f, x0, t0, tf, h):
5     """Metodo de Euler para x' = f(t, x)."""
6     n = int((tf - t0) / h)
7     t = np.zeros(n+1)
8     x = np.zeros(n+1)
9     t[0] = t0
10    x[0] = x0
11
12    for i in range(n):
13        x[i+1] = x[i] + h * f(t[i], x[i])
14        t[i+1] = t[i] + h
15
16    return t, x
17
18 def f(t, x):
19     return x
20
21 # Solucion exacta: x(t) = e^t
22 t_exact = np.linspace(0, 2, 100)
23 x_exact = np.exp(t_exact)
24

```



```

25 # Euler con diferentes pasos
26 for h in [0.5, 0.25, 0.1]:
27     t, x = euler(f, 1, 0, 2, h)
28     plt.plot(t, x, 'o-', label=f'h = {h}')
29
30 plt.plot(t_exact, x_exact, 'k-', label='Solucion exacta')
31 plt.xlabel('t')
32 plt.ylabel('x(t)')
33 plt.title('Metodo de Euler para x\' = x')
34 plt.legend()
35 plt.grid(True)
36 plt.show()

```

Listing 74: Metodo de Euler para $x' = x$

En el grafico, ves que Euler se acerca a la solucion exacta a medida que h se hace mas chico. Pero para $h = 0,5$, el error es considerable.

Metodos de Taylor de orden superior

Si queremos mas precision, podemos usar mas terminos del desarrollo de Taylor:

$$x(t+h) = x(t) + hx'(t) + \frac{h^2}{2}x''(t) + \frac{h^3}{6}x'''(t) + \dots$$

Taylor de orden 2

$$x_{n+1} = x_n + hf(t_n, x_n) + \frac{h^2}{2} (f_t(t_n, x_n) + f_x(t_n, x_n)f(t_n, x_n))$$

Traduccion Matematica de Taylor de orden 2 Taylor de orden 2 se lee en criollo: .^{en} lugar de solo usar la pendiente (como Euler), tambien usamos la curvatura (la segunda derivada)". Esto nos da una mejor aproximacion porque tenemos en cuenta como cambia la pendiente.

Grafico: comparacion de metodos

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def euler(f, x0, t0, tf, h):
5     n = int((tf - t0) / h)
6     t = np.zeros(n+1)
7     x = np.zeros(n+1)
8     t[0] = t0
9     x[0] = x0
10    for i in range(n):
11        x[i+1] = x[i] + h * f(t[i], x[i])
12        t[i+1] = t[i] + h
13    return t, x
14
15 def taylor2(f, ft, fx, x0, t0, tf, h):
16     """Taylor de orden 2 para x' = f(t, x)."""
17     n = int((tf - t0) / h)
18     t = np.zeros(n+1)
19     x = np.zeros(n+1)
20     t[0] = t0

```

```

21     x[0] = x0
22     for i in range(n):
23         x[i+1] = x[i] + h * f(t[i], x[i]) + (h**2/2) * (ft(t[i], x[i]) + fx(t[i], x[i]) * f(t[i], x[i]))
24         t[i+1] = t[i] + h
25     return t, x
26
27 def f(t, x):
28     return x
29
30 def ft(t, x):
31     return 0
32
33 def fx(t, x):
34     return 1
35
36 h = 0.2
37 t_euler, x_euler = euler(f, 1, 0, 2, h)
38 t_taylor, x_taylor = taylor2(f, ft, fx, 1, 0, 2, h)
39
40 t_exact = np.linspace(0, 2, 100)
41 x_exact = np.exp(t_exact)
42
43 plt.figure(figsize=(10, 6))
44 plt.plot(t_exact, x_exact, 'k-', label='Exacta')
45 plt.plot(t_euler, x_euler, 'o-', label='Euler')
46 plt.plot(t_taylor, x_taylor, 's-', label='Taylor orden 2')
47 plt.xlabel('t')
48 plt.ylabel('x(t)')
49 plt.title('Comparacion: Euler vs Taylor de orden 2 para x\'' = x')
50 plt.legend()
51 plt.grid(True)
52 plt.show()

```

Listing 75: Comparacion: Euler vs Taylor de orden 2

Metodos de Runge-Kutta

Los metodos de Runge-Kutta son la familia de metodos mas usada en la practica. La idea es aproximar la solucion usando evaluaciones de f en puntos intermedios, sin tener que calcular derivadas de f .

Runge-Kutta de orden 2 (RK2)

Hay varias versiones de RK2. La mas comun es el **metodo de Heun**:

$$x_{n+1} = x_n + \frac{h}{2} (k_1 + k_2)$$

donde:

$$k_1 = f(t_n, x_n), \quad k_2 = f(t_n + h, x_n + hk_1)$$

Traduccion Matematica de RK2 RK2 se lee en criollo: "primero calculas la pendiente en el punto actual (k_1), luego das un paso de prueba con esa pendiente y calculas la pendiente en ese punto de prueba (k_2), y finalmente tomas el promedio de ambas pendientes". Es como si

estuvieras mirando hacia adelante para ver como cambia el terreno y ajustaras tu direccion en consecuencia.

Runge-Kutta de orden 4 (RK4)

El metodo RK4 es el mas famoso de todos:

$$x_{n+1} = x_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

donde:

$$\begin{aligned} k_1 &= f(t_n, x_n) \\ k_2 &= f(t_n + h/2, x_n + hk_1/2) \\ k_3 &= f(t_n + h/2, x_n + hk_2/2) \\ k_4 &= f(t_n + h, x_n + hk_3) \end{aligned}$$

Traduccion Matematica de RK4 RK4 se lee en criollo: "haces cuatro evaluaciones de la pendiente: al principio (k_1), en el punto medio con la pendiente inicial (k_2), en el punto medio con la pendiente de k_2 (k_3), y al final con la pendiente de k_3 (k_4). Luego combinas todas con pesos: 1, 2, 2, 1". Es como si estuvieras tomando varias muestras de la pendiente a lo largo del paso y haciendo un promedio ponderado para obtener una estimacion mucho mas precisa.

Grafico: comparacion de metodos de Runge-Kutta

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def rk2(f, x0, t0, tf, h):
5     """Metodo de Heun (RK2)."""
6     n = int((tf - t0) / h)
7     t = np.zeros(n+1)
8     x = np.zeros(n+1)
9     t[0] = t0
10    x[0] = x0
11    for i in range(n):
12        k1 = f(t[i], x[i])
13        k2 = f(t[i] + h, x[i] + h * k1)
14        x[i+1] = x[i] + h/2 * (k1 + k2)
15        t[i+1] = t[i] + h
16    return t, x
17
18 def rk4(f, x0, t0, tf, h):
19     """Metodo de Runge-Kutta de orden 4."""
20     n = int((tf - t0) / h)
21     t = np.zeros(n+1)
22     x = np.zeros(n+1)
23     t[0] = t0
24     x[0] = x0
25     for i in range(n):
26         k1 = f(t[i], x[i])
27         k2 = f(t[i] + h/2, x[i] + h*k1/2)
28         k3 = f(t[i] + h/2, x[i] + h*k2/2)
29         k4 = f(t[i] + h, x[i] + h*k3)
30         x[i+1] = x[i] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
31         t[i+1] = t[i] + h

```

```

32     return t, x
33
34 def f(t, x):
35     return x
36
37 h = 0.2
38 t_euler, x_euler = euler(f, 1, 0, 2, h)
39 t_rk2, x_rk2 = rk2(f, 1, 0, 2, h)
40 t_rk4, x_rk4 = rk4(f, 1, 0, 2, h)
41
42 t_exact = np.linspace(0, 2, 100)
43 x_exact = np.exp(t_exact)
44
45 plt.figure(figsize=(10, 6))
46 plt.plot(t_exact, x_exact, 'k-', label='Exacta')
47 plt.plot(t_euler, x_euler, 'o-', label='Euler')
48 plt.plot(t_rk2, x_rk2, 's-', label='RK2')
49 plt.plot(t_rk4, x_rk4, '^--', label='RK4')
50 plt.xlabel('t')
51 plt.ylabel('x(t)')
52 plt.title('Comparacion: Euler vs RK2 vs RK4 para x\' = x')
53 plt.legend()
54 plt.grid(True)
55 plt.show()

```

Listing 76: Comparacion: Euler vs RK2 vs RK4

Analisis del error y convergencia

Para entender por que unos metodos son mejores que otros, necesitamos analizar el error.

Error de truncamiento local

El error de truncamiento local es el error que se comete en un solo paso, asumiendo que el valor anterior es exacto:

$$\tau_n = \frac{x(t_{n+1}) - x(t_n)}{h} - \Phi(t_n, x(t_n), h)$$

donde Φ es la funcion de incremento del metodo.

Traduccion Matematica del error de truncamiento El error de truncamiento se lee en criollo: ^{es} la diferencia entre lo que realmente deberia pasar (la solucion exacta) y lo que predice el metodo en un solo paso, suponiendo que empezaste en el punto correcto". Si este error es $O(h^{p+1})$, el metodo es de orden p .

Error global

El error global es la diferencia entre la solucion numerica y la exacta en el tiempo final:

$$e_n = x(t_n) - x_n$$

Teorema de convergencia

Si el error de truncamiento local es $O(h^{p+1})$ y la funcion de incremento es Lipschitz, entonces el error global es $O(h^p)$.

Grafico: orden de convergencia

Vamos a verificar el orden de convergencia de los metodos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def error_metodo(metodo, f, x0, t0, tf, h):
5     """Calcula el error global de un metodo en tf."""
6     t, x = metodo(f, x0, t0, tf, h)
7     x_exacta = np.exp(tf)
8     return abs(x[-1] - x_exacta)
9
10 h_values = np.array([0.5, 0.25, 0.125, 0.0625, 0.03125])
11 errores_euler = []
12 errores_rk2 = []
13 errores_rk4 = []
14
15 for h in h_values:
16     errores_euler.append(error_metodo(euler, f, 1, 0, 2, h))
17     errores_rk2.append(error_metodo(rk2, f, 1, 0, 2, h))
18     errores_rk4.append(error_metodo(rk4, f, 1, 0, 2, h))
19
20 plt.figure(figsize=(10, 6))
21 plt.loglog(h_values, errores_euler, 'o-', label='Euler (orden 1)')
22 plt.loglog(h_values, errores_rk2, 's-', label='RK2 (orden 2)')
23 plt.loglog(h_values, errores_rk4, '^-', label='RK4 (orden 4)')
24 plt.loglog(h_values, h_values**1 * 0.5, 'k--', label='O(h)')
25 plt.loglog(h_values, h_values**2 * 0.5, 'k-.', label='O(h^2)')
26 plt.loglog(h_values, h_values**4 * 0.5, 'k:', label='O(h^4)')
27 plt.xlabel('Paso h')
28 plt.ylabel('Error global en t=2')
29 plt.title('Orden de convergencia de los metodos')
30 plt.legend()
31 plt.grid(True)
32 plt.show()

```

Listing 77: Orden de convergencia de Euler, RK2 y RK4

En el grafico, ves que Euler tiene pendiente 1 (error $O(h)$), RK2 tiene pendiente 2, y RK4 tiene pendiente 4. Esto confirma los ordenes de convergencia.

Metodos multipaso

Los metodos multipaso usan informacion de varios pasos anteriores para calcular el siguiente. La forma general es:

$$x_{n+k} = \sum_{j=0}^{k-1} \alpha_j x_{n+j} + h \sum_{j=0}^k \beta_j f(t_{n+j}, x_{n+j})$$

Metodos de Adams-Bashforth (explicitos)

Los metodos de Adams-Bashforth usan informacion de pasos anteriores para predecir el siguiente. Por ejemplo, el de 3 pasos:

$$x_{n+3} = x_{n+2} + \frac{h}{12}(23f_{n+2} - 16f_{n+1} + 5f_n)$$

Metodos de Adams-Moulton (implicitos)

Los metodos de Adams-Moulton son implicitos porque el termino f_{n+k} aparece en ambos lados. Por ejemplo, el de 3 pasos:

$$x_{n+3} = x_{n+2} + \frac{h}{12}(5f_{n+3} + 8f_{n+2} - f_{n+1})$$

Traduccion Matematica de los metodos multipaso Los metodos multipaso se leen en criollo: ^{en} lugar de solo usar el punto actual para calcular el siguiente, usamos varios puntos anteriores para hacer una mejor prediccion”. Es como si estuvieras mirando hacia atras para ver la tendencia y predecir el futuro. Los metodos explícitos son mas faciles de implementar, pero los implícitos son mas estables.

Estabilidad de los metodos multipaso

Para que un metodo multipaso sea convergente, necesita:

1. **Consistencia:** el error de truncamiento tiende a cero.
2. **Condicion de la raiz:** todas las raices del polinomio caracteristico estan dentro del circulo unitario, y las del borde son simples.

Grafico: metodo de Adams-Bashforth

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def adams_bashforth_3(f, x0, t0, tf, h):
5     """Adams-Bashforth de 3 pasos."""
6     n = int((tf - t0) / h)
7     t = np.zeros(n+1)
8     x = np.zeros(n+1)
9     t[0] = t0
10    x[0] = x0
11
12    # Usamos RK4 para los primeros pasos
13    for i in range(3):
14        k1 = f(t[i], x[i])
15        k2 = f(t[i] + h/2, x[i] + h*k1/2)
16        k3 = f(t[i] + h/2, x[i] + h*k2/2)
17        k4 = f(t[i] + h, x[i] + h*k3)
18        x[i+1] = x[i] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
19        t[i+1] = t[i] + h
20
21    # Metodo de Adams-Bashforth de 3 pasos
22    for i in range(3, n):
23        f_n = f(t[i], x[i])
24        f_n1 = f(t[i-1], x[i-1])
25        f_n2 = f(t[i-2], x[i-2])
26        x[i+1] = x[i] + h/12 * (23*f_n - 16*f_n1 + 5*f_n2)
27        t[i+1] = t[i] + h
28
29    return t, x
30
31 def f(t, x):
32     return x

```

```

33
34 h = 0.1
35 t_ab, x_ab = adams_bashforth_3(f, 1, 0, 2, h)
36
37 t_exact = np.linspace(0, 2, 100)
38 x_exact = np.exp(t_exact)
39
40 plt.figure(figsize=(10, 6))
41 plt.plot(t_exact, x_exact, 'k-', label='Exacta')
42 plt.plot(t_ab, x_ab, 'o-', label='Adams-Bashforth 3 pasos')
43 plt.xlabel('t')
44 plt.ylabel('x(t)')
45 plt.title('Metodo de Adams-Bashforth de 3 pasos para x\'' = x')
46 plt.legend()
47 plt.grid(True)
48 plt.show()

```

Listing 78: Metodo de Adams-Bashforth de 3 pasos

Estabilidad y problemas rigidos

La estabilidad es un concepto fundamental. Un metodo es **A-estable** si es estable para cualquier paso $h > 0$ cuando se aplica a la ecuacion de prueba $x' = \lambda x$ con $\text{Re}(\lambda) < 0$.

La ecuacion de prueba de Dahlquist

$$x' = \lambda x, \quad x(0) = 1$$

La solucion exacta es $x(t) = e^{\lambda t}$. Si $\text{Re}(\lambda) < 0$, la solucion decae a cero.

Region de estabilidad

La region de estabilidad de un metodo es el conjunto de $z = h\lambda$ para los cuales la solucion numerica no crece.

Grafico: region de estabilidad de Euler explicito e implicito

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def region_estabilidad_euler_explicito(N=200):
5     """Region de estabilidad de Euler explicito: |1+z| <= 1."""
6     x = np.linspace(-3, 3, N)
7     y = np.linspace(-3, 3, N)
8     X, Y = np.meshgrid(x, y)
9     Z = X + 1j*Y
10    mask = np.abs(1 + Z) <= 1
11    return X, Y, mask
12
13 def region_estabilidad_euler_implicito(N=200):
14     """Region de estabilidad de Euler implicito: |1/(1-z)| <= 1."""
15     x = np.linspace(-3, 3, N)
16     y = np.linspace(-3, 3, N)
17     X, Y = np.meshgrid(x, y)
18     Z = X + 1j*Y
19     mask = np.abs(1/(1 - Z)) <= 1

```

```

20     return X, Y, mask
21
22 plt.figure(figsize=(12, 5))
23
24 plt.subplot(1, 2, 1)
25 X, Y, mask = region_estabilidad_euler_explicito()
26 plt.contourf(X, Y, mask, levels=[0, 0.5, 1], colors=['white', 'blue'], alpha
27              =0.5)
28 plt.axhline(0, color='black', linewidth=0.5)
29 plt.axvline(0, color='black', linewidth=0.5)
30 plt.xlabel('Re(z)')
31 plt.ylabel('Im(z)')
32 plt.title('Euler explicito: |1+z| <= 1')
33 plt.grid(True)
34 plt.axis('equal')
35
36 plt.subplot(1, 2, 2)
37 X, Y, mask = region_estabilidad_euler_implicito()
38 plt.contourf(X, Y, mask, levels=[0, 0.5, 1], colors=['white', 'blue'], alpha
39              =0.5)
40 plt.axhline(0, color='black', linewidth=0.5)
41 plt.axvline(0, color='black', linewidth=0.5)
42 plt.xlabel('Re(z)')
43 plt.ylabel('Im(z)')
44 plt.title('Euler implicito: |1/(1-z)| <= 1')
45 plt.grid(True)
46 plt.axis('equal')
47
48 plt.tight_layout()
49 plt.show()

```

Listing 79: Region de estabilidad de Euler

Euler explicito es estable solo para h pequeno (el circulo con centro en -1 y radio 1). Euler implicito es estable para todo el semiplano izquierdo (es A-estable).

Problemas de contorno

Los problemas de contorno son diferentes: en lugar de condiciones iniciales, tenemos condiciones en los bordes del intervalo.

Ejemplo: ecuacion de Poisson 1D

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0$$

Discretizacion con diferencias finitas

Usamos la aproximacion:

$$u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}$$

Esto nos da un sistema lineal tridiagonal.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def poisson_1d(f, n):

```



```

5      """Resuelve -u''(x) = f(x) en (0,1) con u(0)=u(1)=0 usando diferencias
      finitas."""
6      h = 1 / (n + 1)
7      x = np.linspace(0, 1, n + 2)
8
9      # Matriz tridiagonal
10     A = np.zeros((n, n))
11     for i in range(n):
12         A[i, i] = 2 / h**2
13         if i > 0:
14             A[i, i-1] = -1 / h**2
15         if i < n-1:
16             A[i, i+1] = -1 / h**2
17
18     b = f(x[1:-1])
19     u_int = np.linalg.solve(A, b)
20     u = np.zeros(n+2)
21     u[1:-1] = u_int
22     return x, u
23
24 def f(x):
25     return np.pi**2 * np.sin(np.pi * x)
26
27 x, u = poisson_1d(f, 20)
28
29 plt.figure(figsize=(10, 6))
30 plt.plot(x, u, 'o-', label='Solucion numerica')
31 plt.plot(x, np.sin(np.pi * x), 'k-', label='Solucion exacta')
32 plt.xlabel('x')
33 plt.ylabel('u(x)')
34 plt.title('Ecuacion de Poisson 1D: -u\'' = pi^2 sin(pi x)')
35 plt.legend()
36 plt.grid(True)
37 plt.show()

```

Listing 80: Ecuacion de Poisson 1D con diferencias finitas

Metodos espectrales

Los metodos espectrales usan aproximaciones globales (series de Fourier o Chebyshev) para resolver EDOs. Dan precision exponencial para funciones suaves.

Metodo espectral de Chebyshev

Se usa una base de polinomios de Chebyshev para representar la solucion. La derivada se calcula en el espacio de coeficientes.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def chebyshev_spectral_poisson(f, N):
5     """Resuelve -u''(x) = f(x) en (-1,1) con u(-1)=u(1)=0 usando metodo
      espectral de Chebyshev."""
6     # Nodos de Chebyshev (incluyendo bordes)
7     x = np.cos(np.linspace(0, np.pi, N+1))
8
9     # Matriz de derivacion espectral de Chebyshev
10    D = np.zeros((N+1, N+1))

```

```

11     for i in range(N+1):
12         for j in range(N+1):
13             if i != j:
14                 D[i, j] = (-1)**(i+j) / (x[i] - x[j])
15     for i in range(N+1):
16         D[i, i] = -np.sum(D[i, :])
17
18     # Derivada segunda
19     D2 = D @ D
20
21     # Quitamos las condiciones de borde
22     D2_int = D2[1:-1, 1:-1]
23     x_int = x[1:-1]
24
25     b = f(x_int)
26     u_int = np.linalg.solve(D2_int, b)
27
28     u = np.zeros(N+1)
29     u[1:-1] = u_int
30     return x, u
31
32 def f(x):
33     return np.exp(x) * (1 - x**2) * (-2) # Solucion exacta: u(x) = exp(x)*(1-x
34     ^2)
35
36 N = 20
37 x, u = chebyshev_spectral_poisson(f, N)
38
39 x_exact = np.linspace(-1, 1, 100)
40 u_exact = np.exp(x_exact) * (1 - x_exact**2)
41
42 plt.figure(figsize=(10, 6))
43 plt.plot(x, u, 'ro', label='Solucion espectral')
44 plt.plot(x_exact, u_exact, 'k-', label='Solucion exacta')
45 plt.xlabel('x')
46 plt.ylabel('u(x)')
47 plt.title('Metodo espectral de Chebyshev para -u\'\' = f(x)')
48 plt.legend()
49 plt.grid(True)
50 plt.show()

```

Listing 81: Metodo espectral de Chebyshev para Poisson

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Deducion de Euler:** Deducir el metodo de Euler a partir del desarrollo de Taylor. ¿Cual es el error de truncamiento local?
- T2. Convergencia de Euler:** Demuestra que si f es Lipschitz, el metodo de Euler converge con error $O(h)$.
- T3. Taylor de orden 2:** Deducir el metodo de Taylor de orden 2. ¿Cual es el error de truncamiento local?
- T4. Runge-Kutta de orden 2:** Deducir el metodo de Heun (RK2). ¿Por que se llama de orden 2?

- T5. Runge-Kutta de orden 4:** Deduce el metodo RK4. Verifica que tiene orden 4.
- T6. Estabilidad de Euler:** Analiza la estabilidad de Euler explícito e implícito para la ecuación $x' = \lambda x$.
- T7. A-estabilidad:** Demuestra que el metodo de Euler implícito es A-estable.
- T8. Metodos multipaso:** Deduce el metodo de Adams-Bashforth de 2 pasos.
- T9. Condicion de la raiz:** Explica la condicion de la raiz para metodos multipaso y por que es necesaria.
- T10. Metodos espectrales:** Explica por que los metodos espectrales dan convergencia exponencial para funciones suaves.

Parte Computacional (10 ejercicios)

- C1. Euler para una EDO simple:** Resuelve $x' = -x$ con $x(0) = 1$ usando Euler. Compara con la solucion exacta para diferentes h .
- C2. RK4 para un sistema:** Resuelve el sistema $x' = y, y' = -x$ (oscilador armonico) usando RK4. Grafica la trayectoria en el plano de fase.
- C3. Comparacion de metodos:** Compara Euler, RK2 y RK4 para $x' = x^2$ con $x(0) = 1$ en $[0, 0.9]$. ¿Que pasa cerca de $t = 1$?
- C4. Orden de convergencia:** Verifica numericamente el orden de convergencia de RK4 para $x' = \sin(x)$ con $x(0) = 1$.
- C5. Metodo de Adams-Bashforth:** Implementa el metodo de Adams-Bashforth de 2 pasos y compáralo con RK4.
- C6. Problema rígido:** Resuelve $x' = -1000(x - \cos(t))$ con Euler explícito e implícito. ¿Que observas?
- C7. Metodo de punto medio:** Implementa el metodo de punto medio (RK2 con $\alpha = 1/2$) y compáralo con Euler.
- C8. Problema de contorno:** Resuelve $-u'' + u = f(x)$ en $(0, 1)$ con $u(0) = u(1) = 0$ usando diferencias finitas. Prueba con $f(x) = \sin(\pi x)$.
- C9. Metodo espectral:** Usa el metodo espectral de Chebyshev para resolver $-u'' = \cos(\pi x/2)$ en $(-1, 1)$ con condiciones de borde cero.
- C10. Comparacion de metodos para EDOs:** Para $x' = -x + \sin(t)$ en $[0, 10]$, compara Euler, RK4 y Adams-Bashforth. Grafica los errores.

Soluciones

Soluciones Teoricas

T1. De Taylor: $x(t+h) = x(t) + hx'(t) + O(h^2)$. Reemplazando $x'(t) = f(t, x(t))$: $x(t+h) = x(t) + hf(t, x(t)) + O(h^2)$. El error de truncamiento local es $O(h^2)$.

T2. El error global satisface $e_{n+1} = (1+hL)e_n + O(h^2)$. Resolviendo: $e_n \leq \frac{e^{L(t_n-t_0)}-1}{L}O(h) = O(h)$.

T3. Taylor de orden 2: $x(t+h) = x(t) + hx'(t) + \frac{h^2}{2}x''(t) + O(h^3)$. Como $x'' = f_t + f_x f$, queda la formula. El error de truncamiento es $O(h^3)$.

T4. RK2 busca aproximar el desarrollo de Taylor de orden 2 sin calcular derivadas de f . Usando dos evaluaciones de f , se logra orden 2. La formula de Heun es $x_{n+1} = x_n + \frac{h}{2}(k_1 + k_2)$.

T5. RK4 usa cuatro evaluaciones de f y los pesos 1, 2, 2, 1 para cancelar los terminos de error hasta orden $O(h^5)$. El error de truncamiento local es $O(h^5)$.

T6. Para Euler explícito, $x_{n+1} = (1 + h\lambda)x_n$. Estable si $|1 + h\lambda| \leq 1$. Para Euler implícito, $x_{n+1} = \frac{1}{1-h\lambda}x_n$. Estable si $|1/(1-h\lambda)| \leq 1$, que es siempre para $\text{Re}(\lambda) < 0$.

T7. Para Euler implícito, $x_{n+1} = \frac{1}{1-h\lambda}x_n$. Si $\text{Re}(\lambda) < 0$, entonces $|1 - h\lambda| > 1$, así que $|x_{n+1}| < |x_n|$. Es A-estable.

T8. Adams-Bashforth de 2 pasos se obtiene integrando el polinomio interpolante de grado 1 que pasa por (t_{n-1}, f_{n-1}) y (t_n, f_n) . La formula es $x_{n+2} = x_{n+1} + \frac{h}{2}(3f_{n+1} - f_n)$.

T9. La condicion de la raiz dice que todas las raices del polinomio caracteristico $\rho(z)$ deben estar en el circulo unitario, y las del borde deben ser simples. Esto garantiza que los errores no crezcan exponencialmente.

T10. Los metodos espectrales usan bases globales (como polinomios de Chebyshev) que convergen exponencialmente para funciones suaves porque los coeficientes de la serie decaen exponencialmente.

Soluciones Computacionales

C1. Ver el codigo de la Seccion .

C2.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def rk4_sistema(f, x0, t0, tf, h):
5     """RK4 para sistemas."""
6     n = int((tf - t0) / h)
7     t = np.zeros(n+1)
8     X = np.zeros((n+1, len(x0)))
9     t[0] = t0
10    X[0] = x0
11    for i in range(n):
12        k1 = f(t[i], X[i])
13        k2 = f(t[i] + h/2, X[i] + h*k1/2)
14        k3 = f(t[i] + h/2, X[i] + h*k2/2)
15        k4 = f(t[i] + h, X[i] + h*k3)
16        X[i+1] = X[i] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
17        t[i+1] = t[i] + h
18    return t, X
19
20 def f(t, X):
21     x, y = X
22     return np.array([y, -x])
23
24 t, X = rk4_sistema(f, np.array([1, 0]), 0, 10, 0.01)
25
26 plt.figure(figsize=(8, 8))
27 plt.plot(X[:, 0], X[:, 1])
28 plt.xlabel('x')
29 plt.ylabel('y')
30 plt.title('Oscilador armonico: trayectoria en el plano de fase')
31 plt.axis('equal')
32 plt.grid(True)

```

```
33 plt.show()
```

Listing 82: Oscilador armonico con RK4

C3.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def euler(f, x0, t0, tf, h):
5     n = int((tf - t0) / h)
6     t = np.zeros(n+1)
7     x = np.zeros(n+1)
8     t[0] = t0
9     x[0] = x0
10    for i in range(n):
11        x[i+1] = x[i] + h * f(t[i], x[i])
12        t[i+1] = t[i] + h
13    return t, x
14
15 def rk2(f, x0, t0, tf, h):
16     n = int((tf - t0) / h)
17     t = np.zeros(n+1)
18     x = np.zeros(n+1)
19     t[0] = t0
20     x[0] = x0
21     for i in range(n):
22         k1 = f(t[i], x[i])
23         k2 = f(t[i] + h, x[i] + h * k1)
24         x[i+1] = x[i] + h/2 * (k1 + k2)
25         t[i+1] = t[i] + h
26     return t, x
27
28 def rk4(f, x0, t0, tf, h):
29     n = int((tf - t0) / h)
30     t = np.zeros(n+1)
31     x = np.zeros(n+1)
32     t[0] = t0
33     x[0] = x0
34     for i in range(n):
35         k1 = f(t[i], x[i])
36         k2 = f(t[i] + h/2, x[i] + h*k1/2)
37         k3 = f(t[i] + h/2, x[i] + h*k2/2)
38         k4 = f(t[i] + h, x[i] + h*k3)
39         x[i+1] = x[i] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
40         t[i+1] = t[i] + h
41     return t, x
42
43 def f(t, x):
44     return x**2
45
46 h = 0.01
47 t_e, x_e = euler(f, 1, 0, 0.9, h)
48 t_r2, x_r2 = rk2(f, 1, 0, 0.9, h)
49 t_r4, x_r4 = rk4(f, 1, 0, 0.9, h)
50
51 # Solucion exacta:  $x(t) = 1/(1-t)$ 
52 t_exact = np.linspace(0, 0.9, 100)
53 x_exact = 1/(1 - t_exact)
54
55 plt.figure(figsize=(10, 6))
56 plt.plot(t_exact, x_exact, 'k-', label='Exacta')

```

```

57 plt.plot(t_e, x_e, 'o-', label='Euler')
58 plt.plot(t_r2, x_r2, 's-', label='RK2')
59 plt.plot(t_r4, x_r4, '^--', label='RK4')
60 plt.xlabel('t')
61 plt.ylabel('x(t)')
62 plt.title('Comparacion para x\' = x^2 (explota en t=1)')
63 plt.legend()
64 plt.grid(True)
65 plt.show()

```

Listing 83: Comparacion para $x' = x^2$ **C4.**

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def rk4(f, x0, t0, tf, h):
5     n = int((tf - t0) / h)
6     t = np.zeros(n+1)
7     x = np.zeros(n+1)
8     t[0] = t0
9     x[0] = x0
10    for i in range(n):
11        k1 = f(t[i], x[i])
12        k2 = f(t[i] + h/2, x[i] + h*k1/2)
13        k3 = f(t[i] + h/2, x[i] + h*k2/2)
14        k4 = f(t[i] + h, x[i] + h*k3)
15        x[i+1] = x[i] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
16        t[i+1] = t[i] + h
17    return t, x
18
19 def f(t, x):
20     return np.sin(x)
21
22 # Solucion de referencia con h muy chico
23 h_ref = 1e-5
24 _, x_ref = rk4(f, 1, 0, 1, h_ref)
25
26 h_values = np.array([0.1, 0.05, 0.025, 0.0125, 0.00625])
27 errores = []
28
29 for h in h_values:
30     _, x = rk4(f, 1, 0, 1, h)
31     errores.append(abs(x[-1] - x_ref[-1]))
32
33 plt.figure(figsize=(10, 6))
34 plt.loglog(h_values, errores, 'o-')
35 plt.loglog(h_values, h_values**4 * 0.1, 'k--', label='O(h^4)')
36 plt.xlabel('Paso h')
37 plt.ylabel('Error en t=1')
38 plt.title('Orden de convergencia de RK4 para x\' = sin(x)')
39 plt.legend()
40 plt.grid(True)
41 plt.show()

```

Listing 84: Orden de RK4 para $x' = \sin(x)$ **C5.**

```

1 import numpy as np
2 import matplotlib.pyplot as plt

```

```

3
4 def adams_bashforth_2(f, x0, t0, tf, h):
5     """Adams-Bashforth de 2 pasos."""
6     n = int((tf - t0) / h)
7     t = np.zeros(n+1)
8     x = np.zeros(n+1)
9     t[0] = t0
10    x[0] = x0
11
12    # Usamos RK4 para el primer paso
13    k1 = f(t[0], x[0])
14    k2 = f(t[0] + h/2, x[0] + h*k1/2)
15    k3 = f(t[0] + h/2, x[0] + h*k2/2)
16    k4 = f(t[0] + h, x[0] + h*k3)
17    x[1] = x[0] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
18    t[1] = t[0] + h
19
20    # Adams-Bashforth de 2 pasos
21    for i in range(1, n):
22        f_n = f(t[i], x[i])
23        f_n1 = f(t[i-1], x[i-1])
24        x[i+1] = x[i] + h/2 * (3*f_n - f_n1)
25        t[i+1] = t[i] + h
26
27    return t, x
28
29 def f(t, x):
30     return x
31
32 h = 0.05
33 t_ab, x_ab = adams_bashforth_2(f, 1, 0, 2, h)
34 t_rk4, x_rk4 = rk4(f, 1, 0, 2, h)
35
36 t_exact = np.linspace(0, 2, 100)
37 x_exact = np.exp(t_exact)
38
39 plt.figure(figsize=(10, 6))
40 plt.plot(t_exact, x_exact, 'k-', label='Exacta')
41 plt.plot(t_ab, x_ab, 'o-', label='Adams-Bashforth 2 pasos')
42 plt.plot(t_rk4, x_rk4, 's-', label='RK4')
43 plt.xlabel('t')
44 plt.ylabel('x(t)')
45 plt.title('Comparacion: Adams-Bashforth 2 pasos vs RK4')
46 plt.legend()
47 plt.grid(True)
48 plt.show()

```

Listing 85: Adams-Bashforth de 2 pasos

C6.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def euler_implicit(f, x0, t0, tf, h):
5     """Euler implicitito:  $x_{n+1} = x_n + h * f(t_{n+1}, x_{n+1})$ ."""
6     n = int((tf - t0) / h)
7     t = np.zeros(n+1)
8     x = np.zeros(n+1)
9     t[0] = t0
10    x[0] = x0
11    for i in range(n):

```

```

12     # Para este problema particular, resolvemos analiticamente
13     #  $x_{n+1} = x_n + h * (-1000 * (x_{n+1} - \cos(t_{n+1})))$ 
14     #  $x_{n+1} * (1 + 1000*h) = x_n + 1000*h * \cos(t_{n+1})$ 
15     t_next = t[i] + h
16     x[i+1] = (x[i] + 1000*h * np.cos(t_next)) / (1 + 1000*h)
17     t[i+1] = t_next
18     return t, x
19
20 def f(t, x):
21     return -1000 * (x - np.cos(t))
22
23 h = 0.01
24 t_e, x_e = euler(f, 0, 0, 2, h)
25 t_ei, x_ei = euler_implicito(f, 0, 0, 2, h)
26
27 # Solucion exacta (aproximada con RK4 con h muy chico)
28 _, x_ref = rk4(f, 0, 0, 2, 1e-5)
29
30 plt.figure(figsize=(10, 6))
31 plt.plot(t_e, x_e, 'o-', label='Euler explicito')
32 plt.plot(t_ei, x_ei, 's-', label='Euler implicito')
33 plt.plot(t_e, x_ref[:len(t_e)], 'k-', label='Referencia (RK4)')
34 plt.xlabel('t')
35 plt.ylabel('x(t)')
36 plt.title('Problema rigido:  $x' = -1000(x - \cos(t))$ ')
37 plt.legend()
38 plt.grid(True)
39 plt.show()

```

Listing 86: Problema rigido con Euler

C7.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def punto_medio(f, x0, t0, tf, h):
5     """Metodo de punto medio (RK2 con alpha=1/2)."""
6     n = int((tf - t0) / h)
7     t = np.zeros(n+1)
8     x = np.zeros(n+1)
9     t[0] = t0
10    x[0] = x0
11    for i in range(n):
12        k1 = f(t[i], x[i])
13        k2 = f(t[i] + h/2, x[i] + h*k1/2)
14        x[i+1] = x[i] + h * k2
15        t[i+1] = t[i] + h
16    return t, x
17
18 def f(t, x):
19     return x
20
21 h = 0.1
22 t_pm, x_pm = punto_medio(f, 1, 0, 2, h)
23 t_e, x_e = euler(f, 1, 0, 2, h)
24
25 t_exact = np.linspace(0, 2, 100)
26 x_exact = np.exp(t_exact)
27
28 plt.figure(figsize=(10, 6))
29 plt.plot(t_exact, x_exact, 'k-', label='Exacta')

```



```

30 plt.plot(t_e, x_e, 'o-', label='Euler')
31 plt.plot(t_pm, x_pm, 's-', label='Punto medio')
32 plt.xlabel('t')
33 plt.ylabel('x(t)')
34 plt.title('Comparacion: Euler vs Punto Medio')
35 plt.legend()
36 plt.grid(True)
37 plt.show()

```

Listing 87: Metodo de punto medio

C8.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def poisson_reaccion(f, n):
5     """Resuelve -u'' + u = f(x) en (0,1) con u(0)=u(1)=0."""
6     h = 1 / (n + 1)
7     x = np.linspace(0, 1, n + 2)
8
9     A = np.zeros((n, n))
10    for i in range(n):
11        A[i, i] = 2 / h**2 + 1
12        if i > 0:
13            A[i, i-1] = -1 / h**2
14        if i < n-1:
15            A[i, i+1] = -1 / h**2
16
17    b = f(x[1:-1])
18    u_int = np.linalg.solve(A, b)
19    u = np.zeros(n+2)
20    u[1:-1] = u_int
21    return x, u
22
23 def f(x):
24     return (1 + np.pi**2) * np.sin(np.pi * x)
25
26 x, u = poisson_reaccion(f, 20)
27
28 plt.figure(figsize=(10, 6))
29 plt.plot(x, u, 'o-', label='Solucion numerica')
30 plt.plot(x, np.sin(np.pi * x), 'k-', label='Solucion exacta')
31 plt.xlabel('x')
32 plt.ylabel('u(x)')
33 plt.title('-u\'' + u = (1+pi^2) sin(pi x)')
34 plt.legend()
35 plt.grid(True)
36 plt.show()

```

Listing 88: Problema de contorno con termino de reaccion

C9. Ver el codigo de la Seccion .**C10.**

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def adams_bashforth_2(f, x0, t0, tf, h):
5     n = int((tf - t0) / h)
6     t = np.zeros(n+1)
7     x = np.zeros(n+1)

```

```

8     t[0] = t0
9     x[0] = x0
10    # RK4 para el primer paso
11    k1 = f(t[0], x[0])
12    k2 = f(t[0] + h/2, x[0] + h*k1/2)
13    k3 = f(t[0] + h/2, x[0] + h*k2/2)
14    k4 = f(t[0] + h, x[0] + h*k3)
15    x[1] = x[0] + h/6 * (k1 + 2*k2 + 2*k3 + k4)
16    t[1] = t[0] + h
17    for i in range(1, n):
18        f_n = f(t[i], x[i])
19        f_n1 = f(t[i-1], x[i-1])
20        x[i+1] = x[i] + h/2 * (3*f_n - f_n1)
21        t[i+1] = t[i] + h
22    return t, x
23
24 def f(t, x):
25     return -x + np.sin(t)
26
27 # Solucion exacta:  $x(t) = 0.5*(\sin(t) - \cos(t)) + 0.5*e^{-t}$ 
28 def sol_exacta(t):
29     return 0.5 * (np.sin(t) - np.cos(t)) + 0.5 * np.exp(-t)
30
31 h = 0.05
32 t_e, x_e = euler(f, 0, 0, 10, h)
33 t_rk4, x_rk4 = rk4(f, 0, 0, 10, h)
34 t_ab, x_ab = adams_bashforth_2(f, 0, 0, 10, h)
35
36 t_exact = np.linspace(0, 10, 100)
37 x_exact = sol_exacta(t_exact)
38
39 plt.figure(figsize=(10, 6))
40 plt.plot(t_exact, x_exact, 'k-', label='Exacta')
41 plt.plot(t_e, x_e, 'o-', label='Euler', alpha=0.5)
42 plt.plot(t_rk4, x_rk4, 's-', label='RK4', alpha=0.5)
43 plt.plot(t_ab, x_ab, '^--', label='Adams-Bashforth 2', alpha=0.5)
44 plt.xlabel('t')
45 plt.ylabel('x(t)')
46 plt.title('x\'' = -x + sin(t)')
47 plt.legend()
48 plt.grid(True)
49 plt.show()
50
51 # Errores
52 print(f"Error Euler: {abs(x_e[-1] - sol_exacta(10)):.2e}")
53 print(f"Error RK4: {abs(x_rk4[-1] - sol_exacta(10)):.2e}")
54 print(f"Error AB2: {abs(x_ab[-1] - sol_exacta(10)):.2e}")

```

Listing 89: Comparacion de metodos para una EDO lineal no homogenea

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por las ecuaciones diferenciales ordinarias. Viste que no era tan complicado. Arrancamos con Euler, pasamos por Taylor, RK2 y RK4, los metodos multipaso, la estabilidad, los problemas de contorno y los metodos espectrales. Todo esta conectado: la resolucion numerica de EDOs es una de las herramientas mas importantes de la ciencia computacional.

Los puntos clave para llevar:

- Euler es simple pero tiene error $O(h)$ y puede ser inestable.
- Los metodos de Taylor son mas precisos pero requieren derivadas de f .
- RK4 es el metodo de uso general: error $O(h^4)$ y buena estabilidad.
- Los metodos multipaso son eficientes para problemas donde la evaluacion de f es costosa.
- La estabilidad es crucial para problemas rigidos: los metodos implícitos son A-estables.
- Los problemas de contorno se resuelven con diferencias finitas o metodos espectrales.
- Los metodos espectrales dan precision exponencial para funciones suaves.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. Las EDOs son la base de la simulacion numerica en fisica, y dominar estos metodos te va a abrir muchas puertas. ¡Dale para adelante, que vos podes!

Optimizacion Numerica

Como encontrar el minimo de una funcion sin perder la cabeza, con Python y mucho cafe.

Che, ¿como andas? Bienvenido a este capitulo donde vamos a meterle a uno de los temas mas calientes del momento: la optimizacion numerica. Si llegaste hasta aca, seguro ya escuchaste hablar del machine learning, las redes neuronales, o el ajuste de parametros. Todo eso es optimizacion. La idea es simple: tenes una funcion que depende de muchos parametros, y queres encontrar el punto donde esa funcion es minima (o maxima, pero normalmente minima). Y queres hacerlo de forma eficiente.

Cuando curse Metodos Numericos, recuerdo que el profesor nos dijo: “.el 90 por ciento de los problemas de la vida real son problemas de optimizacion”. Y tenia razon. Desde la fisica (minimizar la energia) hasta la economia (maximizar ganancias), la optimizacion esta en todos lados. En este capitulo vamos a ver los metodos mas importantes para resolver problemas de optimizacion.

En este capitulo vamos a cubrir:

- El problema de optimizacion: ¿que significa minimizar una funcion?
- El metodo de Newton para optimizacion: cuando queres convergencia cuadratica.
- El descenso por gradiente: el caballo de batalla del machine learning.
- La eleccion del paso de aprendizaje: muy chico es lento, muy grande explota.
- La comparacion entre metodos: ¿cuando usar cada uno?

La idea es que vos puedas implementar todo esto en Python, entender por que funciona, y saber que metodo usar en cada situacion. Asi que agarra un cafe, ponete comodo, y vamos a darle.

El problema de optimizacion

El problema de optimizacion es simple de enunciar: dada una funcion $f : \mathbb{R}^n \rightarrow \mathbb{R}$, queremos encontrar un punto x^* tal que:

$$f(x^*) \leq f(x) \quad \forall x \in \text{dominio}$$

Es decir, queremos el minimo global de la funcion. En la practica, a menudo nos conformamos con un minimo local.

Traduccion Matematica del problema El problema de optimizacion se lee en criollo: ”tenemos una funcion que depende de muchos numeros (los parametros), y queremos encontrar la combinacion de numeros que hace que la funcion sea lo mas chica posible”. Pensa en esto como si estuvieras ajustando los controles de un equipo de sonido para que suene lo mejor posible: cada control es una variable, y la calidad del sonido.^{es} la funcion que queres maximizar (o minimizar el error).

¿Por que necesitamos metodos numericos?

La respuesta es simple: para funciones complicadas (como las que aparecen en redes neuronales), no hay una formula cerrada para el minimo. Tenemos que ir aproximando iterativamente.

El gradiente y el Hessiano

Para funciones suaves, el gradiente $\nabla f(x)$ apunta en la direccion de maximo crecimiento. Para minimizar, queremos ir en direccion contraria: $-\nabla f(x)$. El Hessiano $H(x) = \nabla^2 f(x)$ nos da informacion sobre la curvatura.

Traduccion Matematica del gradiente y el Hessiano El gradiente se lee en criollo: ".es un vector que te dice en que direccion tenes que moverte para que la funcion crezca mas rapido". Si queres minimizar, te moves en la direccion opuesta. El Hessiano es como una matriz que te dice que tan curvada esta la funcion en cada direccion. Si el Hessiano es definido positivo, estas en un minimo local.

El metodo de Newton para optimizacion

El metodo de Newton para optimizacion es una extension del metodo de Newton para encontrar raices. En lugar de buscar un cero de f , buscamos un cero del gradiente $\nabla f(x) = 0$.

La formula

La iteracion de Newton para optimizacion es:

$$x_{k+1} = x_k - H(x_k)^{-1} \nabla f(x_k)$$

donde $H(x_k)$ es la matriz Hessiana evaluada en x_k .

Traduccion Matematica de Newton para optimizacion Newton para optimizacion se lee en criollo: ".en lugar de usar solo la pendiente (como el gradiente), tambien usamos la curvatura (el Hessiano) para hacer un salto directamente al fondo del valle". Es como si estuvieras en una montania y, en lugar de dar pasitos en la direccion de maximo descenso, pudieras ver la forma del valle y saltar directamente al punto mas bajo.

Convergencia cuadratica

Si f es dos veces diferenciable y el Hessiano es definido positivo en el minimo, Newton tiene convergencia cuadratica:

$$\|x_{k+1} - x^*\| \leq C \|x_k - x^*\|^2$$

Traduccion Matematica de la convergencia cuadratica La convergencia cuadratica se lee en criollo: ".el numero de digitos correctos se duplica en cada iteracion". Es increiblemente rapido, pero el costo es alto: tenes que calcular el Hessiano (que es una matriz $n \times n$) y resolver un sistema lineal en cada paso.

Grafico: Newton para una funcion cuadratica

Vamos a ver como Newton converge para una funcion cuadratica simple.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def newton_optimizacion(f, grad, hess, x0, tol=1e-10, max_iter=100):
5     """Metodo de Newton para optimizacion."""
6     x = np.array(x0, dtype=float)
7     historial = [x.copy()]
8     for i in range(max_iter):
9         # Resolvemos  $H(x) * dx = -grad(x)$ 
10        try:
11            dx = np.linalg.solve(hess(x), -grad(x))
12        except np.linalg.LinAlgError:
13            break
14        x = x + dx
15        historial.append(x.copy())
16        if np.linalg.norm(dx) < tol:
17            break
18    return x, historial
19
20 # Funcion cuadratica:  $f(x, y) = x^2 + 3y^2$ 
21 def f(x):
22     return x[0]**2 + 3*x[1]**2
23
24 def grad(x):
25     return np.array([2*x[0], 6*x[1]])
26
27 def hess(x):
28     return np.array([[2, 0], [0, 6]])
29
30 x0 = np.array([3.0, 2.0])
31 solucion, historial = newton_optimizacion(f, grad, hess, x0)
32
33 print(f"Solucion: {solucion}")
34 print(f"Iteraciones: {len(historial)}")
35
36 # Graficamos la trayectoria
37 historial = np.array(historial)
38 plt.figure(figsize=(8, 8))
39 plt.plot(historial[:, 0], historial[:, 1], 'o-', label='Trayectoria de Newton')
40 plt.plot(solucion[0], solucion[1], 'r*', markersize=15, label='Minimo')
41 # Curvas de nivel
42 x = np.linspace(-4, 4, 100)
43 y = np.linspace(-3, 3, 100)
44 X, Y = np.meshgrid(x, y)
45 Z = X**2 + 3*Y**2
46 plt.contour(X, Y, Z, levels=20, alpha=0.5)
47 plt.xlabel('x')
48 plt.ylabel('y')
49 plt.title('Newton para  $f(x,y) = x^2 + 3y^2$ ')
50 plt.legend()
51 plt.axis('equal')
52 plt.grid(True)
53 plt.show()

```

Listing 90: Newton para optimizacion de una funcion cuadratica

En el grafico, ves como Newton da un solo salto al minimo porque la funcion es cuadratica. Para funciones no cuadraticas, Newton da varios pasos pero converge muy rapido.

El descenso por gradiente

El descenso por gradiente es el metodo mas usado en machine learning. La idea es simple: en cada paso, te moves en la direccion de maximo descenso.

La formula

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

donde $\alpha > 0$ es el **paso de aprendizaje** (learning rate).

Traduccion Matematica del descenso por gradiente El descenso por gradiente se lee en criollo: ^{.en} cada paso, miras hacia donde la pendiente es mas empinada hacia abajo, y das un paso en esa direccion. El tamaño del paso lo controla α . Es como si estuvieras en una montania con los ojos vendados y solo pudieras sentir el terreno con los pies: sentis la pendiente y das un paso en la direccion que mas baja.

Convergencia

Si f es **fuerte convexa** (con constante $\mu > 0$) y tiene gradiente **Lipschitz** (con constante $L > 0$), el descenso por gradiente con paso $\alpha = 1/L$ converge linealmente:

$$\|x_k - x^*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|x_0 - x^*\|^2$$

Traduccion Matematica de la convergencia del gradiente La convergencia del gradiente se lee en criollo: ^{.el} error se reduce en cada paso por un factor constante $1 - \mu/L$. Si L es mucho mas grande que μ (la funcion es muy ^{.a}largada), la convergencia es lenta. Si $L \approx \mu$, la convergencia es rapida.

Grafico: descenso por gradiente para diferentes pasos

Vamos a ver como el paso de aprendizaje afecta la convergencia.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gradiente(f, grad, x0, alpha, tol=1e-10, max_iter=100):
5     """Descenso por gradiente."""
6     x = np.array(x0, dtype=float)
7     historial = [x.copy()]
8     for i in range(max_iter):
9         x_new = x - alpha * grad(x)
10        historial.append(x_new.copy())
11        if np.linalg.norm(x_new - x) < tol:
12            break
13        x = x_new
14    return x, historial
15
16 def f(x):
17     return x[0]**2 + 10*x[1]**2
18
19 def grad(x):
20     return np.array([2*x[0], 20*x[1]])
21

```



```

22 x0 = np.array([3.0, 1.0])
23
24 # Diferentes pasos de aprendizaje
25 alphas = [0.01, 0.05, 0.1, 0.2]
26 plt.figure(figsize=(12, 10))
27
28 for i, alpha in enumerate(alphas):
29     solucion, historial = gradiente(f, grad, x0, alpha, max_iter=50)
30     historial = np.array(historial)
31
32     plt.subplot(2, 2, i+1)
33     plt.plot(historial[:, 0], historial[:, 1], 'o-', label=f'alpha = {alpha}')
34     plt.plot(solucion[0], solucion[1], 'r*', markersize=10)
35     # Curvas de nivel
36     x = np.linspace(-4, 4, 100)
37     y = np.linspace(-2, 2, 100)
38     X, Y = np.meshgrid(x, y)
39     Z = X**2 + 10*Y**2
40     plt.contour(X, Y, Z, levels=20, alpha=0.5)
41     plt.xlabel('x')
42     plt.ylabel('y')
43     plt.title(f'alpha = {alpha}')
44     plt.axis('equal')
45     plt.grid(True)
46
47 plt.tight_layout()
48 plt.show()

```

Listing 91: Descenso por gradiente con diferentes pasos

En el grafico, ves que para α muy chico (0.01), la convergencia es lenta. Para α intermedio (0.05, 0.1), la convergencia es buena. Para α muy grande (0.2), el metodo oscila y no converge.

Eleccion del paso de aprendizaje

El paso de aprendizaje es critico. Si es muy chico, la convergencia es lenta. Si es muy grande, el metodo puede diverger. Una buena regla es usar $\alpha = 1/L$, donde L es la constante de Lipschitz del gradiente. En la practica, se usa una busqueda de linea o metodos adaptativos como Adam.

Newton vs Gradiente

Vamos a comparar los dos metodos para una funcion no cuadratica.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def newton_optimizacion(f, grad, hess, x0, tol=1e-10, max_iter=100):
5     x = np.array(x0, dtype=float)
6     historial = [x.copy()]
7     for i in range(max_iter):
8         try:
9             dx = np.linalg.solve(hess(x), -grad(x))
10        except np.linalg.LinAlgError:
11            break
12        x = x + dx
13        historial.append(x.copy())
14        if np.linalg.norm(dx) < tol:
15            break
16    return x, historial

```

```

17
18 def gradiente(f, grad, x0, alpha, tol=1e-10, max_iter=100):
19     x = np.array(x0, dtype=float)
20     historial = [x.copy()]
21     for i in range(max_iter):
22         x_new = x - alpha * grad(x)
23         historial.append(x_new.copy())
24         if np.linalg.norm(x_new - x) < tol:
25             break
26         x = x_new
27     return x, historial
28
29 # Funcion de Rosenbrock:  $f(x,y) = (1-x)^2 + 100*(y-x^2)^2$ 
30 def f_rosenbrock(x):
31     return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2
32
33 def grad_rosenbrock(x):
34     return np.array([
35         -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2),
36         200*(x[1] - x[0]**2)
37     ])
38
39 def hess_rosenbrock(x):
40     return np.array([
41         [2 - 400*x[1] + 1200*x[0]**2, -400*x[0]],
42         [-400*x[0], 200]
43     ])
44
45 x0 = np.array([-1.0, 1.0])
46
47 # Newton
48 x_newton, hist_newton = newton_optimizacion(f_rosenbrock, grad_rosenbrock,
49     hess_rosenbrock, x0, max_iter=50)
50
51 # Gradiente
52 x_grad, hist_grad = gradiente(f_rosenbrock, grad_rosenbrock, x0, alpha=0.001,
53     max_iter=1000)
54
55 hist_newton = np.array(hist_newton)
56 hist_grad = np.array(hist_grad)
57
58 print(f"Newton: {x_newton}, iteraciones: {len(hist_newton)}")
59 print(f"Gradiente: {x_grad}, iteraciones: {len(hist_grad)}")
60
61 # Graficamos las trayectorias
62 plt.figure(figsize=(12, 5))
63 plt.subplot(1, 2, 1)
64 plt.plot(hist_newton[:, 0], hist_newton[:, 1], 'o-', label='Newton')
65 plt.plot(1, 1, 'r*', markersize=15, label='Minimo')
66 # Curvas de nivel de Rosenbrock
67 x = np.linspace(-2, 2, 100)
68 y = np.linspace(-1, 3, 100)
69 X, Y = np.meshgrid(x, y)
70 Z = (1-X)**2 + 100*(Y-X**2)**2
71 plt.contour(X, Y, Z, levels=np.logspace(-1, 3, 20), alpha=0.5)
72 plt.xlabel('x')
73 plt.ylabel('y')
74 plt.title('Newton')
75 plt.legend()
76 plt.axis('equal')
77

```

```

76 plt.subplot(1, 2, 2)
77 plt.plot(hist_grad[:, 0], hist_grad[:, 1], 'o-', label='Gradiente')
78 plt.plot(1, 1, 'r*', markersize=15, label='Minimo')
79 plt.contour(X, Y, Z, levels=np.logspace(-1, 3, 20), alpha=0.5)
80 plt.xlabel('x')
81 plt.ylabel('y')
82 plt.title('Gradiente')
83 plt.legend()
84 plt.axis('equal')
85
86 plt.tight_layout()
87 plt.show()

```

Listing 92: Comparacion: Newton vs Gradiente

En el grafico, ves que Newton converge en pocas iteraciones (unos 10 pasos), mientras que el gradiente necesita cientos de pasos. Pero Newton es mas costoso por iteracion porque tiene que calcular y resolver el Hessiano.

Descenso por gradiente con momentum

El descenso por gradiente con momentum es una mejora que acelera la convergencia en direcciones con pendiente suave. La idea es mantener un "promedio" de los gradientes anteriores:

$$v_{k+1} = \beta v_k + (1 - \beta) \nabla f(x_k)$$

$$x_{k+1} = x_k - \alpha v_{k+1}$$

donde $\beta \in [0, 1)$ es el factor de momentum.

Traduccion Matematica del momentum El momentum se lee en criollo: ".^{en} lugar de solo seguir el gradiente actual, tambien tenes en cuenta la direccion en la que venias yendo". Es como si estuvieras corriendo cuesta abajo: tu velocidad no cambia instantaneamente, sino que acumulas inercia. Esto ayuda a salir de valles estrechos y a acelerar la convergencia.

Grafico: momentum vs gradiente puro

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gradiente_momentum(f, grad, x0, alpha, beta, tol=1e-10, max_iter=100):
5     """Descenso por gradiente con momentum."""
6     x = np.array(x0, dtype=float)
7     v = np.zeros_like(x)
8     historial = [x.copy()]
9     for i in range(max_iter):
10         v = beta * v + (1 - beta) * grad(x)
11         x_new = x - alpha * v
12         historial.append(x_new.copy())
13         if np.linalg.norm(x_new - x) < tol:
14             break
15         x = x_new
16     return x, historial
17
18 def f(x):
19     return x[0]**2 + 10*x[1]**2

```

```

20
21 def grad(x):
22     return np.array([2*x[0], 20*x[1]])
23
24 x0 = np.array([3.0, 1.0])
25
26 # Gradiente puro
27 _, hist_grad = gradiente(f, grad, x0, alpha=0.05, max_iter=50)
28
29 # Gradiente con momentum
30 _, hist_mom = gradiente_momentum(f, grad, x0, alpha=0.05, beta=0.9, max_iter=50)
31
32 hist_grad = np.array(hist_grad)
33 hist_mom = np.array(hist_mom)
34
35 plt.figure(figsize=(12, 5))
36 plt.subplot(1, 2, 1)
37 plt.plot(hist_grad[:, 0], hist_grad[:, 1], 'o-', label='Gradiente puro')
38 plt.plot(0, 0, 'r*', markersize=15, label='Minimo')
39 x = np.linspace(-4, 4, 100)
40 y = np.linspace(-2, 2, 100)
41 X, Y = np.meshgrid(x, y)
42 Z = X**2 + 10*Y**2
43 plt.contour(X, Y, Z, levels=20, alpha=0.5)
44 plt.xlabel('x')
45 plt.ylabel('y')
46 plt.title('Gradiente puro')
47 plt.legend()
48 plt.axis('equal')
49
50 plt.subplot(1, 2, 2)
51 plt.plot(hist_mom[:, 0], hist_mom[:, 1], 'o-', label='Gradiente con momentum')
52 plt.plot(0, 0, 'r*', markersize=15, label='Minimo')
53 plt.contour(X, Y, Z, levels=20, alpha=0.5)
54 plt.xlabel('x')
55 plt.ylabel('y')
56 plt.title('Gradiente con momentum')
57 plt.legend()
58 plt.axis('equal')
59
60 plt.tight_layout()
61 plt.show()

```

Listing 93: Gradiente con y sin momentum

En el grafico, ves que el momentum acelera la convergencia: el metodo con momentum llega al minimo mucho mas rapido que el gradiente puro.

Ejercicios

Parte Teorica (10 ejercicios)

T1. Condiciones de optimalidad: Enuncia y demuestra las condiciones de optimalidad de primer y segundo orden para una funcion diferenciable.

T2. Newton para optimizacion: Deduce el metodo de Newton para optimizacion a partir de la aproximacion cuadratica de la funcion objetivo.

- T3. Convergencia cuadratica de Newton:** Demuestra que si el Hessiano es definido positivo en el minimo, Newton tiene convergencia cuadratica.
- T4. Descenso por gradiente:** Deduce el metodo de descenso por gradiente y explica por que la direccion $-\nabla f$ es la de maximo descenso.
- T5. Convergencia del gradiente:** Demuestra que si f es fuertemente convexa con constante μ y el gradiente es Lipschitz con constante L , el descenso por gradiente con paso $\alpha = 1/L$ converge linealmente.
- T6. Eleccion del paso:** Explica como elegir el paso de aprendizaje en el descenso por gradiente. ¿Que pasa si es muy chico? ¿Y si es muy grande?
- T7. Momentum:** Deduce el metodo de descenso por gradiente con momentum y explica por que acelera la convergencia.
- T8. Newton vs Gradiente:** Compara el costo computacional y la velocidad de convergencia de Newton y el descenso por gradiente.
- T9. Descenso por gradiente estocastico:** Explica la diferencia entre el descenso por gradiente batch y el estocastico (SGD). ¿Cuando se usa cada uno?
- T10. Metodos de segundo orden:** ¿Por que los metodos de segundo orden como Newton no se usan en machine learning con millones de parametros?

Parte Computacional (10 ejercicios)

- C1. Newton para una funcion cuadratica:** Implementa Newton para optimizacion y prueba con $f(x, y) = x^2 + 3xy + 2y^2$. Encuentra el minimo analiticamente y compara.
- C2. Descenso por gradiente:** Implementa el descenso por gradiente para $f(x) = x^4 - 4x^2 + 4$. Prueba con diferentes pasos de aprendizaje y grafica la convergencia.
- C3. Comparacion de metodos:** Compara Newton y gradiente para la funcion de Rosenbrock. Grafica las trayectorias y el error en funcion de las iteraciones.
- C4. Efecto del paso de aprendizaje:** Para $f(x, y) = x^2 + 10y^2$, prueba diferentes pasos de aprendizaje ($\alpha = 0,01, 0,05, 0,1, 0,2, 0,5$). Grafica la convergencia y explica los resultados.
- C5. Momentum:** Implementa el descenso por gradiente con momentum para $f(x, y) = x^2 + 10y^2$. Compara con el gradiente puro para diferentes valores de β .
- C6. Ajuste de una recta:** Usa el descenso por gradiente para ajustar una recta $y = mx + b$ a un conjunto de datos. Implementa la funcion de costo y el gradiente.
- C7. Regresion logistica:** Implementa el descenso por gradiente para regresion logistica en un conjunto de datos simple. Grafica la frontera de decision.
- C8. Newton con busqueda de linea:** Implementa Newton con busqueda de linea (backtracking) para la funcion de Rosenbrock. Compara con Newton sin busqueda de linea.
- C9. Metodo de gradiente conjugado:** Implementa el metodo de gradiente conjugado para $f(x) = \frac{1}{2}x^T Ax - b^T x$. Compara con el descenso por gradiente.
- C10. Optimizacion de una funcion no convexa:** Prueba el descenso por gradiente en $f(x) = x^4 - 10x^2 + 1$. ¿A que minimo converge? ¿Depende del punto inicial?

Soluciones

Soluciones Teoricas

T1. Primer orden: Si x^* es un minimo local y f es diferenciable, entonces $\nabla f(x^*) = 0$. Segundo orden: Si ademas f es dos veces diferenciable y el Hessiano $H(x^*)$ es definido positivo, entonces x^* es un minimo local estricto.

T2. Aproximamos f por su desarrollo de Taylor de orden 2: $f(x) \approx f(x_k) + \nabla f(x_k)^T(x - x_k) + \frac{1}{2}(x - x_k)^T H(x_k)(x - x_k)$. Minimizando esta aproximacion, obtenemos $x_{k+1} = x_k - H(x_k)^{-1} \nabla f(x_k)$.

T3. Usando el desarrollo de Taylor alrededor del minimo x^* : $\nabla f(x_k) = H(x^*)(x_k - x^*) + O(\|x_k - x^*\|^2)$. Entonces $x_{k+1} - x^* = x_k - x^* - H(x_k)^{-1} \nabla f(x_k) = O(\|x_k - x^*\|^2)$.

T4. La direccion de maximo descenso es la que minimiza la derivada direccional: $\min_{d: \|d\|=1} \nabla f(x)^T d$. El minimo se alcanza cuando $d = -\nabla f(x) / \|\nabla f(x)\|$.

T5. Para $\alpha = 1/L$, se tiene: $\|x_{k+1} - x^*\|^2 \leq (1 - \mu/L) \|x_k - x^*\|^2$. Esto se demuestra usando las propiedades de convexidad fuerte y suavidad.

T6. Si α es muy chico, la convergencia es lenta. Si es muy grande, el metodo puede diverger u oscilar. Una buena eleccion es $\alpha = 1/L$, o usar busqueda de linea.

T7. El momentum acumula informacion de los gradientes anteriores: $v_{k+1} = \beta v_k + (1 - \beta) \nabla f(x_k)$. Esto ayuda a acelerar la convergencia en direcciones con pendiente suave y a suavizar las oscilaciones.

T8. Newton tiene convergencia cuadratica pero cuesta $O(n^3)$ por iteracion (por resolver el sistema lineal). El gradiente tiene convergencia lineal pero cuesta $O(n)$ por iteracion. Newton es mejor cuando n es chico y la precision es critica. El gradiente es mejor cuando n es grande.

T9. El gradiente batch usa todo el dataset para calcular el gradiente. El SGD usa una sola muestra (o un mini-batch). SGD es mas rapido y puede escapar de minimos locales, pero es mas ruidoso.

T10. En machine learning, n puede ser millones. Calcular el Hessiano (una matriz de $n \times n$) y resolver el sistema lineal es imposible. Por eso se usan metodos de primer orden como SGD.

Soluciones Computacionales

C1.

```

1 import numpy as np
2
3 def f(x):
4     return x[0]**2 + 3*x[0]*x[1] + 2*x[1]**2
5
6 def grad(x):
7     return np.array([2*x[0] + 3*x[1], 3*x[0] + 4*x[1]])
8
9 def hess(x):
10    return np.array([[2, 3], [3, 4]])
11
12 x0 = np.array([2.0, 1.0])
13 x, hist = newton_optimizacion(f, grad, hess, x0)
14
15 # Solucion analitica: resolver [2, 3; 3, 4] * x = 0
16 # El unico minimo es (0, 0)
17 print(f"Newton: {x}")

```

Listing 94: Newton para funcion cuadratica

C2.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gradiente_1d(f, grad, x0, alpha, tol=1e-10, max_iter=100):
5     x = x0
6     historial = [x]
7     for i in range(max_iter):
8         x_new = x - alpha * grad(x)
9         historial.append(x_new)
10        if abs(x_new - x) < tol:
11            break
12        x = x_new
13    return x, historial
14
15 def f(x):
16     return x**4 - 4*x**2 + 4
17
18 def grad(x):
19     return 4*x**3 - 8*x
20
21 x0 = 2.0
22 alphas = [0.01, 0.05, 0.1, 0.2]
23
24 plt.figure(figsize=(10, 6))
25 x_plot = np.linspace(-2.5, 2.5, 100)
26 plt.plot(x_plot, f(x_plot), 'k-', label='f(x)')
27
28 for alpha in alphas:
29     _, hist = gradiente_1d(f, grad, x0, alpha, max_iter=20)
30     plt.plot(hist, f(np.array(hist)), 'o-', label=f'alpha = {alpha}')
31
32 plt.xlabel('x')
33 plt.ylabel('f(x)')
34 plt.title('Descenso por gradiente para f(x) = x^4 - 4x^2 + 4')
35 plt.legend()
36 plt.grid(True)
37 plt.show()

```

Listing 95: Descenso por gradiente para $f(x) = x^4 - 4x^2 + 4$ **C3.**

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f_rosenbrock(x):
5     return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2
6
7 def grad_rosenbrock(x):
8     return np.array([
9         -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2),
10        200*(x[1] - x[0]**2)
11    ])
12
13 def hess_rosenbrock(x):
14     return np.array([
15        [2 - 400*x[1] + 1200*x[0]**2, -400*x[0]],
16        [-400*x[0], 200]
17    ])
18
19 x0 = np.array([-1.0, 1.0])

```

```

20
21 # Newton
22 x_newton, hist_newton = newton_optimizacion(f_rosenbrock, grad_rosenbrock,
23     hess_rosenbrock, x0, max_iter=50)
24 hist_newton = np.array(hist_newton)
25
26 # Gradiente
27 x_grad, hist_grad = gradiente(f_rosenbrock, grad_rosenbrock, x0, alpha=0.001,
28     max_iter=500)
29 hist_grad = np.array(hist_grad)
30
31 # Errores
32 errores_newton = np.array([np.linalg.norm(x - np.array([1, 1])) for x in
33     hist_newton])
34 errores_grad = np.array([np.linalg.norm(x - np.array([1, 1])) for x in hist_grad
35     ])
36
37 plt.figure(figsize=(10, 6))
38 plt.semilogy(errores_newton, 'o-', label='Newton')
39 plt.semilogy(errores_grad, 's-', label='Gradiente')
40 plt.xlabel('Iteracion')
41 plt.ylabel('Distancia al minimo')
42 plt.title('Comparacion: Newton vs Gradiente para Rosenbrock')
43 plt.legend()
44 plt.grid(True)
45 plt.show()

```

Listing 96: Comparacion Newton vs Gradiente para Rosenbrock

C4.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return x[0]**2 + 10*x[1]**2
6
7 def grad(x):
8     return np.array([2*x[0], 20*x[1]])
9
10 x0 = np.array([3.0, 1.0])
11 alphas = [0.01, 0.05, 0.1, 0.2, 0.5]
12
13 plt.figure(figsize=(12, 8))
14 for i, alpha in enumerate(alphas):
15     _, hist = gradiente(f, grad, x0, alpha, max_iter=30)
16     hist = np.array(hist)
17     errores = np.array([np.linalg.norm(x - np.array([0, 0])) for x in hist])
18     plt.semilogy(errores, 'o-', label=f'alpha = {alpha}')
19
20 plt.xlabel('Iteracion')
21 plt.ylabel('Distancia al minimo')
22 plt.title('Efecto del paso de aprendizaje en la convergencia')
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```

Listing 97: Efecto del paso de aprendizaje

C5.

```

1 import numpy as np

```



```

2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return x[0]**2 + 10*x[1]**2
6
7 def grad(x):
8     return np.array([2*x[0], 20*x[1]])
9
10 x0 = np.array([3.0, 1.0])
11
12 # Gradiente puro
13 _, hist_grad = gradiente(f, grad, x0, alpha=0.05, max_iter=50)
14
15 # Momentum con diferentes betas
16 betas = [0.3, 0.6, 0.9]
17 plt.figure(figsize=(10, 6))
18 plt.semilogy(np.linalg.norm(np.array(hist_grad) - np.array([0, 0]), axis=1), 'o
19     -', label='Gradiente puro')
20
21 for beta in betas:
22     _, hist_mom = gradiente_momentum(f, grad, x0, alpha=0.05, beta=beta,
23     max_iter=50)
24     hist_mom = np.array(hist_mom)
25     errores = np.linalg.norm(hist_mom - np.array([0, 0]), axis=1)
26     plt.semilogy(errores, 's-', label=f'beta = {beta}')
27
28 plt.xlabel('Iteracion')
29 plt.ylabel('Distancia al minimo')
30 plt.title('Efecto del momentum en la convergencia')
31 plt.legend()
32 plt.grid(True)
33 plt.show()

```

Listing 98: Momentum vs gradiente puro

C6.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def costo(m, b, x, y):
5     """Error cuadrático medio."""
6     return np.mean((y - (m*x + b))**2)
7
8 def grad_costo(m, b, x, y):
9     """Gradiente del costo."""
10    n = len(x)
11    pred = m*x + b
12    dm = -2/n * np.sum(x * (y - pred))
13    db = -2/n * np.sum(y - pred)
14    return np.array([dm, db])
15
16 # Generamos datos
17 np.random.seed(42)
18 n = 50
19 x = np.linspace(0, 10, n)
20 m_real = 2.5
21 b_real = -1.0
22 y = m_real * x + b_real + 2 * np.random.randn(n)
23
24 # Descenso por gradiente
25 theta = np.array([0.0, 0.0]) # [m, b]

```

```

26 alpha = 0.01
27 historial = [theta.copy()]
28
29 for i in range(1000):
30     grad = grad_costo(theta[0], theta[1], x, y)
31     theta = theta - alpha * grad
32     historial.append(theta.copy())
33
34 historial = np.array(historial)
35
36 print(f"m = {theta[0]:.4f}, b = {theta[1]:.4f}")
37
38 plt.figure(figsize=(10, 6))
39 plt.scatter(x, y, label='Datos')
40 plt.plot(x, m_real*x + b_real, 'g--', label='Recta real')
41 plt.plot(x, theta[0]*x + theta[1], 'r-', label='Recta ajustada')
42 plt.xlabel('x')
43 plt.ylabel('y')
44 plt.title('Ajuste de una recta con descenso por gradiente')
45 plt.legend()
46 plt.grid(True)
47 plt.show()

```

Listing 99: Ajuste de una recta con descenso por gradiente

C7.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_classification
4
5 def sigmoid(z):
6     return 1 / (1 + np.exp(-z))
7
8 def costo_logistico(w, X, y):
9     """Costo de regresion logistica."""
10    n = len(y)
11    z = X @ w
12    return -1/n * np.sum(y * np.log(sigmoid(z) + 1e-10) + (1-y) * np.log(1 -
13    sigmoid(z) + 1e-10))
14
15 def grad_logistico(w, X, y):
16     """Gradiente del costo logistico."""
17    n = len(y)
18    z = X @ w
19    return 1/n * X.T @ (sigmoid(z) - y)
20
21 # Generamos datos
22 X, y = make_classification(n_samples=100, n_features=2, n_informative=2,
23    n_redundant=0, n_clusters_per_class=1, random_state
24    =42)
25 X = np.hstack([np.ones((len(X), 1)), X]) # Agregamos el bias
26
27 # Descenso por gradiente
28 w = np.zeros(3)
29 alpha = 0.1
30 historial_costo = []
31
32 for i in range(500):
33     grad = grad_logistico(w, X, y)
34     w = w - alpha * grad
35     historial_costo.append(costo_logistico(w, X, y))

```

```

34
35 # Graficamos la frontera de decision
36 plt.figure(figsize=(10, 6))
37 x_min, x_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
38 y_min, y_max = X[:, 2].min() - 0.5, X[:, 2].max() + 0.5
39 xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max,
40                               100))
41 Z = sigmoid(w[0] + w[1]*xx + w[2]*yy)
42 plt.contourf(xx, yy, Z, levels=[0, 0.5, 1], alpha=0.3, colors=['blue', 'red'])
43 plt.scatter(X[y==0, 1], X[y==0, 2], c='blue', label='Clase 0')
44 plt.scatter(X[y==1, 1], X[y==1, 2], c='red', label='Clase 1')
45 plt.xlabel('x1')
46 plt.ylabel('x2')
47 plt.title('Regresion logistica con descenso por gradiente')
48 plt.legend()
49 plt.grid(True)
50 plt.show()
51
52 # Graficamos la convergencia del costo
53 plt.figure(figsize=(10, 6))
54 plt.semilogy(historial_costo)
55 plt.xlabel('Iteracion')
56 plt.ylabel('Costo')
57 plt.title('Convergencia del costo en regresion logistica')
58 plt.grid(True)
59 plt.show()

```

Listing 100: Regresion logistica con descenso por gradiente

C8.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def backtracking(f, grad, x, dx, alpha=0.5, beta=0.8):
5     """Busqueda de linea con backtracking."""
6     t = 1.0
7     while f(x + t*dx) > f(x) + alpha * t * grad(x) @ dx:
8         t *= beta
9     return t
10
11 def newton_backtracking(f, grad, hess, x0, tol=1e-10, max_iter=100):
12     x = np.array(x0, dtype=float)
13     historial = [x.copy()]
14     for i in range(max_iter):
15         dx = np.linalg.solve(hess(x), -grad(x))
16         t = backtracking(f, grad, x, dx)
17         x = x + t * dx
18         historial.append(x.copy())
19         if np.linalg.norm(t * dx) < tol:
20             break
21     return x, historial
22
23 # Rosenbrock
24 def f_rosenbrock(x):
25     return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2
26
27 def grad_rosenbrock(x):
28     return np.array([
29         -2*(1 - x[0]) - 400*x[0]*(x[1] - x[0]**2),
30         200*(x[1] - x[0]**2)
31     ])

```

```

32
33 def hess_rosenbrock(x):
34     return np.array([
35         [2 - 400*x[1] + 1200*x[0]**2, -400*x[0]],
36         [-400*x[0], 200]
37     ])
38
39 x0 = np.array([-1.0, 1.0])
40 x_newton, hist_newton = newton_backtracking(f_rosenbrock, grad_rosenbrock,
41     hess_rosenbrock, x0, max_iter=50)
42 hist_newton = np.array(hist_newton)
43
44 print(f"Newton con backtracking: {x_newton}, iteraciones: {len(hist_newton)}")
45
46 # Graficamos
47 plt.figure(figsize=(10, 6))
48 plt.plot(hist_newton[:, 0], hist_newton[:, 1], 'o-')
49 plt.plot(1, 1, 'r*', markersize=15, label='Minimo')
50 x = np.linspace(-2, 2, 100)
51 y = np.linspace(-1, 3, 100)
52 X, Y = np.meshgrid(x, y)
53 Z = (1-X)**2 + 100*(Y-X**2)**2
54 plt.contour(X, Y, Z, levels=np.logspace(-1, 3, 20), alpha=0.5)
55 plt.xlabel('x')
56 plt.ylabel('y')
57 plt.title('Newton con busqueda de linea para Rosenbrock')
58 plt.legend()
59 plt.axis('equal')
60 plt.grid(True)
61 plt.show()

```

Listing 101: Newton con busqueda de linea

C9.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gradiente_conjugado(A, b, x0, tol=1e-10, max_iter=100):
5     """Metodo de gradiente conjugado para  $f(x) = 0.5x^T A x - b^T x$ ."""
6     x = np.array(x0, dtype=float)
7     r = A @ x - b
8     p = -r
9     historial = [x.copy()]
10    for i in range(max_iter):
11        Ap = A @ p
12        alpha = - (r @ p) / (p @ Ap)
13        x = x + alpha * p
14        r_new = r + alpha * Ap
15        beta = (r_new @ r_new) / (r @ r)
16        p = -r_new + beta * p
17        r = r_new
18        historial.append(x.copy())
19        if np.linalg.norm(r) < tol:
20            break
21    return x, historial
22
23 # Funcion cuadratica
24 A = np.array([[3, 1], [1, 2]])
25 b = np.array([1, 2])
26 x0 = np.array([0, 0])
27

```

```

28 x_cg, hist_cg = gradiente_conjugado(A, b, x0, max_iter=50)
29 hist_cg = np.array(hist_cg)
30
31 # Solucion exacta
32 x_exact = np.linalg.solve(A, b)
33 print(f"Gradiente conjugado: {x_cg}")
34 print(f"Solucion exacta: {x_exact}")
35
36 # Comparacion con gradiente
37 def f(x):
38     return 0.5 * x.T @ A @ x - b.T @ x
39
40 def grad(x):
41     return A @ x - b
42
43 _, hist_grad = gradiente(f, grad, x0, alpha=0.1, max_iter=50)
44 hist_grad = np.array(hist_grad)
45
46 plt.figure(figsize=(10, 6))
47 plt.semilogy(np.linalg.norm(hist_cg - x_exact, axis=1), 'o-', label='Gradiente
    conjugado')
48 plt.semilogy(np.linalg.norm(hist_grad - x_exact, axis=1), 's-', label='Descenso
    por gradiente')
49 plt.xlabel('Iteracion')
50 plt.ylabel('Distancia a la solucion')
51 plt.title('Gradiente conjugado vs Descenso por gradiente')
52 plt.legend()
53 plt.grid(True)
54 plt.show()

```

Listing 102: Gradiente conjugado para funcion cuadratica

C10.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return x**4 - 10*x**2 + 1
6
7 def grad(x):
8     return 4*x**3 - 20*x
9
10 x0_values = [0.1, 1.0, 2.0, 3.0]
11
12 plt.figure(figsize=(10, 6))
13 x_plot = np.linspace(-3.5, 3.5, 100)
14 plt.plot(x_plot, f(x_plot), 'k-', label='f(x)')
15
16 for x0 in x0_values:
17     _, hist = gradiente_1d(f, grad, x0, alpha=0.01, max_iter=100)
18     plt.plot(hist, f(np.array(hist)), 'o-', label=f'x0 = {x0}')
19
20 plt.xlabel('x')
21 plt.ylabel('f(x)')
22 plt.title('Descenso por gradiente en una funcion no convexa')
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```

Listing 103: Optimizacion de funcion no convexa

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por la optimizacion numerica. Viste que no era tan complicado. Arrancamos con el problema de minimizar funciones, pasamos por Newton (que es rapidisimo pero caro), el descenso por gradiente (que es el caballo de batalla del machine learning), y el momentum (que acelera la convergencia). Todo esta conectado: la optimizacion es la herramienta fundamental para ajustar modelos y encontrar minimos.

Los puntos clave para llevar:

- La optimizacion busca el minimo de una funcion.
- Newton usa el Hessiano para dar saltos grandes y precisos (convergencia cuadratica).
- El descenso por gradiente usa solo el gradiente y es mas barato pero mas lento (convergencia lineal).
- El paso de aprendizaje es critico: muy chico es lento, muy grande diverge.
- El momentum acelera la convergencia acumulando inercia.
- En machine learning, se usa SGD porque es escalable a millones de parametros.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. La optimizacion es una herramienta fundamental que vas a usar en todos los problemas de machine learning, fisica computacional, y mas alla. ¡Dale para adelante, que vos podes!

Aplicaciones Modernas

De la criptografia a la simulacion numerica, pasando por el procesamiento de señales.

Che, ¿como andas? Bienvenido al ultimo capitulo de este viaje por el calculo numerico. Hasta aca vimos un monton de teoria y metodos: desde la aritmetica de punto flotante hasta la resolucion de ecuaciones diferenciales. Pero todo eso no serviria de nada si no lo podemos aplicar a problemas reales. En este capitulo vamos a ver como las herramientas que desarrollamos a lo largo del curso se usan en aplicaciones concretas y modernas.

Cuando curse Analisis Numerico, recuerdo que el profesor nos dijo: "todo lo que aprendieron hasta ahora tiene aplicaciones en la vida real. La pregunta no es si lo van a usar, sino donde". Y tenia razon. Los metodos que vimos estan en todos lados: en la criptografia que protege tus mensajes, en los codigos que corrigen errores en los CDs, en el procesamiento de imagenes de tu celular, y en las simulaciones de fluidos que se usan en la industria aeroespacial.

En este capitulo vamos a cubrir:

- **Shamir:** como compartir un secreto de forma segura usando interpolacion en cuerpos finitos.
- **Reed-Solomon:** como corregir errores en la transmision de datos.
- **Fourier:** como procesar señales y comprimir imagenes.
- **Metodos espectrales:** como resolver ecuaciones diferenciales con precision exponencial.

La idea es que vos puedas implementar todo esto en Python, entender por que funciona, y ver como las herramientas que aprendiste se conectan con problemas reales. Asi que agarra un cafe, ponete comodo, y vamos a darle.

Secreto Compartido de Shamir

El esquema de secreto compartido de Shamir es una aplicacion hermosa de la interpolacion en cuerpos finitos. Fue propuesto por Adi Shamir en 1979 y se usa en criptografia para distribuir un secreto entre varias personas, de forma que se necesite un cierto numero de ellas para reconstruirlo.

La idea

Imaginate que tenes un secreto (una clave, un numero, un mensaje) y queres dividirlo entre n personas de modo que:

- Cualquier grupo de k personas pueda reconstruir el secreto.
- Cualquier grupo de $k - 1$ o menos personas no tenga ninguna informacion sobre el secreto.

Esto se llama un esquema (k, n) -threshold.

Traducción Matematica de Shamir Shamir se lee en criollo: .^{el}egimos un polinomio de grado $k-1$ cuyo termino independiente es el secreto. Cada persona recibe un punto del polinomio. Con k puntos, podemos interpolar el polinomio y recuperar el secreto. Con menos de k puntos, el polinomio no esta determinado, y el secreto puede ser cualquier cosa”. Es como si el secreto estuviera .^{es}condido.^{en} la curva de un polinomio, y cada persona tuviera un punto de esa curva.

El algoritmo

Trabajamos en un cuerpo finito $\mathbb{Z}_p$ con p primo (y p mayor que el numero de participantes y que el secreto).

1. Elegir un polinomio $P(x) = a_0 + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1}$ con $a_0 = s$ (el secreto) y $a_1, \dots, a_{k-1}$ aleatorios en $\mathbb{Z}_p$.
2. Para cada participante $i = 1, \dots, n$, darle el punto $(i, P(i) \bmod p)$.
3. Para reconstruir, interpolar los k puntos y evaluar en $x = 0$.

Implementacion en Python

Vamos a implementar el esquema de Shamir.

```

1 import numpy as np
2 import random
3
4 def shamir_generar_secretos(secreto, n, k, p):
5     """
6     Genera n shares para un secreto en Z_p con umbral k.
7
8     Parametros:
9     - secreto: el secreto a compartir (entero < p)
10    - n: numero de shares a generar
11    - k: umbral (k <= n)
12    - p: primo (p > secreto, p > n)
13
14    Retorna: lista de (x, y) shares
15    """
16    if k > n:
17        raise ValueError("k debe ser <= n")
18    if secreto >= p:
19        raise ValueError("El secreto debe ser menor que p")
20
21    # Coeficientes del polinomio: a0 = secreto, a1...ak-1 aleatorios
22    coefs = [secreto] + [random.randint(1, p-1) for _ in range(k-1)]
23
24    shares = []
25    for i in range(1, n+1):
26        x = i
27        y = 0
28        for j, coef in enumerate(coefs):
29            y = (y + coef * pow(x, j, p)) % p
30        shares.append((x, y))
31
32    return shares, coefs
33
34 def shamir_reconstruir(shares, p):
35     """
36     Reconstruye el secreto a partir de k shares usando interpolacion de Lagrange
37     en Z_p.

```



```

37
38 Parametros:
39 - shares: lista de (x, y) shares
40 - p: primo del cuerpo
41
42 Retorna: el secreto reconstruido
43 """
44 k = len(shares)
45 x_vals = np.array([s[0] for s in shares])
46 y_vals = np.array([s[1] for s in shares])
47
48 # Evaluamos el polinomio interpolante en x=0 usando Lagrange
49 secreto = 0
50 for j in range(k):
51     l_j = 1
52     for m in range(k):
53         if m != j:
54             # (0 - x_m) / (x_j - x_m) en Z_p
55             numerador = (-x_vals[m]) % p
56             denominador = (x_vals[j] - x_vals[m]) % p
57             l_j = (l_j * numerador * pow(denominador, -1, p)) % p
58     secreto = (secreto + y_vals[j] * l_j) % p
59
60 return secreto
61
62 # Ejemplo
63 p = 97 # Primo
64 secreto = 42
65 n = 5 # 5 participantes
66 k = 3 # Umbral de 3
67
68 shares, coefs = shamir_generar_secretos(secreto, n, k, p)
69 print(f"Secreto: {secreto}")
70 print(f"Coeficientes del polinomio: {coefs}")
71 print("Shares:")
72 for share in shares:
73     print(f" {share}")
74
75 # Reconstruimos con k shares
76 secreto_rec = shamir_reconstruir(shares[:k], p)
77 print(f"Secreto reconstruido con {k} shares: {secreto_rec}")
78
79 # Intentamos con k-1 shares (deberia fallar)
80 try:
81     secreto_rec_2 = shamir_reconstruir(shares[:k-1], p)
82     print(f"Secreto reconstruido con {k-1} shares: {secreto_rec_2}")
83 except:
84     print("No se puede reconstruir con menos de k shares")

```

Listing 104: Secreto compartido de Shamir

Grafico: polinomio de Shamir

Vamos a visualizar el polinomio que esconde el secreto.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import random
4
5 # Generamos el polinomio en los reales para visualizar (no en Z_p)

```

```

6 def polinomio_shamir_real(coefs, x):
7     """Evalua el polinomio con coeficientes coefs en x."""
8     y = 0
9     for i, c in enumerate(coefs):
10         y += c * x**i
11     return y
12
13 # Elegimos un secreto y coeficientes
14 secreto_real = 10
15 coefs_real = [secreto_real, 3, -2] # Polinomio: 10 + 3x - 2x^2
16 # coefs_real[0] = secreto
17
18 # Puntos para el grafico
19 x_vals = np.linspace(0, 6, 200)
20 y_vals = polinomio_shamir_real(coefs_real, x_vals)
21
22 # Generamos shares reales (para 5 participantes)
23 x_shares = np.array([1, 2, 3, 4, 5])
24 y_shares = polinomio_shamir_real(coefs_real, x_shares)
25
26 plt.figure(figsize=(10, 6))
27 plt.plot(x_vals, y_vals, 'b-', label='Polinomio P(x)')
28 plt.plot(x_shares[:3], y_shares[:3], 'ro', markersize=10, label='Shares (3 para
    reconstruir)')
29 plt.plot(x_shares[3:], y_shares[3:], 'go', markersize=8, label='Shares
    adicionales')
30 plt.plot(0, secreto_real, 'r*', markersize=15, label='Secreto (P(0))')
31 plt.axhline(y=secreto_real, color='red', linestyle='--', alpha=0.5)
32 plt.axvline(x=0, color='black', linewidth=0.5)
33 plt.xlabel('x')
34 plt.ylabel('P(x)')
35 plt.title('Esquema de Shamir: el secreto es P(0)')
36 plt.legend()
37 plt.grid(True)
38 plt.show()

```

Listing 105: Visualizacion del polinomio de Shamir

Codigos de Reed-Solomon

Los codigos de Reed-Solomon son una aplicacion clasica de la interpolacion en cuerpos finitos para la correccion de errores. Se usan en CDs, DVDs, codigos QR, comunicaciones espaciales, y almacenamiento distribuido.

La idea

Un mensaje de k simbolos se codifica como un polinomio de grado $k - 1$. Luego se evalua en $n > k$ puntos para obtener una palabra de codigo de longitud n . Si algunos simbolos se corrompen, se puede recuperar el mensaje original usando interpolacion, siempre que el numero de errores sea menor que $(n - k)/2$.

Traduccion Matematica de Reed-Solomon Reed-Solomon se lee en criollo: codificamos un mensaje como un polinomio, y lo evaluamos en muchos puntos. Si algunos puntos se pierden o se dañan, podemos recuperar el polinomio original siempre que tengamos suficientes puntos correctos". Es como si escribieras un mensaje en un papel, hicieras varias copias en diferentes lugares, y luego pudieras reconstruir el mensaje original aunque algunas copias esten rotas.

El algoritmo

Trabajamos en $\mathbb{F}_q$ (con q potencia de primo). Elegimos n puntos de evaluacion $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ distintos.

1. **Codificacion:** Dado un mensaje $m = (m_0, \dots, m_{k-1})$, formar el polinomio $P(x) = m_0 + m_1x + \dots + m_{k-1}x^{k-1}$. La palabra de codigo es $(P(\alpha_1), \dots, P(\alpha_n))$.
2. **Decodificacion:** Dada una palabra recibida con errores, encontrar el polinomio de grado $< k$ que mejor se ajusta a los datos (minimizando el numero de errores).

Implementacion simple en Python

Vamos a implementar una version simplificada de Reed-Solomon (sin el algoritmo de decodificacion completo, solo la correccion simple asumiendo que sabemos cuales son los simbolos correctos).

```

1 import numpy as np
2
3 def reed_solomon_codificar(mensaje, n, p):
4     """
5     Codifica un mensaje usando Reed-Solomon en Z_p.
6
7     Parametros:
8     - mensaje: lista de simbolos en Z_p (longitud k)
9     - n: longitud de la palabra de codigo (n > k)
10    - p: primo
11
12    Retorna: palabra de codigo (lista de n simbolos)
13    """
14    k = len(mensaje)
15    if n <= k:
16        raise ValueError("n debe ser > k")
17
18    # Puntos de evaluacion: 0, 1, ..., n-1
19    alphas = list(range(n))
20
21    # Polinomio: P(x) = mensaje[0] + mensaje[1]*x + ... + mensaje[k-1]*x^{k-1}
22    codeword = []
23    for alpha in alphas:
24        y = 0
25        for j, m in enumerate(mensaje):
26            y = (y + m * pow(alpha, j, p)) % p
27        codeword.append(y)
28
29    return codeword, alphas
30
31 def reed_solomon_decodificar(codeword, alphas, k, p, errores_conocidos=None):
32     """
33     Decodifica una palabra de Reed-Solomon.
34
35     Parametros:
36     - codeword: lista de simbolos recibidos (longitud n)
37     - alphas: puntos de evaluacion
38     - k: longitud del mensaje original
39     - p: primo
40     - errores_conocidos: lista de indices donde se sabe que hay errores (para
41       simplificar)
42
43     Retorna: mensaje decodificado

```

```

43 """
44 n = len(codeword)
45
46 # Si conocemos los errores, los corregimos (version simplificada)
47 if errores_conocidos is not None:
48     # Eliminamos los errores y usamos los simbolos correctos
49     indices_correctos = [i for i in range(n) if i not in errores_conocidos]
50 else:
51     # Si no conocemos los errores, asumimos que los primeros k son correctos
52     indices_correctos = list(range(k))
53
54 if len(indices_correctos) < k:
55     raise ValueError("No hay suficientes simbolos correctos para decodificar")
56
57 # Usamos los primeros k indices correctos para interpolar
58 x_vals = [alphas[i] for i in indices_correctos[:k]]
59 y_vals = [codeword[i] for i in indices_correctos[:k]]
60
61 # Interpolamos para recuperar el polinomio (usamos Lagrange)
62 # Solo necesitamos los coeficientes del polinomio
63 # Evaluamos en x = 0, 1, ..., k-1 para obtener el mensaje
64 mensaje = []
65 for j in range(k):
66     # Coeficiente m_j = P^{(j)}(0) / j!
67     # Para simplificar, usamos el metodo de los coeficientes indeterminados
68     pass
69
70 # Usamos un enfoque mas simple: evaluamos el polinomio en los puntos 0...k-1
71 # Para eso, necesitamos el polinomio interpolante
72 # Implementamos Lagrange para evaluar en un punto
73 def lagrange_eval(x, x_vals, y_vals, p):
74     k_ = len(x_vals)
75     result = 0
76     for j in range(k_):
77         term = y_vals[j]
78         for m in range(k_):
79             if m != j:
80                 term = term * (x - x_vals[m]) % p
81                 term = term * pow((x_vals[j] - x_vals[m]) % p, -1, p) % p
82         result = (result + term) % p
83     return result
84
85 # Evaluamos el polinomio en x = 0, 1, ..., k-1 para recuperar el mensaje
86 mensaje = []
87 for x in range(k):
88     mensaje.append(lagrange_eval(x, x_vals, y_vals, p))
89
90 return mensaje
91
92 # Ejemplo
93 p = 97
94 k = 3
95 n = 7
96 mensaje = [5, 10, 15] # 3 simbolos
97
98 codeword, alphas = reed_solomon_codificar(mensaje, n, p)
99 print(f"Codeword: {codeword}")
100
101 # Introducimos errores (corrompemos 2 simbolos)
102 codeword_corrupto = codeword.copy()

```

```

103 codeword_corrupto[1] = (codeword_corrupto[1] + 1) % p
104 codeword_corrupto[3] = (codeword_corrupto[3] + 2) % p
105 print(f"Codeword corrupto: {codeword_corrupto}")
106
107 # Decodificamos (suponemos que sabemos cuales son los errores)
108 errores = [1, 3]
109 mensaje_rec = reed_solomon_decodificar(codeword_corrupto, alphas, k, p, errores)
110 print(f"Mensaje recuperado: {mensaje_rec}")
111 print(f"Mensaje original: {mensaje}")

```

Listing 106: Código de Reed-Solomon simple

Procesamiento de Señales con Fourier

La Transformada de Fourier es una herramienta fundamental en el procesamiento de señales. Permite analizar el contenido de frecuencia de una señal, filtrar ruido, comprimir datos, y mucho mas.

Filtrado de señales

Una aplicacion clasica es el filtrado de ruido. Si una señal tiene ruido de alta frecuencia, podemos eliminar esas frecuencias usando un filtro pasa-bajos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generamos una se al con ruido
5 N = 1024
6 t = np.linspace(0, 1, N)
7 senal_limpia = np.sin(2 * np.pi * 5 * t) + 0.5 * np.sin(2 * np.pi * 12 * t)
8 ruido = 0.3 * np.random.randn(N)
9 senal = senal_limpia + ruido
10
11 # FFT
12 X = np.fft.fft(senal)
13 frecs = np.fft.fftfreq(N, d=1/N)
14
15 # Filtro pasa-bajos (eliminar frecuencias > 15 Hz)
16 filtro = np.zeros(N)
17 filtro[np.abs(frecs) <= 15] = 1
18
19 # Aplicamos el filtro
20 X_filtrada = X * filtro
21 senal_filtrada = np.fft.ifft(X_filtrada).real
22
23 # Graficamos
24 plt.figure(figsize=(12, 8))
25 plt.subplot(3, 1, 1)
26 plt.plot(t, senal)
27 plt.title('Se al original con ruido')
28 plt.grid(True)
29
30 plt.subplot(3, 1, 2)
31 plt.plot(frecs, np.abs(X))
32 plt.xlim(-30, 30)
33 plt.title('Espectro de frecuencias')
34 plt.grid(True)
35

```

```

36 plt.subplot(3, 1, 3)
37 plt.plot(t, senal_filtrada)
38 plt.title('Señal filtrada (pasa-bajos)')
39 plt.grid(True)
40
41 plt.tight_layout()
42 plt.show()

```

Listing 107: Filtrado de una señal con ruido usando FFT

Compresion de imagenes con JPEG

La compresion JPEG usa la Transformada del Coseno Discreto (DCT), que esta relacionada con la DFT. La idea es transformar la imagen al dominio de la frecuencia, cuantificar los coeficientes (eliminando los mas pequenos), y luego comprimir.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generamos una imagen de prueba (patron de rayas)
5 n = 128
6 imagen = np.zeros((n, n))
7 for i in range(n):
8     imagen[i, :] = np.sin(2 * np.pi * i / 8) + 0.5 * np.sin(4 * np.pi * i / 8)
9
10 # SVD
11 U, s, Vt = np.linalg.svd(imagen, full_matrices=False)
12
13 # Comprimimos con diferentes rangos
14 rangos = [2, 5, 10, 20, 50]
15 plt.figure(figsize=(15, 10))
16
17 plt.subplot(2, 3, 1)
18 plt.imshow(imagen, cmap='gray')
19 plt.title('Original')
20 plt.axis('off')
21
22 for i, k in enumerate(rangos):
23     U_k = U[:, :k]
24     s_k = s[:k]
25     Vt_k = Vt[:, :k]
26     img_comp = U_k @ np.diag(s_k) @ Vt_k
27
28     plt.subplot(2, 3, i+2)
29     plt.imshow(img_comp, cmap='gray')
30     plt.title(f'Rango {k}')
31     plt.axis('off')
32
33     # Calculamos el error y la compresion
34     error = np.linalg.norm(imagen - img_comp, 'fro') / np.linalg.norm(imagen, 'fro')
35     compresion = (n*k + k*n) / (n*n)
36     print(f"Rango {k}: Error = {error:.4f}, Compresion = {compresion:.2f}")
37
38 plt.tight_layout()
39 plt.show()

```

Listing 108: Compresion de una imagen usando SVD (similar a JPEG)

Metodos Espectrales para PDEs

Los metodos espectrales son una de las tecnicas mas poderosas para resolver ecuaciones diferenciales parciales. La idea es representar la solucion como una serie de funciones ortogonales (Fourier o Chebyshev) y resolver el sistema en el espacio de coeficientes.

Ejemplo: Ecuacion de Poisson 1D con Chebyshev

Vamos a resolver $-u''(x) = f(x)$ en $(-1,1)$ con condiciones de borde $u(-1) = u(1) = 0$ usando un metodo espectral de Chebyshev.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def chebyshev_spectral_poisson(f, N):
5     """
6     Resuelve  $-u''(x) = f(x)$  en  $(-1,1)$  con  $u(-1)=u(1)=0$ 
7     usando metodo espectral de Chebyshev.
8     """
9     # Nodos de Chebyshev (incluyendo bordes)
10    x = np.cos(np.linspace(0, np.pi, N+1))
11
12    # Matriz de derivacion espectral de Chebyshev
13    D = np.zeros((N+1, N+1))
14    for i in range(N+1):
15        for j in range(N+1):
16            if i != j:
17                D[i, j] = (-1)**(i+j) / (x[i] - x[j])
18    for i in range(N+1):
19        D[i, i] = -np.sum(D[i, :])
20
21    # Derivada segunda
22    D2 = D @ D
23
24    # Quitamos las condiciones de borde
25    D2_int = D2[1:-1, 1:-1]
26    x_int = x[1:-1]
27
28    # Resolvemos
29    b = f(x_int)
30    u_int = np.linalg.solve(D2_int, b)
31
32    u = np.zeros(N+1)
33    u[1:-1] = u_int
34    return x, u
35
36 def f(x):
37     """Funcion fuente."""
38     return np.exp(x) * (1 - x**2) * (-2) # Solucion exacta:  $u(x) = \exp(x) \cdot (1-x^2)$ 
39
40 N = 20
41 x, u = chebyshev_spectral_poisson(f, N)
42
43 # Solucion exacta
44 x_exact = np.linspace(-1, 1, 100)
45 u_exact = np.exp(x_exact) * (1 - x_exact**2)
46
47 plt.figure(figsize=(10, 6))
48 plt.plot(x, u, 'ro', label='Solucion espectral')
```

```

49 plt.plot(x_exact, u_exact, 'k-', label='Solucion exacta')
50 plt.xlabel('x')
51 plt.ylabel('u(x)')
52 plt.title('Metodo espectral de Chebyshev para -u\'\' = f(x)')
53 plt.legend()
54 plt.grid(True)
55 plt.show()
56
57 # Error
58 error = np.max(np.abs(u[1:-1] - np.exp(x[1:-1]) * (1 - x[1:-1]**2)))
59 print(f"Error maximo: {error:.2e}")

```

Listing 109: Metodo espectral de Chebyshev para Poisson

Grafico: convergencia espectral

Vamos a ver como el error decae exponencialmente con el numero de modos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def chebyshev_spectral_poisson(f, N):
5     x = np.cos(np.linspace(0, np.pi, N+1))
6     D = np.zeros((N+1, N+1))
7     for i in range(N+1):
8         for j in range(N+1):
9             if i != j:
10                 D[i, j] = (-1)**(i+j) / (x[i] - x[j])
11     for i in range(N+1):
12         D[i, i] = -np.sum(D[i, :])
13     D2 = D @ D
14     D2_int = D2[1:-1, 1:-1]
15     x_int = x[1:-1]
16     b = f(x_int)
17     u_int = np.linalg.solve(D2_int, b)
18     u = np.zeros(N+1)
19     u[1:-1] = u_int
20     return x, u
21
22 def f(x):
23     return np.exp(x) * (1 - x**2) * (-2)
24
25 Ns = np.arange(4, 30)
26 errores = []
27
28 for N in Ns:
29     x, u = chebyshev_spectral_poisson(f, N)
30     u_exact = np.exp(x) * (1 - x**2)
31     error = np.max(np.abs(u[1:-1] - u_exact[1:-1]))
32     errores.append(error)
33
34 plt.figure(figsize=(10, 6))
35 plt.semilogy(Ns, errores, 'o-')
36 plt.xlabel('Numero de modos N')
37 plt.ylabel('Error maximo')
38 plt.title('Convergencia exponencial del metodo espectral de Chebyshev')
39 plt.grid(True)
40 plt.show()

```

Listing 110: Convergencia espectral de Chebyshev

Ejercicios

Parte Teorica (10 ejercicios)

- T1. Seguridad perfecta de Shamir:** Demuestra que en el esquema de Shamir, con $k - 1$ shares, todos los valores del secreto son igualmente probables.
- T2. Distancia de Reed-Solomon:** Demuestra que la distancia minima del codigo Reed-Solomon (n, k) es $n - k + 1$.
- T3. Capacidad de correccion:** Demuestra que un codigo Reed-Solomon (n, k) puede corregir hasta $\lfloor (n - k)/2 \rfloor$ errores.
- T4. Teorema de convolucion:** Demuestra que $DFT(x * y) = DFT(x) \cdot DFT(y)$.
- T5. Parseval para la DFT:** Demuestra que $\sum |x_n|^2 = \frac{1}{N} \sum |X_k|^2$.
- T6. Principio de incertidumbre:** Demuestra que $|S| \cdot |T| \geq N$ para la DFT.
- T7. Convergencia espectral:** Explica por que los metodos espectrales de Chebyshev tienen convergencia exponencial para funciones suaves.
- T8. Matriz de derivacion espectral:** Deduce la matriz de derivacion espectral de Chebyshev.
- T9. Filtrado de señales:** Explica como funciona un filtro pasa-bajos en el dominio de la frecuencia.
- T10. Compresion de imagenes:** Explica como la SVD se usa para comprimir imagenes.

Parte Computacional (10 ejercicios)

- C1. Shamir:** Implementa el esquema de Shamir y prueba con diferentes valores de n y k .
- C2. Reed-Solomon:** Implementa un codificador y decodificador de Reed-Solomon simple y prueba con diferentes mensajes.
- C3. Filtrado de audio:** Genera una señal de audio sintetica y filtra las frecuencias no deseadas usando FFT.
- C4. Compresion de imagenes:** Usa la SVD para comprimir una imagen en escala de grises y analiza el error en funcion del rango.
- C5. Metodo espectral de Chebyshev:** Resuelve $-u'' + u = f(x)$ en $(-1, 1)$ con condiciones de borde usando el metodo espectral.
- C6. Convergencia espectral:** Compara la convergencia del metodo espectral de Chebyshev con diferencias finitas para la ecuacion de Poisson.
- C7. DFT de una imagen:** Aplica la DFT 2D a una imagen y visualiza el espectro de frecuencias.
- C8. Shamir en $\mathbb{Z}_p$:** *Implementa Shamir en un cuerpo finito y prueba la recuperacion del secreto con diferentes valores de p .*
- C9. Reed-Solomon con errores:** Implementa una simulacion de transmision con errores y decodifica usando Reed-Solomon.
- C10. Aplicacion combinada:** Usa Shamir para compartir una clave y Reed-Solomon para transmitir los shares de forma robusta.

Soluciones

Soluciones Teoricas

T1. Dados $k - 1$ puntos, cualquier valor s' del secreto define un polinomio de grado $k - 1$ que pasa por esos puntos y tiene $P(0) = s'$. Como los coeficientes se eligieron aleatoriamente, todos los s' son igualmente probables.

T2. Dos polinomios distintos de grado $< k$ pueden coincidir en a lo sumo $k - 1$ puntos. Por lo tanto, sus evaluaciones en n puntos difieren en al menos $n - (k - 1) = n - k + 1$ posiciones.

T3. Con $n - k + 1$ distancia minima, se pueden detectar hasta $n - k$ errores y corregir hasta $\lfloor (n - k)/2 \rfloor$ errores.

T4. $DFT(x * y)_k = \sum_n (x * y)_n \omega^{-kn} = \sum_n \sum_m x_m y_{n-m} \omega^{-kn} = \sum_m x_m \omega^{-km} \sum_l y_l \omega^{-kl} = X_k Y_k$.

T5. Por la unitariedad de la DFT: $\frac{1}{\sqrt{N}}F$ es unitaria, asi que $\|x\|^2 = \|\frac{1}{\sqrt{N}}X\|^2 = \frac{1}{N}\|X\|^2$.

T6. Como en el capitulo de transformadas: $\|x\|^2 \leq \frac{|S||T|}{N^2}\|X\|^2$, y por Parseval $\|X\|^2 = N\|x\|^2$, asi que $|S||T| \geq N$.

T7. Los coeficientes de Chebyshev decaen exponencialmente para funciones analiticas. La derivada en el espacio de coeficientes es una matriz que preserva este decaimiento, dando convergencia exponencial.

T8. $D_{ij} = \frac{(-1)^{i+j}}{x_i - x_j}$ para $i \neq j$, y $D_{ii} = -\sum_{j \neq i} D_{ij}$. Se deriva de la interpolacion de Lagrange.

T9. Un filtro pasa-bajos en frecuencia es una multiplicacion punto a punto: $Y_k = H_k X_k$, donde $H_k = 1$ para $|k| \leq k_c$ y 0 en caso contrario. Esto elimina las frecuencias altas.

T10. La SVD da la mejor aproximacion de rango k en norma de Frobenius. Truncando la SVD, se obtiene una imagen comprimida que retiene la informacion mas importante.

Soluciones Computacionales

C1.

```

1 import numpy as np
2 import random
3
4 def shamir_generar_secretos(secreto, n, k, p):
5     if k > n:
6         raise ValueError("k debe ser <= n")
7     if secreto >= p:
8         raise ValueError("El secreto debe ser menor que p")
9     coefs = [secreto] + [random.randint(1, p-1) for _ in range(k-1)]
10    shares = []
11    for i in range(1, n+1):
12        x = i
13        y = 0
14        for j, coef in enumerate(coefs):
15            y = (y + coef * pow(x, j, p)) % p
16        shares.append((x, y))
17    return shares, coefs
18
19 def shamir_reconstruir(shares, p):
20    k = len(shares)
21    x_vals = np.array([s[0] for s in shares])
22    y_vals = np.array([s[1] for s in shares])
23    secreto = 0
24    for j in range(k):
25        l_j = 1
26        for m in range(k):

```

```

27         if m != j:
28             numerador = (-x_vals[m]) % p
29             denominador = (x_vals[j] - x_vals[m]) % p
30             l_j = (l_j * numerador * pow(denominador, -1, p)) % p
31         secreto = (secreto + y_vals[j] * l_j) % p
32     return secreto
33
34 p = 101
35 secretos = [10, 20, 30, 40, 50]
36 n = 6
37 k = 4
38
39 for secreto in secretos:
40     shares, _ = shamir_generar_secretos(secreto, n, k, p)
41     secreto_rec = shamir_reconstruir(shares[:k], p)
42     print(f"Secreto: {secreto}, Recuperado: {secreto_rec}, OK: {secreto == secreto_rec}")

```

Listing 111: Prueba de Shamir con diferentes parametros

C2.

```

1 import numpy as np
2
3 def reed_solomon_codificar(mensaje, n, p):
4     k = len(mensaje)
5     if n <= k:
6         raise ValueError("n debe ser > k")
7     alphas = list(range(n))
8     codeword = []
9     for alpha in alphas:
10         y = 0
11         for j, m in enumerate(mensaje):
12             y = (y + m * pow(alpha, j, p)) % p
13         codeword.append(y)
14     return codeword, alphas
15
16 def reed_solomon_decodificar(codeword, alphas, k, p):
17     n = len(codeword)
18     # Asumimos que los primeros k son correctos (version simple)
19     x_vals = alphas[:k]
20     y_vals = codeword[:k]
21
22     def lagrange_eval(x, x_vals, y_vals, p):
23         k_ = len(x_vals)
24         result = 0
25         for j in range(k_):
26             term = y_vals[j]
27             for m in range(k_):
28                 if m != j:
29                     term = term * (x - x_vals[m]) % p
30                     term = term * pow((x_vals[j] - x_vals[m]) % p, -1, p) % p
31             result = (result + term) % p
32         return result
33
34     mensaje = []
35     for x in range(k):
36         mensaje.append(lagrange_eval(x, x_vals, y_vals, p))
37     return mensaje
38
39 p = 97
40 k = 3

```

```

41 n = 7
42 mensajes = [[1, 2, 3], [5, 10, 15], [20, 30, 40]]
43
44 for mensaje in mensajes:
45     codeword, alphas = reed_solomon_codificar(mensaje, n, p)
46     mensaje_rec = reed_solomon_decodificar(codeword, alphas, k, p)
47     print(f"Original: {mensaje}, Recuperado: {mensaje_rec}")

```

Listing 112: Prueba de Reed-Solomon

C3. Ver el código de filtrado de la Sección .

C4. Ver el código de compresión de la Sección .

C5.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def chebyshev_spectral_poisson_reaccion(f, N):
5     x = np.cos(np.linspace(0, np.pi, N+1))
6     D = np.zeros((N+1, N+1))
7     for i in range(N+1):
8         for j in range(N+1):
9             if i != j:
10                 D[i, j] = (-1)**(i+j) / (x[i] - x[j])
11     for i in range(N+1):
12         D[i, i] = -np.sum(D[i, :])
13     D2 = D @ D
14     # -u'' + u = f
15     A = -D2 + np.eye(N+1)
16     A_int = A[1:-1, 1:-1]
17     x_int = x[1:-1]
18     b = f(x_int)
19     u_int = np.linalg.solve(A_int, b)
20     u = np.zeros(N+1)
21     u[1:-1] = u_int
22     return x, u
23
24 def f(x):
25     # Solucion exacta: u(x) = sin(pi*x/2) * cos(pi*x/2)
26     return (1 + (np.pi/2)**2) * np.sin(np.pi*x/2) * np.cos(np.pi*x/2)
27
28 N = 20
29 x, u = chebyshev_spectral_poisson_reaccion(f, N)
30
31 x_exact = np.linspace(-1, 1, 100)
32 u_exact = np.sin(np.pi*x_exact/2) * np.cos(np.pi*x_exact/2)
33
34 plt.figure(figsize=(10, 6))
35 plt.plot(x, u, 'ro', label='Solucion espectral')
36 plt.plot(x_exact, u_exact, 'k-', label='Solucion exacta')
37 plt.xlabel('x')
38 plt.ylabel('u(x)')
39 plt.title('-u\'' + u = (1+(pi/2)^2) sin(pi x/2) cos(pi x/2)')
40 plt.legend()
41 plt.grid(True)
42 plt.show()

```

Listing 113: Poisson con termino de reaccion usando Chebyshev

C6.

```

1 import numpy as np

```

```

2 import matplotlib.pyplot as plt
3
4 def diferencias_finitas_poisson(f, N):
5     h = 2 / N
6     x = np.linspace(-1, 1, N+1)
7     A = np.zeros((N-1, N-1))
8     for i in range(N-1):
9         A[i, i] = 2 / h**2
10        if i > 0:
11            A[i, i-1] = -1 / h**2
12        if i < N-2:
13            A[i, i+1] = -1 / h**2
14    b = f(x[1:-1])
15    u_int = np.linalg.solve(A, b)
16    u = np.zeros(N+1)
17    u[1:-1] = u_int
18    return x, u
19
20 def f(x):
21     return np.exp(x) * (1 - x**2) * (-2)
22
23 Ns = np.arange(4, 30, 2)
24 errores_cheb = []
25 errores_df = []
26
27 for N in Ns:
28     # Chebyshev
29     x_ch, u_ch = chebyshev_spectral_poisson(f, N)
30     u_exact_ch = np.exp(x_ch) * (1 - x_ch**2)
31     errores_cheb.append(np.max(np.abs(u_ch[1:-1] - u_exact_ch[1:-1])))
32
33     # Diferencias finitas
34     x_df, u_df = diferencias_finitas_poisson(f, N)
35     u_exact_df = np.exp(x_df) * (1 - x_df**2)
36     errores_df.append(np.max(np.abs(u_df[1:-1] - u_exact_df[1:-1])))
37
38 plt.figure(figsize=(10, 6))
39 plt.semilogy(Ns, errores_cheb, 'o-', label='Chebyshev espectral')
40 plt.semilogy(Ns, errores_df, 's-', label='Diferencias finitas')
41 plt.xlabel('Numero de puntos')
42 plt.ylabel('Error maximo')
43 plt.title('Chebyshev vs Diferencias finitas para -u\'\'\' = f(x)')
44 plt.legend()
45 plt.grid(True)
46 plt.show()

```

Listing 114: Comparacion: Chebyshev vs diferencias finitas

C7.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generamos una imagen de prueba
5 n = 128
6 imagen = np.zeros((n, n))
7 for i in range(n):
8     for j in range(n):
9         imagen[i, j] = np.sin(2*np.pi*5*i/n) * np.cos(2*np.pi*3*j/n)
10
11 # DFT 2D
12 F = np.fft.fft2(imagen)

```

```

13 F_shift = np.fft.fftshift(F)
14
15 # Espectro de magnitud
16 magnitud = np.log(np.abs(F_shift) + 1)
17
18 plt.figure(figsize=(12, 5))
19 plt.subplot(1, 2, 1)
20 plt.imshow(imagen, cmap='gray')
21 plt.title('Imagen original')
22 plt.axis('off')
23
24 plt.subplot(1, 2, 2)
25 plt.imshow(magnitud, cmap='gray')
26 plt.title('Espectro de frecuencias (log)')
27 plt.axis('off')
28
29 plt.tight_layout()
30 plt.show()

```

Listing 115: DFT 2D de una imagen

C8. Ver el código de la Sección .

C9.

```

1 import numpy as np
2 import random
3
4 # Usamos las funciones de la Sección 2
5 # Codificamos un mensaje
6 p = 97
7 k = 3
8 n = 7
9 mensaje = [10, 20, 30]
10 codeword, alphas = reed_solomon_codificar(mensaje, n, p)
11
12 # Introducimos errores aleatorios
13 num_errores = 2
14 errores_indices = random.sample(range(n), num_errores)
15 codeword_corrupto = codeword.copy()
16 for idx in errores_indices:
17     codeword_corrupto[idx] = (codeword_corrupto[idx] + random.randint(1, p-1)) %
18     p
19 print(f"Codeword original: {codeword}")
20 print(f"Codeword corrupto: {codeword_corrupto}")
21 print(f"Errores en indices: {errores_indices}")
22
23 # Decodificamos (si sabemos los errores)
24 mensaje_rec = reed_solomon_decodificar(codeword_corrupto, alphas, k, p)
25 print(f"Mensaje recuperado: {mensaje_rec}")
26 print(f"Mensaje original: {mensaje}")

```

Listing 116: Reed-Solomon con errores aleatorios

C10.

```

1 import numpy as np
2 import random
3
4 # 1. Generamos un secreto y sus shares con Shamir
5 p = 101
6 secreto = 42

```

```

7 n_shares = 5
8 k_shares = 3
9
10 shares, _ = shamir_generar_secretos(secreto, n_shares, k_shares, p)
11 print(f"Shares generados: {shares}")
12
13 # 2. Codificamos cada share con Reed-Solomon para transmitirlo de forma robusta
14 k_rs = 2 # Longitud del mensaje RS (cada share se divide en 2 simbolos)
15 n_rs = 5 # Longitud del codigo RS
16
17 shares_codificados = []
18 for share in shares:
19     x, y = share
20     # Codificamos el par (x, y) como un mensaje de Reed-Solomon
21     mensaje_rs = [x, y]
22     codeword_rs, _ = reed_solomon_codificar(mensaje_rs, n_rs, p)
23     shares_codificados.append(codeword_rs)
24
25 print(f"Shares codificados con RS: {shares_codificados}")
26
27 # 3. Simulamos que algunos simbolos se pierden en la transmision
28 # Introducimos errores en algunos shares codificados
29 for i in range(len(shares_codificados)):
30     if i % 2 == 0: # Corrompemos la mitad de los shares
31         shares_codificados[i][1] = (shares_codificados[i][1] + 1) % p
32         shares_codificados[i][3] = (shares_codificados[i][3] + 2) % p
33
34 # 4. Decodificamos cada share usando Reed-Solomon
35 shares_recuperados = []
36 for codeword_rs in shares_codificados:
37     # Asumimos que sabemos cuales son los errores (para simplificar)
38     # En la practica se usaria un decodificador de errores
39     # Usamos los primeros k_rs simbolos (version simple)
40     x_rec = codeword_rs[0]
41     y_rec = codeword_rs[1]
42     shares_recuperados.append((x_rec, y_rec))
43
44 print(f"Shares recuperados: {shares_recuperados}")
45
46 # 5. Reconstruimos el secreto con los shares recuperados
47 if len(shares_recuperados) >= k_shares:
48     secreto_rec = shamir_reconstruir(shares_recuperados[:k_shares], p)
49     print(f"Secreto recuperado: {secreto_rec}")
50     print(f"Secreto original: {secreto}")
51 else:
52     print("No hay suficientes shares para reconstruir el secreto")

```

Listing 117: Shamir + Reed-Solomon: transmision robusta de shares

Palabras Finales

Y bueno, loco, llegamos al final de este recorrido por las aplicaciones modernas. Viste que no era tan complicado. Arrancamos con Shamir para compartir secretos, pasamos por Reed-Solomon para corregir errores, el procesamiento de señales con Fourier, y terminamos con los metodos espectrales para resolver PDEs. Todo esta conectado: las herramientas que desarrollamos a lo largo del curso tienen aplicaciones concretas y poderosas en el mundo real.

Los puntos clave para llevar:

- Shamir usa interpolacion en cuerpos finitos para compartir secretos de forma segura.
- Reed-Solomon usa interpolacion para corregir errores en la transmision de datos.
- Fourier es la herramienta fundamental para el procesamiento de señales y la compresion.
- Los metodos espectrales dan precision exponencial para resolver PDEs suaves.
- Todas estas aplicaciones se basan en los conceptos que aprendiste: interpolacion, ortogonalidad, transformadas, y metodos iterativos.

Ahora solo falta practicar. Los ejercicios estan ahi para que te ensucies las manos. No te frustres si al principio no te salen; a mi me paso lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. Las aplicaciones modernas del calculo numerico son infinitas, y con las herramientas que aprendiste, estas listo para enfrentar cualquier problema que se te presente. ¡Dale para adelante, que vos puedes!

Apéndice

Todo lo que necesitas saber en un solo lugar, con Python y mucho café.

Che, ¿cómo andas? Bienvenido a este mega-apéndice donde vamos a juntar todo lo que vimos a lo largo del libro. La idea es que tengas un lugar donde encontrar las fórmulas, los algoritmos, las implementaciones, las tablas de complejidad, y todo el material de referencia que necesitas para resolver problemas de cálculo numérico. Es como tener una navaja suiza: no siempre la usas, pero cuando la necesitas, la tienes a mano.

Cuando estaba preparando el final de Análisis Numérico, me acuerdo de haber pasado horas buscando entre mis apuntes una fórmula que había visto en la primera clase. Desde ese día, siempre tengo un resumen de fórmulas y algoritmos a mano. Este apéndice es exactamente eso: un resumen de todo lo importante.

Así que agarra un café, ponete cómodo, y vamos a repasar todo de forma rápida y práctica.

Apéndice A: Formulario de Cálculo Numérico

Errores y precisión numérica

- **Error absoluto:** $E_a = |x - \tilde{x}|$
 - Mide la diferencia real entre el valor exacto y el aproximado.
 - Es útil cuando los valores son de magnitud similar.
- **Error relativo:** $E_r = \frac{|x - \tilde{x}|}{|x|}$ (para $x \neq 0$)
 - Permite comparar errores cuando los valores tienen escalas diferentes.
 - Es la medida más significativa en cálculo numérico.
- **Epsilon de máquina:** $\epsilon_{mach} = \beta^{1-t}$
 - Límite de precisión del sistema IEEE-754.
 - Para double precisión: $\epsilon_{mach} \approx 2,22 \times 10^{-16}$.
 - Unit roundoff: $u = \frac{1}{2}\epsilon_{mach}$.
- **Modelo de aritmética de punto flotante:**

$$\text{fl}(a \circ b) = (a \circ b)(1 + \delta), \quad |\delta| \leq u$$

- Cada operación introduce un error relativo de a lo sumo u .

Normas vectoriales y matriciales

- **Norma 1:** $\|x\|_1 = \sum_{i=1}^n |x_i|$
- **Norma 2 (Euclidiana):** $\|x\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2}$
- **Norma infinito:** $\|x\|_\infty = \max_i |x_i|$
- **Norma matricial inducida:** $\|A\| = \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|}$

- $\|A\|_1 = \max_j \sum_i |a_{ij}|$ (maximo por columnas)
- $\|A\|_\infty = \max_i \sum_j |a_{ij}|$ (maximo por filas)
- $\|A\|_2 = \sigma_{\max}(A)$ (maximo valor singular)
- **Norma de Frobenius:** $\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2} = \sqrt{\sum_i \sigma_i^2}$

Numero de condicion

$$\kappa(A) = \|A\| \|A^{-1}\|$$

- Para norma 2: $\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$
- Siempre $\kappa(A) \geq 1$
- Si $\kappa(A) \gg 1$, la matriz esta mal condicionada.
- Cota de error: $\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\Delta b\|}{\|b\|}$

Factorizaciones matriciales

- **LU:** $A = LU$, L triangular inferior, U triangular superior.
- **PLU:** $PA = LU$, P matriz de permutacion.
- **QR:** $A = QR$, $Q^T Q = I$, R triangular superior.
- **SVD:** $A = U \Sigma V^T$, U, V ortogonales, Σ diagonal.
- **Cholesky:** $A = LL^T$, A simetrica definida positiva.

Ecuaciones no lineales (una variable)

- **Biseccion:** $c = \frac{a+b}{2}$, error $\leq \frac{b-a}{2^n}$
- **Newton:** $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$, convergencia cuadratica.
- **Secante:** $x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$, orden $\approx 1,618$
- **Punto fijo:** $x_{n+1} = g(x_n)$, converge si $|g'(x)| < 1$.

Integracion numerica

- **Trapezio:** $\int_a^b f(x)dx \approx \frac{b-a}{2}(f(a) + f(b))$, error $O(h^3)$
- **Trapezio compuesto:** $T_m = \frac{h}{2}[f(a) + 2 \sum_{i=1}^{m-1} f(x_i) + f(b)]$, error $O(h^2)$
- **Simpson:** $\int_a^b f(x)dx \approx \frac{b-a}{6}(f(a) + 4f(\frac{a+b}{2}) + f(b))$, error $O(h^5)$
- **Simpson compuesto:** $S_m = \frac{h}{3}[f(a) + 4 \sum_{i=0}^{m-1} f(x_{2i+1}) + 2 \sum_{i=1}^{m-1} f(x_{2i}) + f(b)]$, error $O(h^4)$
- **Gauss-Legendre:** $\int_{-1}^1 f(x)dx \approx \sum_{i=0}^n A_i f(x_i)$, x_i raices de Legendre, error $O(h^{2n+1})$

Ecuaciones diferenciales ordinarias

- **Euler:** $x_{n+1} = x_n + hf(t_n, x_n)$, error $O(h)$
- **Euler implícito:** $x_{n+1} = x_n + hf(t_{n+1}, x_{n+1})$, A-estable.
- **Taylor orden 2:** $x_{n+1} = x_n + hf + \frac{h^2}{2}(f_t + f_x f)$, error $O(h^2)$
- **RK4:** $x_{n+1} = x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$, error $O(h^4)$

Interpolacion

- **Lagrange:** $p(x) = \sum_{i=0}^n y_i \ell_i(x)$, $\ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$
- **Forma baricentrica:** $p(x) = \frac{\sum_{j=0}^n \frac{w_j y_j}{x - x_j}}{\sum_{j=0}^n \frac{w_j}{x - x_j}}$, $w_j = \frac{1}{\prod_{k \neq j} (x_j - x_k)}$
- **Error de interpolacion:** $f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{k=0}^n (x - x_k)$
- **Chebyshev:** $T_n(x) = \cos(n \arccos x)$, nodos $x_k = \cos(\frac{2k+1}{2n}\pi)$

Transformada Discreta de Fourier

- **DFT:** $X_k = \sum_{n=0}^{N-1} x_n \omega_N^{-kn}$, $\omega_N = e^{2\pi i/N}$
- **IDFT:** $x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \omega_N^{kn}$
- **Teorema de convolucion:** $DFT(x * y) = DFT(x) \cdot DFT(y)$
- **Parseval:** $\sum |x_n|^2 = \frac{1}{N} \sum |X_k|^2$

Optimizacion

- **Newton:** $x_{k+1} = x_k - H(x_k)^{-1} \nabla f(x_k)$, convergencia cuadratica.
- **Descenso por gradiente:** $x_{k+1} = x_k - \alpha \nabla f(x_k)$, convergencia lineal.
- **Momentum:** $v_{k+1} = \beta v_k + (1 - \beta) \nabla f(x_k)$, $x_{k+1} = x_k - \alpha v_{k+1}$

Apéndice B: Algoritmos en Pseudocodigo

Eliminacion Gaussiana

```

1 Para k = 1 hasta n-1:
2     // Buscar pivote
3     p = argmax(|A[i,k]| para i de k a n)
4     Si p != k:
5         Intercambiar filas k y p
6
7     // Eliminar debajo del pivote
8     Para i = k+1 hasta n:
9         factor = A[i,k] / A[k,k]
10        Para j = k hasta n:
11            A[i,j] = A[i,j] - factor * A[k,j]
12        b[i] = b[i] - factor * b[k]
```

```

13
14 // Sustitucion hacia atras
15 x[n] = b[n] / A[n,n]
16 Para i = n-1 hasta 1:
17     suma = 0
18     Para j = i+1 hasta n:
19         suma = suma + A[i,j] * x[j]
20     x[i] = (b[i] - suma) / A[i,i]
21 Fin

```

Listing 1: Eliminacion Gaussiana (pseudocodigo)

Factorizacion LU

```

1 Para k = 1 hasta n-1:
2     Para i = k+1 hasta n:
3         L[i,k] = A[i,k] / A[k,k]
4         Para j = k+1 hasta n:
5             A[i,j] = A[i,j] - L[i,k] * A[k,j]
6         A[i,k] = 0
7
8 // U es la parte triangular superior de A
9 // L es la triangular inferior con 1 en diagonal

```

Listing 2: Factorizacion LU (pseudocodigo)

Factorizacion QR (Gram-Schmidt)

```

1 Para k = 1 hasta n:
2     v_k = a_k
3     Para j = 1 hasta k-1:
4         R[j,k] = <v_k, q_j>
5         v_k = v_k - R[j,k] * q_j
6     R[k,k] = ||v_k||
7     q_k = v_k / R[k,k]

```

Listing 3: QR via Gram-Schmidt (pseudocodigo)

Jacobi

```

1 Para iter = 1 hasta max_iter:
2     Para i = 1 hasta n:
3         x_new[i] = (b[i] - sum_{j != i} A[i,j] * x[j]) / A[i,i]
4     x = x_new
5     Si error < tol:
6         break

```

Listing 4: Jacobi (pseudocodigo)

Gauss-Seidel

```

1 Para iter = 1 hasta max_iter:
2     Para i = 1 hasta n:
3         x[i] = (b[i] - sum_{j < i} A[i,j] * x[j] - sum_{j > i} A[i,j] * x[j]) /
4             A[i,i]

```

```

4     Si error < tol:
5         break

```

Listing 5: Gauss-Seidel (pseudocodigo)

Newton (1D)

```

1 x = x0
2 Para iter = 1 hasta max_iter:
3     x_new = x - f(x) / f'(x)
4     Si |x_new - x| < tol:
5         break
6     x = x_new

```

Listing 6: Newton (pseudocodigo)

FFT (Cooley-Tukey)

```

1 Si N == 1:
2     retornar x
3
4 // Separar pares e impares
5 x_par = x[0::2]
6 x_impar = x[1::2]
7
8 // FFT recursiva
9 X_par = FFT(x_par)
10 X_impar = FFT(x_impar)
11
12 // Combinar
13 Para k = 0 hasta N/2-1:
14     factor = exp(-2*pi*i*k/N)
15     X[k] = X_par[k] + factor * X_impar[k]
16     X[k+N/2] = X_par[k] - factor * X_impar[k]

```

Listing 7: FFT (pseudocodigo)

Simpson Compuesto

```

1 h = (b - a) / n // n debe ser par
2 suma = f(a) + f(b)
3 Para i = 1 hasta n-1 con paso 2:
4     suma = suma + 4 * f(a + i*h)
5 Para i = 2 hasta n-2 con paso 2:
6     suma = suma + 2 * f(a + i*h)
7 resultado = (h/3) * suma

```

Listing 8: Simpson compuesto (pseudocodigo)

Euler

```

1 t = t0
2 x = x0
3 Mientras t < tf:
4     x = x + h * f(t, x)

```

```
5 t = t + h
```

Listing 9: Euler (pseudocodigo)

Apéndice C: Implementaciones en Python

Eliminacion Gaussiana con pivoteo

```
1 import numpy as np
2
3 def gauss_pivoteo(A, b):
4     """
5     Resuelve  $Ax = b$  con eliminacion gaussiana y pivoteo parcial.
6     """
7     n = len(b)
8     A = np.array(A, dtype=float)
9     b = np.array(b, dtype=float)
10
11     # Eliminacion
12     for k in range(n-1):
13         # Pivoteo parcial
14         p = np.argmax(np.abs(A[k:n, k])) + k
15         if p != k:
16             A[[k, p]] = A[[p, k]]
17             b[[k, p]] = b[[p, k]]
18
19         # Eliminar debajo del pivote
20         for i in range(k+1, n):
21             factor = A[i, k] / A[k, k]
22             A[i, k:] -= factor * A[k, k:]
23             b[i] -= factor * b[k]
24
25     # Sustitucion hacia atras
26     x = np.zeros(n)
27     for i in range(n-1, -1, -1):
28         x[i] = (b[i] - np.dot(A[i, i+1:], x[i+1:])) / A[i, i]
29
30     return x
31
32 # Ejemplo
33 A = np.array([[2, 1], [1, 3]], dtype=float)
34 b = np.array([5, 6], dtype=float)
35 x = gauss_pivoteo(A, b)
36 print("Solucion:", x)
37
38 # Verificacion
39 print("Verificacion  $Ax - b =$ ", A @ x - b)
```

Listing 10: Gauss con pivoteo (desde cero)

Factorizacion LU

```
1 def lu_factor(A):
2     """
3     Factorizacion LU sin pivoteo.
4     """
5     n = len(A)
```

```

6     A = np.array(A, dtype=float)
7     L = np.eye(n)
8     U = np.zeros((n, n))
9
10    for k in range(n):
11        # Calcular U[k, k:]
12        for j in range(k, n):
13            U[k, j] = A[k, j] - np.dot(L[k, :k], U[:k, j])
14        # Calcular L[k+1:, k]
15        for i in range(k+1, n):
16            L[i, k] = (A[i, k] - np.dot(L[i, :k], U[:k, k])) / U[k, k]
17
18    return L, U
19
20 # Ejemplo
21 A = np.array([[2, 1, 1], [4, -6, 0], [-2, 7, 2]], dtype=float)
22 L, U = lu_factor(A)
23 print("L =")
24 print(L)
25 print("U =")
26 print(U)
27 print("L U =")
28 print(L @ U)

```

Listing 11: LU (desde cero)

QR via Gram-Schmidt modificado

```

1 def qr_gram_schmidt(A):
2     """
3     Factorizacion QR via Gram-Schmidt modificado.
4     """
5     m, n = A.shape
6     Q = np.zeros((m, n))
7     R = np.zeros((n, n))
8
9     for j in range(n):
10        v = A[:, j].copy()
11        for i in range(j):
12            R[i, j] = np.dot(v, Q[:, i])
13            v = v - R[i, j] * Q[:, i]
14        R[j, j] = np.linalg.norm(v)
15        Q[:, j] = v / R[j, j]
16
17    return Q, R
18
19 # Ejemplo
20 A = np.array([[1, 1], [1, 0], [0, 1]], dtype=float)
21 Q, R = qr_gram_schmidt(A)
22 print("Q =")
23 print(Q)
24 print("R =")
25 print(R)
26 print("Q R =")
27 print(Q @ R)
28 print("Q^T Q =")
29 print(Q.T @ Q)

```

Listing 12: QR (desde cero)

SVD desde cero

```

1 def svd(A):
2     """
3     Calcula la SVD de A usando numpy para autovalores.
4     """
5     A = np.array(A, dtype=float)
6     m, n = A.shape
7
8     # Calcular  $A^T A$ 
9     ATA = A.T @ A
10
11     # Autovalores y autovectores de  $A^T A$ 
12     autovalores, V = np.linalg.eigh(ATA)
13
14     # Ordenar de mayor a menor
15     idx = np.argsort(autovalores)[::-1]
16     autovalores = autovalores[idx]
17     V = V[:, idx]
18
19     # Valores singulares
20     sigma = np.sqrt(np.maximum(autovalores, 0))
21     Sigma = np.zeros((m, n))
22     min_dim = min(m, n)
23     Sigma[:min_dim, :min_dim] = np.diag(sigma[:min_dim])
24
25     # Vectores singulares izquierdos
26     U = np.zeros((m, m))
27     for i in range(min_dim):
28         if sigma[i] > 1e-12:
29             U[:, i] = A @ V[:, i] / sigma[i]
30     # Completar U (si es necesario)
31     if m > min_dim:
32         # Gram-Schmidt para completar la base
33         for i in range(min_dim, m):
34             v = np.random.randn(m)
35             for j in range(i):
36                 v = v - np.dot(v, U[:, j]) * U[:, j]
37             U[:, i] = v / np.linalg.norm(v)
38
39     return U, Sigma, V.T
40
41 # Ejemplo
42 A = np.array([[1, 1], [1, 0], [0, 1]], dtype=float)
43 U, S, Vt = svd(A)
44 print("U =")
45 print(U)
46 print("Sigma =")
47 print(S)
48 print("V^T =")
49 print(Vt)
50 print("U Sigma V^T =")
51 print(U @ S @ Vt)

```

Listing 13: SVD (desde cero)

Apéndice D: Implementaciones desde cero (sin NumPy)

Eliminacion Gaussiana sin NumPy

```

1 def gauss_sin_numpy(A, b):
2     """
3     Eliminacion gaussiana sin usar NumPy.
4     """
5     n = len(b)
6     # Copiar matrices
7     A = [fila[:] for fila in A]
8     b = b[:]
9
10    # Eliminacion
11    for k in range(n-1):
12        # Pivoteo
13        p = k
14        for i in range(k, n):
15            if abs(A[i][k]) > abs(A[p][k]):
16                p = i
17        if p != k:
18            A[k], A[p] = A[p], A[k]
19            b[k], b[p] = b[p], b[k]
20
21        # Eliminar
22        for i in range(k+1, n):
23            factor = A[i][k] / A[k][k]
24            for j in range(k, n):
25                A[i][j] -= factor * A[k][j]
26            b[i] -= factor * b[k]
27
28    # Sustitucion hacia atras
29    x = [0] * n
30    for i in range(n-1, -1, -1):
31        suma = 0
32        for j in range(i+1, n):
33            suma += A[i][j] * x[j]
34        x[i] = (b[i] - suma) / A[i][i]
35
36    return x
37
38 # Ejemplo
39 A = [[2, 1], [1, 3]]
40 b = [5, 6]
41 x = gauss_sin_numpy(A, b)
42 print("Solucion:", x)

```

Listing 14: Gauss sin NumPy

Metodo de Newton sin NumPy

```

1 def newton_sin_numpy(f, df, x0, tol=1e-10, max_iter=100):
2     """Newton sin NumPy."""
3     x = x0
4     for i in range(max_iter):
5         x_new = x - f(x) / df(x)
6         if abs(x_new - x) < tol:
7             break
8         x = x_new

```

```

9     return x
10
11 # Ejemplo: sqrt(2)
12 def f(x):
13     return x**2 - 2
14 def df(x):
15     return 2*x
16
17 raiz = newton_sin_numpy(f, df, 1.5)
18 print("Raiz de x^2 - 2 =", raiz)
19 print("Raiz exacta =", 2**0.5)

```

Listing 15: Newton sin NumPy

Simpson compuesto sin NumPy

```

1 def simpson_sin_numpy(f, a, b, n):
2     """Simpson compuesto sin NumPy."""
3     if n % 2 == 1:
4         n += 1
5     h = (b - a) / n
6     suma = f(a) + f(b)
7     for i in range(1, n, 2):
8         suma += 4 * f(a + i*h)
9     for i in range(2, n-1, 2):
10        suma += 2 * f(a + i*h)
11    return (h/3) * suma
12
13 # Ejemplo: integral de x^2 de 0 a 1
14 def f(x):
15     return x**2
16
17 resultado = simpson_sin_numpy(f, 0, 1, 10)
18 print("Integral de x^2 de 0 a 1 =", resultado)
19 print("Exacto =", 1/3)

```

Listing 16: Simpson sin NumPy

Apéndice E: Implementaciones con NumPy

Sistemas lineales

```

1 import numpy as np
2
3 # Metodo directo: solve
4 A = np.array([[2, 1], [1, 3]], dtype=float)
5 b = np.array([5, 6], dtype=float)
6 x = np.linalg.solve(A, b)
7 print("solve:", x)
8
9 # Inversa
10 A_inv = np.linalg.inv(A)
11 x_inv = A_inv @ b
12 print("inversa:", x_inv)
13
14 # Factorizacion LU
15 from scipy.linalg import lu

```

```

16 P, L, U = lu(A)
17 print("LU:")
18 print("P =", P)
19 print("L =", L)
20 print("U =", U)
21
22 # QR
23 Q, R = np.linalg.qr(A)
24 print("QR:")
25 print("Q =", Q)
26 print("R =", R)
27
28 # SVD
29 U, s, Vt = np.linalg.svd(A)
30 print("SVD:")
31 print("U =", U)
32 print("s =", s)
33 print("Vt =", Vt)
34
35 # Autovalores
36 autovalores = np.linalg.eigvals(A)
37 print("Autovalores:", autovalores)

```

Listing 17: Sistemas lineales con NumPy

Ecuaciones diferenciales

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4
5 def f(t, x):
6     return -x + np.sin(t)
7
8 sol = solve_ivp(f, [0, 10], [0], method='RK45', t_eval=np.linspace(0, 10, 100))
9
10 plt.plot(sol.t, sol.y[0])
11 plt.xlabel('t')
12 plt.ylabel('x(t)')
13 plt.title('Solucion de x\' = -x + sin(t)')
14 plt.grid(True)
15 plt.show()

```

Listing 18: EDOs con SciPy

FFT con NumPy

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 N = 1000
5 t = np.linspace(0, 1, N)
6 x = np.sin(2*np.pi*50*t) + 0.5*np.sin(2*np.pi*120*t)
7
8 X = np.fft.fft(x)
9 freqs = np.fft.fftfreq(N, d=1/N)
10
11 plt.subplot(2,1,1)

```

```

12 plt.plot(t, x)
13 plt.title('Se al en tiempo')
14
15 plt.subplot(2,1,2)
16 plt.stem(freqs[:N//2], np.abs(X[:N//2]))
17 plt.title('Espectro de frecuencia')
18 plt.xlabel('Frecuencia (Hz)')
19 plt.tight_layout()
20 plt.show()

```

Listing 19: FFT con NumPy

Apéndice F: Tabla de Algoritmos

—p3cm—p3cm—p3cm—p3cm—p3cm—

Problema	Metodo	Complejidad	Error	Convergencia
Ax=b	Gauss	$O(n^3)$	Estable	Directa
Ax=b	LU	$O(n^3)$	Estable	Directa
Ax=b	QR	$O(n^3)$	Muy estable	Directa
Ax=b	Jacobi	$O(n^2)$ por iteracion	Depende	Lineal (si $\rho < 1$)
Ax=b	Gauss-Seidel	$O(n^2)$ por iteracion	Depende	Lineal (si $\rho < 1$)
f(x)=0	Biseccion	$O(\log(1/\epsilon))$	$O(2^{-n})$	Lineal
f(x)=0	Newton	$O(1)$ por iteracion	$O(h^2)$	Cuadratica
f(x)=0	Secante	$O(1)$ por iteracion	$O(h^{1.618})$	Orden 1.618
Integracion	Trapezio	$O(n)$	$O(h^2)$	—
Integracion	Simpson	$O(n)$	$O(h^4)$	—
Integracion	Gauss-Legendre	$O(n)$	$O(h^{2n+1})$	Exponencial
EDO	Euler	$O(n)$	$O(h)$	Lineal
EDO	RK4	$O(n)$	$O(h^4)$	Lineal
EDO	Euler implicito	$O(n)$	$O(h)$	A-estable
Interpolacion	Lagrange	$O(n^2)$	Depende	—
Interpolacion	Chebyshev	$O(n^2)$	$O(2^{-n})$	Exponencial
DFT	Directa	$O(N^2)$	Exacto	—
DFT	FFT	$O(N \log N)$	Exacto	—
Optimizacion	Newton	$O(n^3)$ por iteracion	$O(h^2)$	Cuadratica
Optimizacion	Gradiente	$O(n)$ por iteracion	$O(h)$	Lineal

Apéndice G: Complejidad Computacional

—p5cm—p5cm—

Algoritmo	Complejidad
Eliminacion Gaussiana	$O(n^3)$
Factorizacion LU	$O(n^3)$

Factorizacion QR (Gram-Schmidt) $O(n^3)$
 Factorizacion QR (Householder) $O(n^3)$
 SVD $O(n^3)$
 Cholesky $O(n^3/3)$
 FFT $O(N \log N)$
 Biseccion $O(\log(1/\epsilon))$
 Newton (1D) $O(1)$ por iteracion
 Newton (nD) $O(n^3)$ por iteracion
 Descenso por gradiente $O(n)$ por iteracion
 Jacobi $O(n^2)$ por iteracion
 Gauss-Seidel $O(n^2)$ por iteracion
 Trapecio compuesto $O(n)$
 Simpson compuesto $O(n)$
 Gauss-Legendre $O(n)$
 Euler $O(n)$
 RK4 $O(n)$
 Metodos espectrales $O(N^3)$ o $O(N \log N)$

Apéndice H: Errores mas Comunes

1. Cancelacion Catastrofica

```

1 # Ejemplo de cancelacion
2 x = 1.234567890123456
3 y = 1.234567890123455
4 print(x - y) # Resultado: 1e-15 (pierde precision)
5
6 # Solucion: evitar restar numeros casi iguales
7 # Usar formulas equivalentes

```

Listing 20: Cancelacion catastrofica

2. Overflow

```

1 # Overflow en punto flotante
2 try:
3     x = 1e308 * 1e308
4     print(x) # Infinito
5 except OverflowError:
6     print("Overflow!")
7
8 # Evitar: reescalar o usar logaritmos

```

Listing 21: Overflow

3. Underflow

```

1 # Underflow en punto flotante
2 x = 1e-308 / 1e308
3 print(x) # 0.0 (pierde informacion)
4

```

```
5 # Evitar: reescalar o usar logaritmos
```

Listing 22: Underflow

4. Perdida de Precision al sumar numeros de diferente magnitud

```
1 # Perdida de precision
2 x = 1e16 + 1
3 print(x - 1e16) # 0.0 (el 1 se pierde)
4
5 # Solucion: sumar primero los numeros pequenos
6 suma = 0
7 for i in range(1000000):
8     suma += 1
9 resultado = 1e16 + suma
10 print(resultado - 1e16) # 1000000 (aproximadamente)
```

Listing 23: Suma de numeros de diferente magnitud

5. Matriz mal condicionada

```
1 import numpy as np
2
3 # Matriz de Hilbert
4 H = np.array([[1, 1/2, 1/3], [1/2, 1/3, 1/4], [1/3, 1/4, 1/5]])
5 b = np.array([1, 1, 1])
6 x = np.linalg.solve(H, b)
7 print("Solucion:", x)
8 print("Condicion:", np.linalg.cond(H)) # Muy grande
9
10 # Pequena perturbacion
11 H_pert = H + 1e-10 * np.random.randn(3, 3)
12 x_pert = np.linalg.solve(H_pert, b)
13 print("Solucion perturbada:", x_pert)
14 print("Diferencia:", np.linalg.norm(x - x_pert))
```

Listing 24: Matriz mal condicionada

6. No verificar convergencia en metodos iterativos

```
1 # Jacobi sin criterio de parada
2 def jacobi_sin_tol(A, b, max_iter=100):
3     # ... implementacion
4     pass
5 # Problema: puede no converger o tardar demasiado
```

Listing 25: Convergencia de Jacobi

7. Error en la eleccion del paso en optimizacion

```
1 # Paso muy grande o muy chico
2 alpha = 1e-10 # Muy lento
3 alpha = 10    # Puede diverger
4
5 # Solucion: usar busqueda de linea o paso adaptativo
```

Apéndice I: Chuletas Matematicas

Normas

- $\|x\|_1 = \sum |x_i|$
- $\|x\|_2 = \sqrt{\sum |x_i|^2}$
- $\|x\|_\infty = \max |x_i|$
- $\|A\|_1 = \max_j \sum_i |a_{ij}|$
- $\|A\|_\infty = \max_i \sum_j |a_{ij}|$
- $\|A\|_2 = \sigma_{max}(A)$
- $\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2} = \sqrt{\sum_i \sigma_i^2}$

Producto interno

- $\langle x, y \rangle = \sum_i x_i y_i$ (espacio real)
- $\langle x, y \rangle = \sum_i x_i \overline{y_i}$ (espacio complejo)
- $\langle f, g \rangle = \int_a^b f(x)g(x)dx$
- $\langle f, g \rangle_w = \int_a^b f(x)g(x)w(x)dx$

Gradiente, Jacobiano, Hessiano

- $\nabla f(x) = \begin{pmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{pmatrix}$
- $J_F(x) = (\partial F_i / \partial x_j)_{i,j}$
- $H_f(x) = (\partial^2 f / \partial x_i \partial x_j)_{i,j}$

Matrices especiales

- **Matriz de Vandermonde:** $V = \begin{pmatrix} 1 & x_0 & \cdots & x_0^n \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \cdots & x_n^n \end{pmatrix}$
- **Matriz de Hilbert:** $H_{ij} = \frac{1}{i+j-1}$

- **Matriz companera:** $C = \begin{pmatrix} 0 & \cdots & 0 & -a_0 \\ 1 & \cdots & 0 & -a_1 \\ \vdots & & \vdots & \vdots \\ 0 & \cdots & 1 & -a_{n-1} \end{pmatrix}$
- **Matriz diagonal:** $D = \text{diag}(d_1, \dots, d_n)$
- **Matriz identidad:** $I = \text{diag}(1, \dots, 1)$

Descomposiciones

- LU: $A = LU$
- PLU: $PA = LU$
- QR: $A = QR$
- SVD: $A = U\Sigma V^T$
- Cholesky: $A = LL^T$ (SPD)

Familias de polinomios ortogonales

- **Legendre:** $w(x) = 1, [-1, 1]$
- **Chebyshev:** $w(x) = \frac{1}{\sqrt{1-x^2}}, [-1, 1]$
- **Laguerre:** $w(x) = e^{-x}, [0, \infty)$
- **Hermite:** $w(x) = e^{-x^2}, (-\infty, \infty)$

Apéndice J: Bibliografía Comentada

Bibliografía base

- **Duran, Lassalle y Rossi — Calculo Numerico.** Es la base del desarrollo clasico que usamos en este curso. Excelente para errores numericos, estabilidad, interpolacion, integracion y EDOs. Tiene muchos ejemplos y ejercicios resueltos. Ideal para entender los conceptos fundamentales con un enfoque practico.
- **Martin Maas — Apuntes de Calculo Numerico.** Complementa el libro anterior con una vision moderna del algebra lineal numerica: SVD, FFT, optimizacion, computabilidad y aplicaciones. Muy bueno para entender la conexion entre la teoria y las aplicaciones actuales como machine learning y procesamiento de señales.

Algebra Lineal Numerica

- **Golub y Van Loan — Matrix Computations.** Referencia clasica sobre factorizaciones, autovalores y metodos numericos matriciales. Es un libro avanzado, pero es la biblia del algebra lineal numerica. Si quieres profundizar en QR, SVD, y metodos iterativos, este es el libro.
- **Trefethen y Bau — Numerical Linear Algebra.** Excelente libro para entender QR, SVD y estabilidad. Es muy didactico y tiene un enfoque geometrico que ayuda a visualizar los conceptos. Perfecto para estudiantes que quieren ir mas alla de los calculos.

Analisis Numerico

- **Atkinson — Numerical Analysis.** Muy didactico para metodos clasicos: interpolacion, integracion, EDOs, ecuaciones no lineales. Tiene muchos ejemplos y ejercicios. Es un libro de referencia para los temas fundamentales.
- **Chapra y Canale — Numerical Methods for Engineers.** Enfoque aplicado con numerosos ejemplos de ingenieria. Excelente para ver como se usan los metodos numericos en problemas reales. Tiene mucho codigo y aplicaciones practicas.

Optimizacion

- **Nocedal y Wright — Numerical Optimization.** Referencia moderna sobre optimizacion numerica. Cubre desde metodos de gradiente hasta metodos de region de confianza y optimizacion restringida. Es el libro de referencia para temas avanzados en optimizacion.

Metodos Cientificos en Python

- **McKinney — Python for Data Analysis.** Excelente para aprender a usar Python con NumPy, pandas y matplotlib. Muy util para implementar los metodos numericos en la practica.
- **Kiusalaas — Numerical Methods in Engineering with Python.** Enfoque practico con muchos ejemplos de metodos numericos implementados en Python. Perfecto para ver como se aplican los algoritmos en codigo.

Apéndice K: Indice de Simbolos

—p3cm—p8cm—

Simbolo	Significado
---------	-------------

Simbolo	Significado
---------	-------------

ϵ	Epsilon de maquina
ϵ_{mach}	Epsilon de maquina
u	Unit roundoff ($u = \epsilon_{mach}/2$)
$\kappa(A)$	Numero de condicion de A
$\ A\ $	Norma matricial
$\ x\ $	Norma vectorial
$\det(A)$	Determinante de A
$\text{tr}(A)$	Traza de A
λ	Autovalor
σ	Valor singular
∇f	Gradiente de f
H	Matriz Hessiana
J_F	Matriz Jacobiana
ω_N	Raiz N -esima primitiva de la unidad ($e^{2\pi i/N}$)
$\rho(A)$	Radio espectral de A
ϕ	Funcion de incremento (metodos de un paso)
τ	Error de truncamiento local
h	Paso de discretizacion

α Paso de aprendizaje (optimizacion)
 β Factor de momentum
 μ Constante de convexidad fuerte
 L Constante de Lipschitz del gradiente
 $\mathbb{F}$ Cuerpo finito
 $\mathbb{Z}_p$ Cuerpo de enteros modulo p
 $\mathcal{P}_n$ Espacio de polinomios de grado $\leq n$
 $\ell_i(x)$ Polinomio base de Lagrange
 $T_n(x)$ Polinomio de Chebyshev de grado n
 $P_n(x)$ Polinomio de Legendre de grado n
 $f[x_0, \dots, x_n]$ Diferencia dividida
 $W_{n+1}(x)$ Polinomio nodal ($\prod_{k=0}^n (x - x_k)$)
 Λ_n Constante de Lebesgue
 $\text{fl}(x)$ Redondeo de punto flotante
 $\hat{f}_k$ Coeficiente de Fourier
 X_k DFT de x
 $x * y$ Convolucion discreta

Apéndice L: Guia de Software

Instalacion de Python y bibliotecas

Instalacion basica Para ejecutar todos los codigos de este libro, necesitas Python 3.7 o superior. La forma mas facil de instalar todo es usando Anaconda:

```

1 # Descargar e instalar Anaconda desde https://www.anaconda.com/download
2 # Luego, abrir una terminal y ejecutar:
3 conda install numpy scipy matplotlib jupyter

```

Listing 27: Instalacion con Anaconda

O usando pip:

```

1 pip install numpy scipy matplotlib jupyter

```

Listing 28: Instalacion con pip

NumPy

NumPy es la biblioteca fundamental para calculo numerico en Python. Proporciona arrays multidimensionales y funciones matematicas.

```

1 import numpy as np
2
3 # Crear arrays
4 a = np.array([1, 2, 3])
5 b = np.array([[1, 2], [3, 4]])
6
7 # Operaciones
8 c = a + a
9 d = b @ b # Producto matricial
10
11 # Funciones utiles
12 x = np.linspace(0, 1, 10)

```

```

13 y = np.sin(x)
14 z = np.random.randn(100)

```

Listing 29: Ejemplo basico de NumPy

SciPy

SciPy extiende NumPy con funciones para algebra lineal, integracion, optimizacion, EDOs, y mas.

```

1 from scipy.linalg import lu, svd
2 from scipy.integrate import quad, solve_ivp
3 from scipy.optimize import minimize, root
4
5 # Integracion
6 resultado, error = quad(lambda x: x**2, 0, 1)
7
8 # EDOs
9 sol = solve_ivp(lambda t, y: -y, [0, 1], [1])
10
11 # Optimizacion
12 res = minimize(lambda x: x**2, 1)

```

Listing 30: Ejemplo basico de SciPy

Matplotlib

Matplotlib es la biblioteca para graficos.

```

1 import matplotlib.pyplot as plt
2
3 x = np.linspace(0, 2*np.pi, 100)
4 y = np.sin(x)
5
6 plt.plot(x, y)
7 plt.xlabel('x')
8 plt.ylabel('sin(x)')
9 plt.title('Funcion seno')
10 plt.grid(True)
11 plt.show()

```

Listing 31: Ejemplo basico de Matplotlib

Jupyter Notebook

Jupyter Notebook es un entorno interactivo para escribir y ejecutar codigo en Python. Es ideal para aprender y experimentar.

```

1 # En la terminal:
2 jupyter notebook
3 # Luego abrir el navegador y crear un nuevo notebook

```

Listing 32: Iniciar Jupyter

VS Code

VS Code es un editor de código que soporta Python con depuración, completado de código, y notebooks integrados.

```
1 # Instalar VS Code desde https://code.visualstudio.com/  
2 # Instalar la extension de Python  
3 # Abrir la carpeta del proyecto  
4 # Crear un archivo .py o .ipynb
```

Listing 33: Configurar VS Code

Palabras Finales

Y bueno, loco, llegamos al final de este mega-apéndice. Con esto tenes todas las herramientas que necesitas para resolver problemas de cálculo numérico: fórmulas, algoritmos, implementaciones en Python, tablas de complejidad, errores comunes, chuletas matemáticas, bibliografía, símbolos, y una guía de software.

La idea es que esto sea tu compañero de viaje: cuando no te acuerdes de una fórmula, cuando necesites implementar un algoritmo, cuando quieras saber cual método usar, o cuando tengas un error y no sepas por qué, este apéndice está ahí para ayudarte.

Recordá que el cálculo numérico es una herramienta poderosa, pero también es un arte. No te limites a usar las fórmulas y los algoritmos como cajas negras. Entendé por qué funcionan, cuando fallan, y como adaptarlos a tus problemas específicos.

Ahora, a practicar. Los ejercicios que vimos a lo largo del libro están ahí para que te ensucies las manos. No te frustres si al principio no te salen; a mí me pasó lo mismo. Lo importante es que entiendas las ideas y sepas implementarlas. ¡Dale para adelante, que vos podés!

